



Documentação técnica

LPS — Plataforma de Gestão do Trabalho

Para desenvolvedores · Gerado em 03/10/2026 a partir de `README.md` e `docs/*.md`

Repositório: github.com/Lucasrafael703/workflow

Versão: add5acd

Sumário

—	Visão geral	3
—	Telas da aplicação	5
01	Arquitetura	33
02	Ambiente local	38
03	Configuração	41
04	Modelos de dados	44
05	Autorização	63
06	Atividades (Demandas) e tarefas	72
07	Contas e autenticação	88
08	Notificações e auditoria	92
09	Rotas	98
10	Frontend	112
11	Testes	121
12	Deploy no Render	130
13	Pendências conhecidas	133
—	Apêndices	141
A	Elementos, Subelementos e Integração com Processos	142
B	Especificação Completa das Colunas Dinâmicas	202
C	Especificação da Visualização por Calendário	266
D	Especificação da Visualização por Kanban	310
E	Especificação da Visualização por Tabela (Quadro)	333

Visão geral

Aplicação Django que implementa o núcleo D0 da LPS: uma plataforma de gestão do trabalho **multi-organização** que organiza **Demandas** (no código, `Activity`) e suas tarefas, torna responsabilidades visíveis, registra o que acontece durante a execução e transforma esse histórico em informação para gestão. As Demandas e as tarefas aparecem em quadros dinâmicos com visões Tabela, Kanban e Calendário.

Atualizado em 03/10/2026 (código no commit `add5acd`). Vocabulário da interface: "Atividade" virou **Demanda**, "Situação" virou **Etapa** e "Condição" virou **Status** (glossário no começo de [docs/01_ARQUITETURA.md](#)).

- O **produto** (regras de negócio e telas) está em `Regras/` e `Telas/`.
- O **código** (como essas regras foram implementadas) está documentado em [docs/](#).

Stack

Item	Versão / escolha
Framework	Django 5.2 (LTS)
Python	3.13/3.14 no desenvolvimento local (o <code>.venv</code> atual é 3.13) · 3.12.7 no Render
Banco	SQLite (dev) · PostgreSQL via <code>DATABASE_URL</code> (prod)
Servidor	<code>runserver</code> (dev) · <code>gunicorn</code> + <code>whitenoise</code> (prod)
Frontend	Templates Django + CSS/JS estáticos, sem bundler
Dependências	<code>django-environ</code> , <code>nh3</code> (sanitização de HTML), <code>gunicorn</code> , <code>whitenoise</code> , <code>psycopy</code> — ver <code>requirements.txt</code>
Testes	Django <code>manage.py test</code> (Python) · <code>node --test</code> com <code>jsdom</code> (JavaScript, <code>npm run test:js</code>)

Início rápido (Windows / PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env          # e define SECRET_KEY
python manage.py migrate
python manage.py seed_lps_demo  # organização "Biasi", setores, ações e perfis
python manage.py createsuperuser
python manage.py runserver
```

Antes de usar as telas, **vincule seu usuário a uma organização** (Admin do Django → Accounts → Perfis → campo "organização"); sem isso toda tela redireciona para o perfil. Detalhes em [docs/02_AMBIENTE_LOCAL.md](#).

O login é feito **por e-mail**, não por nome de usuário. A Caixa de Entrada (`/entrada/`) vem **inativa** (`INTAKE_ENABLED` , ver [docs/03_CONFIGURACAO.md](#)) e o fluxo operacional antigo de tarefas (`/fila/` , `/processos/` , `/tarefas/<id>/...`) está desativado (responde 410): as tarefas vivem no quadro de cada Demanda.

Documentação técnica

#	Documento	Conteúdo
01	Arquitetura	Apps, camadas, multi-tenancy, convenções do código

#	Documento	Conteúdo
02	Ambiente local	Instalação, dados de demonstração, management commands (incluindo os de <code>boards</code>)
03	Configuração	Settings dev/prod, variáveis de ambiente, estáticos e download seguro de anexos
04	Modelos de dados	Todos os models por app, enums, constraints, diagrama ER
05	Autorização	Motor de acessos: ações, perfis, escopos, <code>AuthorizationService</code>
06	Atividades e tarefas	Demandas e tarefas: ciclos de vida, fila, prazos, sessões, processos (o fluxo operacional de tarefas está desativado)
07	Contas e autenticação	Cadastro, confirmação de e-mail, login por e-mail, recuperação de senha, perfil
08	Notificações e auditoria	O que gera notificação, e-mail e registro de auditoria
09	Rotas	Referência de todas as URLs
10	Frontend	Templates, JS, CSS, context processors, template tags
11	Testes	Como rodar, organização, helpers, teste JS
12	Deploy no Render	<code>render.yaml</code> , primeiro deploy, limitações do plano free, histórico de correções
13	Pendências conhecidas	Defeitos e dívidas técnicas mapeados

Especificações dos quadros dinâmicos

Além dos docs 01–13 (que descrevem o **código atual**), a pasta `docs/` guarda cinco especificações `LPS_*.md` dos quadros dinâmicos (app `boards`). As de colunas dinâmicas, Tabela e Kanban trazem, no começo, uma tabela de **status de implementação** (o que está implementado, parcial ou não implementado, e onde está no código); a de Calendário é texto de especificação, sem essa tabela, e a de Elementos/Subelementos descreve uma evolução arquitetural **proposta**, ainda não implementada:

Documento	Conteúdo
LPS_ESPECIFICACAO_COLUNAS_DINAMICAS.md	Motor de colunas: tipos de coluna, configurações, etiquetas, células tipadas
LPS_VISUALIZACAO_TABELA_QUADRO.md	Tabela do Quadro (o "Quadro principal"): grupos, colunas, itens, edição inline
LPS_VISUALIZACAO_KANBAN.md	Kanban: agrupamento, raias, cartões, movimentação
LPS_VISUALIZACAO_CALENDARIO.md	Calendário: itens posicionados por uma coluna de Data
LPS_ELEMENTOS_SUBELEMENTOS_PROCESSOS.md	Conceito de Elemento e Subelemento e a integração com Processos

Uma versão compilada em PDF da documentação técnica está em `docs/LPS_Documentacao_Tecnica.pdf`. Ela foi gerada em 29–30/09/2026 e **não** acompanha as mudanças posteriores (Demandas, quadros dinâmicos, desativação do fluxo operacional de tarefas, download seguro de anexos): para o estado atual, vale o texto dos `.md` .

Telas da aplicação

Capturas geradas automaticamente a partir da aplicação rodando localmente, com dados fictícios de demonstração. Geradas em 03/10/2026.

Acesso

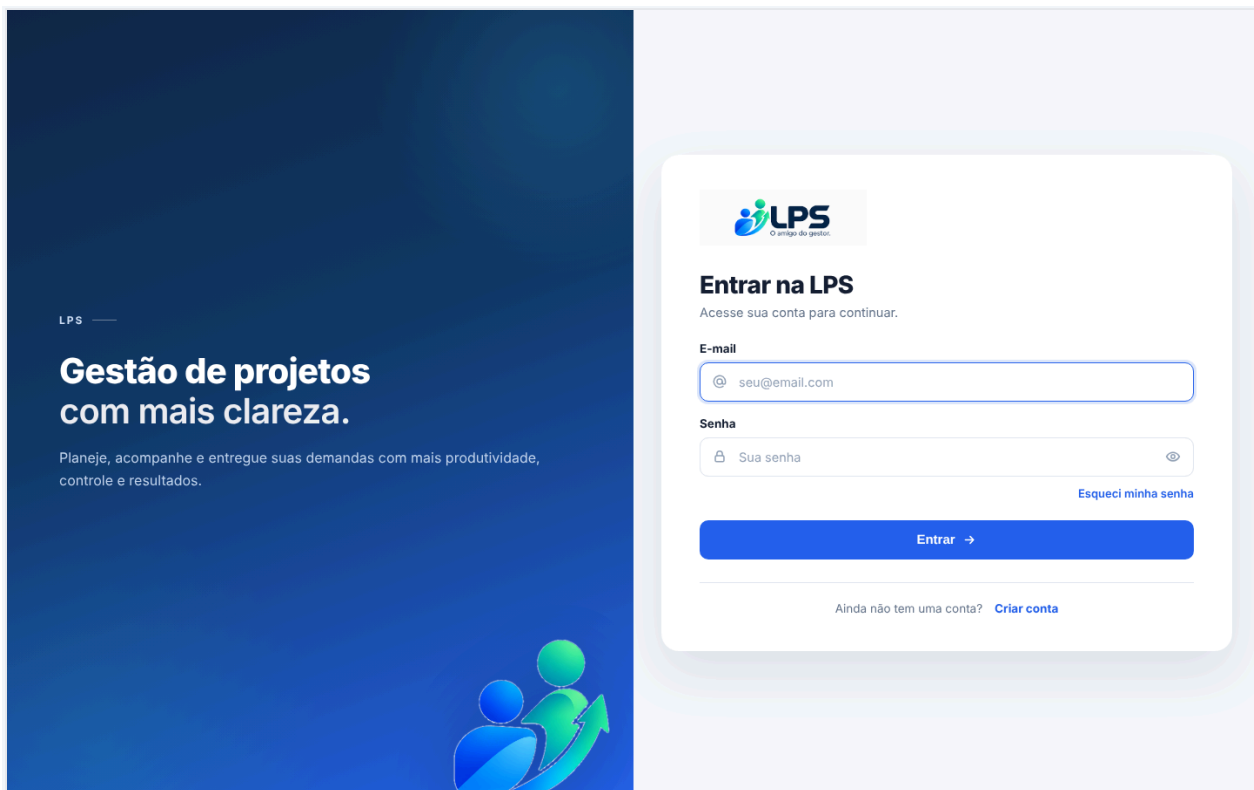


Figura 1 — Login (/accounts/login/)

Entrada na LPS por e-mail e senha, com links para recuperar a senha e criar conta.



Criar sua conta

Comece criando seu acesso à LPS.

Nome de exibição

E-mail

Senha

Confirmar senha

☐ Li e aceito os [Termos de Uso](#) e a [Política de Privacidade](#).

[Criar conta](#) →

Já possui uma conta? [Entrar](#)

Figura 2 — Criar conta (/accounts/criar-conta/)
Autocadastro: nome, e-mail, senha e aceite dos termos. A conta só fica ativa após confirmar o e-mail.



Confirme seu e-mail

Enviamos um código de confirmação para:
vi***@exemplo.com

[Confirmar e-mail](#)

[Não recebeu o código? Reenviar](#) [E-mail errado? Alterar e-mail](#)

Figura 3 — Confirmar e-mail (/accounts/confirmar-email/)
Confirmação do e-mail com o código de 6 dígitos enviado após o cadastro (em desenvolvimento, o código aparece no console do servidor).

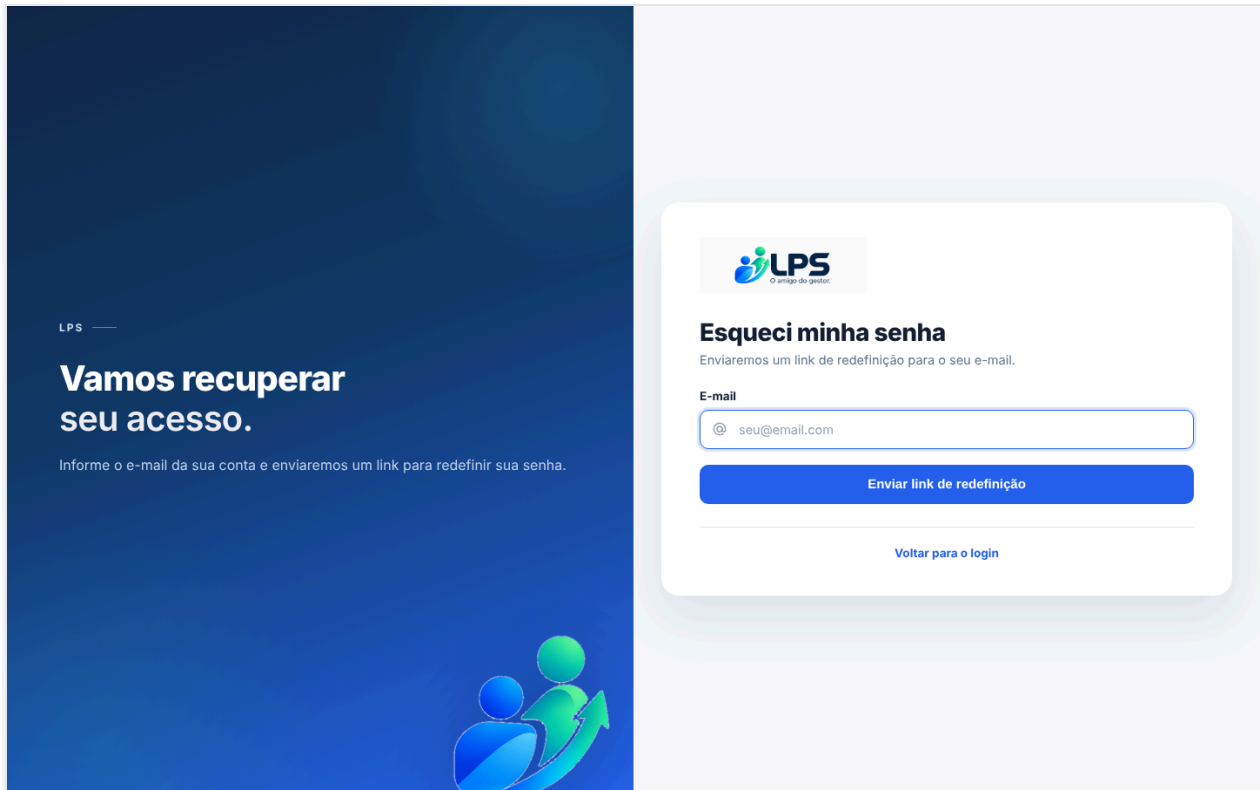


Figura 4 — Recuperar senha (/accounts/esqueci-senha/)
Pedido de redefinição de senha: a pessoa informa o e-mail e recebe um link.

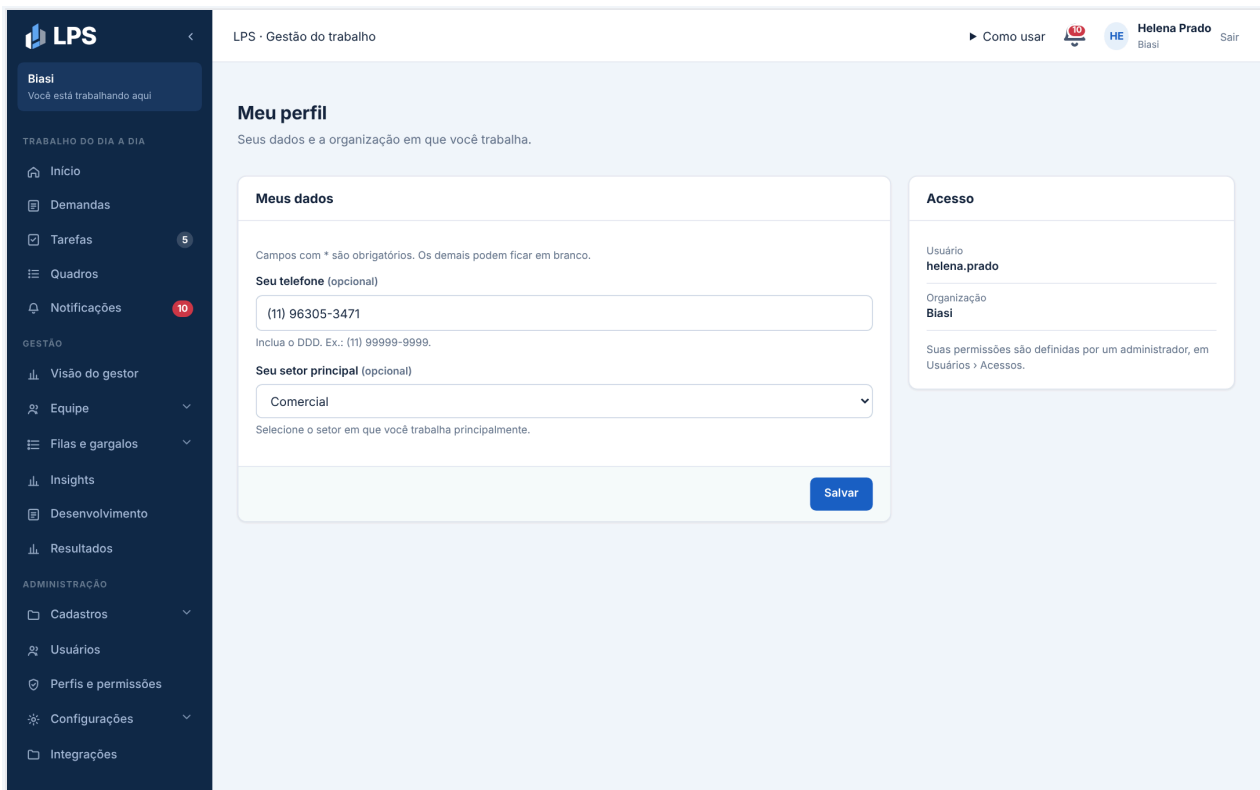


Figura 5 — Meu perfil (/accounts/me/)
Dados da própria conta: telefone e setor principal usado como padrão nos filtros.

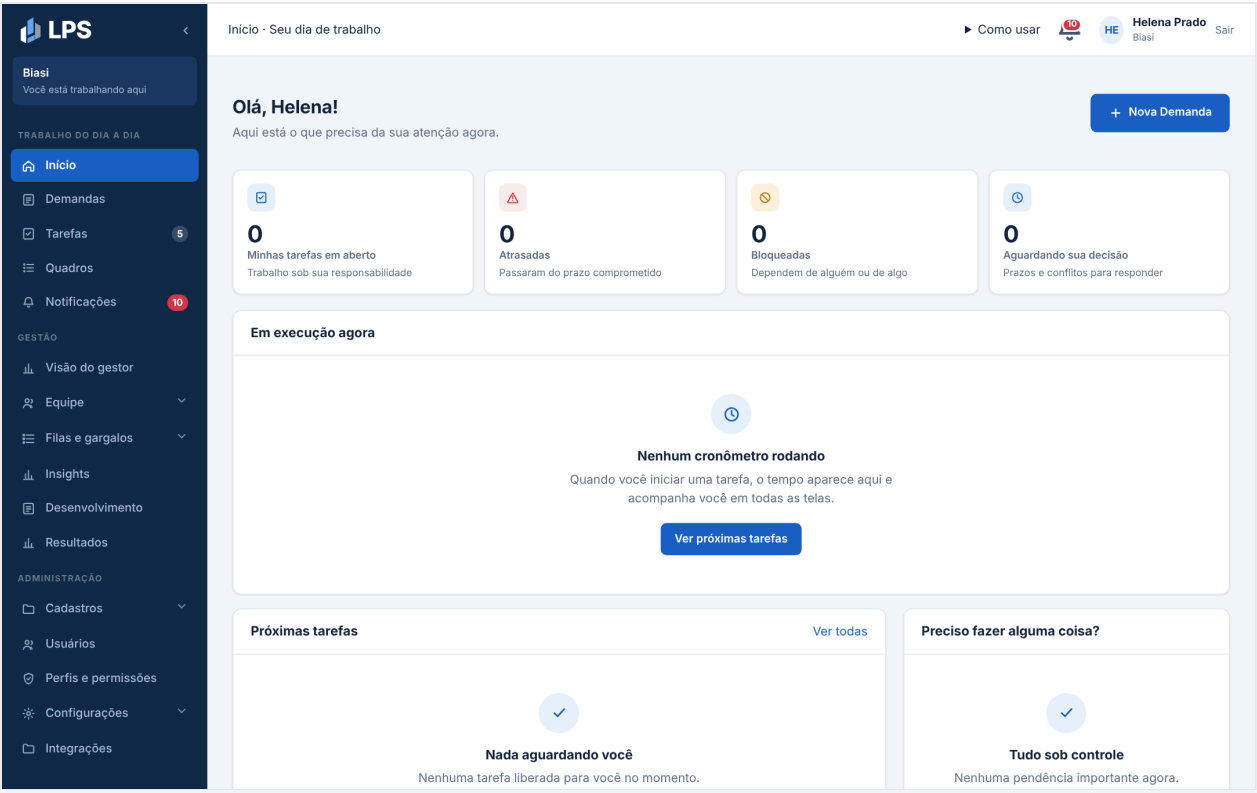


Figura 6 — Início (/)

Painel inicial "O que precisa da minha atenção agora?", com indicadores, tarefas próximas e pendências.

Operação — demandas

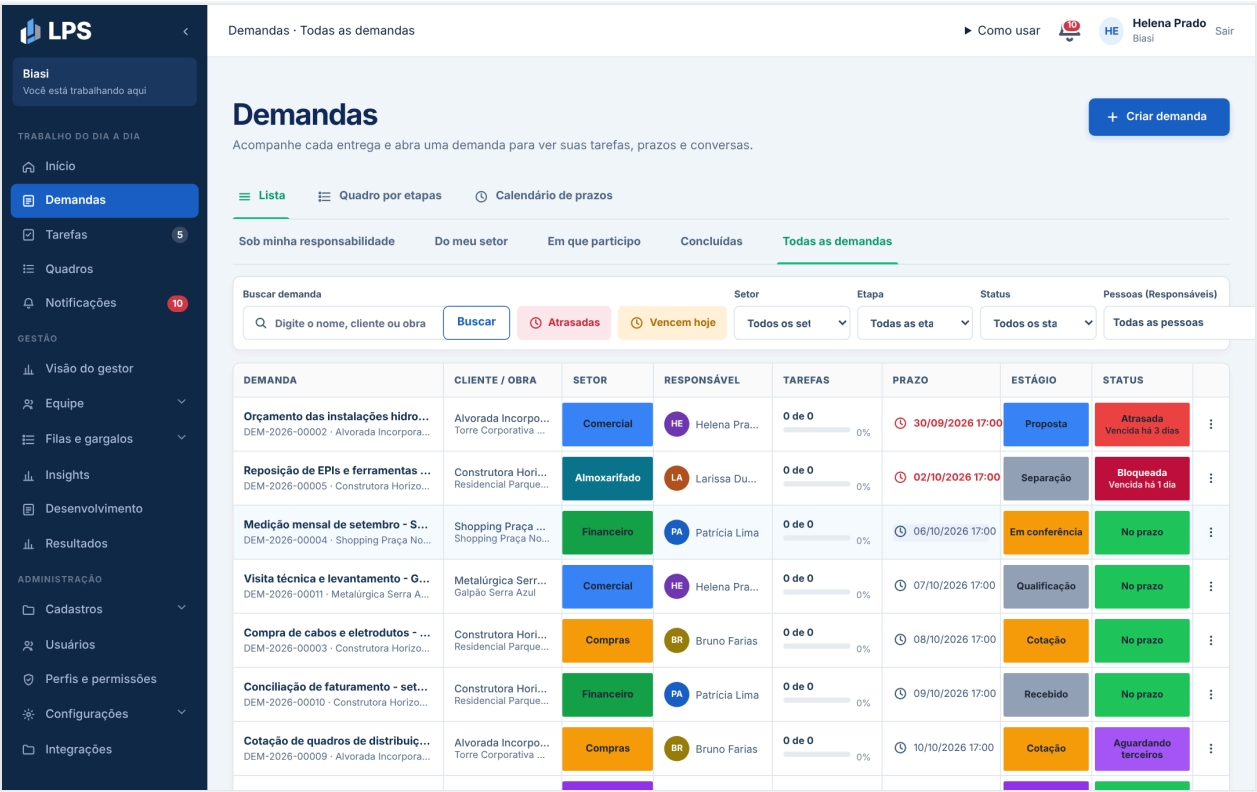


Figura 7 — Lista de demandas (/demandas/)

Lista operacional de demandas com abas de escopo, filtros e edição direta de setor, etapa e status.

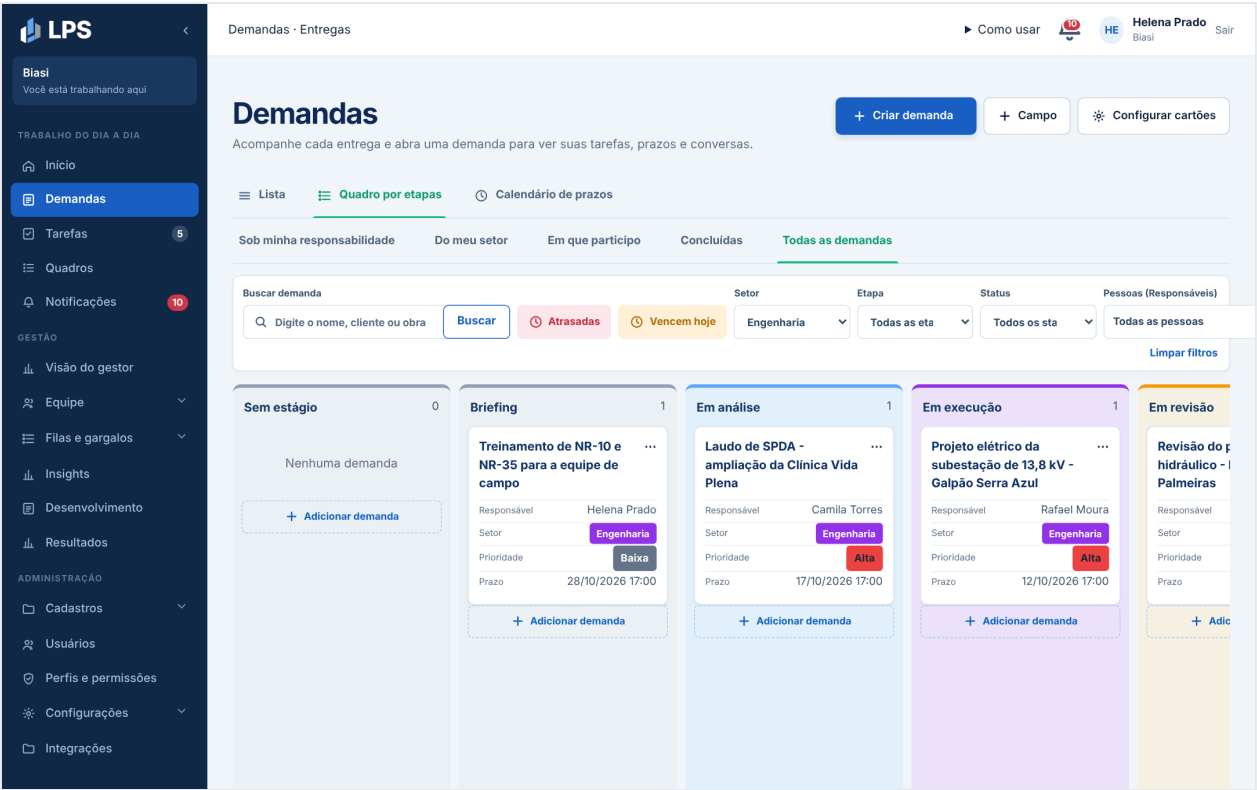


Figura 8 — Quadro por etapas (/demandas/kanban/)
Quadro das demandas organizadas por etapa do fluxo do setor, com cartões arrastáveis.

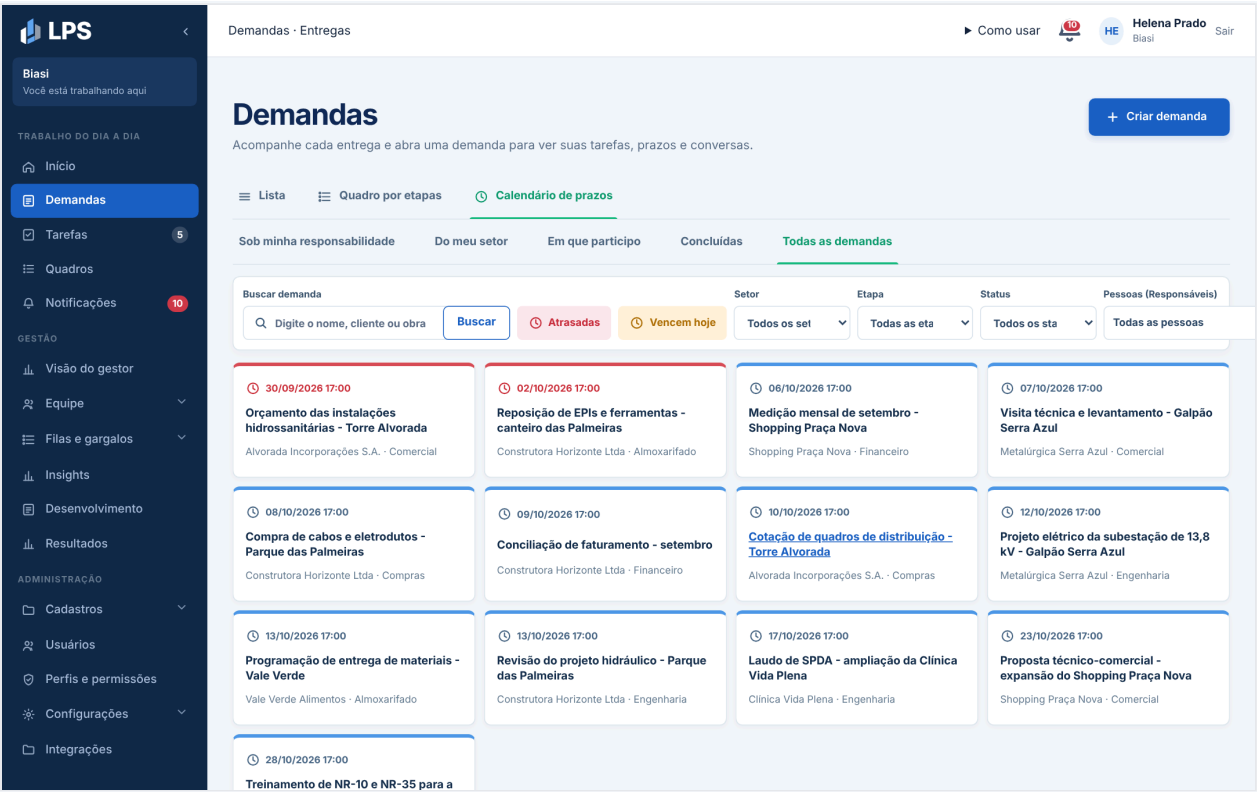


Figura 9 — Calendário de prazos (/demandas/calendario/)
Demandas ordenadas por prazo; as atrasadas aparecem em destaque.

LPS

Biagi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Biagi

Sair

Nova demanda

X

Preencha as informações para criar uma nova demanda.

1

2

3

4

Informações principais

Cliente e obra

Quadro de tarefas

Descrição e arquivos

1

Informações principais

Dados básicos da demanda e responsáveis.

Nome da demanda *

Manutenção elétrica preventiva - Clínica Vida Plena

Descreva a entrega esperada. Ex.: Orçamento do gerador aprovado.

Atribuído a *

Helena Prado

Essa pessoa será responsável pela entrega e acompanhará as tarefas.

Setor responsável *

Engenharia

Escolha o setor que será responsável por esta demanda.

Etapa

Briefing

Etapa visual deste setor.

Status

No prazo

Status manual deste setor; não muda o status operacional.

Prazo de vencimento

30/10/2026

17:00

Data e horário para conclusão da demanda. Sem horário, vale até o fim do dia.

Urgência

Baixa

Média

Alta

Indique o quanto esta demanda precisa de atenção.

Organização

Figura 10 — Nova demanda — passo 1 (/demandas/nova/)
Passo 1 da janela de criação: informações principais (nome, responsável, setor, etapa, prazo e urgência).

LPS

Biagi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Biagi

Sair

Nova demanda

X

Preencha as informações para criar uma nova demanda.

1

2

3

4

Informações principais

Cliente e obra

Quadro de tarefas

Descrição e arquivos

2

Cliente e obra

Dados relacionados ao cliente, obra e centro de custo.

Cliente

Clínica Vida Plena

Selecione o cliente relacionado a esta demanda.

Obra

Clínica Vida Plena - Ampliação

Selecione a obra relacionada, se houver.

Centro de custo

CC-1005 - Clínica Vida Plena - Ampliação

Escolha o centro de custo para apropriação desta demanda.

Solicitante (Externo)

Nome do solicitante

Pessoa que solicitou a demanda (cliente, fornecedor, etc.).

Endereço complementar

Ex.: Rua, número, complemento, bairro, cidade

Informações adicionais do local, se necessário.

Cancelar

Voltar

Continuar

Figura 11 — Nova demanda — passo 2 (/demandas/nova/)
Passo 2: cliente, obra e centro de custo, encadeados entre si.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usarHEHelena PradoBiassiSair

Nova demanda

X

Preencha as informações para criar uma nova demanda.

1

2

3

4

Informações principais

Cliente e obra

Quadro de tarefas

Descrição e arquivos

3

Quadro de tarefas

Defina como as tarefas desta demanda serão organizadas.

Como deseja organizar as tarefas?

☐ Começar em branco

Crie um novo quadro de tarefas do zero para esta demanda.

☒ Usar quadro existente

Use um quadro já configurado como modelo.

Modelo de quadro

Orçamentos 2026

Modelos de qualquer setor podem ser usados; a cópia será independente.

Cancelar

< Voltar

Continuar →

Figura 12 — Nova demanda — passo 3 (/demandas/nova/)

Passo 3: escolha entre começar com o quadro de tarefas em branco ou copiar um quadro existente.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usarHEHelena PradoBiassiSair

Nova demanda

X

Preencha as informações para criar uma nova demanda.

1

2

3

4

Informações principais

Cliente e obra

Quadro de tarefas

Descrição e arquivos

4

Descrição e arquivos

Informações complementares para a execução da demanda.

Observações

B / S

Contrato de manutenção preventiva trimestral dos quadros, SPDA e iluminação de emergência.

90/2000

Inclua todas as informações necessárias para a execução desta demanda.

Link / caminho dos arquivos

https://arquivos.exemplo.com/vida-plena/manutencao-2026

Você pode colar o link ou o caminho da pasta/arquivo relacionado a esta demanda (Google Drive, OneDrive, Dropbox, etc.), ou um caminho de rede.

Cancelar

< Voltar

Criar demanda →

Figura 13 — Nova demanda — passo 4 (/demandas/nova/)

Passo 4: descrição com editor de texto e o local onde os arquivos do projeto estão guardados.

LPS — Documentação técnica

11 / 359

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Sair

Demandas > Projeto elétrico da subestação de 13,8 kV - Ga...

DEM-2026-00001

RESPONSÁVEL

Rafael Moura

PRAZO

12/10/2026 17:00

Assumir

Aberta

Alta

Engenharia

Etapa atual

14%

1 de 7 concluídas

3 em andamento · 3 a fazer

Tarefas

Abrir quadro

7tarefas

1concluída

3em andamento

1menção nova

NOME DA TAREFA	RESPONSÁVEL	STATUS	PRAZO
Aprovar aditivo de esc...	HP Helena Prado	Em andamento	02/10/2026 VENCIDO
Diagrama unifilar da su...	CT Camila Torres	Em andamento	06/10/2026
Estudo de seletividade ...	DN Diego Nogueira	Em andamento	08/10/2026
Memorial descritivo	DN Diego Nogueira	Não iniciado	09/10/2026

Conversa

Histórico

Escreva uma atualização e mencione pessoas com

Todas as pessoas com

Mensagem

28/09/2026

CA Camila Torres 4 dias, 15 horas atrás 28/09/2026 08:31

@rafael.moura conclui o levantamento de cargas do Bloco B. A demanda contratada subiu para 750 kVA; vale revisar o transformador antes de fechar o unifilar.

Responder 1 Reagir

RA Rafael Moura 4 dias, 13 horas atrás 28/09/2026 11:14

Obrigado, @camila.torres. Vou validar com o cliente ainda hoje. @helena.prado, isso pode impactar o valor da proposta aprovada.

Responder Reagir

HE Helena Prado 4 dias, 9 horas atrás 28/09/2026 14:59

Vamos tratar como aditivo. @thiago.rocha, prepara o comparativo de custos para o cliente até quarta.

Figura 14 — Ficha da demanda (/demandas/<pk>/)
Ficha da demanda: responsável, prazo, etapa atual, andamento das tarefas e conversa da equipe.

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Sair

7tarefas

1concluída

3em andamento

1menção nova

NOME DA TAREFA	RESPONSÁVEL	STATUS	PRAZO
Aprovar aditivo de esc...	HP Helena Prado	Em andamento	02/10/2026 VENCIDO
Diagrama unifilar da su...	CT Camila Torres	Em andamento	06/10/2026
Estudo de seletividade ...	DN Diego Nogueira	Em andamento	08/10/2026
Memorial descritivo	DN Diego Nogueira	Não iniciado	09/10/2026
Revisão técnica do proj...	RM Rafael Moura	Não iniciado	11/10/2026

+ 2 tarefas no quadro.

Esta é uma prévia. Para editar, atribuir pessoas e organizar o trabalho completo, abra o quadro.

Detalhes da demanda

Cliente	Obra	Centro de custo
Metalúrgica Serra Azul	Galpão Serra Azul	CC-1004 - Galpão Serra Azul
Empresa	Criador	
Biasi Engenharia	rafael.moura em 18/09/2026 14:39	
Marcadores		
Elétrica	Urgente	
Descrição		
Projeto executivo da subestação abrigada de 13,8 kV, com cálculo de demanda, diagrama unifilar, memorial		

Conversa

Histórico

Escreva uma atualização e mencione pessoas com

Todas as pessoas com

Mensagem

28/09/2026

CA Camila Torres 4 dias, 15 horas atrás 28/09/2026 08:31

@rafael.moura conclui o levantamento de cargas do Bloco B. A demanda contratada subiu para 750 kVA; vale revisar o transformador antes de fechar o unifilar.

Responder 1 Reagir

RA Rafael Moura 4 dias, 13 horas atrás 28/09/2026 11:14

Obrigado, @camila.torres. Vou validar com o cliente ainda hoje. @helena.prado, isso pode impactar o valor da proposta aprovada.

Responder Reagir

HE Helena Prado 4 dias, 9 horas atrás 28/09/2026 14:59

Vamos tratar como aditivo. @thiago.rocha, prepara o comparativo de custos para o cliente até quarta.

Responder 1 Reagir

29/09/2026

DI Diego Nogueira 3 dias, 13 horas atrás 29/09/2026 10:28

Estudo de seletividade em andamento.

@camila.torres me passa as correntes de curto quando fechar o unifilar?

Responder Reagir

Figura 15 — Ficha da demanda — tarefas e conversa (/demandas/<pk>/)
Parte inferior da ficha: resumo e lista das tarefas do quadro e a conversa com @menções e reações.

LPS — Documentação técnica

12 / 359

LPS

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Sair

Demandas > Projeto elétrico da subestação de 13,8 kV - Ga...

DEM-2026-00001

RESPONSÁVEL

Rafael Moura

PRAZO

12/10/2026 17:00

Assumir

Projeto elétrico da subestação de 13,8 kV - Galpão Serra Azul

Cliente: Metalúrgica Serra Azul

Obra: Galpão Serra Azul

Centro de custo: CC-1004 - Galpão Serra Azul

Aberta

Alta

Engenharia

Etapa atual

14%

1 de 7 concluídas

3 em andamento · 3 a fazer

Tarefas

Abrir quadro

7 tarefas

1 concluída

3 em andamento

1 menção nova

NOME DA TAREFA	RESPONSÁVEL	STATUS	PRAZO
Aprovar aditivo de esc...	HP Helena Prado	Em andamento	02/10/2026 VENCIDO
Diagrama unifilar da su...	CT Camila Torres	Em andamento	06/10/2026
Estudo de seletividade ...	DN Diego Nogueira	Em andamento	08/10/2026
Memorial descritivo	DN Diego Nogueira	Não iniciado	09/10/2026

Conversa

Histórico

Helena Prado alterou anexo

anexo Unifilar_subestacao_13_8kV.dwg

25/09/2026 14:01

Helena Prado alterou anexo

anexo Memorial_descritivo_rev0.pdf

25/09/2026 10:51

Helena Prado alterou etapa

etapa Briefing → Em execução

18/09/2026 14:49

Rafael Moura criou a demanda

Projeto elétrico da subestação de 13,8 kV - Galpão Serra Azul

18/09/2026 14:45

Figura 16 — Ficha da demanda — histórico (/demandas/<pk>/)
Aba Histórico da ficha: todas as alterações da demanda, com autor e horário.

LPS

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Sair

Demandas > Projeto elétrico da subestação de 13,8 kV - Ga...

DEM-2026-00001

RESPONSÁVEL

Rafael Moura

PRAZO

12/10/2026 17:00

Assumir

Projeto elétrico da subestação de 13,8 kV - Galpão Serra Azul

Cliente: Metalúrgica Serra Azul

Obra: Galpão Serra Azul

Centro de custo: CC-1004 - Galpão Serra Azul

Aberta

Alta

Engenharia

Etapa atual

14%

1 de 7 concluídas

3 em andamento · 3 a fazer

Tarefas

Abrir quadro

7 tarefas

1 concluída

3 em andamento

1 menção nova

NOME DA TAREFA	RESPONSÁVEL	STATUS	PRAZO
Aprovar aditivo de esc...	HP Helena Prado	Em andamento	02/10/2026 VENCIDO
Diagrama unifilar da su...	CT Camila Torres	Em andamento	06/10/2026
Estudo de seletividade ...	DN Diego Nogueira	Em andamento	08/10/2026
Memorial descritivo	DN Diego Nogueira	Não iniciado	09/10/2026

Conversa

Editar

Transferir responsável

Alterar prazo

Marcar como pendente

Concluir

Cancelar

28/09/2026

CA Camila Torres 4 dias, 15 horas atrás 28/09/2026 08:31

@rafael.moura conclui o levantamento de cargas do Bloco B. A demanda contratada subiu para 750 kVA; vale revisar o transformador antes de fechar o unifilar.

Responder 1 Reagir

RA Rafael Moura 4 dias, 13 horas atrás 28/09/2026 11:14

Obrigado, @camila.torres. Vou validar com o cliente ainda hoje. @helena.prado, isso pode impactar o valor da proposta aprovada.

Responder Reagir

HE Helena Prado 4 dias, 9 horas atrás 28/09/2026 14:59

Vamos tratar como aditivo. @thiago.rocha, prepara o comparativo de custos para o cliente até quarta.

Figura 17 — Ficha da demanda — ações (/demandas/<pk>/)
Menu de ações da ficha: alterar prazo, trocar responsável, marcar pendência, finalizar e editar.

LPS — Documentação técnica

13 / 359

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Demandas · Entregas

Como usar

HE

Helena Prado

Sair

Editar demanda

Atualize as informações da demanda. DEM-2026-00001

1

2

3

4

Informações principais

Cliente e obra

Quadro de tarefas

Descrição e arquivos

2

Cliente e obra

Dados relacionados ao cliente, obra e centro de custo.

Cliente

Metalúrgica Serra Azul

Selecione o cliente relacionado a esta demanda.

Obra

Galpão Serra Azul

Selecione a obra relacionada, se houver.

Centro de custo

CC-1004 - Galpão Serra Azul

Escolha o centro de custo para apropriação desta demanda.

Solicitante (Externo)

Nome do solicitante

Pessoa que solicitou a demanda (cliente, fornecedor, etc.).

Endereço complementar

Ex.: Rua, número, complemento, bairro, cidade

Informações adicionais do local, se necessário.

Cancelar

< Voltar

Continuar →

Figura 18 — Editar demanda (/demandas/<pk>/editar/)

A mesma janela de quatro etapas aberta para edição, aqui direto em "Cliente e obra".

Operação — tarefas e fila

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Tarefas · Trabalho a fazer

Como usar

HE

Helena Prado

Sair

Tarefas

Veja o que está nos quadros em que você é responsável e abra o quadro da demanda quando precisar organizar o trabalho.

4

Atrasadas

Prazo anterior a hoje

1

Vencem hoje

Prazo = hoje

16

Em andamento

Já iniciadas

4

Mencões

Não lidas

Lista

Kanban

Calendário

Q

Buscar tarefa, demanda, cliente ou obra...

Pessoa

Todas as pessoas

Status

Em aberto

Prazo

Todos os prazos

Filtrar

NOME DA TAREFA	DEMANDA	CLIENTE / OBRA	RESPONSÁVEL	STATUS	PRAZO
Cotação de tubulaç...	Orçamento das instal... DEM-2026-00002	Alvorada Incorporaçõ... Torre Corporativa Alvorada	BF Bruno Farias	Em andamento	30/09/2026 VENCIDO
Aprovar aditivo de ...	Projeto elétrico da su... DEM-2026-00001	Metalúrgica Serra Azul Galpão Serra Azul	HP Helena Prado	Em andamento	02/10/2026 VENCIDO
Composição de pre...	Orçamento das instal... DEM-2026-00002	Alvorada Incorporaçõ... Torre Corporativa Alvorada	PL Patrícia Lima	Em andamento	02/10/2026 VENCIDO
Separar itens dispo...	Reposição de EPIs e f... DEM-2026-00005	Construtora Horizont... Residencial Parque das Palmeiras	LD Larissa Duarte	Em andamento	02/10/2026 VENCIDO
Preparar pauta da r...	Visita técnica e levan... DEM-2026-00011	Metalúrgica Serra Azul Galpão Serra Azul	HP Helena Prado	Em andamento	03/10/2026
Conferir quantidad...	Medição mensal de s... DEM-2026-00004	Shopping Praça Nova Shopping Praça Nova -	PL Patrícia Lima	Em andamento	04/10/2026

Figura 19 — Tarefas — lista (/tarefas/)

Painel Tarefas: o que está nos quadros das demandas, com indicadores e filtros por pessoa, status e prazo.

LPS — Documentação técnica

14 / 359

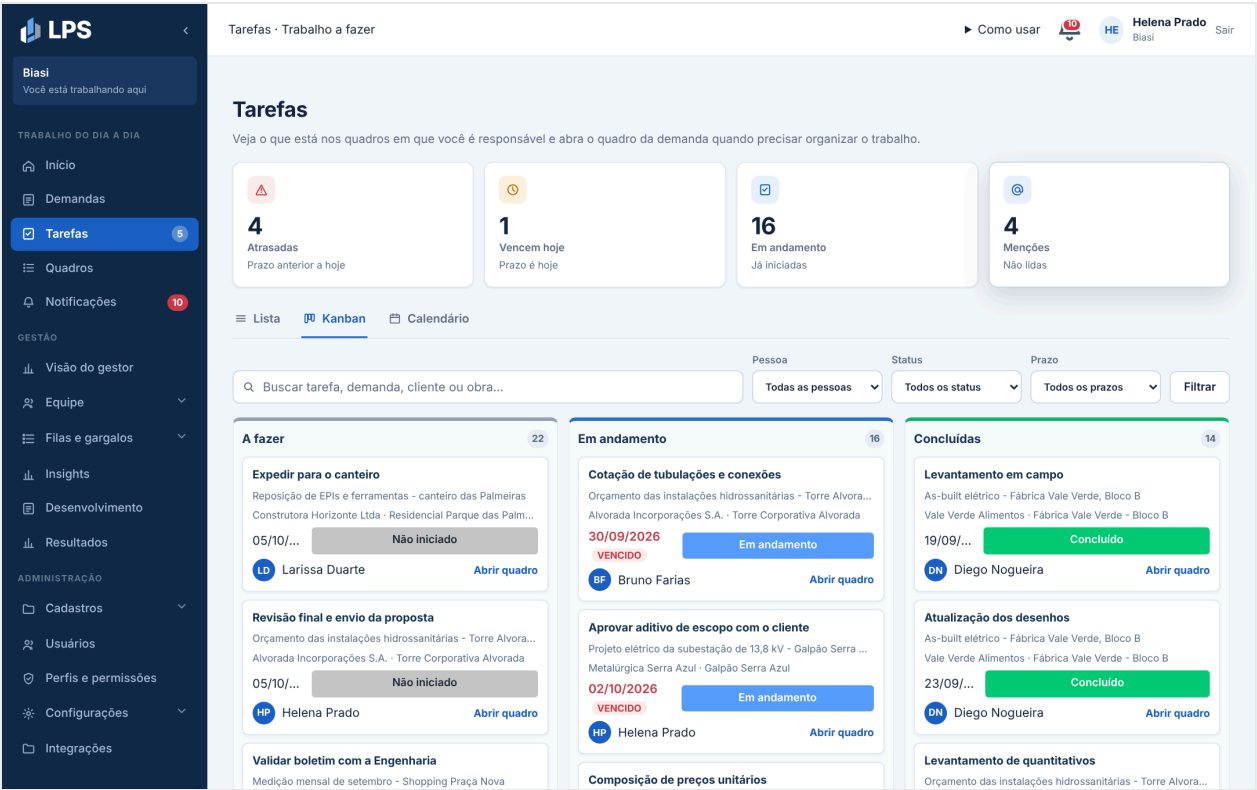


Figura 20 — Tarefas — Kanban (/tarefas/kanban/) Tarefas em colunas A fazer, Em andamento e Concluídas, derivadas da etiqueta de status do quadro.

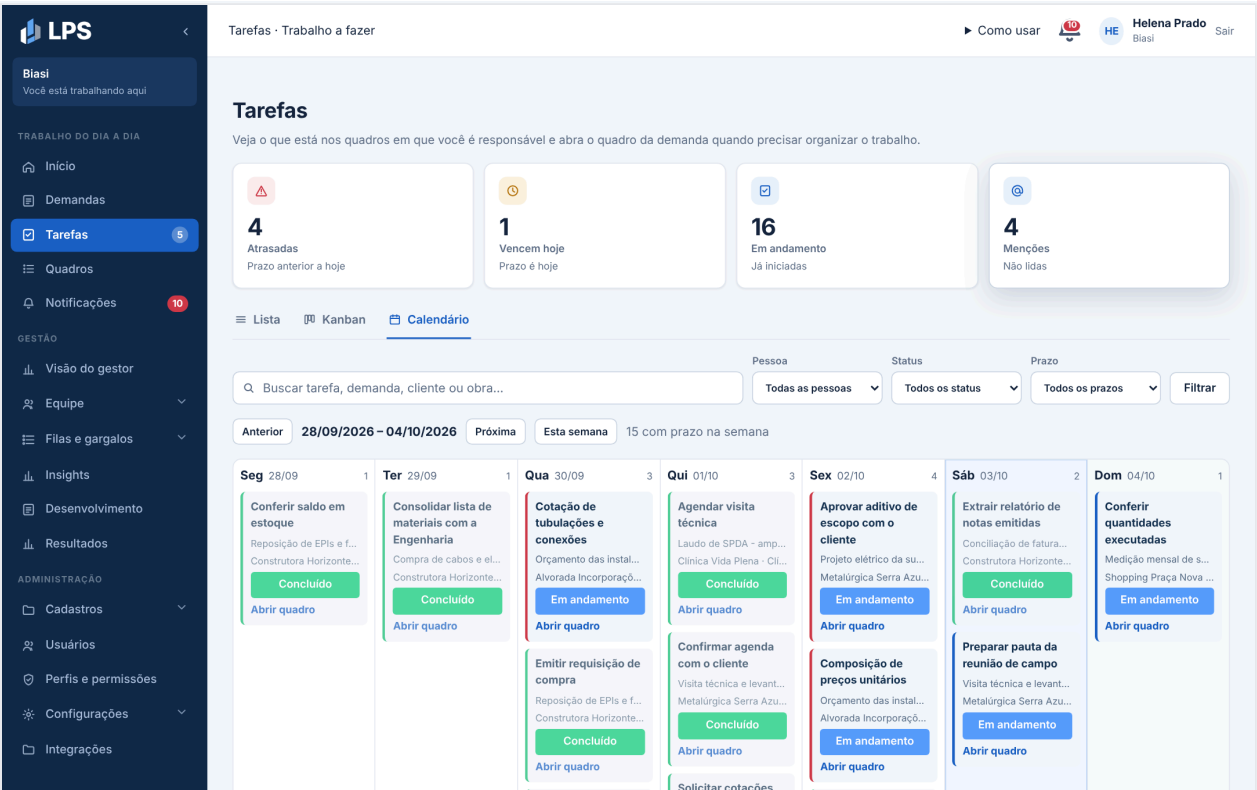


Figura 21 — Tarefas — calendário semanal (/tarefas/calendario/) Calendário semanal das tarefas por prazo.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Quadros · Monte do seu jeito

Como usar

HE

Helena Prado

Biassi

Sair

Quadros

Projeto elétrico da subestação de 13,8 kV - Galpão Se

Histórico

Quadro principal

Kanban

Calendário

Buscar neste quadro...

Todas as pessoas

Buscar

Tarefas

7 tarefas

Nome da Tarefa	Responsável	Status	Data	Horas previs...	Observação	
Levantamento de cargas do ...	CT Camila Torres	Concluído	27/09/2026	16 h	Carga total: 750 kVA	
Diagrama unifilar da subesta...	CT Camila Torres	Em andamento	06/10/2026	24 h	Revisar após valida...	
Estudo de seletividade e coo...	DN Diego Nogueira	Em andamento	08/10/2026	20 h	Usar curvas do fabr...	
Memorial descritivo	DN Diego Nogueira	Não iniciado	09/10/2026	12 h		
Revisão técnica do projeto	RM Rafael Moura	Não iniciado	11/10/2026	8 h	Revisão pelo gestor	
Emissão da ART e document...	RM Rafael Moura	Não iniciado	12/10/2026	6 h		
Aprovar aditivo de escopo c...	HP Helena Prado	Em andamento	02/10/2026	4 h	Reunião quinta-feira	

+ Adicionar tarefa

+ Novo grupo

Figura 22 — Quadro da demanda — tabela (/quadros/<id>/)
Quadro de tarefas de uma demanda em tabela: responsável, status, data e colunas extras, com edição direta nas células.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Quadros · Monte do seu jeito

Como usar

HE

Helena Prado

Biassi

Sair

Quadros

Projeto elétrico da subestação de 13,8 kV - Galpão Se

Histórico

Quadro principal

Kanban

Calendário

Buscar neste quadro...

Todas as pessoas

Buscar

Agrupar por

Status

Ordenar

Ordem do quadro

Configurar cartões

Filtros

EM BREVE

Não iniciado

3

Memorial descritivo

DN Diego Nogueira

09/10/2026

12 h

—

Em andamento

3

Diagrama unifilar da subestação

CT Camila Torres

06/10/2026

24 h

Revisar após validação do cliente

Concluído

1

Levantamento de cargas do Bloco B

CT Camila Torres

27/09/2026

16 h

Carga total: 750 kVA

+ Adicionar nome da tarefa

Revisão técnica do projeto

RM Rafael Moura

11/10/2026

8 h

Revisão pelo gestor

Estudo de seletividade e coordenação

DN Diego Nogueira

08/10/2026

20 h

Usar curvas do fabricante

Emissão da ART e documentação final

RM Rafael Moura

12/10/2026

6 h

—

Aprovar aditivo de escopo com o cliente

HP Helena Prado

02/10/2026

4 h

Reunião quinta-feira

Figura 23 — Quadro da demanda — Kanban (/quadros/visoes/<id>/)
O mesmo quadro de tarefas agrupado por etiqueta de status.

LPS — Documentação técnica

16 / 359

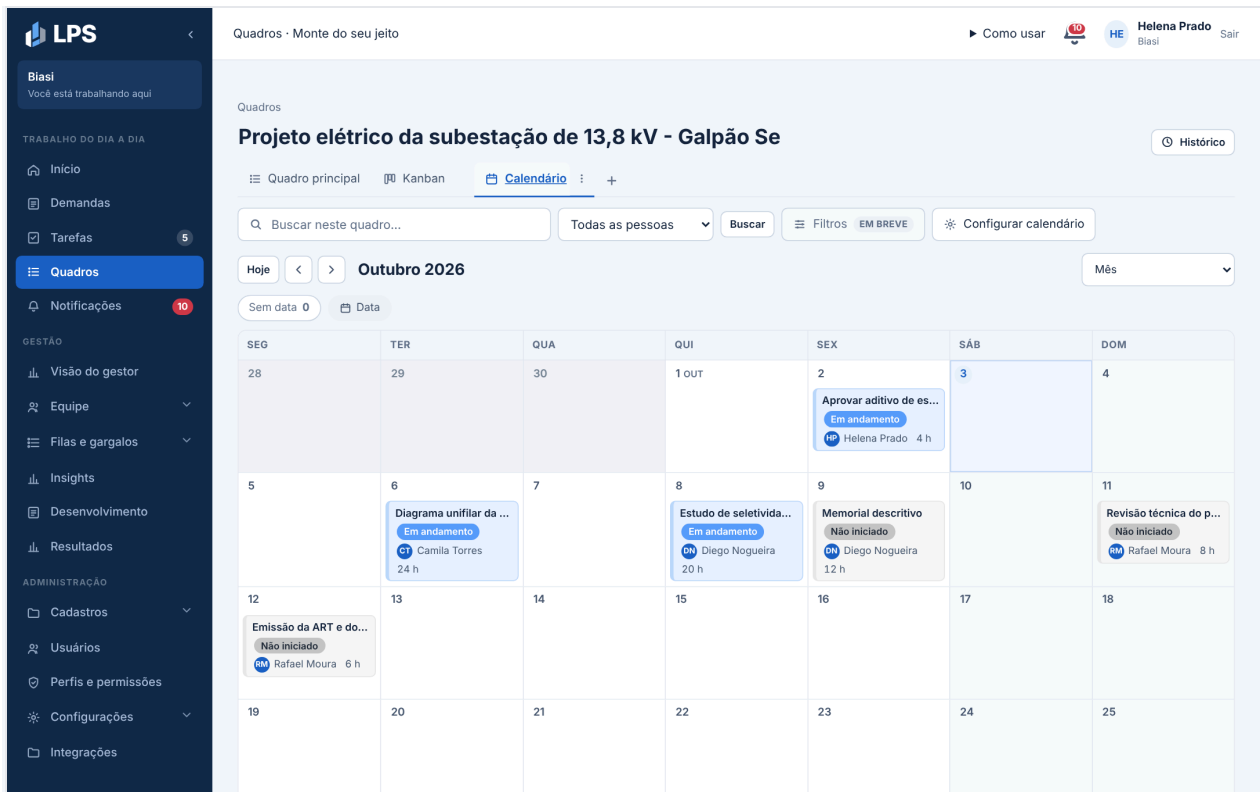


Figura 24 — Quadro da demanda — calendário (/quadros/visoes/<id>/)
O mesmo quadro de tarefas em visão de calendário mensal, posicionado pela coluna de data.

Quadros dinâmicos

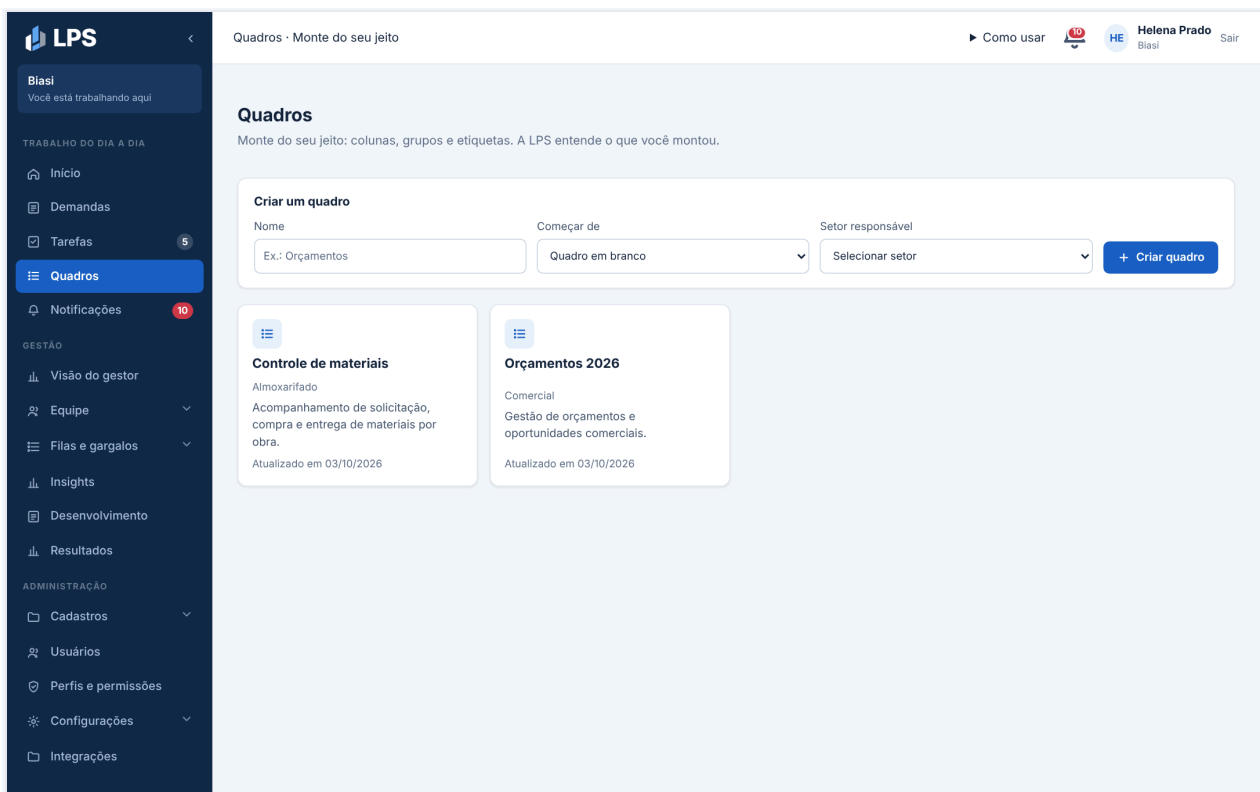


Figura 25 — Quadros (/quadros/)
Lista de quadros da organização e formulário para criar um quadro em branco ou a partir de um modelo.

Quadros · Monte do seu jeito

Como usar

HE Helena Prado

Sair

Quadros

Orçamentos 2026

Histórico

Excluir quadro

Quadro principal

Kanban por Status

Calendário de prazos

Kanban por setor

+

Buscar neste quadro...

Todas as pessoas

Buscar

Oportunidades 3 tarefas

Nome da Tarefa	Cliente	Responsável	Setor	Prazo	Status	Valor	Probabi..
Galpão Serra Azul - subesta...	Metalúrgica Serra Azul	TR Thiago Rocha	Engenharia	17/10/2026	Levantamento	R\$ 2.100.000,00	
Shopping Praça Nova - nova...	Shopping Praça Nova	HP Helena Prado	Comercial	24/10/2026	Novo	R\$ 4.600.000,00	
Galpão logístico Itatiba	Metalúrgica Serra Azul	TR Thiago Rocha	Suprimentos	31/10/2026	Novo	R\$ 1.300.000,00	

+ Adicionar nome da tarefa

Em andamento 2 tarefas

Nome da Tarefa	Cliente	Responsável	Setor	Prazo	Status	Valor	Probabi..
Torre Corporativa Alvorada - ...	Alvorada Incorporações	TR Thiago Rocha	Engenharia	09/10/2026	Cotação	R\$ 3.250.000,00	
Parque das Palmeiras - fase 2	Construtora Horizonte	HP Helena Prado	Suprimentos	12/10/2026	Em revisão	R\$ 890.000,00	

+ Adicionar nome da tarefa

Propostas enviadas 2 tarefas

Nome da Tarefa	Cliente	Responsável	Setor	Prazo	Status	Valor	Probabi..
Clinica Vida Plena - ampliaç...	Clinica Vida Plena	HP Helena Prado	Comercial	05/10/2026	Proposta enviada	R\$ 1.480.000,00	
Hotel Marina Azul - retrofit el...	Hotel Marina Azul	TR Thiago Rocha	Engenharia	14/10/2026	Revisão cliente	R\$ 2.780.000,00	

Figura 26 — Quadro — tabela (/quadros/<id>/)
Quadro de orçamentos em tabela, agrupado por fase, com colunas de texto, pessoa, lista, data, status, moeda e número.

Quadros · Monte do seu jeito

Como usar

HE Helena Prado

Sair

Quadros

Orçamentos 2026

Histórico

Excluir quadro

Quadro principal

Kanban por Status

Calendário de prazos

Kanban por setor

+

Buscar neste quadro...

Todas as pessoas

Buscar

Agrupar por

Status

Ordenar

Ordem do quadro

Configurar cartões

Filtros

EM BREVE

Novo 2

R\$ 5.900.000,00

Shopping Praça Nova - nova ala

Shopping Praça Nova

HP Helena Prado

24/10/2026

R\$ 4.600.000,00

Galpão logístico Itatiba

Metalúrgica Serra Azul

TR Thiago Rocha

31/10/2026

R\$ 1.300.000,00

+ Adicionar nome da tarefa

Levantamento 1

R\$ 2.100.000,00

Galpão Serra Azul - subestação 13,8 kV

Metalúrgica Serra Azul

TR Thiago Rocha

17/10/2026

R\$ 2.100.000,00

+ Adicionar nome da tarefa

Cotação 1

R\$ 3.250.000,00

Torre Corporativa Alvorada - hidrossanitário

Alvorada Incorporações

TR Thiago Rocha

09/10/2026

R\$ 3.250.000,00

+ Adicionar nome da tarefa

Em revisão 1

R\$ 890.000,00

Parque das Palmeiras - fase 2

Construtora Horizonte

HP Helena Prado

12/10/2026

R\$ 890.000,00

+ Adicionar nome da tarefa

Validação direta

Nenhum item.

+ Adicionar r

Figura 27 — Quadro — Kanban (/quadros/visoes/<id>/)
Visão Kanban agrupada por Status, com a soma do valor no topo de cada raia.

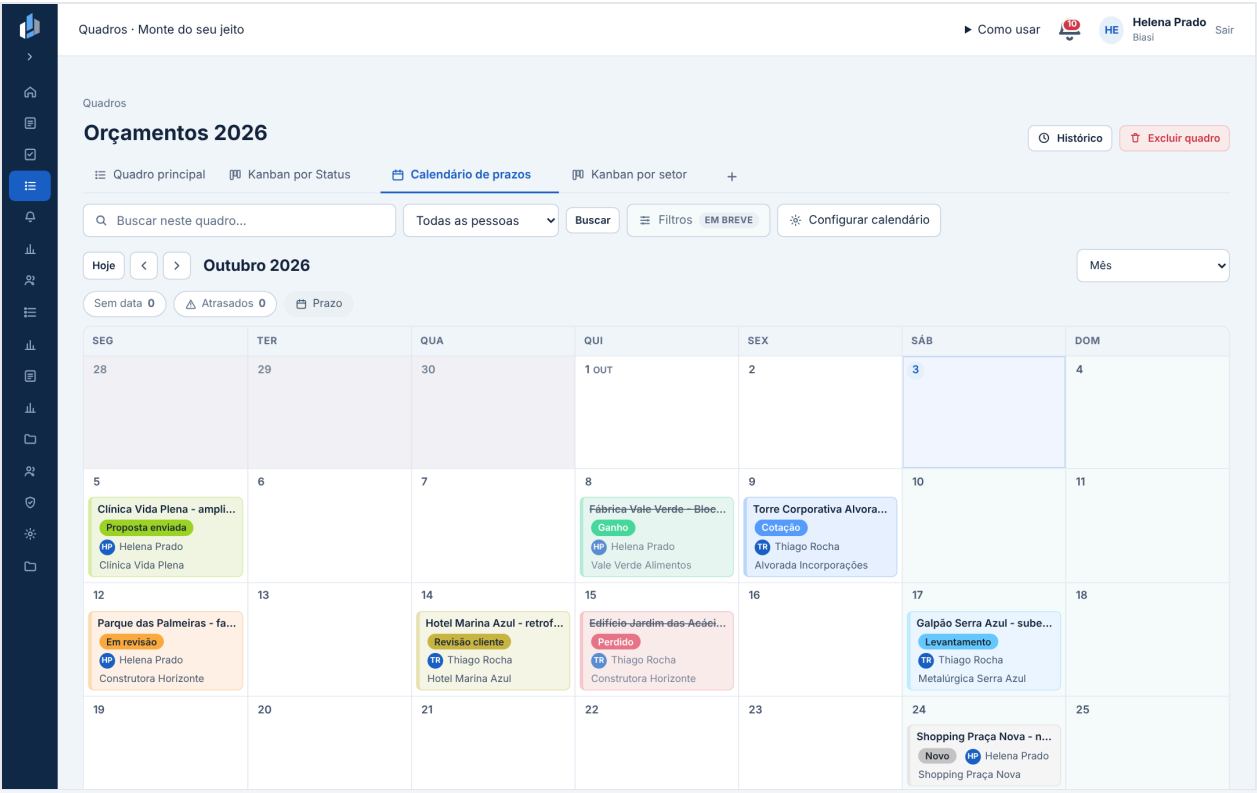


Figura 28 — Quadro — calendário (/quadros/visoes/<id>/)
Visão Calendário mensal posicionando os itens pela coluna de prazo.

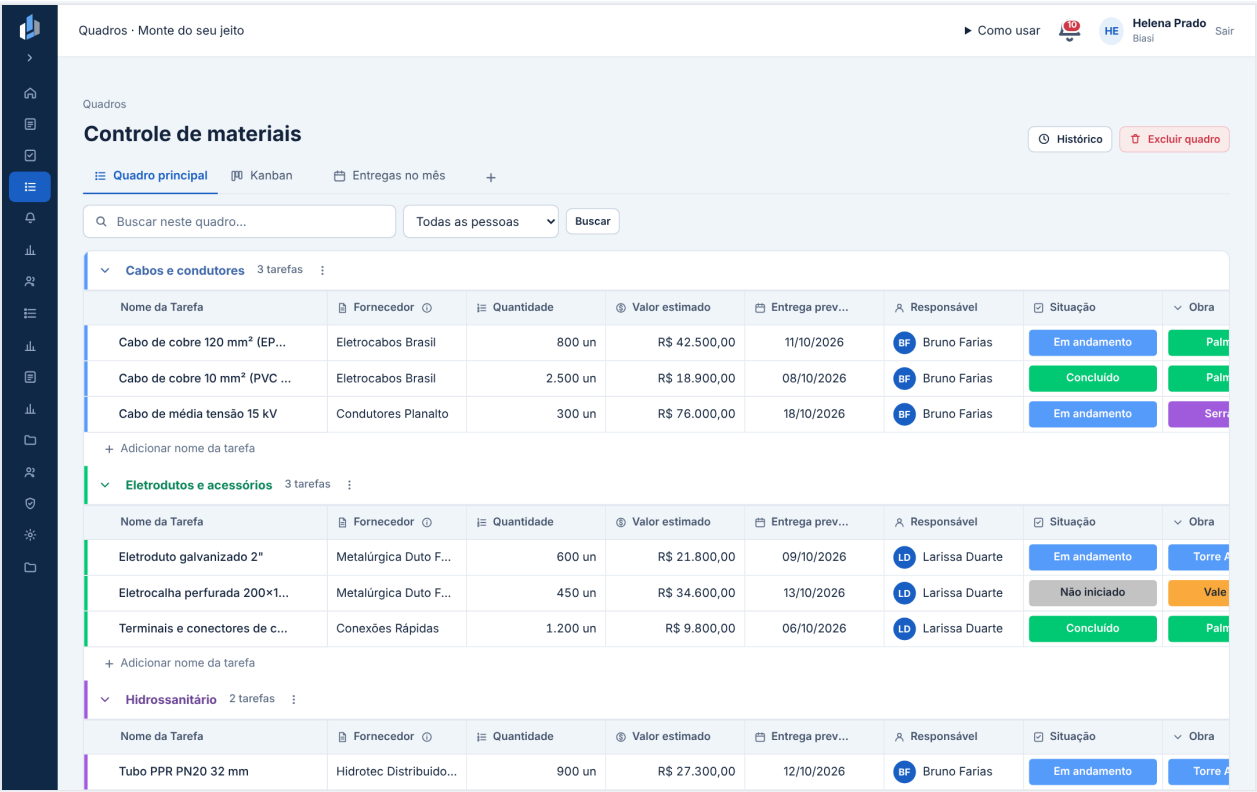


Figura 29 — Quadro — um exemplo de cada tipo de coluna (/quadros/<id>/)
Quadro de controle de materiais com todos os tipos de coluna: texto, número, moeda, data, pessoa, status, lista e confirmação.

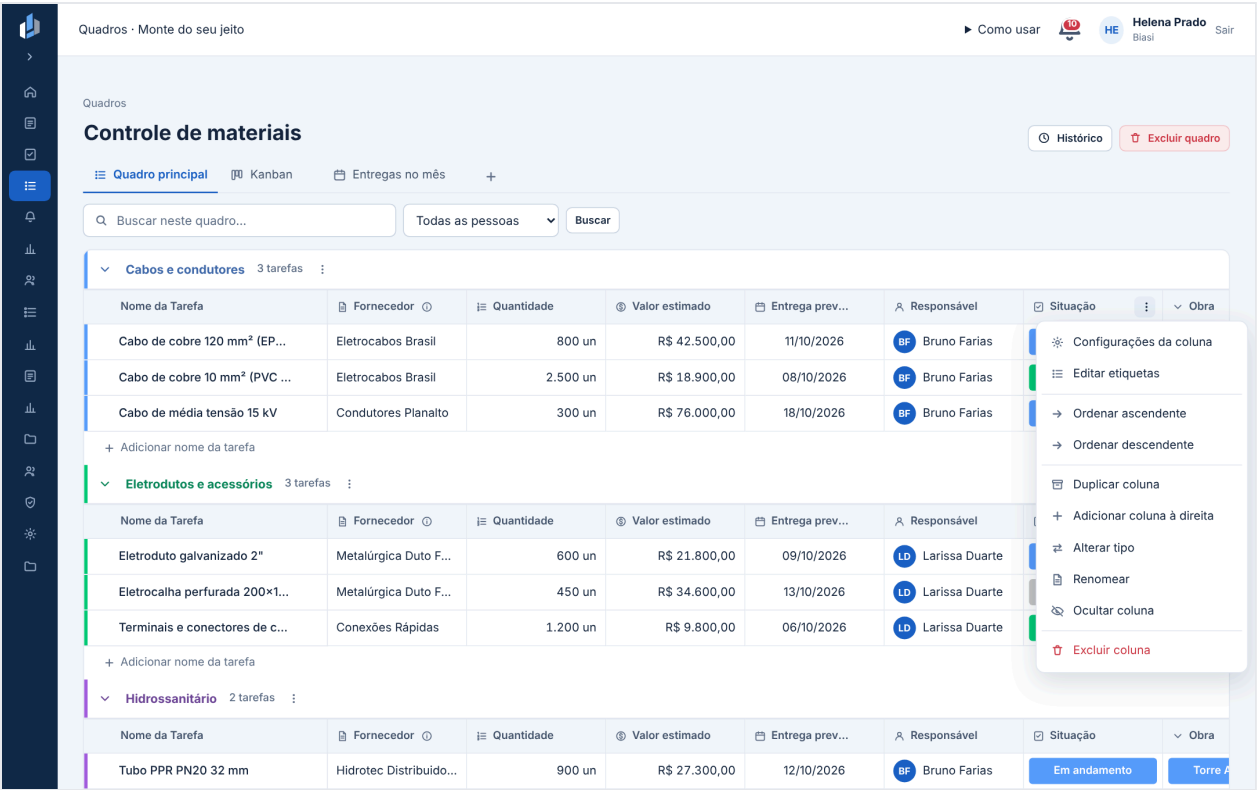


Figura 30 — Menu da coluna (/quadros/<id>/)
Menu da coluna: configurações, edição de etiquetas e ordenação.

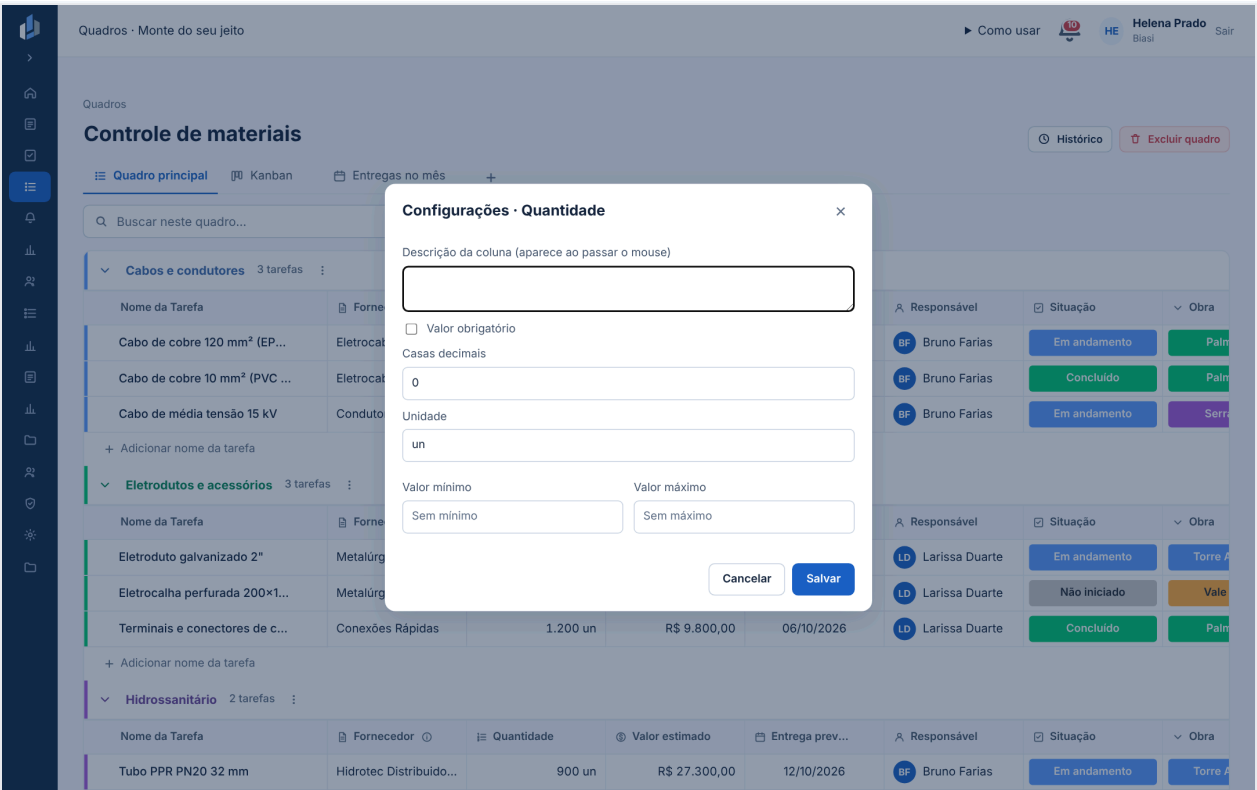


Figura 31 — Configurações da coluna (/quadros/<id>/)
Janela de configuração de uma coluna numérica: descrição, obrigatoriedade, casas decimais e unidade.

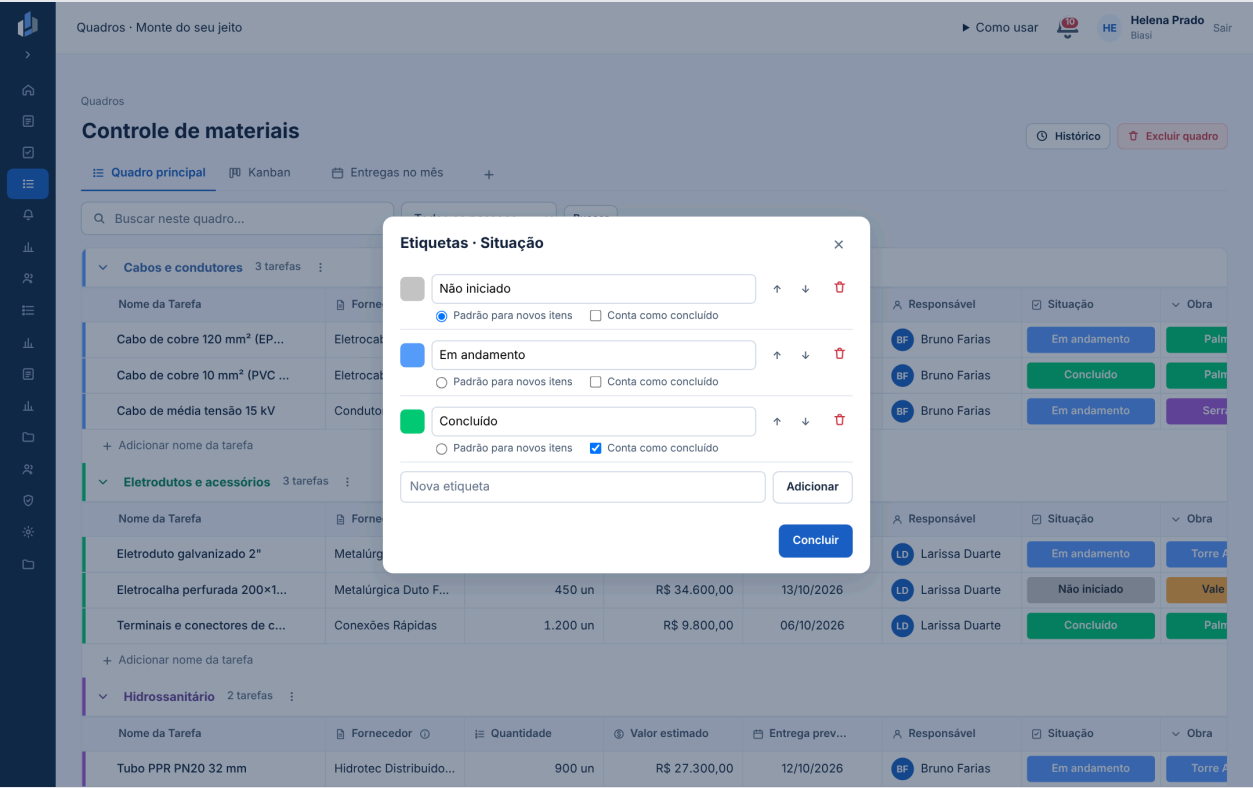


Figura 32 — Etiquetas da coluna Status (/quadros/<id>/)
Editor de etiquetas de uma coluna de Status: nome, cor, etiqueta padrão e de conclusão.

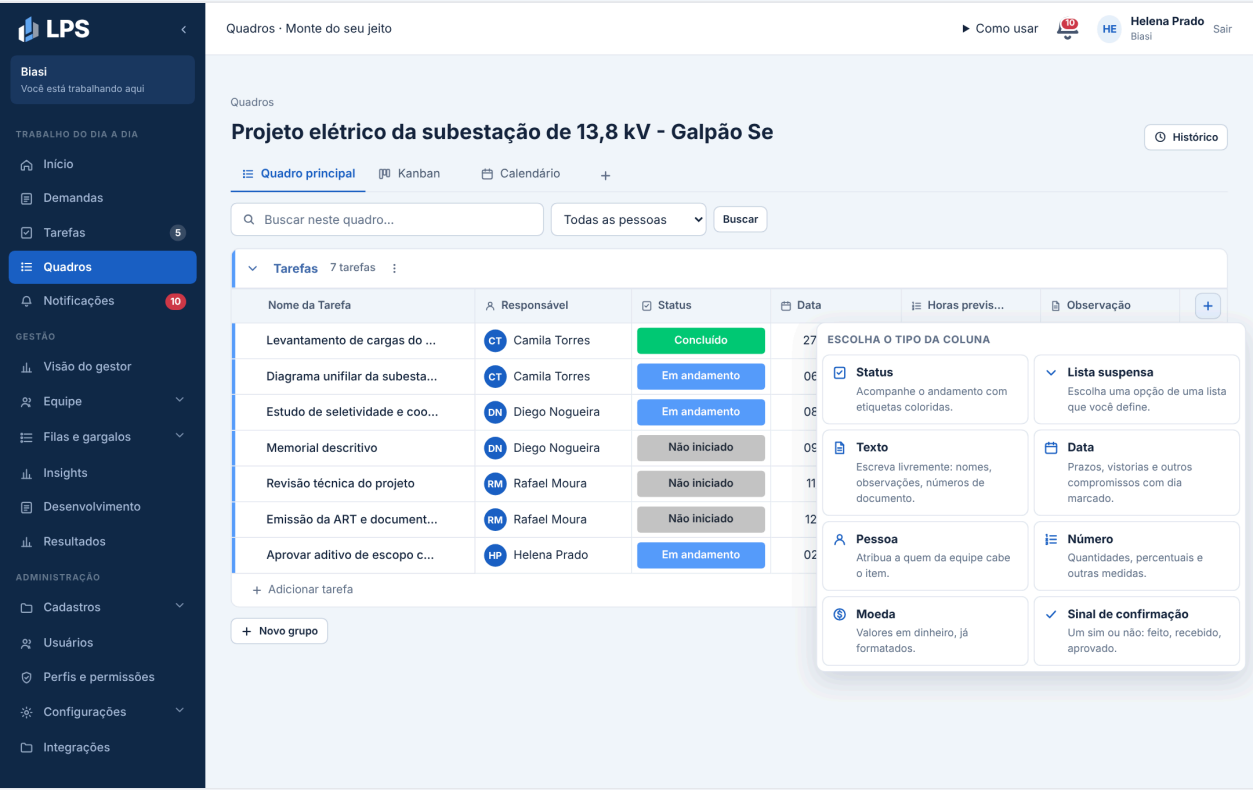


Figura 33 — Nova coluna (/quadros/<id>/)
Seletor de tipo ao adicionar uma coluna ao quadro.

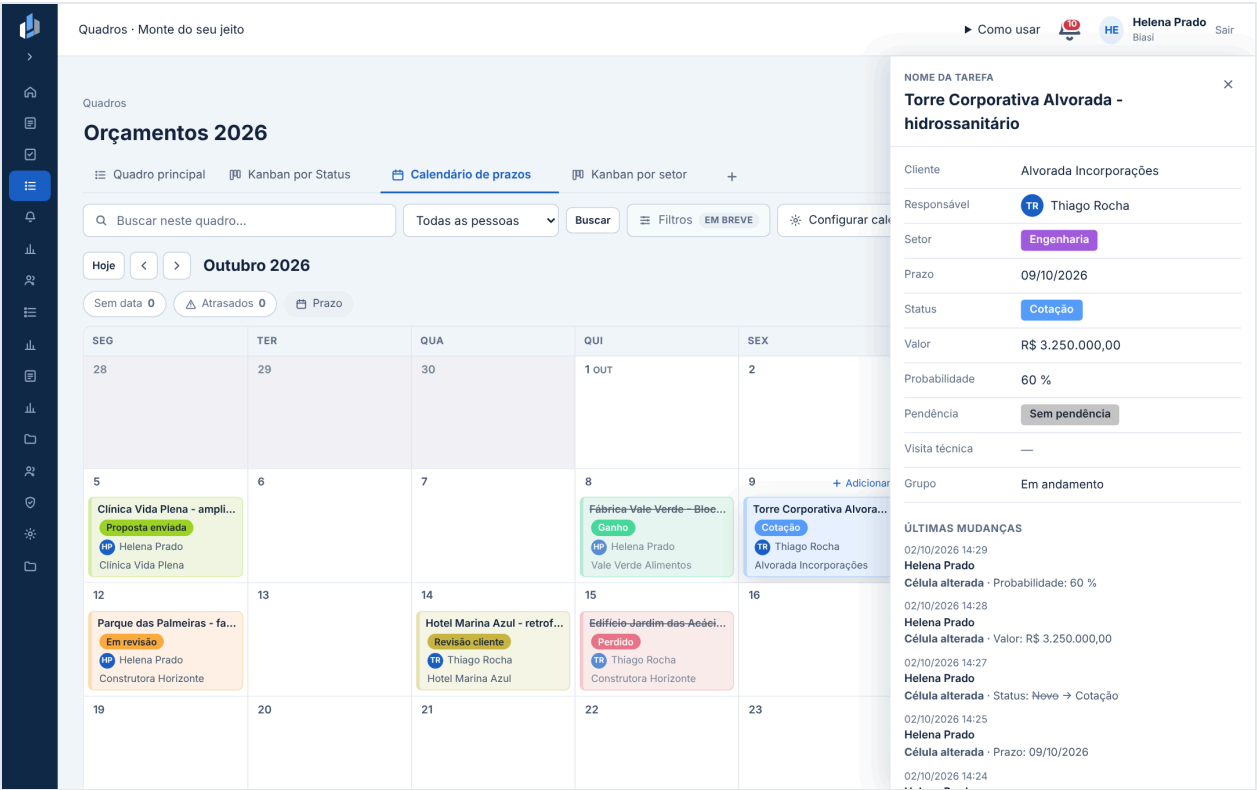


Figura 34 — Gaveta do item (/quadros/<id>/)
Gaveta lateral de um item com todos os campos editáveis e as últimas alterações.

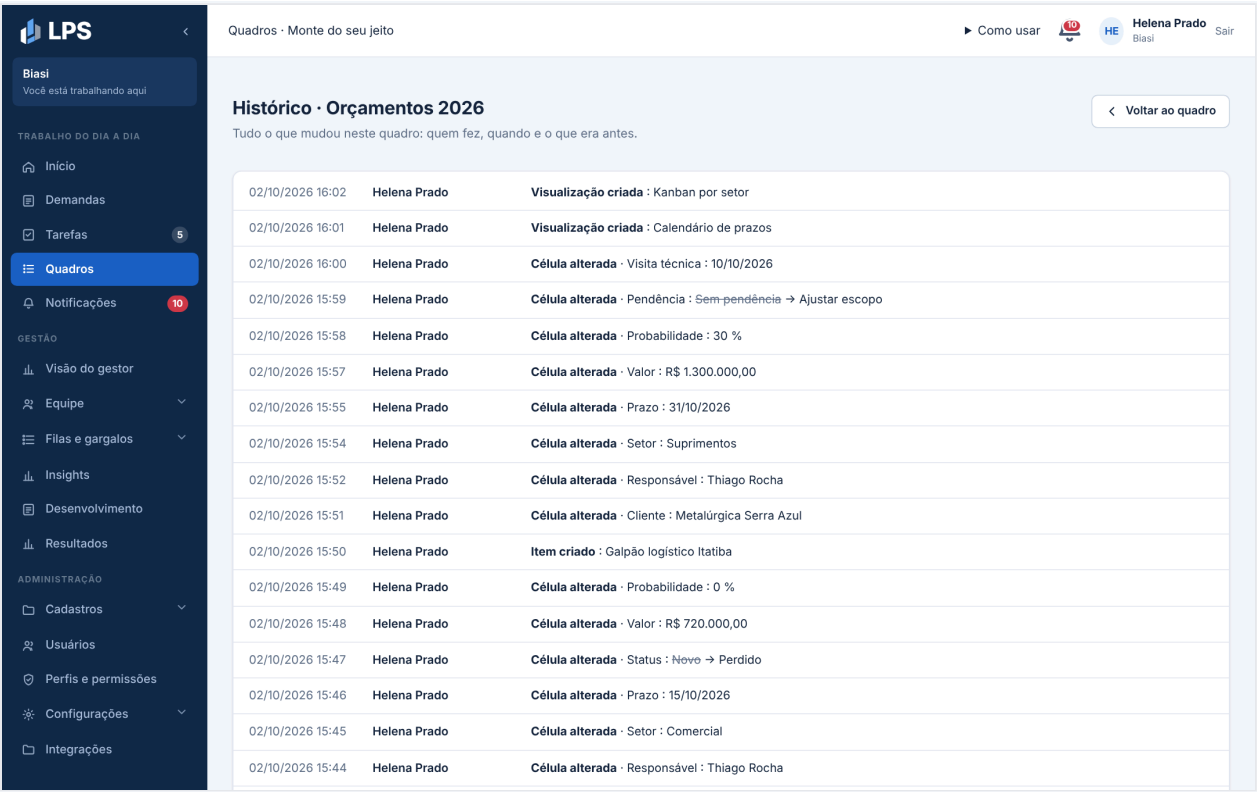


Figura 35 — Histórico do quadro (/quadros/<id>/historico/)
Registro de auditoria do quadro: criação de colunas, itens e alterações de células.

Acompanhamento

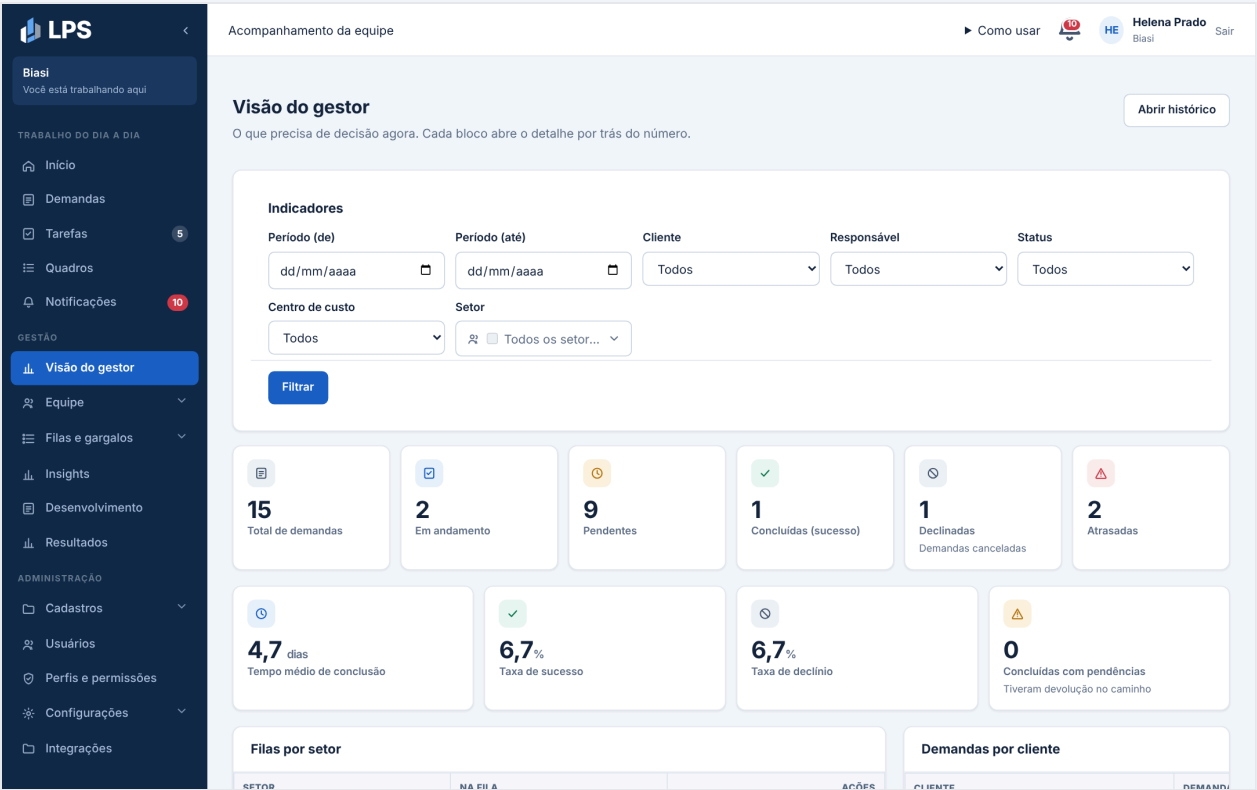


Figura 36 — Visão do gestor (/gestao/)

Gestão por exceção: indicadores de demandas, filtros por período, cliente, responsável e setor.

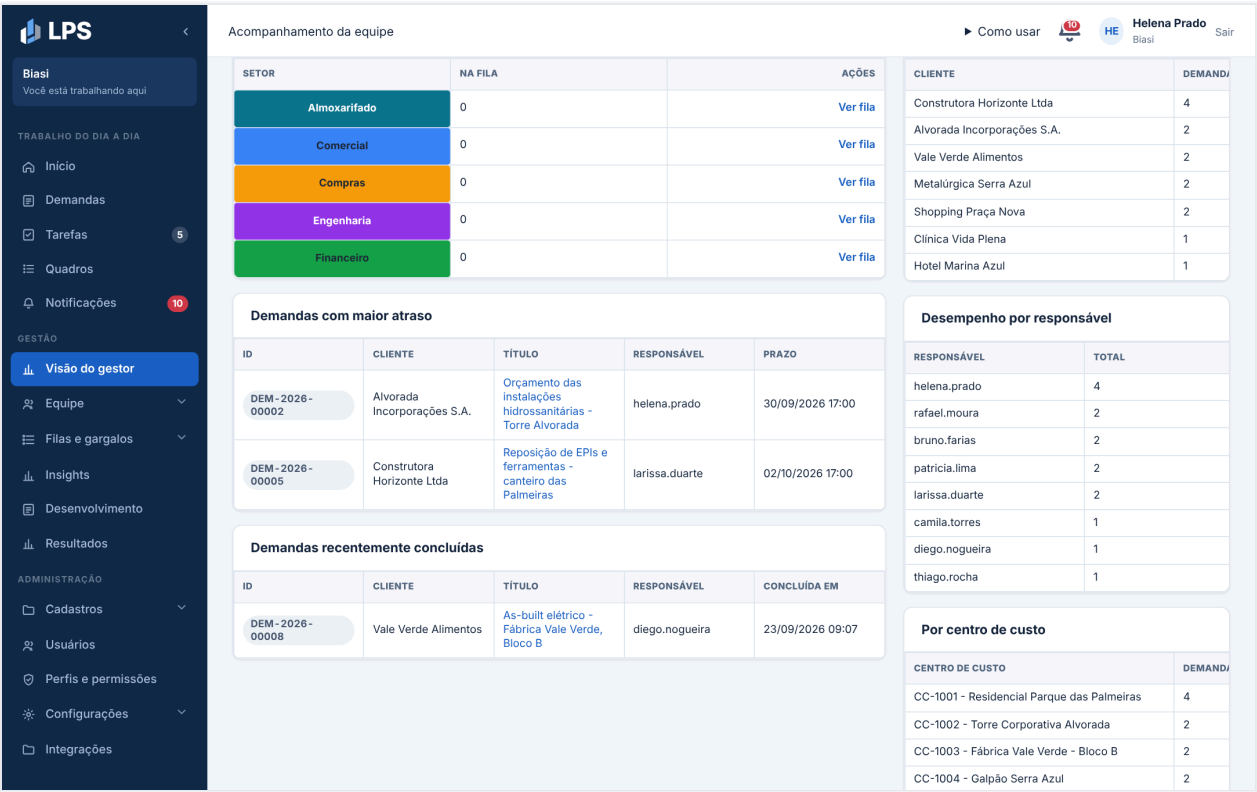


Figura 37 — Visão do gestor — por setor e por cliente (/gestao/)

Continuação da visão do gestor: tabelas por setor e por cliente.

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Notificações · Avisos para você

Como usar

HE

Helena Prado

Biasi

Sair

Notificações

O que precisa de sua atenção.

Para mim1Não lidas10Menções4Prazos0Sistema3Todas14

Mais recentes

Tipo	Título / descrição	Demanda	Sector	De / para	Data / prazo	Ação
Ontem1						
Menção	Visita técnica e levantamento - Galpão Serr... Thiago Rocha mencionou você em um comentário. @helena.prado a visita ao galpão foi confirmada para q...	Visita técnica e levantam DEM-2026-00011	Comercial	João Serr...TH	02/10/2026 13:38 Não lida	Responder
Esta semana5						
Menção	Revisão do projeto hidráulico - Parque das P... Rafael Moura mencionou você em um comentário. Reaberta por mudança do cliente. @helena.prado, o adi...	Revisão do projeto hidrául... DEM-2026-00012	Engenharia	João P...RA	01/10/2026 15:25 Não lida	Responder
Informativo	Reposição de EPIs e ferramentas - canteiro ... larissa.duarte: @bruno.farias o pedido dos EPIs está ...	Reposição de EPIs e ferra DEM-2026-00005	Almoxarifado	Leandro ...LA	30/09/2026 14:39 Não lida	Ver demanda
Informativo	Orçamento das instalações hidrossanitária... thiago.rocha: Avisei. O cliente aceitou a nova data, m... thiago.rocha: Avisei. O cliente aceitou a nova data, mas ...	Orçamento das instalaçõ... DEM-2026-00002	Comercial	Thiago ...TH	30/09/2026 08:07 Não lida	Ver demanda
Menção	Orçamento das instalações hidrossanitária... Bruno Farias mencionou você em um comentário. @helena.prado a cotação das tubulações de PPR ainda ...	Orçamento das instalaçõ... DEM-2026-00002	Comercial	Bruno ...BR	29/09/2026 14:38 Não lida	Responder
Menção	Projeto elétrico da subestação de 13,8 kV - ... Rafael Moura mencionou você em um comentário. Obrigado, @camila.torres. Vou validar com o cliente aih...	Projeto elétrico da subest... DEM-2026-00001	Engenharia	Rafael ...RA	28/09/2026 13:51 Não lida	Responder

Figura 38 — Notificações (/notificacoes/)

Caixa de notificações com abas (para mim, não lidas, menções, prazos, sistema), busca e agrupamento por data.

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Acompanhamento da equipe

Como usar

HE

Helena Prado

Biasi

Sair

Histórico de alterações

Veja quem fez cada mudança e quando ela aconteceu. Escolha abaixo o tipo de registro que quer consultar.

TodosTarefasPrazosFilaDevoluçõesTempo trabalhado

Helena Prado alterou anexo

anexo

Fotos_visita_tecnica.zip

25/09/2026 17:03

Helena Prado alterou anexo

anexo

Unifilar_subestacao_13_8kV.dwg

25/09/2026 14:01

Helena Prado alterou anexo

anexo

Memorial_descritivo_rev0.pdf

25/09/2026 10:51

Thiago Rocha marcou como pendente: PENDENTE

ABERTA → PENDENTE

Motivo: Aguardando ajustes/revisões

24/09/2026 13:10

Bruno Farias marcou como pendente: PENDENTE

ABERTA → PENDENTE

Motivo: Aguardando material

24/09/2026 11:02

Helena Prado reabriu

CONCLUIDA → EM_ANDAMENTO

Motivo: Cliente mudou a posição dos reservatórios; incluir nova rede de recalque.

24/09/2026 09:07

Helena Prado concluiu

Figura 39 — Histórico de alterações (/historico/)

Trilha de auditoria: quem fez cada mudança nas demandas e quando.

LPS — Documentação técnica

24 / 359

Administração

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadrados

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Sectores

Clientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em Configurações.

Processos

Sectores

Empresas

Obras

Centros de custo

Clientes

Tipos de demanda

Tipos de tarefa

Tags

Motivos

Q

Buscar pelo nome...

Buscar

NOME	COR	DESCRIÇÃO	TAREFAS ABERTAS	ETAPA	AÇÕES
Comercial		Propostas, orçamentos e relacionamento com clientes	0	Ativo	Editar Inativar
Engenharia		Projetos, laudos e acompanhamento técnico	0	Ativo	Editar Inativar
Compras		Cotações, pedidos e fornecedores	0	Ativo	Editar Inativar
Financeiro		Medições, faturamento e conciliação	0	Ativo	Editar Inativar
Almoxarifado		Estoque, separação e expedição de materiais	0	Ativo	Editar Inativar

Inativar preserva o histórico: o setor deixa de receber novas demandas, mas o que já passou por ele continua registrado.

Figura 40 — Cadastros — setores (/cadastros/?tab=setores)
Cadastro de setores da organização, com cor e situação.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadrados

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Sectores

Clientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em Configurações.

Processos

Sectores

Empresas

Obras

Centros de custo

Clientes

Tipos de demanda

Tipos de tarefa

Tags

Motivos

Q

Buscar pelo nome...

Buscar

NOME	CNPJ/CPF	ETAPA	AÇÕES
Biassi Engenharia	12.345.678/0001-90	Ativa	Editar Inativar
Biassi Instalações	11.222.333/0001-44	Ativa	Editar Inativar

Figura 41 — Cadastros — empresas (/cadastros/?tab=empresas)
Cadastro das empresas do grupo.

LPS — Documentação técnica

25 / 359

LPS

Biagi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadrados

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Cientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em Configurações.

Processos

Setores

Empresas

Obras

Centros de custo

Cientes

Tipos de demanda

Tipos de tarefa

Tags

Motivos

Q

Buscar pelo nome...

Buscar

NOME	CNPJ/CPF	TELEFONE	ETAPA	AÇÕES
Alvorada Incorporações S.A.	32.456.789/0001-21	(11) 4002-2002	Ativo	Editar Inativar
Clinica Vida Plena	54.678.901/0001-43	(11) 3004-4004	Ativo	Editar Inativar
Construtora Horizonte Ltda	21.345.678/0001-10	(11) 4002-1001	Ativo	Editar Inativar
Hotel Marina Azul	87.901.234/0001-76	(13) 3007-7007	Ativo	Editar Inativar
Metalúrgica Serra Azul	65.789.012/0001-54	(11) 3005-5005	Ativo	Editar Inativar
Shopping Praça Nova	76.890.123/0001-65	(11) 3006-6006	Ativo	Editar Inativar
Vale Verde Alimentos	43.567.890/0001-32	(19) 3003-3003	Ativo	Editar Inativar

Figura 42 — Cadastros — clientes (/cadastros/?tab=clientes)
Cadastro de clientes com documento, telefone e e-mail.

LPS

Biagi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demanda

Tarefas

Quadrados

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Cientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em Configurações.

Processos

Setores

Empresas

Obras

Centros de custo

Cientes

Tipos de demanda

Tipos de tarefa

Tags

Motivos

Q

Buscar pelo nome...

Buscar

NOME	CLIENTE	ETAPA	AÇÕES
Clinica Vida Plena - Ampliação	Clinica Vida Plena	Ativa	Editar Inativar
Fábrica Vale Verde - Bloco B	Vale Verde Alimentos	Ativa	Editar Inativar
Galpão Serra Azul	Metalúrgica Serra Azul	Ativa	Editar Inativar
Hotel Marina Azul - Retrofit	Hotel Marina Azul	Ativa	Editar Inativar
Residencial Parque das Palmeiras	Construtora Horizonte Ltda	Ativa	Editar Inativar
Shopping Praça Nova - Expansão	Shopping Praça Nova	Ativa	Editar Inativar
Torre Corporativa Alvorada	Alvorada Incorporações S.A.	Ativa	Editar Inativar

Figura 43 — Cadastros — obras (/cadastros/?tab=obras)
Cadastro de obras, cada uma ligada a um cliente.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Cientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em [Configurações](#).

ProcessosSetoresEmpresasObrasCentros de custoCientesTipos de demandaTipos de tarefaTagsMotivos

Q. Buscar pelo nome...

Buscar

NOME	OBRA	ETAPA	AÇÕES
CC-0001 - Administração central	—	Ativo	Editar Inativar
CC-1001 - Residencial Parque das Palmeiras	Residencial Parque das Palmeiras	Ativo	Editar Inativar
CC-1002 - Torre Corporativa Alvorada	Torre Corporativa Alvorada	Ativo	Editar Inativar
CC-1003 - Fábrica Vale Verde - Bloco B	Fábrica Vale Verde - Bloco B	Ativo	Editar Inativar
CC-1004 - Galpão Serra Azul	Galpão Serra Azul	Ativo	Editar Inativar
CC-1005 - Clínica Vida Plena - Ampliação	Clínica Vida Plena - Ampliação	Ativo	Editar Inativar
CC-1006 - Shopping Praça Nova - Expansão	Shopping Praça Nova - Expansão	Ativo	Editar Inativar

Figura 44 — Cadastros — centros de custo (/cadastros/?tab=centros-de-custo)
Cadastro de centros de custo, opcionalmente ligados a uma obra.

LPS

Biassi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas

Quadros

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Cientes

Obras

Centros de custo

Cadastros

Como usar

HE

Helena Prado

Sair

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em [Configurações](#).

ProcessosSetoresEmpresasObrasCentros de custoCientesTipos de demandaTipos de tarefaTagsMotivos

Q. Buscar pelo nome...

Buscar

NOME	COR	ETAPA	AÇÕES
Elétrica		Ativo	Editar Inativar
Hidrossanitária		Ativo	Editar Inativar
Medição		Ativo	Editar Inativar
Orçamento		Ativo	Editar Inativar
Retrabalho		Ativo	Editar Inativar
Urgente		Ativo	Editar Inativar

Figura 45 — Cadastros — tags (/cadastros/?tab=tags)
Cadastro de marcadores (tags) coloridos usados nas demandas.

LPS — Documentação técnica

27 / 359

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

LPS · Gestão do trabalho

Como usar

HEHelena PradoBiasi

Sair

Novo: cliente

×

Preencha os dados abaixo. Os campos opcionais podem ficar para depois.

Campos com * são obrigatórios. Os demais podem ficar em branco.

Nome*

CNPJ/CPF (opcional)

Telefone (opcional)

E-mail (opcional)

Endereço (opcional)

Cancelar

Salvar cliente

Figura 46 — Novo cliente (/cadastros/clientes/novo/)
Formulário de cadastro de um cliente.

LPS

Biasi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

LPS · Gestão do trabalho

Como usar

HEHelena PradoBiasi

Sair

Etapas e status

Organize a leitura do trabalho por setor. Etapa e status não alteram fila, prazo, bloqueio, cronômetro ou conclusão.

O status operacional continua protegido pelas regras da LPS.
Etapa organiza o fluxo visual; status registra a leitura manual atual do trabalho.

Setor

Engenharia

Demandas

Tarefas

Etapas

+ Nova etapa

Colunas e leitura visual do fluxo de Engenharia.

Briefing

Padrão deste setor

Briefing

Em análise

Ativa

Em análise

Em execução

Ativa

Em execução

Em revisão

Ativa

Em revisão

Entregue

Ativa

Entregue

Status

+ Nova status

Leitura manual, independente do status operacional.

No prazo

Sem descrição · Padrão deste setor

No prazo

Atenção

Sem descrição · Ativa

Atenção

Atrasada

Sem descrição · Ativa

Atrasada

Aguardando terceiros

Sem descrição · Ativa

Aguardando terceiros

Bloqueada

Sem descrição · Ativa

Bloqueada

Figura 47 — Etapas e status (/painel/configuracoes/etapas-e-status/)
Configuração do fluxo visual por setor: etapas (colunas do Kanban) e status manuais, cada um com cor.

LPS — Documentação técnica

28 / 359

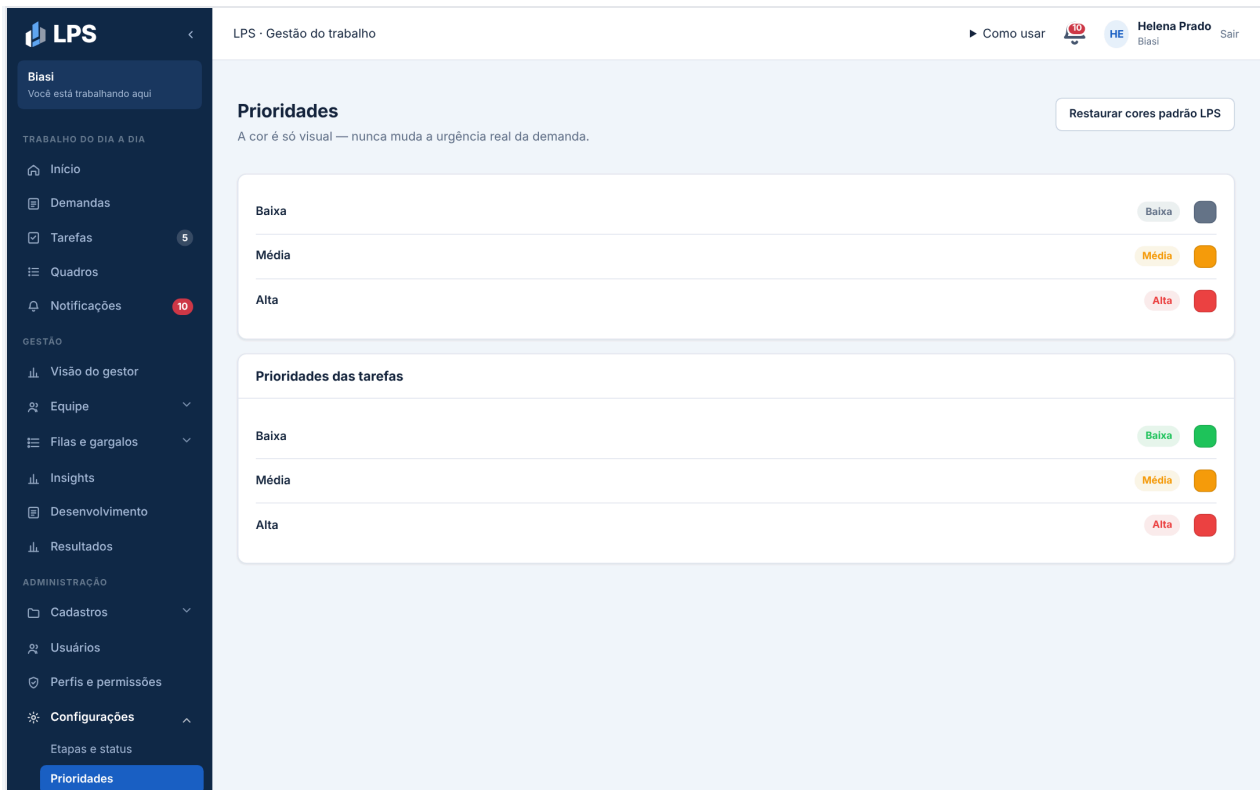


Figura 48 — Prioridades e cores (/painel/configuracoes/prioridades/)
Cores e nomes exibidos para as prioridades das demandas e tarefas.

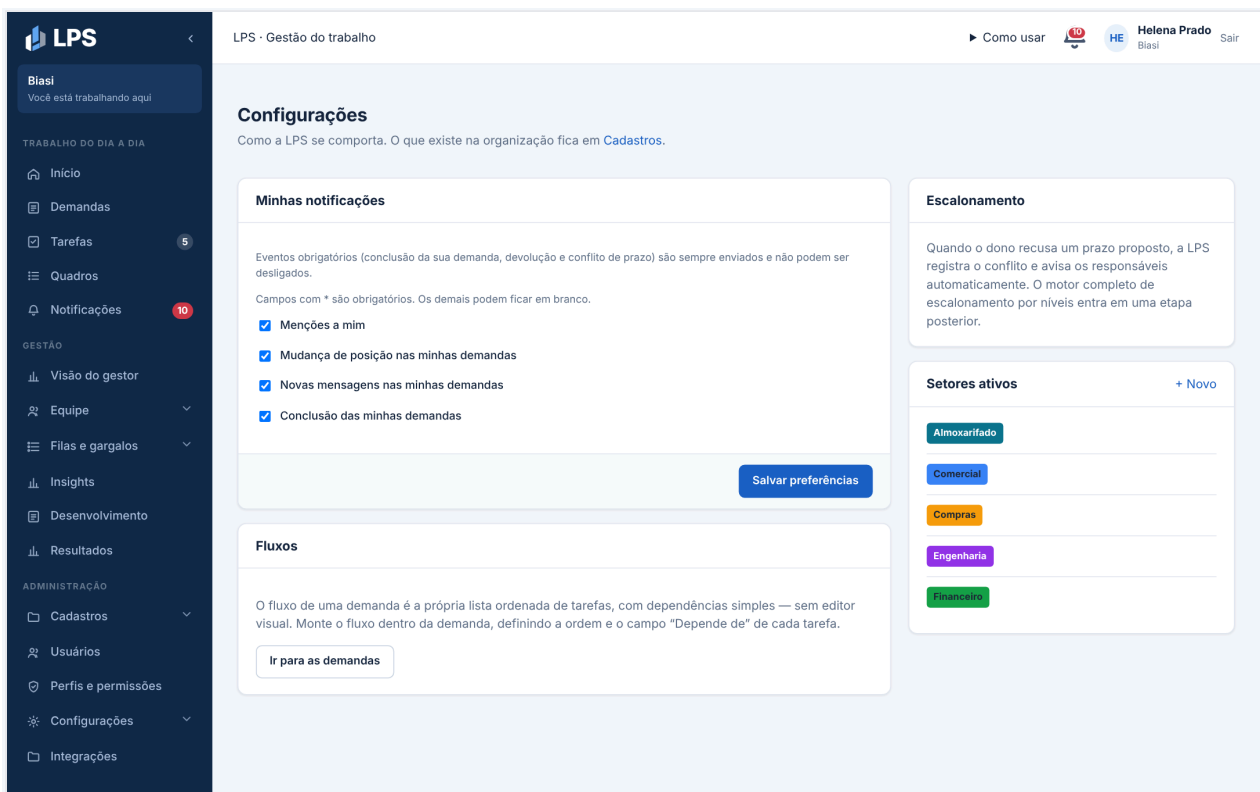


Figura 49 — Configurações — notificações (/configuracoes/)
Preferências de notificação da pessoa.

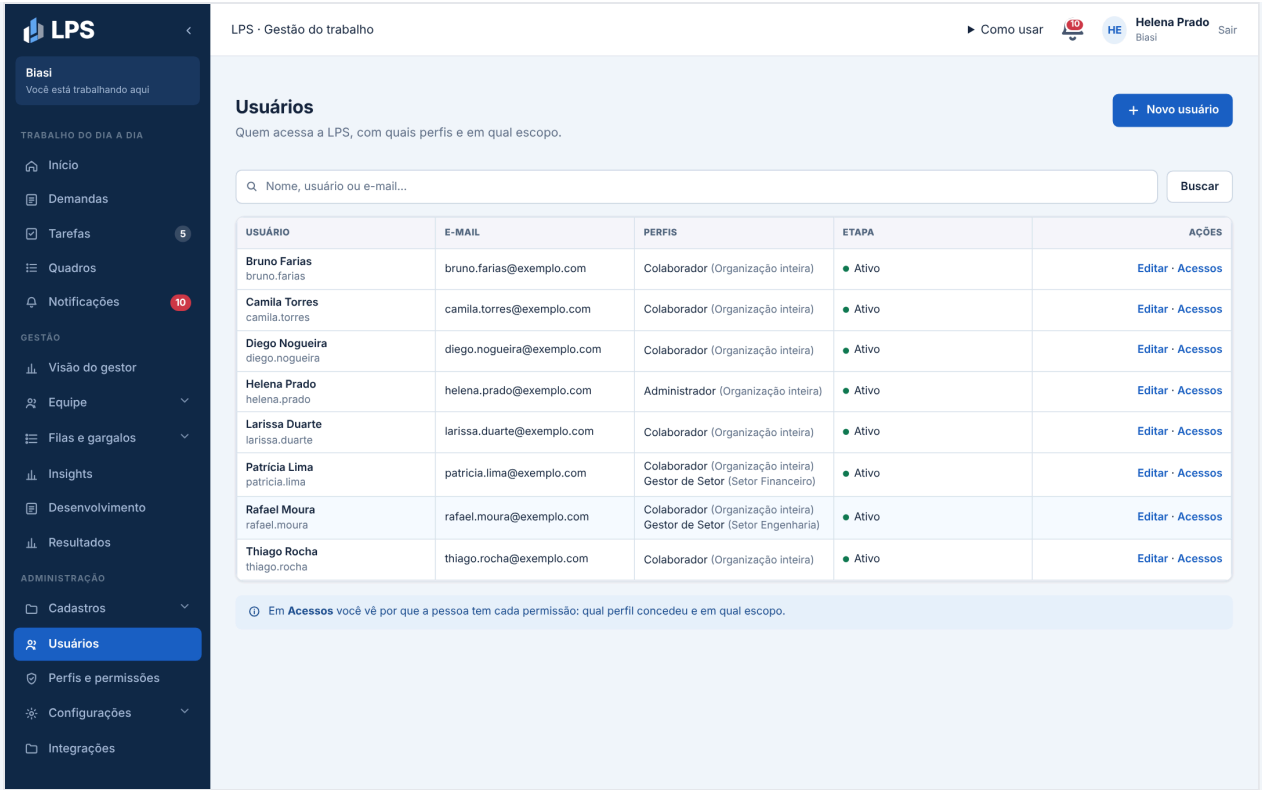


Figura 50 — Usuários (/usuarios/)
Usuários da organização com perfil, setores e situação.

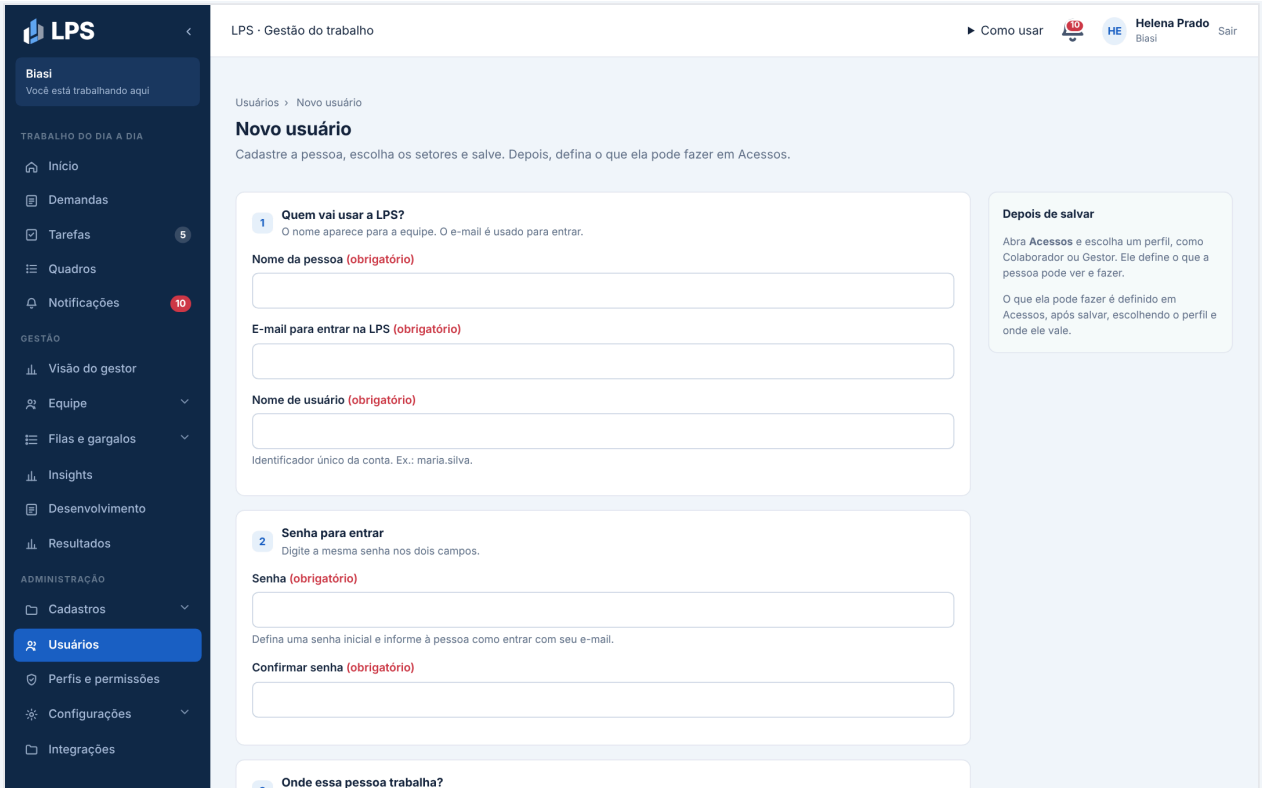


Figura 51 — Novo usuário (/usuarios/novo/)
Cadastro de um usuário pelo administrador.

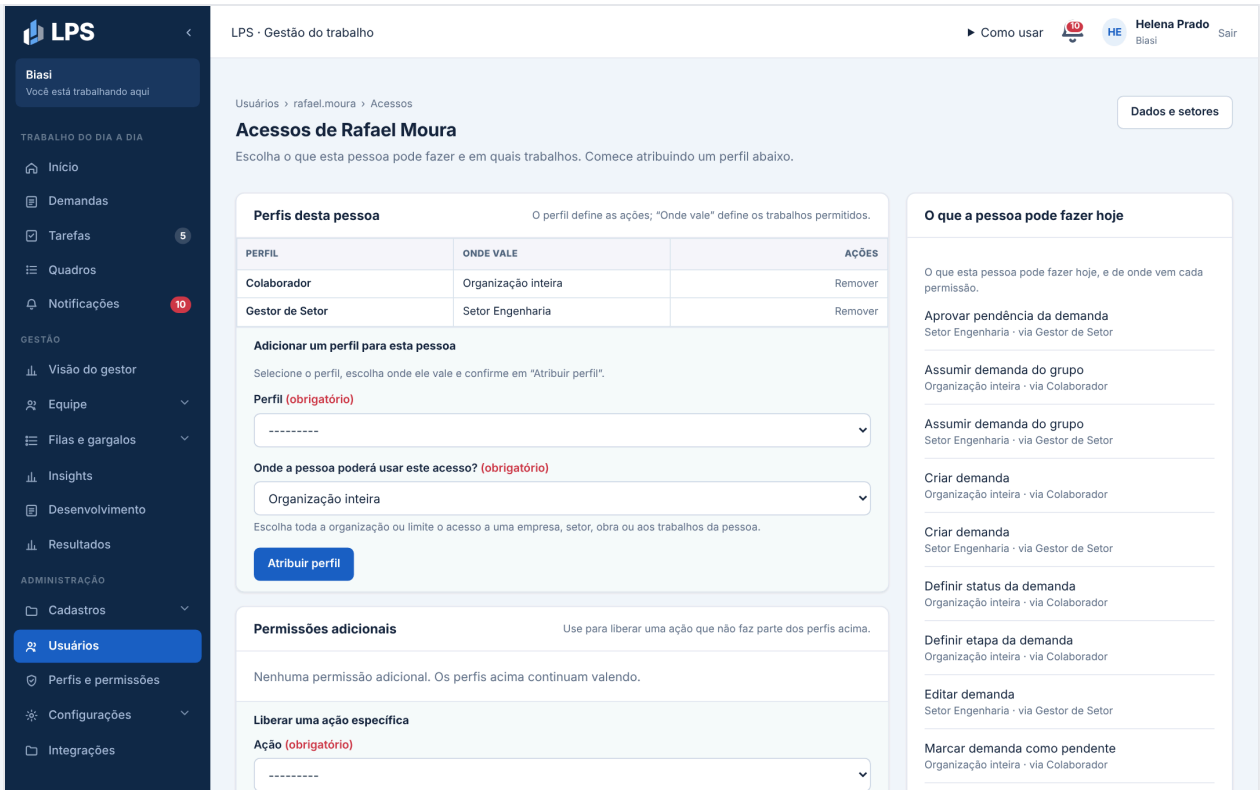


Figura 52 — Acessos de uma pessoa (/usuarios/<pk>/acessos/)
Acessos de uma pessoa: perfis atribuídos por escopo, concessões diretas e a origem de cada permissão.

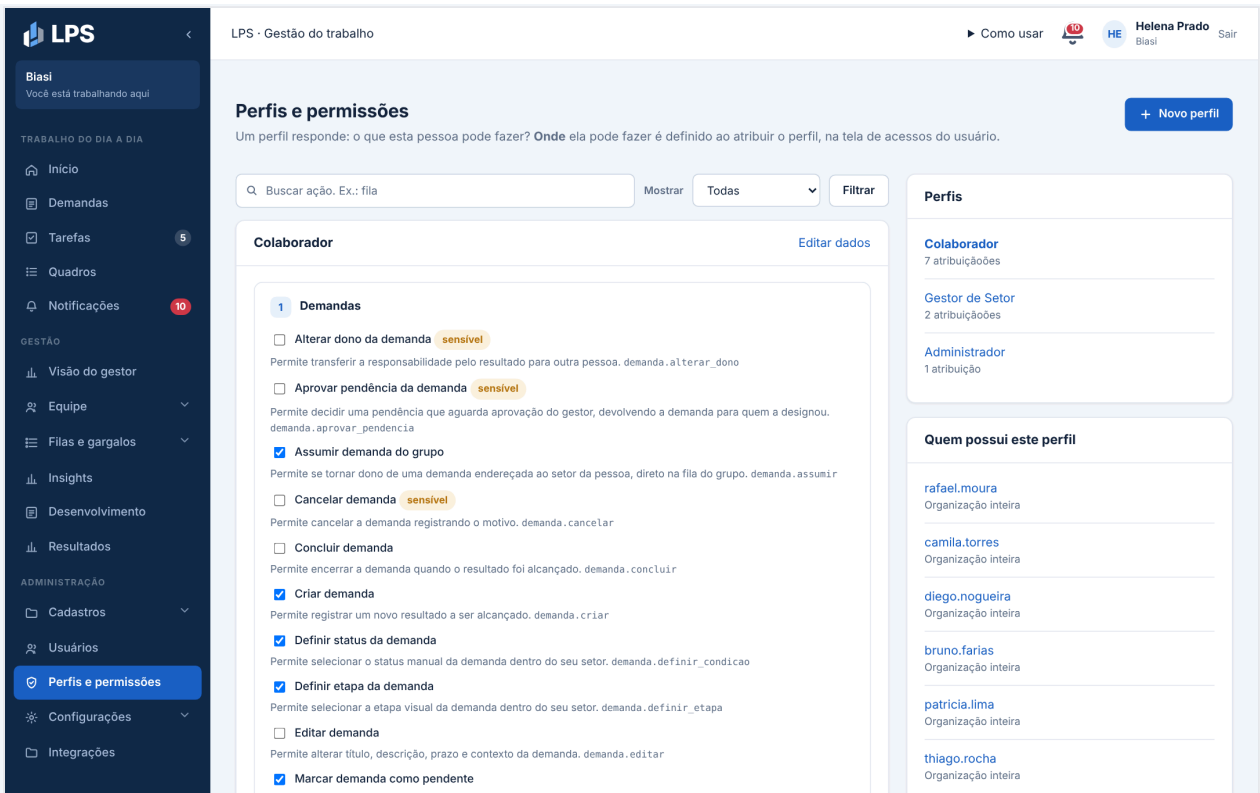


Figura 53 — Perfis e permissões (/permissoes/)
Matriz de ações do catálogo por perfil e quem possui cada perfil.



Biagi

Você está trabalhando aqui

TRABALHO DO DIA A DIA

Início

Demandas

Tarefas5

Quadros

Notificações10

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

LPS · Gestão do trabalho

Como usar



HEHelena Prado
Biagi

Sair

Criar perfil de acesso

Um perfil reúne as permissões de uma função, como Colaborador ou Gestor.

Campos com * são obrigatórios. Os demais podem ficar em branco.

Nome do perfil*

Ex.: Colaborador, Gestor de setor ou Administrador.

Para quem é este perfil? (opcional)

Depois de salvar, marque as ações permitidas na tela de permissões. Para usar o perfil, abra Usuários → Acessos e escolha a pessoa e onde ele vale.

Cancelar

Salvar perfil

Figura 54 — Novo perfil de acesso (/perfis/novo/)
Formulário de criação de um perfil de acesso.

LPS — Documentação técnica

32 / 359

01 — Arquitetura

Visão geral de como o código da LPS está organizado: quais apps existem, o que cada um faz, como as camadas se relacionam e quais convenções o código segue. As regras de negócio que motivam cada decisão estão em `Regras/` ; os docstrings do código citam essas seções (ex.: "Regras 05 §43"). Atualizado em 03/10/2026 (código no commit `add5acd`).

Glossário (01/10/2026): na interface, "Atividade" passou a se chamar "Demanda". Mudou tudo o que a pessoa vê: menus, títulos, botões, formulários, mensagens, avisos, e-mails, nomes de ações e de grupos de permissão (inclusive as chaves mostradas em `/permissoes/` : `demanda.criar` , `demanda.editar` ...), rótulos de modelos (no Admin) e as notificações já gravadas. **O endereço e o código também mudaram:** as páginas vivem em `/demandas/` ... (os links `/atividades/` ... de antes redirecionam, `core/legacy_redirects.py`), o código gerado é `DEM-AAAA-NNNNN` (os já emitidos foram reescritos, junto com as pastas de anexos, pela migração `activities/0020`) e os slugs de aba na URL passaram a `?tab=estagios-demanda` , `?tab=status-demanda` , `?visao=demanda` , `?sort=demanda` (os valores antigos continuam valendo). Os anexos **não** têm mais endereço público: são entregues por `/demandas/<id>/anexos/<id>/download/` , que exige login, mesma organização e permissão de ver a demanda (ver [03_CONFIGURACAO.md](#) §4); o antigo `/atividade-arquivos/` ... responde 410. Fica como está, de propósito, por ser nome técnico e não o que a pessoa vê: o código (`Activity` , `activities/` , `ActivityService` , as constantes `ATIVIDADE_*`), as tabelas, os nomes de rota (`activity-*`), a pasta em disco `atividade_arquivos/` (variável `ACTIVITY_FILES_ROOT`) e os valores internos (`MINHAS_ATIVIDADES` , `ACTIVITY_CREATED`). Este documento e os demais em `docs/` seguem dizendo "atividade" ao falar do modelo `Activity` .

Glossário (02/10/2026): "Situação" virou "Etapa" e "Condição" virou "Status" nos textos da interface. A troca foi textual: a "Condição" manual por setor (modelo `WorkflowStatus` , campo `condition` , rota `activity-set-condition`) aparece como **Status**, e o que a tela chamava de "Situação" aparece como **Etapa** (inclusive a coluna ativo/inativo das telas de cadastro). Os nomes de código, tabelas e rotas não mudaram (`ActivityStage` , `TaskStage` , `WorkflowStatus` , `ATIVIDADE_DEFINIR_CONDICAO` ...). O **estado operacional** (`Activity.status` , `Task.status`) continua fixo no código e é outra coisa: ver a convenção "Status x etapa" na seção 4.

1. Apps

O projeto Django fica em `config/` (settings, urls, wsgi/asgi). Os apps de negócio estão na raiz:

App	Responsabilidade	Principais arquivos
core	Tenant (<code>Organization</code>) e cadastros da organização: empresas, setores (com cor), obras, clientes, centros de custo, tags, etapas (<code>ActivityStage</code> , <code>TaskStage</code>) e status (<code>WorkflowStatus</code>) por setor, cores e rótulos configuráveis (<code>EnumColor</code>). Hospeda também as telas de usuários, de permissões (<code>/permissoes/</code> , <code>/perfis/</code>), de cadastros (inclusive o de motivos de devolução, cujo modelo <code>ReturnReason</code> fica em <code>activities</code>) e de configurações, os mixins base e os redirecionamentos de endereços antigos.	<code>models.py</code> , <code>services.py</code> (<code>StageService</code> , <code>ConditionService</code>), <code>mixins.py</code> , <code>colors.py</code> , <code>sanitize.py</code> , <code>widgets.py</code> , <code>legacy_redirects.py</code>
accounts	Identidade: Profile (1:1 com <code>User</code> , liga a pessoa a uma organização), participação em setores (<code>UserSector</code>), cadastro com confirmação de e-mail (<code>EmailVerification</code>), login por e-mail.	<code>models.py</code> , <code>auth_backends.py</code> , <code>forms.py</code> , <code>signals.py</code> , <code>views.py</code>
acessos	Motor de autorização: catálogo de ações, perfis de acesso, escopos, concessões. Responde "quem pode fazer o quê, onde e por quê". Não tem telas próprias (ficam em <code>core</code>); o <code>context processor</code> <code>navigation</code> monta o menu lateral.	<code>catalog.py</code> , <code>models.py</code> , <code>services.py</code> , <code>context_processors.py</code> , <code>testing.py</code>

App	Responsabilidade	Principais arquivos
activities	Núcleo operacional das Demandas (Activity): lista e quadro de Demandas (a tela /demandas/ é desenhada por boards), criação em etapas (wizard) e edição em janela, edição inline, Kanban por setor (kanban.py), pendências, conversa com @menções e reações, anexos, histórico. Guarda também o modelo de Tarefa (Task) com fila, sessões de trabalho, bloqueios, devoluções, prazos e checklist; esse fluxo operacional foi desativado (todas as rotas /tarefas/<id>/... , /fila/... , /prazos/... e /conflitos/... respondem 410) e as tarefas de uma demanda passaram a ser itens do quadro dela (boards). Ver 06_ATIVIDADES_E_TAREFAS.md .	models.py , services.py , views.py , policies.py , activity_editor.py , inline_edit.py , inline_views.py , kanban.py , filtering.py , errors.py , process_application.py , process_state.py , templatetags/lps.py
processes	Modelos de processo reutilizáveis e versionados (inputs, output, critérios de aceite, fluxo por setor). Os dados continuam preservados, mas o app está desativado para uso : todas as rotas de /processos/ respondem 410 e /demandas/<id>/processo/... também ("Processos foram desativados. Use o quadro da Demanda.").	models.py , services.py , views.py
intake	Caixa de Entrada (inativa por padrão , INTAKE_ENABLED , doc 03): solicitações que chegaram (e-mail, Teams, pedido verbal) e ainda não viraram demanda. A pessoa decide Criar demanda , Editar as sugestões ou Ignorar ; nada vira demanda sozinho. Sugere título, cliente, obra, setor e prazo com regras locais (sem IA). Ver 06_ATIVIDADES_E_TAREFAS.md para o que a demanda criada recebe.	models.py , services.py , suggestions.py , textparse.py , policies.py , views.py
boards	Quadros , em duas camadas. (1) <i>Quadros dinâmicos</i> (Board): o usuário monta o quadro (grupos, colunas de 8 tipos ativos, etiquetas) e preenche itens numa tabela editável, com autosave por célula e as visualizações Kanban e Calendário (BoardView). Um quadro pode ser modelo (Board.Kind.TEMPLATE , de um setor) ou o quadro de uma Demanda (Board.Kind.DEMAND , 1:1 com Activity , instanciado a partir de um modelo ou em branco): esses itens são as tarefas operacionais de hoje. (2) <i>Quadros de domínio</i> (DomainBoard , Domain.DEMAND e Domain.TASK): só guardam a configuração das lentes (colunas, campos do cartão, visões) sobre os objetos operacionais; hoje a tela /demandas/ os usa (Tabela, Kanban e Calendário de Activity). O domínio TASK e TaskWorkBoardView continuam no código, mas /tarefas/ é servida pelo painel Tarefas (TaskCenterView : lista, Kanban e Calendário das tarefas — itens dos quadros das Demandas — em que a pessoa é responsável, sem model próprio). A ficha da Demanda mostra uma prévia só de leitura do quadro dela (demand_preview.py). Auditoria genérica (target_type / target_id / metadata). Ver 04_MODELOS_DE_DADOS.md e 09_ROTAS.md ; especificações em docs/LPS-*.md .	models.py , validators.py , services.py , queries.py , presentation.py , starter_templates.py , kanban.py , calendar_view.py , demand_services.py , domain_defaults.py , domain_services.py , work_views.py , task_center.py , task_center_views.py , demand_preview.py , views.py , templatetags/lps_board.py
notifications	Notificações in-app, caixa de entrada com categorias, e-mails síncronos.	models.py , services.py , recipients.py , views.py
audit	AuditLog : trilha de auditoria somente-inclusão de eventos de negócio e de segurança.	models.py , services.py
painel	Telas "em construção" que mantêm o menu lateral fiel a Telas/MENU_E_SUBMENUS_LPS.md , mais duas telas de configuração emprestadas de core (EtapasEStatusView , PrioridadesView).	views.py , urls.py

Fora dos apps: config/ (settings, urls, wsgi/asgi), templates/ (um diretório por app, mais base.html e parciais), static/ (CSS e JS por tela, sem bundler), tests/ (testes JavaScript) e Regras/ , Telas/ (especificação do produto). A estrutura completa de pastas está na seção 5.

Não há mais as pastas demands/ e workflows/ (restos de uma versão anterior, citados em edições antigas deste documento).

Dependências entre apps

Levantado a partir dos imports (fora de testes e migrations, inclusive os feitos dentro de funções) e das FKs entre apps:

App	Usa
audit	nenhum app (só FKs para <code>activities</code> e <code>User</code>)
accounts	acessos (invalidação de cache, em <code>signals.py</code>); FKs para <code>core</code>
painel	<code>core</code>
processes	<code>accounts</code> , <code>acessos</code> , <code>core</code> ; FKs para <code>activities</code> e <code>core</code>
notifications	<code>accounts</code> ; FKs para <code>activities</code>
intake	<code>acessos</code> , <code>activities</code> (cria a demanda por <code>ActivityService.save_draft + publish_draft</code>), <code>core</code> . <code>activities</code> não importa <code>intake</code> : a ligação de volta é só o <code>OneToOneField IntakeItem.activity</code> (<code>activity.intake_item</code>).
boards	<code>acessos</code> , <code>activities</code> (modelos e serviços: o quadro da Demanda e os quadros de domínio leem e mudam <code>Activity / Task</code> por <code>ActivityService / TaskService</code>), <code>audit</code> , <code>core</code> , <code>notifications</code> (só leitura de <code>Notification</code> , no painel Tarefas e na prévia). FK <code>Board.activity</code> (1:1) para <code>activities</code> .
acessos	<code>accounts</code> , <code>audit</code> , <code>core</code> . Não importa <code>activities</code> , <code>boards</code> nem <code>intake</code> para decidir acesso: <code>ResourceContext.of</code> reconhece cada recurso pelo nome do modelo (<code>_meta.model_name</code>). Só o <code>context processor navigation</code> importa <code>intake.services</code> e <code>boards.task_center</code> (dentro de funções), para os contadores do menu.
core	<code>accounts</code> , <code>acessos</code> , <code>activities</code> , <code>audit</code>
activities	<code>acessos</code> , <code>audit</code> , <code>boards</code> (criação do quadro da Demanda, <code>DemandBoardAccess</code> , e as telas <code>DemandWorkBoardView / TaskCenterView</code> ligadas em <code>activities/urls.py</code> ; FK de <code>Board</code> para <code>Activity</code>), <code>core</code> , <code>notifications</code> , <code>processes</code> (FKs <code>Activity.process_version</code> e <code>Task.process_step</code> ; código de aplicação do processo em <code>activities/process_application.py</code> , hoje sem rota ativa)
config	<code>accounts</code> (<code>EmailAuthenticationForm</code>) e <code>core</code> (<code>legacy_redirects</code>), em <code>urls.py</code>

Há dependências circulares (`core` ↔ `acessos` ↔ `activities` e `activities` ↔ `boards`). Elas são resolvidas com imports **dentro das funções** e FKs por string (`"core.Organization"`, `"activities.Activity"`). Ao mover código entre apps, mantenha esse padrão para não criar erro de import circular na inicialização.

2. Camadas

URL → View (fina) → Service (regra + autorização + auditoria + notificação) → Model

- **Views** resolvem a organização, carregam objetos já filtrados por ela, validam o formulário e chamam um serviço. Views de ação herdam de `ServiceActionView` (em `activities/views.py`): chamam o serviço, convertem o erro em mensagem (`django.contrib.messages`) e redirecionam. Os endpoints JSON (edição inline da lista de Demandas, Kanban, células e itens dos quadros) seguem o mesmo princípio: view fina, regra no serviço. Entre a view e o serviço pode haver um *adapter* que só traduz dados de tela em chamadas ao serviço, sem regra própria (`activities/inline_edit.py`, `boards/domain_services.py`).
- **Services** (`*/services.py`) concentram a regra de negócio. Cada método público:
 1. verifica a autorização (`require_action(user, catalog.X, recurso)`);
 2. valida a transição de estado;
 3. grava;
 4. registra em `AuditService.log(...)`;
 5. dispara `NotificationService.notify(...)` e, quando aplicável, e-mail.
- **Models** guardam dados e propriedades de exibição; quase não têm regra.

Erros de negócio são exceções específicas por app, sempre com mensagem pronta para o usuário:

Exceção	Onde
<code>ActivityError</code>	<code>activities/errors.py</code> (reexportada por <code>activities/services.py</code>)
<code>ActivityPermissionError</code>	<code>activities/errors.py</code> ; subclasse de <code>ActivityError</code> levantada por <code>require_action</code> quando o motor nega (as respostas JSON usam 403 para ela e 400 para os demais)
<code>CadastroError</code>	<code>core/services.py</code>
<code>ProcessError</code>	<code>processes/services.py</code>
<code>IntakeError</code>	<code>intake/services.py</code> (a falha de <code>ActivityService</code> ao converter vira <code>IntakeError</code>)
<code>BoardError</code> , <code>BoardPermissionError</code>	<code>boards/services.py</code> (a segunda é subclasse da primeira e é a recusa do motor de autorização; as views respondem 400/409 e 403)
<code>DomainBoardError</code> , <code>DomainBoardConflict</code>	<code>boards/domain_services.py</code> (alterações nas lentes de Demandas e Tarefas; o conflito é o valor que mudou desde a leitura)
<code>AuthorizationError</code>	<code>acessos/services.py</code> (convertida em <code>ActivityPermissionError</code> por <code>require_action</code>)

3. Multi-tenancy

`core.Organization` é o tenant. Tudo pertence a uma organização:

1. Uma pessoa pertence a **uma** organização via `accounts.Profile.organization` (Regras 05 §7). O `Profile` é criado vazio por sinal quando o `User` é criado; quem define a organização é um administrador (tela `/usuarios/`).
2. Cadastros e registros de acesso têm FK `organization`. `Activity` tem FK direta; `Task` herda pela atividade; `QueueEntry`, pelo setor. Os modelos de `boards` expõem `organization_id` por um caminho curto (por exemplo, `BoardItem.organization_id` vem de `board.organization_id`), que é o endereço usado pelo motor de autorização.
3. `core.mixins.OrganizationRequiredMixin` resolve `self.organization` em toda view da LPS. Sem organização, redireciona para `/accounts/me/` com a mensagem "Seu usuário ainda não está vinculado a uma organização".
4. As views filtram tudo por `organization=self.organization` (ou `activity__organization=...`); objetos de outra organização dão 404. Isso vale também para o download de anexos de demanda.
5. O motor de autorização aplica um **teto de tenant**: se o recurso é de outra organização, nenhuma concessão vale (doc 05 §43). Ver [05_AUTORIZACAO.md](#).

O Admin do Django (`/admin/`) **não** é filtrado por organização — é a ferramenta do operador da plataforma.

4. Convenções do código

- **Autorização nega por padrão.** `ActionRequiredMixin` exige o atributo `required_action`; sem ele, levanta `ImproperlyConfigured`.
- **Esconder no menu é UX, não segurança.** O context processor `acessos.context_processors.navigation` só decide o que aparece; a verificação real acontece na view ou no serviço (doc 09 §11).
- **Auditoria só por** `AuditService.log`. Nenhum código cria `AuditLog` diretamente.
- **HTML de usuário só entra por** `core/sanitize.py` (`sanitize_description`, baseado em `nh3`, com lista fixa de tags permitidas).
- **Nada é apagado de verdade** quando há histórico: cadastros usam `is_active`, vínculos usam `removed_at` (`UserSector`, `TaskExecutor`), concessões de acesso usam `is_active=False`. Mudanças de dono/responsável geram linhas de log (`OwnerChangeLog`, `TaskResponsavelChangeLog`).
- **Nomes duplicados** em cadastros são checados sem diferenciar maiúsculas e espaços (`core.services._assert_unique_name`, Regras 05 §22-23).

- **Status x etapa x status manual.** `Activity.status` e `Task.status` são o estado operacional (fixo no código, governam fila, bloqueio e conclusão). A **etapa** (`stage : ActivityStage / TaskStage`) é a coluna visual do Kanban e o **status** da interface (`WorkflowStatus` , "condição" no código) é uma leitura manual; ambos são configuráveis **por setor** e nunca alteram o estado operacional: mover um cartão de etapa grava só a etapa e escolher o status grava só o status (`set_stage / set_condition`).
- **Cores e rótulos** de status, urgência e prioridade são configuráveis por organização (`core.EnumColor`) mas o **código** do enum nunca muda de significado; a resolução passa por `core.colors.EnumColorResolver` . Etapas, status, setores e etiquetas de quadro guardam a própria cor (hex, paleta de `core/colors.py`).
- **Posição sem reescrever listas.** Nos quadros, grupos, colunas, itens e etiquetas têm posição decimal (passo 1000): mover algo grava UMA posição entre as vizinhas, e o cliente manda os vizinhos, nunca um número (`boards/services.py` , `PositionService`).
- **Rotas desativadas respondem 410, não 404.** O que foi retirado de uso (fluxo operacional de tarefa, fila, processos, links de anexo antigos) continua reconhecível e responde `410 Gone` com uma mensagem (`RetiredFeatureView` , `RetiredProcessView` , `MovedActivityFileView`); endereços que apenas mudaram de nome redirecionam (301; 308 em POST). Não escreva código novo que dependa das rotas desativadas.
- **Recurso opcional por settings.** A Caixa de Entrada só existe com `INTAKE_ENABLED=true` (rotas dão 404 e o item some do menu; ver [03_CONFIGURACAO.md](#)).
- **Idioma.** Interface, docstrings e mensagens em português; nomes de classes e funções em inglês; valores de enum em português maiúsculo (`EM_FILA` , `CONCLUIDA`).

5. Estrutura de pastas

```
workflow/
├── config/           # settings (base/dev/prod), urls, wsgi, asgi
├── core/ accounts/ acessos/ activities/ boards/ processes/
├── intake/ notifications/ audit/ painel/      # apps (seção 1)
├── templates/       # base.html, parciais (_icon, _sector_picker...) e um diretório por app
├── static/          # css/, js/, img/ (sem bundler; collectstatic gera staticfiles/)
├── tests/           # testes JavaScript (*.test.cjs, rodam com Node + jsdom)
├── docs/            # esta documentação, os docs LPS_*.md e o PDF compilado
├── Regras/          # regras de negócio (o "porquê"; citadas nos docstrings)
├── Telas/           # especificação das telas e do menu
├── manage.py requirements.txt package.json render.yaml
├── .env.example     # modelo das variáveis de ambiente (o .env real não vai para o Git)
├── iniciar_lps.bat  # atalho para subir o servidor no Windows
└── atividade_arquivos/ db.sqlite3 staticfiles/ # gerados em execução, ignorados pelo Git
```

Os testes Python ficam ao lado do código de cada app (`<app>/tests.py` e `<app>/test_*.py`); ver [11_TESTES.md](#).

02 — Ambiente local

Como colocar a LPS para rodar na sua máquina, do zero, com dados de demonstração e um usuário capaz de usar todas as telas. Atualizado em 03/10/2026 (código no commit `add5acd`).

1. Pré-requisitos

- Python 3.12 ou superior (o `.venv` local atual usa 3.13; o Render usa 3.12.7 — ver [12_DEPLOY_RENDER.md](#)).
- Git.
- Node 18+ **apenas** se for rodar os testes JavaScript (`tests/*.test.cjs` , com `jsdom` ; `npm install` e `npm run test:js` — ver [11_TESTES.md](#)).

Os comandos abaixo são para PowerShell no Windows. Em Linux/macOS, troque `.venv\Scripts\Activate.ps1` por `source .venv/bin/activate` e `copy` por `cp` .

2. Instalação

```
git clone https://github.com/Lucasrafael703/workflow.git
cd workflow
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
```

Abra o `.env` e defina ao menos `SECRET_KEY` (qualquer string longa e aleatória serve em dev). O resto pode ficar como está: em dev o e-mail sai no console e o banco é SQLite (`db.sqlite3` , ignorado pelo Git). Todas as variáveis estão em [03_CONFIGURACAO.md](#).

Use `.venv/` , não `venv/` (a pasta `venv/` antiga, de outra máquina, não existe mais, mas `.claude/launch.json` ainda aponta para `venv/Scripts/python.exe` ; ver [13_PENDENCIAS_CONHECIDAS.md](#)).

3. Banco e dados de demonstração

```
python manage.py migrate
python manage.py seed_lps_demo
```

`seed_lps_demo` é idempotente (pode rodar várias vezes) e:

1. roda `seed_acoes` , que grava o catálogo de ações de `acessos/catalog.py` ;
2. cria a organização **Biasi** e a empresa **Biasi Engenharia** (mude com `--org-name` e `--company-name`);
3. cria os setores Comercial, Engenharia, Compras, Financeiro e Almoxarifado;
4. cria os perfis de acesso sugeridos — **Colaborador**, **Gestor de Setor** e **Administrador** — com suas ações.

Ele **não** cria usuários nem quadros. Os quadros padrão de Demandas e Tarefas (`DomainBoard`) são criados sozinhos na primeira vez que a tela é aberta; para criá-los de uma vez em todas as organizações, rode `seed_default_boards` (seção 6).

4. Primeiro usuário

```
python manage.py createsuperuser
```

Preencha o **e-mail**: o login da LPS é por e-mail (`accounts.auth_backends.EmailBackend`), então um superusuário sem e-mail não consegue entrar pela tela de login.

Em seguida, **vincule o usuário a uma organização**. Sem isso, toda tela da LPS redireciona para `/accounts/me/` com a mensagem "Seu usuário ainda não está vinculado a uma organização":

1. `python manage.py runserver` e entre em `http://127.0.0.1:8000/admin/` ;
2. seção **Accounts** → **Perfis** (não confundir com "Perfis de acesso", do app Acessos) → seu usuário → campo **organização** = Biasi → Salvar.

Ou, pelo shell:

```
python manage.py shell -c "from django.contrib.auth import get_user_model; from core.models import Organization; u = get_user_model().objects.get(username='SEU_USUARIO'); u.profile.organization = Organization.objects.get(name='Biasi'); u.profile.save()"
```

Um superusuário ativo passa por todas as verificações de autorização (`AuthorizationService.is_platform_admin`), então não precisa de perfil de acesso. Para testar como um usuário comum, crie outra pessoa em `/usuarios/novo/` e atribua um perfil em `/usuarios/<id>/acessos/`.

5. Rodando

```
python manage.py runserver
```

Ou dê dois cliques em `iniciar_lps.bat` : ele sobe o servidor em `127.0.0.1:8000` numa janela minimizada (usando `.venv`) e abre o Chrome.

`manage.py` usa `config.settings.dev` por padrão (DEBUG ligado, SQLite, e-mail no console).

Duas coisas aparecem **desligadas** num ambiente novo, de propósito:

- a **Caixa de Entrada** (`/entrada/`): inativa por padrão; as rotas dão 404 e o item "Entrada" some do menu. Para ligar, `INTAKE_ENABLED=true` no `.env` ([03_CONFIGURACAO.md](#));
- o **fluxo operacional de Tarefas** (`/fila/` , `/processos/` , `/tarefas/<id>/...`): foi desativado em 02/10/2026 e responde **410**. As tarefas de uma Demanda vivem no quadro dela (`/quadros/...` , painel **Tarefas**); ver [06_ATIVIDADES_E_TAREFAS.md](#).

6. Management commands

Comando	App	O que faz	Quando usar
<code>seed_acoes</code>	<code>acessos</code>	Sincroniza o catálogo de <code>acessos/catalog.py</code> com o banco (<code>update_or_create</code>). Ações que saíram do catálogo ficam <code>is_active=False</code> ; nunca são apagadas.	Sempre que <code>catalog.py</code> mudar. Já é chamado por <code>seed_lps_demo</code> .
<code>seed_lps_demo</code>	<code>core</code>	Organização, empresa, setores e perfis sugeridos (ver seção 3). Opções <code>--org-name</code> , <code>--company-name</code> .	Primeiro setup; ambientes de demonstração.

Comando	App	O que faz	Quando usar
<code>ensure_superuser</code>	<code>accounts</code>	Cria um superusuário a partir de <code>DJANGO_SUPERUSER_USERNAME</code> , <code>DJANGO_SUPERUSER_EMAIL</code> e <code>DJANGO_SUPERUSER_PASSWORD</code> , só se esse username ainda não existir. Sem as variáveis, não faz nada.	Deploy automatizado (roda no build do Render).
<code>check_overdue_tasks</code>	<code>activities</code>	Marca tarefas com prazo comprometido vencido, notifica (in-app + e-mail) uma única vez por tarefa.	Pode ser agendado (Agendador de Tarefas do Windows, cron); não há worker assíncrono. Como o fluxo operacional de <code>Task</code> foi desativado (a migration <code>boards/0008</code> cancela as tarefas operacionais), hoje tende a não encontrar nada.
<code>seed_default_boards</code>	<code>boards</code>	Cria, de forma idempotente, os quadros de domínio padrão de Demandas e de Tarefas (<code>DomainBoard</code>) em todas as organizações.	Opcional: a tela <code>/demandas/</code> já cria o quadro sob demanda (<code>ensure_domain_board</code>).
<code>seed_board_templates</code>	<code>boards</code>	Cria um quadro a partir de um modelo (hoje só <code>orcamentos</code>). Opções: <code>--organization</code> e <code>--user</code> (obrigatórias), <code>--modelo</code> , <code>--nome</code> , <code>--exemplos</code> . Não faz nada se já existir um quadro ativo com o mesmo nome.	Demonstração ou ponto de partida; não roda no deploy.
<code>backfill_demand_boards</code>	<code>boards</code>	Desativado: sempre termina com erro orientando a usar a migration <code>boards/0008</code> .	Não usar; fica no código só como registro da migração antiga.

7. Problemas comuns

Sintoma	Causa	Solução
<code>ImproperlyConfigured: Set the SECRET_KEY environment variable</code>	<code>.env</code> ausente ou sem <code>SECRET_KEY</code>	Copie <code>.env.example</code> para <code>.env</code> e preencha.
Toda tela volta para "Meu perfil" com aviso de organização	<code>Profile.organization</code> vazio	Seção 4.
Login diz que e-mail/senha estão errados para o superusuário	Superusuário criado sem e-mail	Defina o e-mail no Admin ou recrie.
Conta criada em "Criar conta" não entra	A conta fica inativa até confirmar o código de e-mail; o código aparece no console do <code>runserver</code> em dev	Ver 07_CONTAS_E_AUTENTICACAO.md .
Botões/menus não aparecem para um usuário comum	Falta perfil de acesso com a ação no escopo certo	Ver 05_AUTORIZACAO.md .
<code>/entrada/</code> dá 404 e não há "Entrada" no menu	A Caixa de Entrada está inativa por padrão	<code>INTAKE_ENABLED=true</code> no <code>.env</code> (seção 5).
<code>/fila/</code> , <code>/processos/</code> ou <code>/tarefas/<id>/...</code> respondem 410	Fluxo operacional de Tarefas e Processos desativado (02/10/2026)	Não é erro: use o quadro da Demanda.
<code>InconsistentMigrationHistory</code> depois de um <code>git pull</code>	Banco local criado com migrations antigas que foram reescritas	Em dev, apague <code>db.sqlite3</code> e rode <code>migrate + seed_lps_demo</code> de novo.

03 — Configuração

Como os settings estão divididos, todas as variáveis de ambiente que o código lê, e como arquivos estáticos e anexos são guardados e servidos. Atualizado em 03/10/2026 (código no commit `add5acd`).

1. Settings

```
config/settings/
├─ base.py  # comum: apps, middleware, templates, auth, i18n, estáticos, e-mail
├─ dev.py   # DEBUG=True, SQLite, e-mail no console
└─ prod.py  # DEBUG=False, Postgres via DATABASE_URL, whitenoise, HTTPS
```

`base.py` lê o arquivo `.env` na raiz do projeto (`environ.Env.read_env`). Variáveis já definidas no ambiente têm prioridade sobre o `.env` .

Qual settings é usado depende de **como** o Django é iniciado:

Entrada	Default de <code>DJANGO_SETTINGS_MODULE</code>
<code>manage.py</code>	<code>config.settings.dev</code>
<code>config/wsgi.py</code> (gunicorn)	<code>config.settings.prod</code>
<code>config/asgi.py</code>	<code>config.settings.prod</code>

Por isso, no Render, `DJANGO_SETTINGS_MODULE=config.settings.prod` precisa estar definido: o build roda `manage.py` (`collectstatic` , `migrate`), que sozinho usaria dev.

2. Variáveis de ambiente

Variável	Default	Lida em	Observação
<code>SECRET_KEY</code>	— obrigatória	<code>base.py</code>	Sem ela o Django não inicia.
<code>ALLOWED_HOSTS</code>	<code>[]</code>	<code>base.py</code>	Lista separada por vírgula. Em dev com <code>DEBUG=True</code> , <code>localhost</code> já é aceito.
<code>ACTIVITY_FILES_ROOT</code>	<code><projeto>/atividade_de_arquivos</code>	<code>base.py</code>	Pasta dos anexos das demandas (seção 4). O nome <code>atividade_arquivos</code> não mudou com a troca para "Demanda": é local de armazenamento, não endereço.
<code>INTAKE_ENABLED</code>	<code>false</code>	<code>base.py</code>	Liga a Caixa de Entrada (<code>/entrada/</code> , app <code>intake</code>). Desligada (padrão), o item "Entrada" some do menu (<code>acessos.context_processors.navigation</code>) e todas as rotas dela dão 404 (<code>intake/urls.py</code>); o código, as tabelas e as permissões <code>entrada.*</code> continuam. Para reativar: <code>INTAKE_ENABLED=true</code> (no Render, em Environment do serviço web; não está no <code>render.yaml</code> para um novo deploy não desfazer o valor do painel).
<code>EMAIL_HOST</code>	<code>""</code>	<code>base.py</code>	
<code>EMAIL_PORT</code>	<code>587</code>	<code>base.py</code>	

Variável	Default	Lida em	Observação
EMAIL_HOST_USER	""	base.py	
EMAIL_HOST_PASSWORD	""	base.py	
EMAIL_USE_TLS	True	base.py	
DEFAULT_FROM_EMAIL	no-reply@example.com	base.py	Remetente dos e-mails.
ADMIN_GROUP_NAME	ADMIN	base.py	Lida, mas não usada por nenhum código hoje (grep só encontra a definição).
EMAIL_BACKEND	console (dev) · SMTP (prod)	dev.py , prod.py	
DATABASE_URL	— obrigatória em prod	prod.py	Ex.: postgres://user:senha@host:5432/banco .
SECURE_SSL_REDIRECT	True	prod.py	Redireciona HTTP → HTTPS.
SECURE_HSTS_SECONDS	3600	prod.py	
CSRF_TRUSTED_ORIGINS	[]	prod.py	Com esquema: https://*.onrender.com ; O hostname público do Render também é incluído automaticamente por RENDER_EXTERNAL_HOSTNAME .
RENDER_EXTERNAL_HOSTNAME	""	prod.py	Injetada pelo próprio Render em todo Web Service; não se define à mão. Se presente, https://<hostname> entra em CSRF_TRUSTED_ORIGINS .
DJANGO_SUPERUSER_USERNAME	—	ensure_superuser	
DJANGO_SUPERUSER_EMAIL	""	ensure_superuser	Preencha: o login é por e-mail.
DJANGO_SUPERUSER_PASSWORD	—	ensure_superuser	
DJANGO_SETTINGS_MODULE	ver seção 1	Django	

Todas, exceto `RENDER_EXTERNAL_HOSTNAME` e `DJANGO_SETTINGS_MODULE` , estão listadas (as opcionais comentadas) em `.env.example`. Nunca coloque valores reais em documentação nem no Git: o `.env` é ignorado pelo Git.

3. Outras configurações relevantes

Setting	Valor	Por quê
LANGUAGE_CODE / TIME_ZONE	pt-br / America/Sao_Paulo	
USE_TZ	True	Datas gravadas em UTC, exibidas no fuso local.
AUTHENTICATION_BACKENDS	accounts.auth_backends.EmailBackend	Login por e-mail (<code>Telas/09_01_LOGIN.md</code>). Único backend: <code>ModelBackend</code> por username não está ativo.
LOGIN_URL / LOGIN_REDIRECT_URL / LOGOUT_REDIRECT_URL	login / home / login	
DEFAULT_AUTO_FIELD	BigAutoField	

Setting	Valor	Por quê
ACTIVITY_FILES_URL	/demanda-arquivos/	base_url do storage dos anexos (activity_files_storage()). Não há rota que sirva esse prefixo (seção 4).
INSTALLED_APPS próprios	accounts , core , acessos , activities , processes , notifications , intake , boards , audit , painel	Ver 01_ARQUITETURA.md.
Context processors próprios	notifications...unread_notifications_count , acessos...navigation , activities...my_active_sessions	Ver 10_FRONTEND.md.
SECURE_PROXY_SSL_HEADER (prod)	("HTTP_X_FORWARDED_PROTO", "https")	O Render termina o TLS no proxy. Sem isso, SECURE_SSL_REDIRECT entra em loop de redirecionamento.
Cookies (prod)	SESSION_COOKIE_SECURE , CSRF_COOKIE_SECURE	
HSTS (prod)	INCLUDE_SUBDOMAINS e PRELOAD ligados	

4. Arquivos estáticos, mídia e anexos

Tipo	Pasta de origem	URL	Quem serve
Estáticos do projeto	static/ (STATICFILES_DIRS)	/static/	dev: runserver · prod: whitenoise a partir de staticfiles/
Estáticos coletados	staticfiles/ (STATIC_ROOT , gerado por collectstatic , ignorado pelo Git)	/static/	whitenoise
Mídia	media/ (MEDIA_ROOT ; a pasta não existe no projeto)	/media/	django.views.static.serve somente com DEBUG=True (dev); em prod não há rota
Anexos das demandas	ACTIVITY_FILES_ROOT	/demandas/<id>/anexos/<id>/download/	ActivityAttachmentDownloadView (view própria, com autorização)

- Em prod, STORAGES["staticfiles"] usa whitenoise.storage.CompressedManifestStaticFilesStorage (nomes com hash e compressão). Por isso é preciso rodar collectstatic no build. Em dev o storage padrão é mantido para não exigir collectstatic .
- Anexos são gravados pelo storage activity_files_storage() (activities/models.py) no caminho <empresa>/<código da demanda>/<arquivo> (DEM-AAAA-NNNNN) (Regras 12 e 13); sem empresa, vai para sem-empresa/ .
- **Os anexos não têm mais endereço público.** Até 01/10/2026 config/urls.py servia /demanda-arquivos/ (antes /atividade-arquivos/) com django.views.static.serve , sem login. Hoje essa rota não existe: o arquivo só é entregue por /demandas/<id>/anexos/<id>/download/ (ActivityAttachmentDownloadView), que exige login, o anexo ser da mesma organização, a ação demanda.visualizar sobre a demanda e, para anexo de mensagem da conversa, que a mensagem seja visível à pessoa. O download sai como attachment (força baixar). O antigo /atividade-arquivos/... responde 410 (MovedActivityFileView , core/legacy_redirects.py) com a orientação de abrir a demanda.
- No plano free do Render o disco não é persistente: anexos enviados se perdem a cada deploy ou reinício.

04 — Modelos de dados

Todos os modelos da aplicação, agrupados por app, com os campos que importam, relacionamentos (`related_name`), enums e constraints. Campos triviais (`created_at` , `name`) aparecem só quando têm regra. A modelagem conceitual do produto está em `Regras/08_BANCO_DE_DADOS.md` .

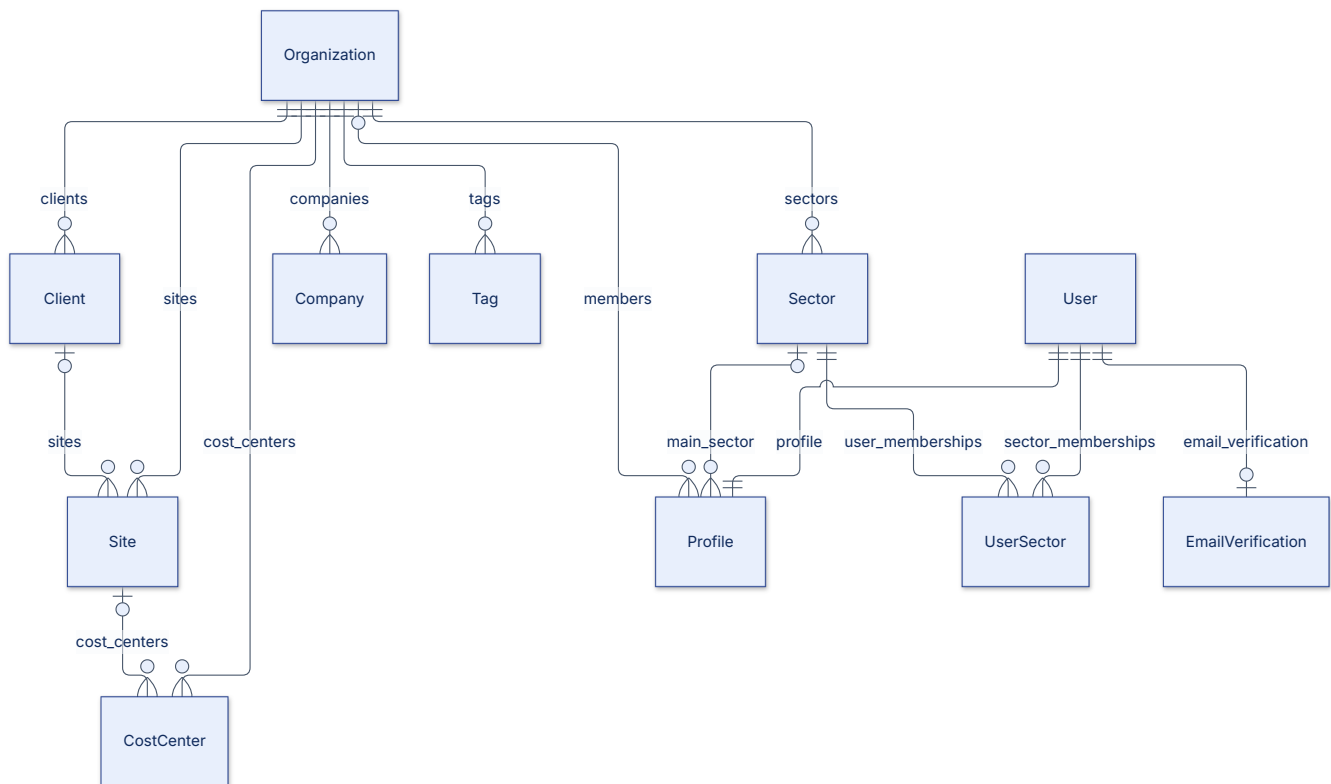
Vocabulário. Desde 01/10/2026 a interface e as URLs dizem **Demanda**, mas o código segue com `Activity` (models, tabelas, `related_name` , chaves de enum e de permissão no código); este documento usa os nomes do código e escreve "demanda" no texto corrido. Desde 02/10/2026 os textos da interface dizem **Etapas** (`ActivityStage` / `TaskStage`) e **Status** (`WorkflowStatus` , que o código e os docs mais antigos chamam de *condição*). Esse "Status" manual por setor **não** é o `Activity.status` / `Task.status` operacional.

Estado descrito: HEAD `add5acd` (02/10/2026). Atualizado em 03/10/2026.

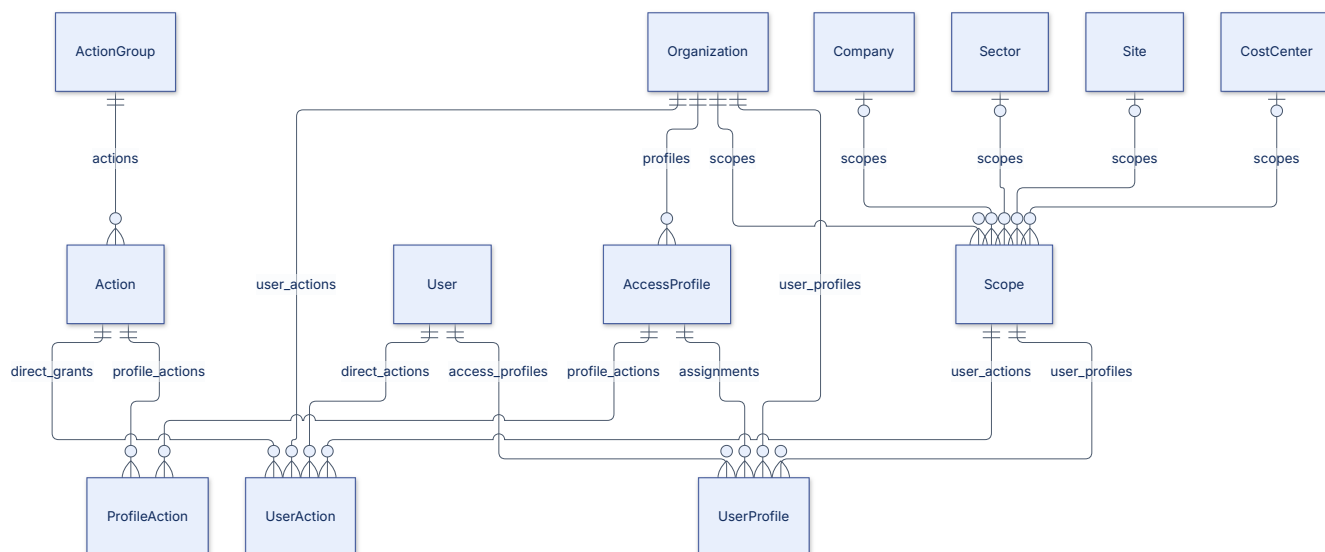
1. Visão geral

Os diagramas estão divididos por domínio para caberem numa página. Só aparecem entidades e relacionamentos; os campos estão nas seções 2 a 7. `User` é o `auth.User` do Django. Em todos os diagramas `AccessProfile` é `acessos.Profile` e `Profile` é `accounts.Profile` .

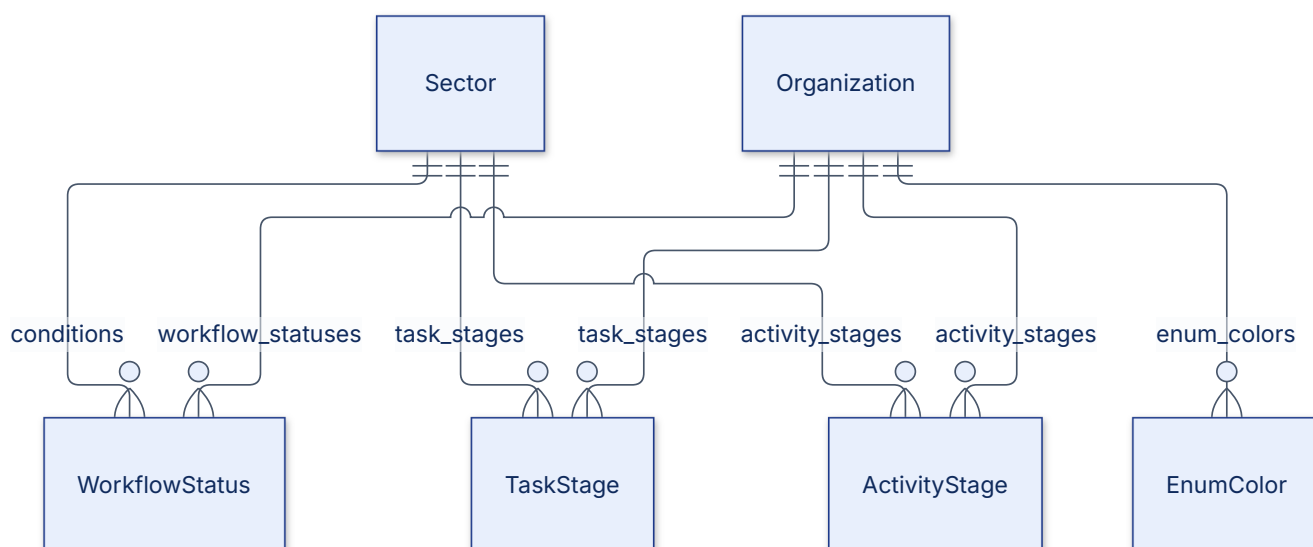
1.1 Pessoas, organização e cadastros



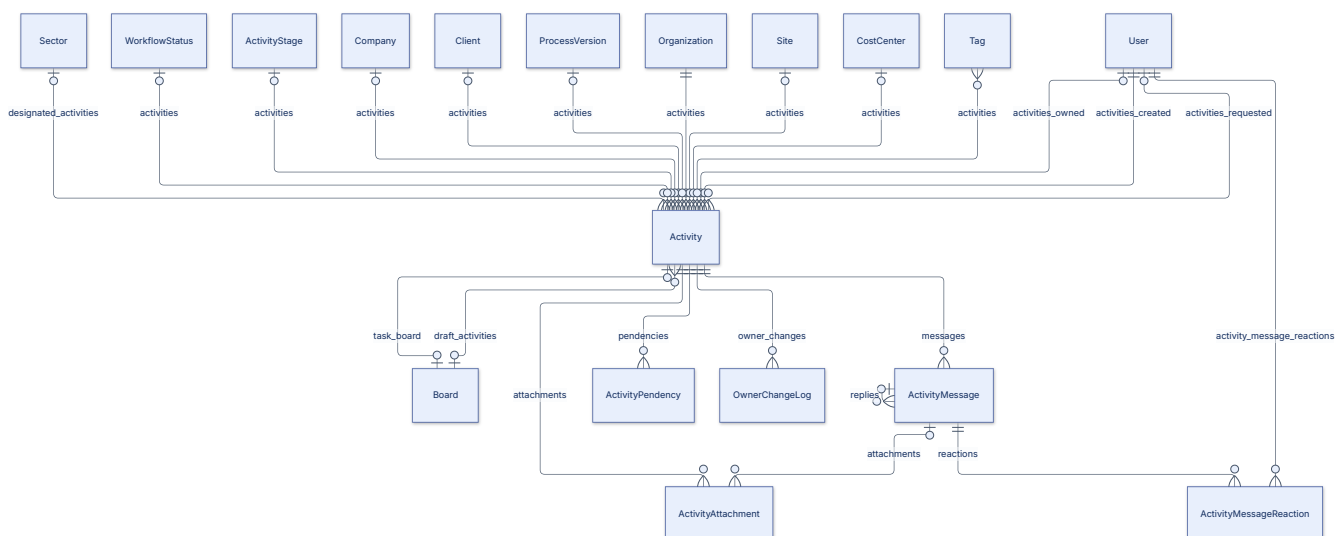
1.2 Acesso (perfis, ações e escopos)



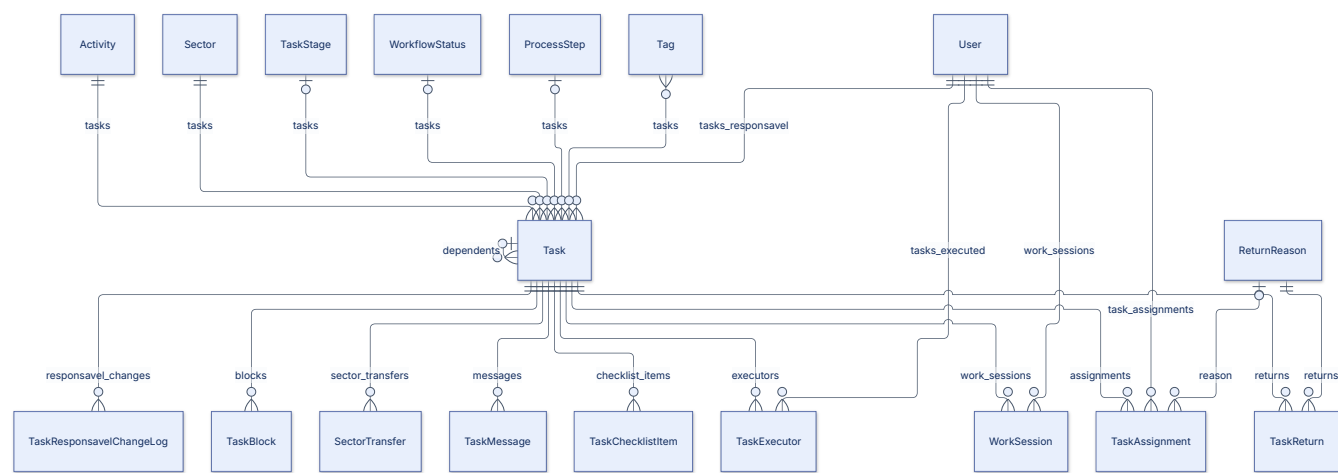
1.3 Etapas, status manuais e cores por setor



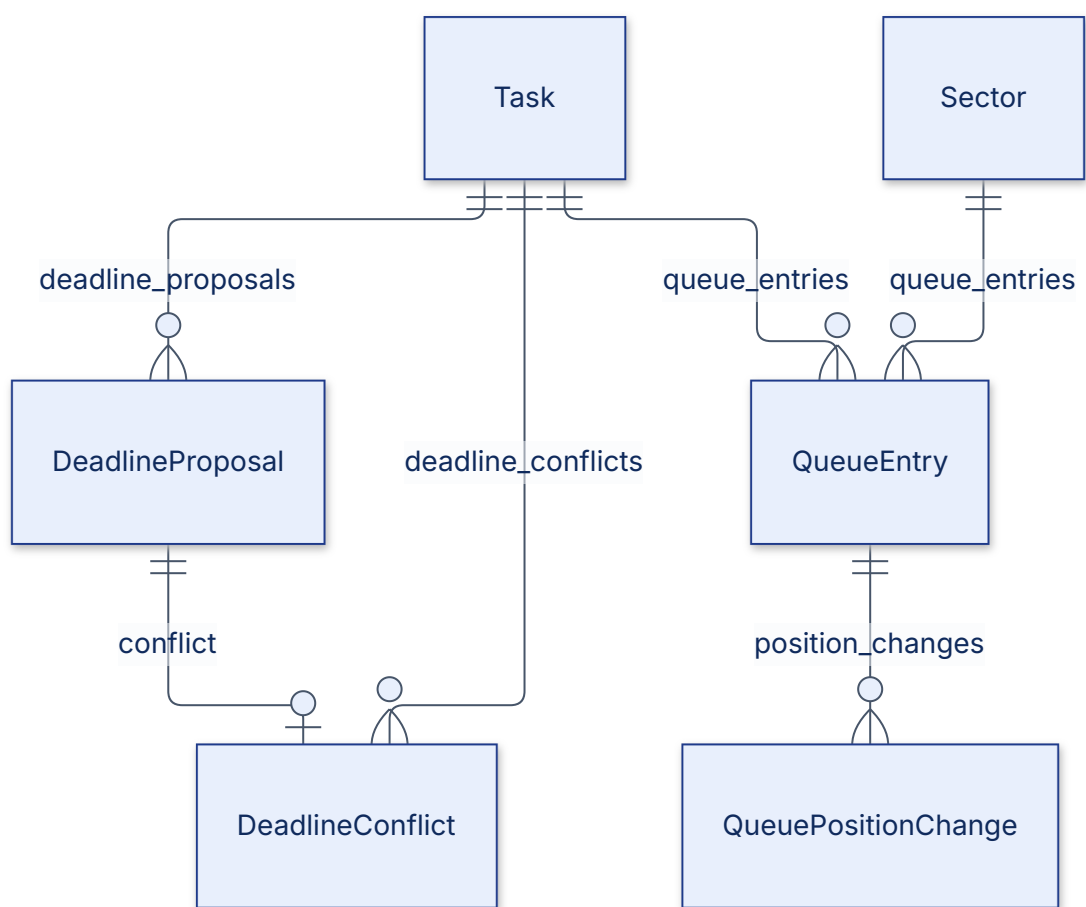
1.4 Demanda (Activity)



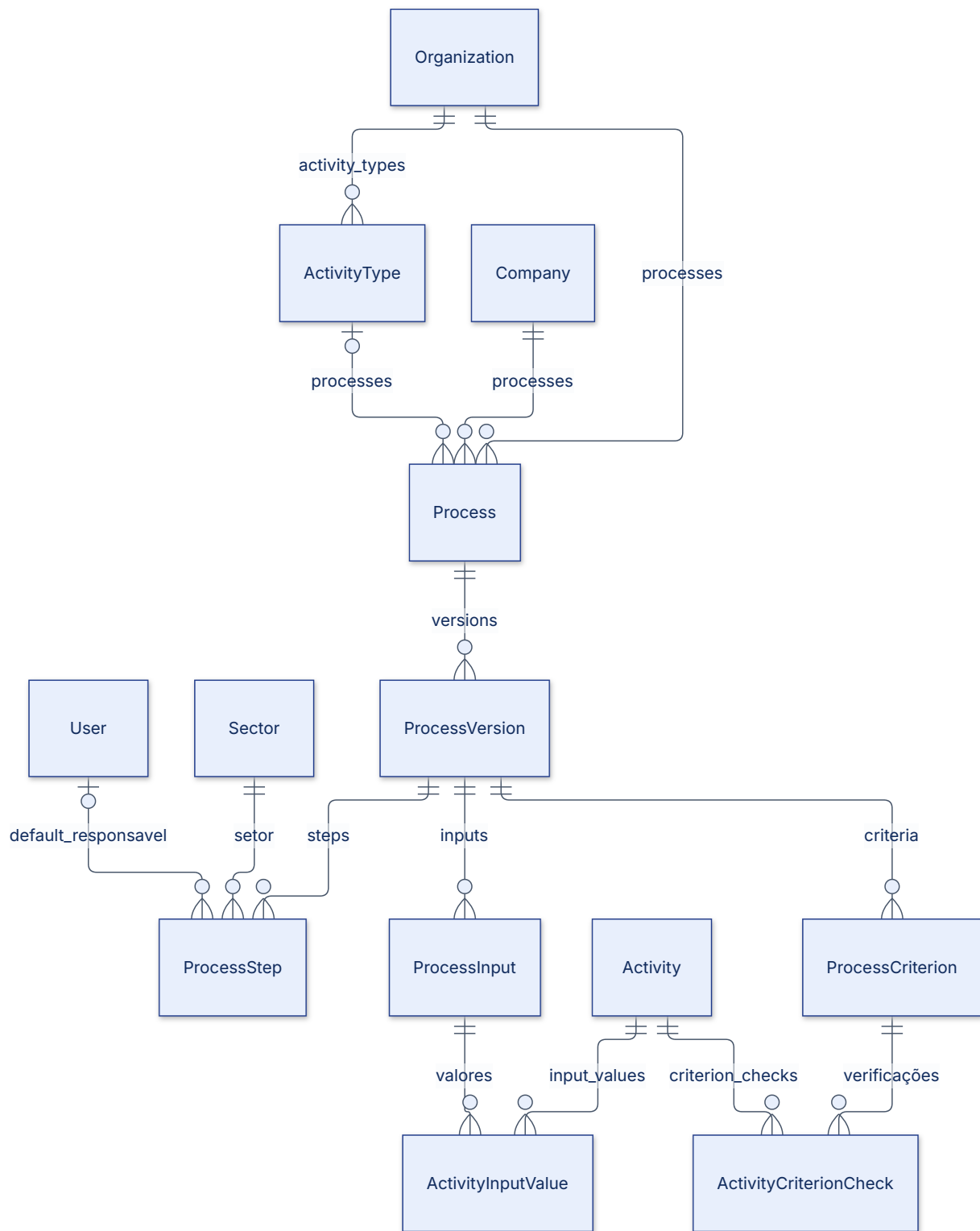
1.5 Tarefa (Task)



1.6 Fila e prazo



1.7 Processos



Os diagramas de notificações e auditoria (§7), Caixa de Entrada (§7.1) e quadros (§7.2 e §7.3) estão nas respectivas seções.

2. core

Todos, exceto `Organization`, têm `organization` → `Organization` (CASCADE).

Model	Campos relevantes	Constraints / regras
Organization	name (único), is_active	O tenant. Ordenação por name .
Company	name , document , is_active	Único por (org, name). Empresa operacional interna (Regras 05 §9).
Sector	name , description , color (hex, padrão #3B82F6 ; migração core/0015), is_active , created_by (null, SET_NULL)	Único por (org, name). Nunca fixo no código.
Client	name , document , phone , email , address , is_active	Único por (org, name). Quem solicita o serviço; diferente de Company .
Site (obra)	client → Client (null, SET_NULL, sites), name , is_active	Regras 10 §11. Não tem mais company (migração 0010). Sem unicidade de nome.
CostCenter	site → Site (null, SET_NULL, cost_centers), name , is_active	Sem unicidade de nome.
Tag	name , color (hex da paleta de 36 cores), is_active , created_by	Único por (org, name).
ActivityStage	sector → Sector (CASCADE, activity_stages), name , order , color , is_active , is_default , column_limit (inteiro curto, null), created_by	Único por (org, sector, name); uma etapa padrão por (org, sector) (constraint parcial is_default). Coluna do Kanban de demandas; nunca altera Activity.status . column_limit só avisa (nunca impede mover ou criar). Na interface: Etapas .
TaskStage	sector → Sector (CASCADE, task_stages), name , order , color , is_active , is_default , column_limit , created_by	Igual a ActivityStage , para tarefas; nunca altera Task.status .
WorkflowStatus	sector → Sector (CASCADE, conditions), domain (activity / task), name , description , behavior (legado : vazio, não é usado), color , order , is_active , is_default , created_by	Único por (org, sector, domain, name); um padrão por (org, sector, domain) (constraint parcial). Nunca movimenta fila, cronômetro, bloqueio, conclusão ou transição. Cada setor ganhou um "Normal" padrão (migração 0014). Na interface: Status . É referenciado por Activity.condition e Task.condition (SET_NULL).
EnumColor	domain (activity_status , task_status , activity_urgency , task_priority), code , color , label , description , is_hidden , updated_by (null, SET_NULL), updated_at	Único por (org, domain, code). Sobrescreve cor/rótulo de um valor de enum fixo (code não é FK: o service o valida contra o TextChoices); o significado do código não muda. is_hidden só tira o código das opções oferecidas.

Organization , Company , Sector , Client , Site , CostCenter e Tag têm ordering = ["name"] . ActivityStage , TaskStage e WorkflowStatus ordenam por (org, sector, [domain,] order, name).

As etapas e os status são **por setor** desde 01/10/2026 (migração core/0012_sector_scoped_stages_conditions , que copiou o cadastro antigo da organização para cada setor). Uma demanda sem setor perdeu a etapa global antiga.

A paleta e as cores padrão ficam em core/colors.py (PALETTE , DEFAULTS), espelhadas em static/js/color-utils.js .

3. accounts

Model	Campos relevantes	Constraints / regras
Profile	user 1:1 (CASCADE, profile), organization (null, PROTECT, members), phone , main_sector (null, SET_NULL, sem related_name), created_at	Criado vazio por signal (accounts/signals.py) ao criar um User . main_sector é só o filtro padrão das telas (Regras 05 §29).

Model	Campos relevantes	Constraints / regras
UserSector	user (CASCADE, sector_memberships), sector (CASCADE, user_memberships), role (MEMBRO padrão, GESTOR), joined_at , removed_at (null)	Único por (user, sector) enquanto removed_at é nulo (constraint parcial unique_active_user_sector) — sair e voltar abre novo período. Ordenação -joined_at . Ser GESTOR não concede autorização por si só. Salvar/apagar invalida o cache de escopos do motor de autorização (signal).
EmailVerification	user 1:1 (CASCADE, email_verification), code (6 dígitos), created_at , expires_at , attempts , confirmed_at (null)	CODE_TTL =15 min, RESEND_COOLDOWN =60 s, MAX_ATTEMPTS =5. issue(user) substitui o código anterior (mesma linha).

4. acessos

Model	Campos relevantes	Constraints / regras
ActionGroup	key (slug, único), name , description , order , is_active	Só organiza a interface.
Action	group (PROTECT, actions), key (único, ex. fila.reordenar), name , description , is_sensitive , is_active	Definida pelo produto em catalog.py (semeada por seed_acoes), nunca pelo cliente. is_sensitive só destaca na UI/auditoria. Índice (group, is_active).
Profile (perfil de acesso)	organization (CASCADE, profiles), name , description , actions (M2M via ProfileAction , profiles), is_active , created_by , updated_at	Único por (org, name). Índice (org, is_active).
ProfileAction	profile (profile_actions), action (profile_actions), created_by	Único por (profile, action).
Scope	organization (scopes), type , company / sector / site / cost_center (opcionais, CASCADE, scopes), relation , is_active	clean() exige exatamente o campo do tipo; chamado no save() . Linhas reutilizáveis. Índice (org, type).
UserProfile	organization (user_profiles), user (access_profiles), profile (CASCADE, assignments), scope (PROTECT, user_profiles), is_active , created_by	Único por (user, profile, scope) enquanto ativo (constraint parcial). Índices (user, is_active) e (profile, is_active).
UserAction (concessão direta)	organization (user_actions), user (direct_actions), action (direct_grants), scope (PROTECT, user_actions), is_active , created_by	Único por (user, action, scope) enquanto ativo. Só positiva — não existe "negar".

Scope.type : ORGANIZACAO , EMPRESA , SETOR , OBRA , CENTRO_CUSTO , RELACIONAL . Scope.relation : MINHAS_ATIVIDADES (rótulo "Demandas das quais sou dono"), MINHAS_TAREFAS , MEUS_SETORES , SETORES_GERENCIADOS . O significado de cada um está em [05_AUTORIZACAO.md](#).

O catálogo ganhou, no ciclo de 01-02/10/2026, as chaves demanda.* (renomeadas de atividade.* ; grupo demandas), *.definir_etapa / *.definir_condicao , etapa.gerir , condicao.gerir e o grupo quadros com quadro.visualizar , quadro.criar , quadro.editar , quadro.excluir (sensível), quadro.gerir_colunas , quadro.criar_item , quadro.editar_item e quadro.excluir_item — ver migrações acessos/0003 a 0005 na seção 8.

5. activities

Demanda

Activity (rótulo "demanda"; Meta.ordering = ["-created_at"]):

Campo	Tipo / relação	Regras
code	Char(20), único, null/blank, editable=False	DEM-AAAA-NNNNN , gerado no primeiro save() (Regra 12); até 01/10/2026 era ATV-... , reescrito pela migração activities/0020 . A busca ainda aceita o prefixo antigo (code_search_term).
title	Char(200)	O resultado esperado. Rascunho sem título usa DRAFT_TITLE_PLACEHOLDER ("Rascunho sem título").
description	Text, blank	HTML sanitizado no formulário.
internal_notes	Text, blank	Nunca aparece para o cliente.
status	Char(14), Status , padrão ABERTA	Verdade operacional.
urgency	Char(6), Urgency , padrão MEDIA	Na interface: Prioridade.
organization	FK Organization (PROTECT, activities)	
client	FK Client (null, SET_NULL, activities)	Quem solicitou o serviço.
company	FK Company (null, PROTECT, activities)	
site	FK Site (null, SET_NULL, activities)	Obra.
cost_center	FK CostCenter (null, SET_NULL, activities)	
sector	FK Sector (null, SET_NULL, designated_activities)	"Grupo designado": setor a que a demanda é endereçada, distinto do setor de cada tarefa.
stage , stage_changed_at	FK ActivityStage (null, SET_NULL, activities); datetime null	Camada visual; não altera status . stage_changed_at alimenta o indicador de tempo parado do Kanban.
condition	FK WorkflowStatus (null, SET_NULL, activities)	"Status" manual por setor (leitura do trabalho); não altera status .
owner	FK User (null/blank, PROTECT, activities_owned)	Único dono; nulo só em rascunho.
requested_by	FK User (null, SET_NULL, activities_requested)	Solicitante interno .
external_requester	Char(150), blank	Quem pediu fora da organização — texto livre, não é cadastro.
created_by	FK User (PROTECT, activities_created)	
address	Char(255), blank	Endereço adicional ao do cadastro do cliente.
files_location	Char(500), blank	Link ou caminho dos arquivos — a LPS não guarda esses arquivos (os anexos de ActivityAttachment são outra coisa).

Campo	Tipo / relação	Regras
tags	M2M Tag (blank, activities)	
requested_deadline	datetime null	
board_setup_mode	Char(12), BoardSetupMode , padrão BLANK	Escolha do passo "Quadro de tarefas" da criação: em branco ou a partir de um modelo. O quadro real (Board.source_template) é quem manda.
board_template	FK boards.Board (null, SET_NULL, draft_activities)	Modelo escolhido no rascunho.
process_version	FK ProcessVersion (null, PROTECT, activities)	Gravado uma única vez (ver seção 6).
completion_outcome	Char(28), blank, CompletionOutcome	Escolhido ao finalizar.
created_at , updated_at	datetime (auto_now_add / auto_now)	updated_at mostra há quanto tempo um rascunho está parado.
first_action_at , completed_at , cancelled_at , reopened_at	datetime null	
completed_by	FK User (null, SET_NULL, sem related_name)	
cancelled_reason	Text, blank	

Satélites da demanda:

Model	Campos relevantes
OwnerChangeLog	activity (CASCADE, owner_changes), previous_owner , new_owner , changed_by (todos PROTECT), changed_at — nunca sobrescrito
ActivityPendency	activity (pendencies), reason , status , comment , decision_deadline (null), notify_client , previous_owner / approver (null, SET_NULL), opened_by (PROTECT), opened_at , resolved_by / resolved_at (null), resolution_comment
ActivityMessage	activity (messages), author (PROTECT), body , kind , parent → self (null, CASCADE, replies : respostas em fio), visibility (MessageVisibility , padrão ALL : aplicada na leitura, não só no seletor), created_at
ActivityAttachment	activity (attachments), message → ActivityMessage (null, SET_NULL, attachments), file (storage em ACTIVITY_FILES_ROOT , caminho <empresa>/<código>/<arquivo>), original_name , uploaded_by (PROTECT), uploaded_at
ActivityMessageReaction	message (CASCADE, reactions), user (CASCADE, activity_message_reactions), emoji (até 12 caracteres), created_at — único por (message, user, emoji): clicar de novo remove a reação
ReturnReason	organization (return_reasons), name , is_active — único por (org, name)

Enums:

Enum	Valores
Activity.Status	RASCUNHO , ABERTA (padrão), EM_ANDAMENTO , BLOQUEADA , PENDENTE , CONCLUIDA , CANCELADA
Activity.Urgency	BAIXA , MEDIA (padrão), ALTA
Activity.CompletionOutcome	SUCESSO , CONCLUIDO_COM_PENDENCIAS , DECLINADO , CANCELADO

Enum	Valores
Activity.BoardSetupMode	BLANK (padrão; "Começar em branco"), TEMPLATE ("Usar quadro existente")
ActivityPendency.Reason	APROVACAO_GESTOR , AJUSTES_REVISOES (as duas exigem aprovação), MATERIAL , INFORMACOES_CLIENTE
ActivityPendency.Status	ABERTA , APROVADA , ENCERRADA
MessageKind (demanda e tarefa)	NORMAL , DECISAO , RISCO , COMPROMISSO , IMPEDIMENTO
MessageVisibility (só ActivityMessage)	ALL (todas as pessoas com acesso), PARTICIPANTS , SECTOR

Tarefa

Task (Meta.ordering = ["activity", "order"]):

Campo	Tipo / relação	Regras
activity	FK Activity (CASCADE, tasks)	
sector	FK Sector (PROTECT, tasks)	Setor responsável.
stage , stage_changed_at	FK TaskStage (null, SET_NULL, tasks); datetime null	Camada visual; não altera status .
condition	FK WorkflowStatus (null, SET_NULL, tasks)	"Status" manual por setor.
responsavel	FK User (PROTECT, tasks_responsavel)	Obrigatório desde a migração 0015 .
created_by	FK User (PROTECT)	
depends_on	FK self (null, SET_NULL, dependents)	Única dependência entre tarefas irmãs.
process_step	FK processes.ProcessStep (null, PROTECT, tasks)	Vazio nas tarefas manuais.
order	PositiveInt, padrão 1	
title , description	Char(200); Text blank	
tags	M2M Tag (blank, tasks)	
status	Char(14), Status , padrão NAO_INICIADA	Verdade operacional.
priority	Char(8), Priority , padrão MEDIA	Migração 0022 (02/10/2026). Cor/rótulo configuráveis por EnumColor (task_priority).
requested_deadline , committed_deadline	datetime null	
created_at , first_action_at , completed_at , cancelled_at	datetime	
completed_by	FK User (null, SET_NULL)	Quem concluiu — limpo ao reabrir.
overdue_notified_at	datetime null	Evita reenviar a notificação de atraso.

Model	Campos relevantes
TaskExecutor (participante)	task (CASCADE, executors), user (PROTECT, tasks_executed), added_by (PROTECT), added_at, removed_at (remoção lógica). Sem constraint de unicidade
TaskAssignment	task (assignments), user (PROTECT, task_assignments), assigned_by, assigned_at, status (PENDENTE padrão, ACEITA, RECUSADA), decided_at, reason → ReturnReason (null, PROTECT), observation
TaskResponsavelChangeLog	task (responsavel_changes), previous_responsavel, new_responsavel, changed_by (PROTECT), changed_at
WorkSession	task (work_sessions), user (PROTECT, work_sessions), started_at, ended_at (null; quando o trabalho aconteceu), is_manual (período informado pela pessoa, não cronometrado), logged_at (quando o sistema soube ; nulo no cronômetro e nas sessões antigas), manual_reason (WorkSession.ManualReason : ESQUECI_INICIAR, FORA_DA_LPS, AJUSTE_PERIODO, OUTRO ; vazio no cronômetro e em "Adicionar tempo trabalhado"), note (até 255 caracteres) — propriedade duration
TaskBlock	task (blocks), reason (Char 255), observation, started_by (PROTECT)/ started_at, ended_by (null, SET_NULL)/ ended_at (null)
SectorTransfer	task (sector_transfers), from_sector (null, PROTECT), to_sector (PROTECT), moved_by, moved_at, note
TaskReturn	task (returns), from_sector, to_sector (PROTECT), reason → ReturnReason (PROTECT, returns), observation, returned_by, returned_at
TaskMessage	task (messages), author, body, kind (MessageKind), created_at — sem fio nem visibilidade (isso é só de ActivityMessage)
TaskCheckListItem	task (checklist_items), text, is_done, order, created_by (PROTECT), done_by (null, SET_NULL), done_at

Task.Status : NAO_INICIADA (padrão do model, mas nenhum fluxo deixa a tarefa nele), DISPONIVEL, EM_FILA, EM_EXECUCAO, BLOQUEADA, DEVOLVIDA, CONCLUIDA, CANCELADA. Task.Priority : BAIXA, MEDIA (padrão), ALTA.

Constraint de Task : unique_task_per_activity_process_step — UniqueConstraint em (activity, process_step) com condition=process_step IS NOT NULL. Uma etapa de processo gera no máximo uma tarefa por demanda; tarefas manuais (process_step nulo) não participam. Índice parcial, suportado por SQLite e PostgreSQL.

Tarefa que aguarda a etapa anterior: é DISPONIVEL, **sem** QueueEntry, com depends_on apontando para uma tarefa ainda não CONCLUIDA. Ao concluir a predecessora, TaskService a enfileira e ela vira EM_FILA. A propriedade Task.waiting_for devolve a predecessora pendente (só para exibição) e Task.status_label mostra "Aguardando etapa anterior" nesse caso.

Tarefas e quadros da demanda (02/10/2026). A migração boards/0008 cancelou todas as Task ainda operacionais (CANCELADA, com AuditLog de motivo "Tarefa operacional arquivada na centralização do quadro."; encerrou sessões e saídas de fila abertas). Os registros Task e seus satélites continuam no banco como histórico, mas o fluxo ativo de tarefas de uma demanda vive no **quadro da demanda** (Board com kind = DEMAND, itens BoardItem; ver §7.2). Não confundir as duas coisas ao ler o código: Task é o modelo operacional original (fila, cronômetro, prazo); BoardItem é a tarefa do quadro.

Fila e prazo

Model	Campos relevantes
QueueEntry	task (CASCADE, queue_entries), sector (PROTECT, queue_entries), position, queue_size_at_entry, entered_at, left_at (null) — uma linha por passagem pela fila (Regras 04 §37); propriedade is_active = left_at nulo. Ordenação (sector, position)
QueuePositionChange	queue_entry (position_changes), old/new_position, old/new_total, reason (AUTOMATICA_CONCLUSAO, AUTOMATICA_ENTRADA, MANUAL, ENTRADA_PRIORITARIA), note, changed_by (null, SET_NULL; nulo = automático), changed_at
DeadlineProposal	task (deadline_proposals), proposed_deadline, proposed_by (PROTECT), proposed_at, status (PENDENTE padrão, ACEITO, RECUSADO), decided_by (null, SET_NULL), decided_at, decision_note

Model	Campos relevantes
DeadlineConflict	task (deadline_conflicts), proposal 1:1 (CASCADE, conflict), status (ABERTO padrão, RESOLVIDO), opened_at, resolution_note, resolved_by (null, SET_NULL), resolved_at

Activity, Task e DeadlineConflict ainda declaram Meta.permissions do Django (can_view_all_activities, can_change_owner, can_reopen_activity, can_cancel_activity; can_assume_task, can_assign_task, can_reorder_queue, can_view_full_queue; can_resolve_deadline_conflict). São legado: a autorização real é o motor do acessos .

6. processes

Desativado na interface desde 02/10/2026. Todas as rotas de processes/ respondem **410 Gone** ("Processos foram desativados. Use o quadro da Demanda.") por RetiredProcessView . Os modelos e os dados continuam preservados; o que segue descreve o que está no banco.

Model	Campos relevantes
ActivityType	organization (CASCADE, activity_types), name, is_active, created_at — único por (org, name); rótulo "tipo de demanda"
Process	organization (processes), company (PROTECT, processes), activity_type (null, SET_NULL, processes), name, description, is_active, created_by (PROTECT), created_at — único por (company, name); propriedades published_version, draft_version
ProcessVersion	process (CASCADE, versions), number, status (RASCUNHO padrão, PUBLICADO, SUBSTITUIDO), output_description, output_evidence_type (ARQUIVO, LINK, CHECKLIST, CONFIRMACAO; blank), created_by (PROTECT), created_at, published_by (null, SET_NULL)/ published_at — único por (process, number); só editável em rascunho (is_editable)
ProcessInput	version (inputs), name, input_type (TEXTO padrão, ARQUIVO, DATA, NUMERO, LINK, SELECAO), is_required (padrão verdadeiro), source (SOLICITANTE, EXECUTOR, TERCEIRO; blank), help_text, order
ProcessCriterion	version (criteria), name, is_required, order
ProcessStep	version (steps), sector (PROTECT), name, order, depends_on_previous (padrão verdadeiro), default_responsavel → User (null, SET_NULL)
ActivityInputValue	activity (CASCADE, input_values), process_input (PROTECT), value, is_received, received_at, received_by (null, SET_NULL) — único por (activity, process_input)
ActivityCriterionCheck	activity (CASCADE, criterion_checks), process_criterion (PROTECT), is_met, met_at, met_by (null, SET_NULL) — único por (activity, process_criterion)

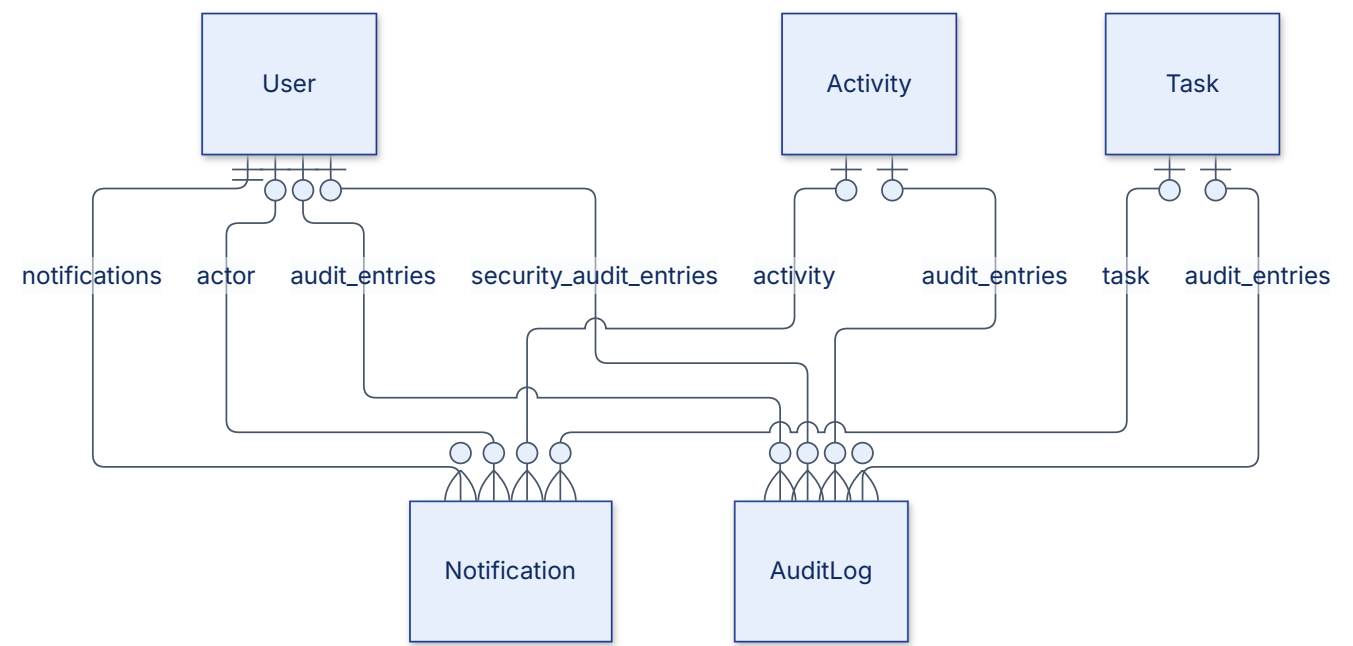
ProcessInput, ProcessCriterion e ProcessStep ordenam por (version, order); ProcessVersion, por (process, -number). Os related_name de ProcessInput → ActivityInputValue, ProcessCriterion → ActivityCriterionCheck, ProcessStep.sector e default_responsavel são + (sem acesso reverso).

ProcessStep.default_responsavel — quem responde pela tarefa gerada pela etapa. Opcional; precisa ser uma pessoa **ativa da mesma organização** do processo (validado em ProcessStep.clean() / save() e em ProcessStepService), mas **não** precisa participar do setor da etapa — a criação manual de tarefas já aceita qualquer pessoa da organização como responsável. Só se altera em rascunho; create_new_version copia o valor.

ActivityInputValue.value é texto. TEXTO guarda o texto; DATA, a data ISO (AAAA-MM-DD); NUMERO, o decimal normalizado (1500.5); LINK, uma URL http(s). ARQUIVO e SELECAO guardam apenas a confirmação de recebimento (is_received) e uma observação livre em value — não há vínculo com anexos nem lista de opções no molde (ver [13_PENDENCIAS_CONHECIDAS.md](#)).

Vínculo permanente: `Activity.process_version` é gravado uma única vez, por `ProcessApplicationService.apply` (`activities/process_application.py`), e nunca é trocado nem removido. Publicar a versão 2 de um processo não altera demandas que receberam a versão 1.

7. notifications e audit



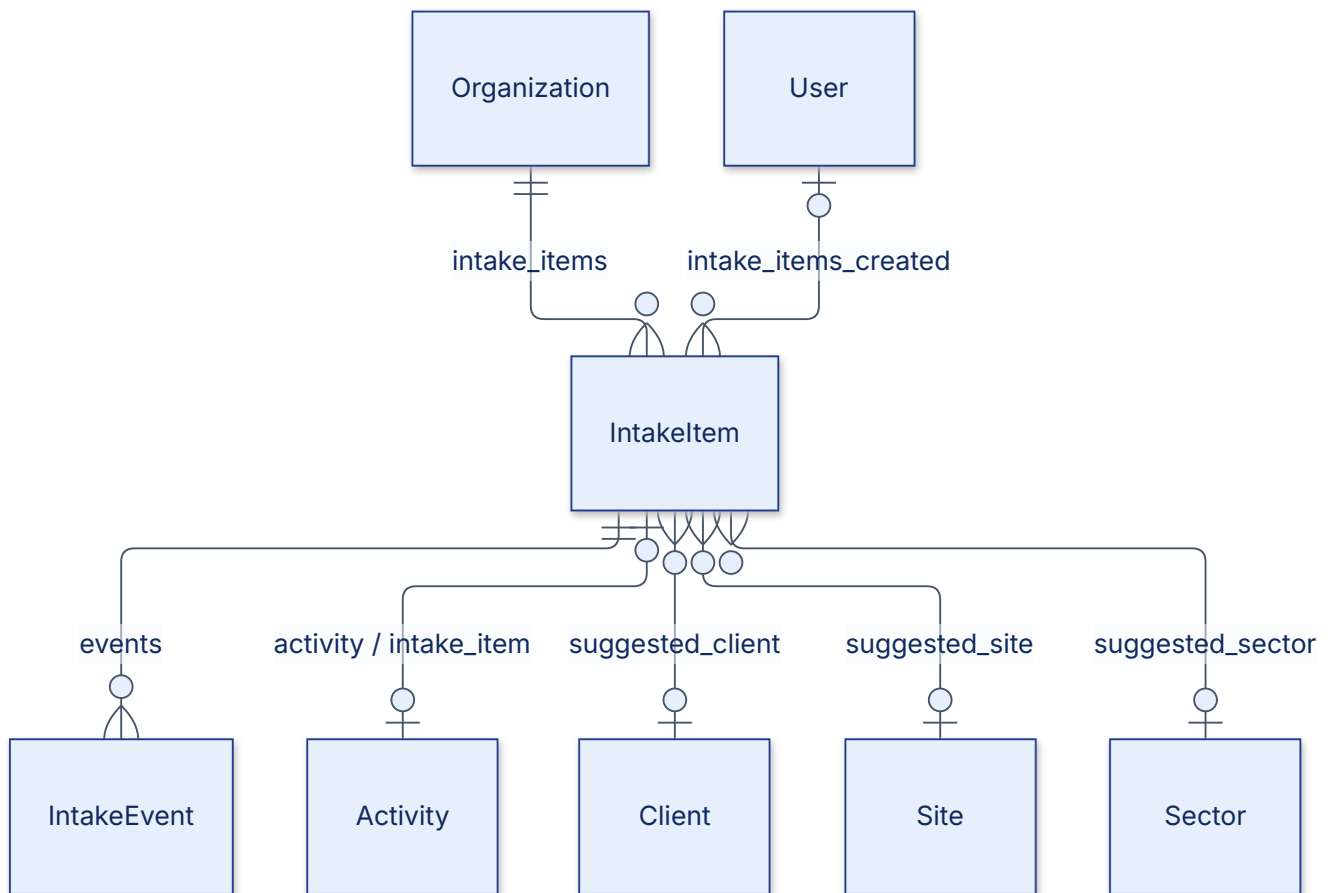
Model	Campos relevantes
Notification	<code>recipient</code> (CASCADE, notifications), <code>event_type</code> (Char 24), <code>actor</code> (null, SET_NULL, + : quem delegou, propôs, mencionou ou concluiu), <code>activity</code> (null, CASCADE, +), <code>task</code> (null, CASCADE, +), <code>title</code> (150), <code>message</code> (255), <code>url</code> (255, blank), <code>is_read</code> , <code>created_at</code> , <code>read_at</code> — índice (recipient, is_read); ordenação - created_at
AuditLog	<code>user</code> (null, SET_NULL, audit_entries), <code>activity</code> (null, SET_NULL, audit_entries), <code>task</code> (null, SET_NULL, audit_entries), <code>target_user</code> (null, SET_NULL, security_audit_entries ; eventos de segurança), <code>action</code> (Char 24), <code>field_name</code> , <code>old_value</code> , <code>new_value</code> , <code>reason</code> , <code>target_type</code> (Char 64, blank), <code>target_id</code> (inteiro grande, null), <code>metadata</code> (JSON, padrão {}), <code>timestamp</code> — índices (activity, timestamp), (task, timestamp) e (target_type, target_id, timestamp)

`target_type` / `target_id` / `metadata` (migração `audit/0010` , 02/10/2026) existem para os eventos que não pertencem a uma demanda nem a uma tarefa — hoje, os dos quadros (`BOARD_*` , com `metadata["board_id"]`).

Os valores de `Notification.EventType` e `AuditLog.Action` estão em [08_NOTIFICACOES_E_AUDITORIA.md](#).

7.1 intake (Caixa de Entrada)

Inativa por padrão desde 01/10/2026 (`INTAKE_ENABLED=false`): o item "Entrada" some do menu e `/entrada/...` dá 404. O app, as tabelas, as migrações e as permissões `entrada.*` continuam no banco; o que segue descreve os models.



Model	Campos relevantes
IntakeItem	organization (PROTECT, intake_items), source (EMAIL padrão, TEAMS, USUARIO, FORMULARIO), external_id, subject, sender_name, sender_email (minúsculas), raw_content (texto puro , nunca renderizado como HTML; máx. 20.000 caracteres), content_hash (sha256 do conteúdo normalizado, anti-duplicata; db_index, editable=False), received_at (padrão agora), status (NOVO padrão, CONVERTIDO, IGNORADO), suggested_title / suggested_client / suggested_site / suggested_sector (null, SET_NULL, +) / suggested_deadline, confidence (0 a 100), suggestion_reasons (JSON: lista de frases), activity (1:1, null, SET_NULL, intake_item), created_by (null, SET_NULL, intake_items_created), created_at, resolved_by (null, SET_NULL, +) / resolved_at / resolution_note
IntakeEvent	item (CASCADE, events), user (null, SET_NULL, +), kind (REGISTRADA, EDITADA, CONVERTIDA, IGNORADA, RESTAURADA), note, created_at

- **Constraints:** `unique_intake_external_id_per_source` — único por (organização, origem, `external_id`) quando `external_id` não é vazio (a mesma mensagem não entra duas vezes por um canal); `intake_confidence_0_100`. Índice `intake_org_status_idx` em (organization, status, -received_at). Ordenação -received_at, -id.
- **Confiança:** `confidence_level` é ALTA (≥ 70), MEDIA (40–69) ou BAIXA (< 40).
- **IntakeEvent em vez de AuditLog:** AuditLog só liga a Activity e Task, então o que acontece com a solicitação antes de a demanda existir (e depois) fica em IntakeEvent. A demanda criada tem a sua auditoria normal (CREATE). A nota do evento CONVERTIDA guarda o código da demanda (DEM-...; migração intake/0003 trocou o prefixo antigo ATV-).
- **Estados:** NOVO → CONVERTIDO (terminal), NOVO → IGNORADO, IGNORADO → NOVO (intake/policies.py).

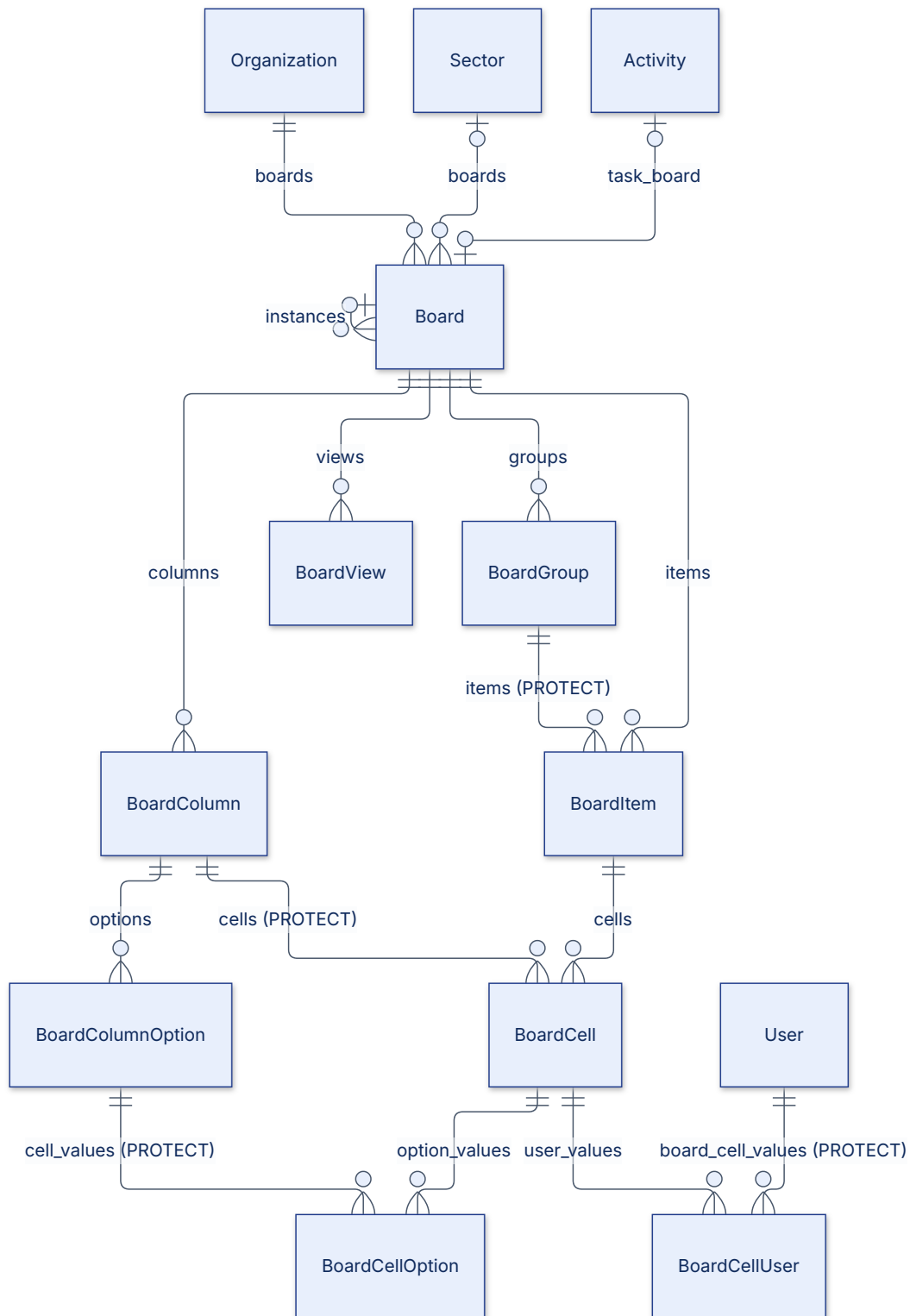
7.2 boards (Quadros dinâmicos)

Há dois tipos de quadro (Board.kind):

- **TEMPLATE (Modelo)** — quadro avulso da organização, com setor (sector), usado como modelo. Não tem demanda.

- **DEMAND (Demanda)** — o quadro de tarefas **de uma demanda**: `Board.activity` é 1:1 com `Activity` (CASCADE, `related_name="task_board"`). Nasce em branco (com as colunas padrão, o Kanban e o Calendário) ou como **cópia de um modelo** (`source_template`, `SET_NULL`, `instances`), por `BoardInstantiationService.create_for_activity` (`boards/demand_services.py`). O setor do quadro tem de ser o setor da demanda (`Board.clean()`).

Desde `boards/0008` o quadro é a única fonte de verdade das tarefas da demanda: o título e o responsável são o nome do item e uma coluna de Pessoa comuns (os campos `BoardColumn.binding` e `BoardItem.task`, criados em `0007`, foram removidos). Quem pode ver ou editar é decidido por `DemandBoardAccess` (dono, criador e quem aparece numa célula de Pessoa participam; as ações `quadro.*` seguem valendo).



Modelo	Campos principais
Board	organization (CASCADE, boards), name (160), description , item_label (como a primeira coluna se chama; padrão "Nome da Tarefa" — era "Elemento" até 0007 , que migrou os existentes), kind (TEMPLATE padrão, DEMAND), sector (null, SET_NULL, boards ; obrigatório para novos modelos, quadros legados podem não ter), activity (1:1, null, CASCADE, task_board), source_template (self, null, SET_NULL, instances), is_active , created_by (PROTECT, boards_created), created_at , updated_at . Ordenação (name , id)
BoardGroup	board (CASCADE, groups), name , color (#RRGGBB , padrão #579BFC), position , is_active . É organização visual, não um status: mover de grupo não muda dado
BoardView	board (views), name (até 80), type (KANBAN padrão, CALENDAR), position , settings (JSON, ver abaixo), is_active , created_by (PROTECT, board_views_created). Não guarda dado de negócio : é só a configuração de uma forma de enxergar os mesmos itens. A tabela ("Quadro principal") é implícita e não tem registro
BoardColumn	board (columns), name (120), type (ver tipos abaixo), position , width (96–640, padrão 160), description , settings (JSON), is_required , is_visible , is_active , created_by (PROTECT, board_columns_created), updated_at
BoardColumnOption	column (options), label , color (padrão #C4C4C4), position , is_default , is_done , is_active (etiqueta de Status e Lista suspensa)
BoardItem	board (items), group (PROTECT, items), name (255, pode ficar vazio), position , is_active , created_by (PROTECT, board_items_created), updated_by (PROTECT, board_items_updated), created_at , updated_at
BoardCell	item (cells), column (PROTECT, cells), value_text , value_number (Decimal 24,6), value_date , value_datetime , value_boolean , value_json (padrão {}), updated_by (PROTECT, board_cells_updated), updated_at . Única por (item, column) (uniq_board_cell_item_column); índices (column, value_number), (column, value_date), (column, value_boolean)
BoardCellUser / BoardCellOption	Liga a célula a uma pessoa (PROTECT, board_cell_values) / uma etiqueta (PROTECT, cell_values), com position ; únicas por par (uniq_board_cell_user , uniq_board_cell_option). Uma célula guarda um valor hoje; a tabela já comporta vários

Todos expõem `organization_id` por um caminho curto (propriedade), porque a autorização e o isolamento entre organizações dependem dele. As posições são `DecimalField(20, 6)`, padrão 1000, e todas as listas ordenam por `position`.

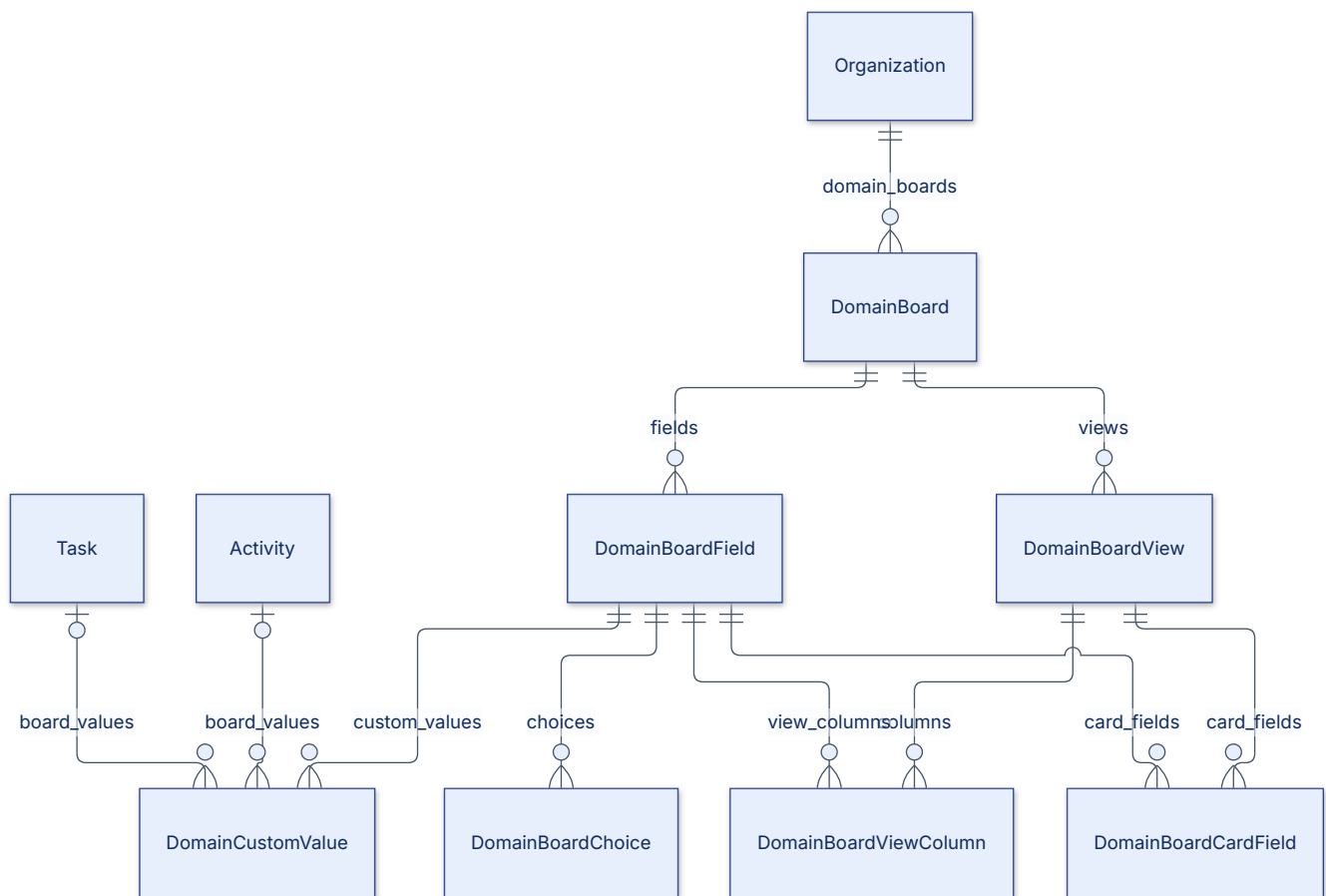
Constraints e índices de Board: `board_kind_matches_activity` (`CheckConstraint`: `TEMPLATE` exige `activity` nulo; `DEMAND` exige `activity` preenchido); índices (organization, is_active) e (organization, kind, is_active).

- **Tipos de coluna** (`BoardColumn.Type`): ativos hoje `STATUS`, `DROPDOWN`, `TEXT`, `DATE`, `PERSON`, `NUMBER`, `CURRENCY`, `CHECKBOX` (`ACTIVE_TYPES`). Existem no enum, mas ainda **não** se criam nem se editam: `FILE`, `TIMELINE`, `PRIORITY`, `CONFIRMATION`, `RELATION`, `FORMULA`, `AI_EXTRACT`.
- **Valor tipado, não JSON**: número ordena como número e data como data **no banco**; relatórios e filtros futuros precisam disso. A conversão do que o usuário digitou fica em `boards/validators.py` (`normalize_cell_value`): moeda vira `Decimal` (aceita `1.234,56` e `1234.56`), data aceita ISO e `dd/mm/aaaa`, pessoa tem de ser usuário **ativo da mesma organização**, etiqueta tem de ser **ativa e da mesma coluna**, e `is_required` recusa limpar. O valor antigo só é apagado depois de o novo passar na validação.
- **settings por tipo** (só chaves conhecidas; as demais são descartadas): `DATE` → `show_time`, `allow_weekends`, `is_deadline` (destaca "vencido"), `format`; `NUMBER` → `decimal_places` (0–6), `unit`, `minimum`, `maximum`; `CURRENCY` → `currency` (`BRL` / `USD` / `EUR`), `decimal_places`, `minimum`, `maximum`; `PERSON` → `multiple` (só `false` por ora).
- **Posição decimal** (passo 1000): mover algo grava **uma** posição entre as vizinhas (ponto médio); quando a distância some ($\leq 0,001$) as irmãs são renumeradas (`PositionService.rebalance`). O cliente manda os vizinhos (`before_id` à esquerda/acima, `after_id` à direita/abaixo), nunca o número; se os dois não são mais adjacentes, vale a da esquerda.
- **Etiquetas em tabela própria** (não no JSON da coluna) para poder ordenar, referenciar e trocar a cor sem regravar as células. Status nasce com *Não iniciado* (padrão), *Em andamento* e *Concluído* (`is_done`); item novo já nasce com a etiqueta padrão de cada coluna. Excluir uma etiqueta é lógico e **limpa** as células que a usavam.

- **Conversão de tipo** (`ColumnService.conversion_plan / change_type`): `safe` (Número↔Moeda, Status↔Lista, qualquer tipo → Texto, escreve o texto que a tela mostrava), `parsed` (Texto → Número/Moeda/Data: o que não converte é limpo) e `destructive` (demais: limpa os valores). Fora de `safe` exige `confirm` (a view responde **409** com a contagem).
- **Visualizações (Kanban)** (`boards/kanban.py`): o Kanban **lê as colunas e os itens do quadro**; cada etiqueta da coluna agrupadora vira uma raia (então renomear ou recolorir a etiqueta muda a raia na hora: não há outro cadastro de nome ou de cor) e cada item é o mesmo item da tabela, nunca uma cópia. Todo quadro nasce com um Kanban (`ViewService.create_default_kanban`); o modelo Orçamentos já traz o seu configurado (por Status, soma do Valor). `BoardView.settings` aceita só estas chaves (as demais são descartadas e toda referência a coluna tem de ser de uma coluna **ativa deste quadro** do tipo certo): `group_by` (id de coluna de Status ou Lista suspensa; vazio = a primeira de Status, depois de Lista), `show_empty` (raias sem cartão), `blank_lane` (`auto` só quando há itens sem etiqueta · `always` · `never`), `sort` ({by: manual|name|created|<id de coluna>, dir}), `sum_column` (Número ou Moeda: soma no cabeçalho da raia), `card_fields` (ids na ordem do cartão; vazio = os 4 primeiros campos visíveis, fora o agrupador), `show_field_names` . Item sem etiqueta (ou com a etiqueta apagada) cai na raia **"Em branco"**. Arrastar o cartão grava a etiqueta da coluna agrupadora pelo mesmo serviço da célula (`CellService.set_value`): mesma validação, mesma auditoria.
- **Visualizações (Calendário)** (`boards/calendar_view.py`): o Calendário **lê os mesmos itens** e só decide em que **dia** mostrá-los, pela coluna de Data escolhida. Mover um cartão grava a **mesma célula** de data (`CellService.set_value` : a mesma permissão, validação e auditoria da tabela, que registra a data anterior e a nova), então não há cópia nem sincronização. `BoardView.type` = `CALENDAR` ; `settings` aceita só: `date_field` (id de coluna de Data; vazio = a coluna marcada como prazo ou a primeira Data), `period` (hoje só `month`), `color_by` (id de coluna de Status/Lista, `group` ou `none` ; vazio = o primeiro Status), `card_fields` (até 6 ids, na ordem; vazio = Status, a primeira Pessoa e mais um campo visível, no máximo 3; a coluna de Data da lente não entra), `show_weekends` , `show_completed` . Criar exige uma coluna de Data (`BoardError` se o quadro não tem) e já grava a coluna de Data e a de cor que o servidor resolveria, para criar outra Data depois não trocar a lente de lugar. Só se busca o intervalo da grade (o mês e as semanas de ponta). Item **sem data** não está na grade: tem contador e lista "Sem data". **Concluído** = etiqueta com `is_done` na primeira coluna de **Status**; **atrasado** = coluna marcada como prazo (`is_deadline`) com data anterior a hoje e item aberto. A hora só existe quando a coluna tem `show_time` e a célula tem hora (dia sem hora é "dia inteiro": nunca se inventa 00:00).
- **Exclusão lógica** em tudo (`is_active`). Excluir coluna **mantém** as células no banco; grupo só sai se estiver sem itens.
- **Auditoria genérica**: `audit.AuditLog` ganhou `target_type` (ex.: `board_cell`), `target_id` e `metadata` (JSON). Todo registro do quadro leva `metadata["board_id"]` , que alimenta o histórico do quadro (`/quadros/<id>/historico/`). Migração `audit/0010` ; `audit/0011` acrescentou as ações `BOARD_VIEW_*` .
- **Migrações do app**: ver a seção 8.

7.3 boards — quadros de domínio (Demandas e Tarefas)

Camada que guarda **só a configuração** das lentes (Tabela, Kanban, Calendário) da tela de Demandas e da de Tarefas. Os registros continuam sendo `Activity` e `Task` ; não há cópia de dado operacional (`boards/domain_defaults.py` , `domain_services.py`). Criada em `boards/0004` e semeada para cada organização por `boards/0005` (e sob demanda por `ensure_domain_board` , idempotente).



Modelo	Campos principais
DomainBoard	organization (CASCADE, domain_boards), domain (DEMAND, TASK), name, created_at, updated_at. Único por (organization, domain) — uniq_domain_board_org_domain
DomainBoardField	board (CASCADE, fields), key (slug, 64), label, type (TEXT, NUMBER, CURRENCY, DATETIME, PERSON, STAGE, SECTOR, PRIORITY, SELECT, BOOLEAN, CHECKLIST, RELATION), position, settings (JSON), is_system (campo nativo da demanda/tarefa), is_active. Único por (board, key)
DomainBoardChoice	field (choices), label, color (padrão #94A3B8), position, is_active — opções de um campo SELECT
DomainBoardView	board (views), name (80), type (TABLE "Quadro principal", KANBAN, CALENDAR), position, settings (JSON), is_default, is_active. Único por (board, name)
DomainBoardViewColumn	view (columns), field (view_columns), position, width (padrão 160), is_visible. Único por (view, field)
DomainBoardCardField	view (card_fields), field (card_fields), position, is_visible. Único por (view, field)
DomainCustomValue	field (custom_values), activity (null, CASCADE, board_values), task (null, CASCADE, board_values), value (JSON), updated_by (null, SET_NULL, domain_board_values_updated), updated_at

- **DomainCustomValue tem exatamente um alvo:** domain_custom_value_one_target (CheckConstraint: ou activity ou task, nunca os dois nem nenhum), e valor único por campo e alvo (uniq_domain_value_activity, uniq_domain_value_task, constraints parciais).
- **Campos de sistema semeados** — demandas: title, owner, sector, urgency, requested_deadline, stage, condition, tasks; tarefas: title, activity, responsavel, sector, priority, requested_deadline, stage, checklist. Visualizações padrão: "Quadro principal" (TABLE, group_by=stage, a padrão), "Kanban" (group_by=stage, show_empty) e "Calendário" (date_field=requested_deadline).

8. Migrations

- Cada app tem suas migrations em `<app>/migrations/`. `painel` e o pacote de projeto `config` não têm models nem migrations.
- Estado em 03/10/2026 (HEAD `add5acd`):

App	Migrations (última)	Observação
accounts	0001 a 0003_emailverification	
acessos	0001 a 0005_quadros_actions	0002 rótulos; 0003, 0004 e 0005 mexem em dados (abaixo)
activities	0001 a 0024_activity_board_setup	dois 0005_* unidos por 0006_merge_*
audit	0001 a 0011_visualizacoes_de_quadro	0002 acrescenta <code>target_user</code> ; 0003 – 0008 e 0011 só ampliam <code>AuditLog.Action</code> ; 0009 rótulos; 0010 campos genéricos (<code>target_type</code> , <code>target_id</code> , <code>metadata</code>) e índice
boards	0001 a 0008_centralize_demand_board_items	
core	0001 a 0016_enumcolor_prioridade_datarefa	
intake	0001 a 0003_codigo_da_demanda_nas_notas	
notifications	0001 a 0008_demanda_nas_mensagens_gravadas	0002 acrescenta <code>actor</code> ; 0002 – 0007 ampliam <code>Notification.EventType</code> ; 0008 ajusta texto gravado
processes	0001 a 0003_atividade_vira_demanda_nos_rotulos	

- Migrations de dados e mudanças de esquema que valem conhecer:
 - `activities/0012_fix_stuck_em_execucao_tasks` : corrige tarefas `EM_EXECUCAO` sem sessão aberta.
 - `activities/0014_populate_task_responsavel` e `0015_task_responsavel_not_null` : preenchem e tornam obrigatório o responsável.
 - `core/0006 – 0007` : migram cores de tag para hex.
 - `core/0010_remove_site_company_site_client` : obra passa a ter cliente em vez de empresa.
 - Aplicação de processo (30/09/2026): `processes/0002_processstep_default_responsavel`, `activities/0016_task_process_step` (campo + constraint parcial), `audit/0007_process_execution_actions` (novos valores de `AuditLog.Action`) e `notifications/0006_process_applied_event` (novo `Notification.EventType`).
 - Trabalho informado depois (30/09/2026): `activities/0017_trabalho_informado_depois` (`WorkSession.logged_at`, `manual_reason`, `note`) e `audit/0008`.
 - Demanda externa e arquivos (30/09/2026): `activities/0018` (`external_requester`, `files_location`).
 - Caixa de Entrada (01/10/2026): `intake/0001_initial`. Só cria tabelas novas; **não** altera `activities`, `audit` nem `notifications`.
 - **Atividade → Demanda (01/10/2026)**: `activities/0019`, `core/0011`, `acessos/0002`, `audit/0009`, `intake/0002`, `notifications/0007` e `processes/0003` só trocam rótulos (`verbose_name`, `choices`, `help_text`).
`activities/0020_codigo_da_demanda` **reescreve dados**: o prefixo do código `ATV-` vira `DEM-` e a pasta de anexos `<empresa>/<código>/` é renomeada (reversível e idempotente); `intake/0003` acompanha nas notas de evento; `notifications/0008` ajusta o texto fixo já gravado nas notificações; `acessos/0003_chaves_demanda` renomeia as

chaves de permissão (`atividade.*` → `demanda.*` , grupo `atividades` → `demandas`) no próprio registro, preservando perfis e concessões.

- o **Etapas e status por setor (01/10/2026):** `core/0012_sector_scoped_stages_conditions` (dados: copia o cadastro da organização para cada setor, remapeia as etapas das demandas e tarefas; exige o `SET CONSTRAINTS` no PostgreSQL, corrigido em 02/10/2026), `core/0013_stage_column_limit` , `core/0014_seed_default_conditions` (dados: um status "Normal" padrão por setor e domínio), `core/0015_sector_color` , `activities/0021_activity_task_condition` (`Activity.condition` , `Task.condition`) e `acessos/0004_etapas_condicoes_actions` (dados: novas ações, concedidas a quem já tinha `demanda.mover_estagio` / `tarefa.mover_estagio` , `estagio_tarefa.gerir` ou `cor_status.gerir`).
- o **Quadros dinâmicos (01-02/10/2026):** `boards/0001_initial` , `boards/0002_boardview` , `boards/0003_kanban_padrao_nos_quadros` (dados: dá um Kanban aos quadros existentes; reversível), `boards/0006_visualizacao_calendario` (só acrescenta `CALENDAR` às escolhas de `BoardView.type`), `audit/0010_auditoria_generica_de_quadros` e `audit/0011_visualizacoes_de_quadro` , e `acessos/0005_quadros_actions` (cria o grupo `quadros` e **concede por mapeamento** aos perfis existentes: quem tem `demanda.criar` recebe visualizar/criar item/editar item; quem tem `demanda.aprovar_pendencia` , também criar/editar quadro, gerir colunas e excluir item; quem tem `seguranca.gerir_perfis` , tudo; reversível).
- o **Quadros de domínio e prioridade da tarefa (02/10/2026):** `boards/0004_quadros_de dominio` (cria as sete tabelas `DomainBoard*` / `DomainCustomValue`), `boards/0005_seed_domain_workboards` (dados: semeia Demandas e Tarefas em cada organização), `activities/0022_task_priority` (`Task.priority`) e `core/0016_enumcolor_prioridade_da_tarefa` (valor `task_priority` em `EnumColor.domain`).
- o **Quadro por demanda e conversa (02/10/2026):** `activities/0023_activity_message_threads` (`ActivityMessage.parent` / `visibility` , `ActivityAttachment.message` , `ActivityMessageReaction`), `activities/0024_activity_board_setup` (`Activity.board_setup_mode` , `board_template`), `boards/0007_demand_boards` (`Board.kind` / `sector` / `activity` / `source_template` , `BoardColumn.binding` e `BoardItem.task` provisórios, constraint `board_kind_matches_activity` , `item_label` "Elemento" → "Nome da Tarefa") e `boards/0008_centralize_demand_board_items` (**dados:** converte as tarefas ligadas a itens em itens comuns, **cancela todas as Task ainda operacionais**, encerra sessões e saídas de fila abertas e remove `binding` e `BoardItem.task` ; **não é reversível** em dados).
- Depois de mudar um model: `python manage.py makemigrations` e confira com `python manage.py makemigrations --check --dry-run` antes de commitar.

05 — Autorização

Como a LPS decide se uma pessoa pode fazer algo. O modelo é **SUJEITO + AÇÃO + ESCOPO + ORIGEM**: toda permissão é uma ação, vale em um escopo e tem uma origem explicável (um perfil ou uma concessão direta). Nega por padrão e não existe "negar explícito". Regras de produto em `Regras/05_USUARIOS_SETORES_E_AUTORIZACOES.md` ; implementação em `acessos/` . Atualizado em 03/10/2026 (código no commit `add5acd`): na interface "Atividade" virou **Demanda** (as constantes continuam `ATIVIDADE_*` , as chaves gravadas são `demanda.*`), e fila, processos e as telas de tarefa foram desativados em favor dos **quadros** (seções 4.4 e 8).

1. Peças

Peça	Onde	O que é
Catálogo de ações	<code>acessos/catalog.py</code>	Constantes (<code>catalog.TAREFA_INICIAR = "tarefa.iniciar"</code>) e a lista <code>GROUPS</code> . Única fonte da verdade : o código referencia as constantes; <code>seed_acoes</code> grava no banco.
<code>Action</code> / <code>ActionGroup</code>	<code>acessos/models.py</code>	O catálogo materializado. Uma ação só existe se a aplicação tiver comportamento correspondente (doc 05 §14).
Perfil de acesso (<code>acessos.Profile</code>)	por organização	Conjunto reutilizável de ações (<code>ProfileAction</code>).
<code>Scope</code>	por organização	"Onde vale": a organização toda, uma empresa, um setor, uma obra, um centro de custo ou uma relação.
<code>UserProfile</code>	por pessoa	Atribuição: pessoa + perfil + escopo.
<code>UserAction</code>	por pessoa	Concessão direta: pessoa + ação + escopo. Exceção pontual, não a forma padrão (doc 05 §27).

Participar de um setor (`accounts.UserSector`), mesmo como GESTOR, **não concede nada** por si só. Só alimenta os escopos relacionais `MEUS_SETORES` e `SETORES_GERENCIADOS` e os destinatários de notificações.

2. Escopos

<code>Scope.type</code>	Cobre o recurso quando...
<code>ORGANIZACAO</code>	sempre (dentro da organização)
<code>EMPRESA</code>	<code>recurso.company_id == scope.company_id</code>
<code>SETOR</code>	<code>recurso.sector_id == scope.sector_id</code> (para tarefa, é o setor da tarefa)
<code>OBRA</code>	<code>recurso.site_id == scope.site_id</code>
<code>CENTRO_CUSTO</code>	<code>recurso.cost_center_id == scope.cost_center_id</code>
<code>RELACIONAL</code> + <code>MINHAS_ATIVIDADES</code>	a pessoa é dona da atividade
<code>RELACIONAL</code> + <code>MINHAS_TAREFAS</code>	a pessoa é participante ativa (<code>TaskExecutor</code>) da tarefa
<code>RELACIONAL</code> + <code>MEUS_SETORES</code>	o setor do recurso é um dos setores da pessoa
<code>RELACIONAL</code> + <code>SETORES_GERENCIADOS</code>	o setor do recurso é um que a pessoa gerencia (GESTOR)

`MINHAS_TAREFAS` olha só os **participantes** (`TaskExecutor`); O `responsavel` da tarefa não entra nesse conjunto. Desde a centralização em quadros (02/10/2026) a `Task` operacional é histórico: quem trabalha numa tarefa é a pessoa na célula **Pessoa** do item do quadro da Demanda, e isso **não** passa por escopo (ver `DemandBoardAccess` , seção 4.3). Na prática `MINHAS_TAREFAS` ficou sem efeito sobre o trabalho novo.

O "endereço" de um recurso é montado por `ResourceContext.of(recurso)` em `acessos/services.py` :

Recurso	Endereço
<code>Activity</code>	org, empresa, setor designado, obra, centro de custo, dono
<code>Task</code>	setor da tarefa + resto da atividade + participantes ativos
<code>QueueEntry</code>	setor da fila + dados da atividade + participantes (legado: a fila foi desativada)
<code>DeadlineProposal</code> , <code>DeadlineConflict</code>	o da tarefa (legado: prazos de tarefa desativados)
<code>Sector</code> , <code>Company</code> , <code>Site</code> , <code>CostCenter</code>	org + o próprio id
<code>IntakeItem</code> (Caixa de Entrada)	org + setor sugerido + obra sugerida. Sem setor sugerido só casam escopos de organização
<code>ActivityStage</code> , <code>TaskStage</code> , <code>WorkflowStatus</code> (etapas e status do setor)	org + setor do cadastro
<code>Board</code> , <code>BoardGroup</code> , <code>BoardColumn</code> , <code>BoardColumnOption</code> , <code>BoardItem</code> , <code>BoardCell</code> , <code>BoardView</code> e os modelos de quadro de domínio (<code>DomainBoard</code> , <code>DomainBoardField</code> , <code>DomainBoardChoice</code> , <code>DomainBoardView</code> , <code>DomainBoardViewColumn</code> , <code>DomainBoardCardField</code> , <code>DomainCustomValue</code>)	só a org , mesmo que o <code>Board</code> tenha <code>sector</code> (setor do modelo ou da Demanda): o setor não entra no endereço
outro model com <code>organization_id</code>	só a org
qualquer outra coisa	<code>unresolved</code> → nega
<code>None</code>	vazio: só escopos <code>ORGANIZACAO</code> (e relacionais) casam

Para **criar** algo que ainda não existe, os serviços passam `ResourceContext.for_new(organization=... , sector=... , ...)` .

3. Como `AuthorizationService.can()` decide

```
AuthorizationService.can(user, catalog.TAREFA_INICIAR, task)
```

1. Superusuário ativo → **permitido** (`is_platform_admin`). É o operador da plataforma; ainda precisa de organização no `Profile` para usar as telas.
2. Usuário anônimo, inativo ou sem organização → **negado**.
3. Monta o endereço com `ResourceContext.of(resource)` .
4. **Teto de tenant**: recurso de outra organização → **negado** (doc 05 §43).
5. Busca atribuições ativas (`UserProfile`) cujo perfil ativo contém a ação ativa, e concessões diretas ativas (`UserAction`) para a ação.
6. Mantém as que têm escopo ativo que **cobre** o endereço (`scope_contains`).
7. Sobrou alguma → **permitido**.

8. Nenhuma sobrou: só para `demanda.visualizar` sobre uma `Activity` da própria organização vale a **relação** (`_demand_relation_allows`): o dono, o criador ou o responsável de uma `Task` não cancelada da demanda pode **ler** a própria Demanda, sem concessão. A regra é só de leitura; nenhuma outra ação usa relação.

Outras funções úteis de `AuthorizationService` :

Função	Uso
<code>require(user, key, resource)</code>	Levanta <code>AuthorizationError</code> com mensagem legível.
<code>grants_for(user, key, resource)</code>	Lista de <code>Grant</code> com a origem de cada permissão.
<code>explain(user, key, resource)</code>	Motivos em texto ("Perfil: Gestor de Setor · Escopo: Setor Compras").
<code>effective_actions(user)</code>	Todas as ações da pessoa com origem — usado nas telas de segurança.
<code>can_many(user, [(ação, recurso), ...])</code>	Várias verificações com uma consulta de perfis/concessões por chamada; devolve <code>{(ação, pk): bool}</code> . Usada nas listas e quadros para evitar uma consulta por linha (<code>can_many</code> aplica a mesma relação de leitura da Demanda).
<code>can_anywhere(user, key)</code>	Permitido em algum lugar (org ou algum setor)? Usado para menus e listas.
<code>accessible_sector_ids(user, key)</code>	Setores em que a ação vale.

Os escopos de setor são memoizados na instância do usuário; qualquer mudança em `UserSector` invalida o cache via signal (`invalidate_sector_cache`).

4. Onde a verificação acontece

Camada	Mecanismo
Views de <code>core</code> (cadastros, usuários, segurança)	<code>ActionRequiredMixin</code> com <code>required_action = catalog.X</code> . Deve vir depois de <code>OrganizationRequiredMixin</code> nas bases. Nega com <code>PermissionDenied</code> . Escopo padrão: organização (<code>get_scope_object()</code> retorna <code>None</code>).
Serviços de <code>activities</code>	<code>require_action(user, catalog.X, recurso)</code> no início de cada método; converte a negação em <code>ActivityError</code> . Os de Demanda (<code>ActivityService</code>) estão em uso; os de Tarefa , fila e prazo (<code>TaskService</code> , <code>QueueService</code> , <code>DeadlineService</code>) continuam no código e na autorização, mas as telas que os chamavam respondem 410 (seção 4.4).
Ficha, lista e anexos da Demanda	<code>ActivityDetailView</code> exige <code>DemandBoardAccess.can_view_activity</code> (participante do quadro ou <code>demanda.visualizar</code>); o download de anexo exige <code>demanda.visualizar</code> e a visibilidade da mensagem; a lista <code>/demandas/</code> filtra as linhas com <code>can_many(demanda.visualizar)</code> .
Quadros (<code>boards</code>)	<code>boards.services._require</code> (e <code>BoardAPIView.require</code>): ação <code>quadro.*</code> sobre o quadro; para quadro de Demanda (<code>Board.Kind.DEMAND</code>) a regra relacional de <code>DemandBoardAccess</code> (seção 4.3).
Central de tarefas (<code>/tarefas/</code> , <code>TaskCenterQuery</code>)	Sem ação própria: mostra os itens de quadro em que a pessoa é a Pessoa responsável; ver "outras pessoas/todas" exige <code>quadro.visualizar</code> ou <code>demanda.visualizar_todas</code> na organização; os quadros de outras pessoas passam por <code>can_many(demanda.visualizar)</code> .
Serviços de <code>processes</code>	<code>_require(...)</code> com endereço org + empresa; levanta <code>ProcessError</code> . Inalcançáveis pela interface : <code>/processos/*</code> responde 410 (<code>RetiredProcessView</code>).
Aplicar processo (<code>ProcessApplicationService.apply</code>)	<code>require_action(user, PROCESSO_APLICAR, atividade)</code> — o recurso é a atividade . Desativado : <code>/demandas/<pk>/processo/aplicar/</code> e as rotas de inputs e critérios respondem 410. Ver a seção 4.1.

Camada	Mecanismo
Inputs e critérios do processo aplicado (<code>ActivityProcessService</code>)	Sem ação nova: <code>ActivityProcessService.can_update</code> aceita quem pode <code>demanda.editar</code> na atividade, o dono e quem é responsável/participante de alguma tarefa dela, sempre dentro da organização e nunca em atividade encerrada. Hoje sem rota ativa.
Caixa de Entrada (<code>IntakeService</code>) — inativa por padrão (<code>INTAKE_ENABLED=false</code> : toda rota <code>/entrada/</code> dá 404)	<code>entrada.registrar</code> e <code>entrada.visualizar</code> na porta da tela usam <code>can_anywhere</code> (<code>AnywhereActionMixin</code> , em <code>intake/views.py</code>): registrar não tem setor ainda, e <code>ActionRequiredMixin</code> só olharia o escopo de organização e negaria quem cuida de um setor. Itens: <code>entrada.triar</code> / <code>entrada.visualizar</code> sobre o próprio <code>IntakeItem</code> (outro <code>tenant = 404</code> , sem a ação = 403). A lista é filtrada por <code>IntakeService.visible_queryset</code> . Ver a seção 4.2.
Menu lateral	<code>acessos.context_processors.navigation</code> — só UX , não protege nada.

Algumas ações do catálogo não são verificadas em ponto nenhum, e outras só são verificadas em código que a interface já não alcança — a lista completa está na seção 8 e as pendências em [13_PENDENCIAS_CONHECIDAS.md](#).

4.1 processo.aplicar × tarefa.criar

Desativado em 02/10/2026. Aplicar processo, os inputs e critérios do processo aplicado e todo o CRUD de processos respondem 410; o texto abaixo descreve o desenho que continua no código (`ProcessApplicationService`) e nos testes, útil caso o recurso volte.

Aplicar um processo cria tarefas em **vários setores**, e quem aplica (o `Colaborador` sugerido, por exemplo) normalmente não tem `tarefa.criar` em todos eles. O desenho é:

1. `ProcessApplicationService.apply` exige **processo.aplicar** na atividade (e que a pessoa seja da mesma organização dela);
2. as tarefas são criadas por `TaskService._create_task_core` , o mesmo núcleo que `create_task` usa **depois** de exigir `tarefa.criar` . O núcleo **não autoriza** — por isso é privado, só `create_task` e `apply` o chamam e nenhuma view o chama —, mas valida organização, setor, título e responsável como qualquer criação;
3. a auditoria registra as tarefas como criadas por quem aplicou o processo.

Ou seja, `processo.aplicar` autoriza materializar **o fluxo de uma versão já publicada** (o molde é imutável e foi aprovado por quem tem `processo.publicar`); não autoriza criar tarefas avulsas, e `tarefa.criar` continua sendo exigido para elas. As Regras não dizem se aplicar processo dispensa o escopo de setor de `tarefa.criar` ; a decisão segue [Regras/05 §2057](#) (usar o fluxo padrão não exige poder editar o fluxo) e está registrada em [Regras/12 §41](#).

4.2 Caixa de Entrada: entrada.* × demanda.criar

Inativa desde 01/10/2026 (`INTAKE_ENABLED` , padrão `false`): o item "Entrada" some do menu e `/entrada/...` dá 404. As regras abaixo valem quando ela for religada (`INTAKE_ENABLED=true`).

Ação	Permite
<code>entrada.visualizar</code>	Ver a Caixa de Entrada e o texto das solicitações do escopo.
<code>entrada.registrar</code>	Colocar uma solicitação na Caixa de Entrada (e-mail, Teams, pedido verbal).
<code>entrada.triar</code>	Corrigir as sugestões, criar a demanda , ignorar e restaurar.

- **Criar demanda exige também** `demanda.criar` no contexto final (setor, obra e organização escolhidos). Quem garante é o `ActivityService` (`save_draft` + `publish_draft`), que a `IntakeService.convert` chama; o `intake` não repete a regra. Quem só tem `entrada.triar` recebe a recusa e a solicitação continua nova, sem atividade nem efeito pela metade (tudo roda numa transação).

- **Visibilidade:** escopo de organização enxerga tudo, inclusive solicitações **sem setor sugerido** (a triagem geral). Escopo de setor enxerga só as do seu setor. Uma solicitação sem setor é invisível para o gestor de setor até alguém com escopo de organização atribuí-la a um.
- **Editar sem perder o acesso:** ao corrigir o setor sugerido, a `IntakeService` reverifica `entrada.triar` no setor novo; quem gerencia só um setor não consegue mandar o item para um setor que não enxerga (erro claro, em vez de o item sumir da lista).
- **Limitação conhecida:** a lista usa `accessible_sector_ids`, que só entende escopos de organização e de setor (inclusive `MEUS_SETORES` e `SETORES_GERENCIADOS`). Escopos de empresa, obra e centro de custo não ampliam o que se vê na Caixa de Entrada.
- **Perfis sugeridos:** Colaborador recebe `entrada.registrar`; Gestor de Setor recebe as três; Administrador, todas.

4.3 Quadros dinâmicos: `quadro.*`

Ação	O que permite	Sensível
<code>quadro.visualizar</code>	Ver a lista e o conteúdo dos quadros da organização, o histórico e a busca/ordem/filtro.	não
<code>quadro.criar</code>	Criar quadro (em branco ou de um modelo).	não
<code>quadro.editar</code>	Renomear o quadro, gerir grupos (criar, renomear, cor, mover, excluir vazio) e gerir visualizações (criar, renomear, configurar e excluir Kanbans e Calendários).	não
<code>quadro.excluir</code>	Excluir o quadro inteiro.	sim
<code>quadro.gerir_colunas</code>	Criar, renomear, redimensionar, reordenar, configurar, ocultar, duplicar, converter e excluir colunas; gerir etiquetas.	não
<code>quadro.criar_item</code>	Adicionar item a um grupo.	não
<code>quadro.editar_item</code>	Renomear item, mover (de grupo e de posição) e preencher células.	não
<code>quadro.excluir_item</code>	Excluir item.	não

- **Escopo só de organização.** O endereço do quadro não inclui setor (mesmo que o `Board` tenha `sector`), então `ResourceContext.of` devolve só a organização e as ações valem para a organização inteira; um escopo de setor concedido a `quadro.*` **não** casa (a lista dá 403 e o menu não mostra "Quadros", que usa `can` com `ResourceContext.for_new(org)`, a mesma pergunta da lista). Quem quiser quadros por setor precisa de uma decisão nova (doc 13, F31).
- **Quadro de Demanda (`Board.Kind.DEMAND`): regra relacional.** Cada Demanda publicada tem um quadro próprio, e `boards.demand_services.DemandBoardAccess` decide o acesso **antes** do motor de escopos (que continua valendo como alternativa): *participa* quem é dono ou criador da Demanda ou aparece em uma célula **Pessoa** de um item ativo do quadro. Participante vê o quadro (`quadro.visualizar`) e colabora (`quadro.criar_item`, `quadro.editar_item` : criar item, editar célula, mover). Mexer na **estrutura** (`quadro.editar`, `quadro.gerir_colunas`, `quadro.excluir` : colunas, grupos, visualizações) é só do dono, do criador ou de quem tem `quadro.gerir_colunas` na organização; participante comum não vira administrador de colunas. `quadro.excluir_item` num quadro de Demanda não tem regra relacional: segue o escopo de organização. Quadros-modelo (`Board.Kind.TEMPLATE`) seguem só o escopo de organização.
- **Visualizações são compartilhadas:** quem vê o quadro vê todas as abas e usa qualquer Kanban (arrastar cartão, preencher campo e criar item seguem `criar_item` / `editar_item`); mexer na **configuração** da visualização (agrupar por, ordem, campos do cartão, soma, nome, criar e excluir aba) é `quadro.editar`, e mexer nas **etiquetas das raia**s é `quadro.gerir_colunas`. Não há visualização pessoal (doc 13, F31).
- **A permissão é checada no serviço**, a cada ação, e de novo a cada gravação de célula. Esconder botão (a tela recebe `permissions` calculado uma vez por `can_many`) é só experiência de uso. Endpoints que só **leem** (`.../fragmento/`, a prévia de conversão de tipo, `.../configuracoes/` sem campos) também exigem a ação (`BoardAPIView.require`): sem isso eles vazariam o desenho e a contagem de valores.
- **Outra organização recebe 404** em tudo (página e API): o quadro é carregado sempre por `organization=<a da pessoa>`. Um objeto de **outro quadro da mesma organização** usado como âncora (coluna, grupo) também dá 404.

- **Perfis sugeridos** (`SUGGESTED_PROFILES`): Colaborador recebe `visualizar`, `criar_item`, `editar_item`; Gestor de Setor soma `criar`, `editar`, `gerir_colunas` e `excluir_item`; Administrador, todas. Em bases que já existem, a migração `acessos/0005` concede por mapeamento (ver doc 04 §7.2) e `seed_acoes` cria as ações; marque o que faltar em `/permissoes/`.

4.4 O que mudou com a desativação de fila, processos e tarefas (02/10/2026)

As tarefas passaram a viver em **quadros por Demanda** (`boards`): a "tarefa" é um item do quadro da Demanda, com a pessoa em uma célula **Pessoa** e o andamento numa coluna de **Status**. A migração `boards/0008` cancelou as `Task` operacionais (ficam só como histórico). Efeito sobre a autorização:

Rota	Resposta	Consequência
<code>/fila/...</code> , <code>/processos/...</code> , <code>/prazos/...</code> , <code>/conflitos/...</code> , <code>/checklist/...</code> , <code>/tarefas/<pk>/...</code> (todas as ações, painel, tempo, prazo, devolução...), <code>/demandas/<pk>/processo/...</code> , <code>/demandas/<pk>/tarefas/rapida/</code> , <code>/tarefas/lista-legada/</code> , <code>/tarefas/nova-rapida/</code>	410 (<code>RetiredFeatureView</code> / <code>RetiredProcessView</code>)	As ações de tarefa, fila, prazo e processo não têm mais ponto de entrada na interface.
<code>/tarefas/</code> , <code>/tarefas/kanban/</code> , <code>/tarefas/calendario/</code>	<code>TaskCenterView</code>	Central de tarefas sobre itens de quadro : o recorte é "onde sou a Pessoa responsável"; sem ação própria (ver seção 4).
<code>/demandas/</code> , <code>/demandas/kanban/</code> , <code>/demandas/calendario/</code>	<code>DemandWorkBoardView</code>	Lista/Kanban/Calendário de Demandas; linhas filtradas por <code>demanda.visualizar</code> .

- Os **serviços** (`TaskService`, `QueueService`, `DeadlineService`, `ProcessApplicationService`, `ProcessService`) continuam exigindo suas ações; só não há tela que os chame (restam as rotas auxiliares `kanban/tarefas/<pk>/etapa|condicao/`, que operam sobre as `Task` arquivadas, e a criação de `TaskService.create_task` com `board=`, que usa `DemandBoardAccess` em vez de `tarefa.criar`).
- **Quem trabalha numa tarefa** não precisa de nenhuma ação `tarefa.*`: basta participar do quadro da Demanda (acima). Por isso conceder `tarefa.*` ou `fila.*` a um perfil hoje não muda nada para o usuário.
- A **visibilidade** da Demanda deixou de ser só pela organização: ficha, anexos e lista exigem `demanda.visualizar` (ou a relação de leitura do dono, criador e responsável).
- **Grupos de acesso por tela** (commit `be4b9ee`) foram revertidos em `6fb95b9`: não existem no código; o modelo é só perfil + escopo.

5. Administração pela interface

Tela	URL	Ação exigida
Matriz de permissões (perfis × ações)	<code>/permissoes/</code>	<code>seguranca.gerir_perfis</code>
Salvar ações de um perfil	<code>/permissoes/<pk>/salvar/</code>	<code>seguranca.gerir_autorizacoes</code>
Criar/editar perfil	<code>/perfis/novo/</code> , <code>/perfis/<pk>/</code>	<code>seguranca.gerir_perfis</code>
Acessos de uma pessoa (perfis e concessões, com origem)	<code>/usuarios/<pk>/acessos/</code>	<code>seguranca.gerir_autorizacoes</code>
Concessão direta	<code>/usuarios/<pk>/concessoes/</code>	<code>seguranca.gerir_autorizacoes</code>

Todas as alterações passam por `AccessService` e são auditadas (`PROFILE_CREATED`, `PROFILE_ASSIGNED`, `ACTION_GRANTED` etc., com `target_user`). `AccessService.set_profile_actions` só altera as ações que estavam **visíveis** na tela, para um filtro na matriz não apagar o resto.

6. Perfis sugeridos

Criados por `seed_lps_demo` a partir de `SUGGESTED_PROFILES`. São ponto de partida — a organização pode renomear e alterar (doc 05 §11).

Perfil	Resumo
Colaborador	Demandas: ver, criar, assumir, marcar pendente, mover estágio, definir etapa e status. Tarefas (<code> tarefa.*</code> , hoje sem tela): ver, assumir, aceitar, recusar, iniciar, pausar, retomar, concluir, devolver, bloquear. Ver a própria posição na fila; propor/aceitar/recusar prazo; conversar; ver e aplicar processos; gerir clientes; registrar solicitações na Caixa de Entrada (<code> entrada.registrar</code>). Quadros: ver, criar item, editar item.
Gestor de Setor	Demandas: ver, ver todas, criar, editar, assumir, marcar pendente, aprovar pendência. Não recebe mover estágio, definir etapa/status, alterar dono, concluir, cancelar nem reabrir demanda (ficam com o Administrador ou com concessão direta). Tarefas: as do Colaborador (menos mover estágio de demanda) + criar, editar, atribuir, mover de setor, mover estágio, definir etapa e status, reabrir (<code> tarefa.reabrir</code>), lançar tempo manual; não recebe cancelar nem alterar responsável. Fila completa e reordenar; propor/aceitar/recusar/alterar prazo solicitado e resolver conflito; métricas e auditoria; todas as ações de processo; clientes, estágios de tarefa, etapas, status, marcadores, cores de status e de prioridade; ver, registrar e triar a Caixa de Entrada (<code> entrada.*</code>). Quadros: ver, criar, editar, gerir colunas, criar/editar/excluir item (não excluir quadro).
Administrador	Todas as ações do catálogo (84). Setores, empresas, obras, centros de custo, motivos de devolução, usuários e segurança só ele recebe por perfil.

Ação nova em organização já implantada: vale para qualquer ação nova (também `quadro.*` , `etapa.gerir` , `condicao.gerir` , `entrada.*`). `seed_acoes` cria a ação no catálogo, mas **não** a acrescenta a perfis que já existem no banco — nem ao Administrador. Depois de rodar `seed_acoes` , marque `tarefa.reabrir` nos perfis que devem ter (Matriz de permissões, `/permissoes/`). Rodar `seed_lps_demo` de novo também a acrescentaria, mas ele re-adiciona **todas** as ações sugeridas (desfazendo remoções feitas de propósito) e recria setores de demonstração que faltarem — não use em produção. Reabrir uma tarefa de atividade já concluída também exige `demanda.reabrir` (hoje só no Administrador). A reabertura de tarefa saiu da interface junto com as rotas `/tarefas/<pk>/` (seção 4.4); a de demanda continua. Ver [06_ATIVIDADES_E_TAREFAS.md](#) §2.11.

7. Receita: adicionar uma ação nova

- Em `acessos/catalog.py` , crie a constante (`MINHA_ACAO = "grupo.minha_acao"`) e adicione a tupla (chave, nome, descrição, sensível) ao grupo certo em `GROUPS` . Se fizer sentido, inclua em `SUGGESTED_PROFILES` .
- Use a constante no código:
 - view de `core` : `required_action = catalog.MINHA_ACAO` ;
 - serviço: `require_action(user, catalog.MINHA_ACAO, recurso)` .
- Se o recurso for um model novo, ensine `ResourceContext.of` a endereçá-lo — senão ele só terá o escopo da organização (ou será negado, se não tiver `organization_id`).
- Rode `python manage.py seed_acoes` (local e no deploy) para gravar a ação.
- Teste com os helpers de `acessos/testing.py` (`grant_action(user, catalog.MINHA_ACAO, organization=org)`).

Para **remover** uma ação: tire do catálogo e rode `seed_acoes` ; ela fica `is_active=False` e deixa de conceder, mas o histórico é preservado.

8. Catálogo completo de ações (84) e onde são verificadas

Fonte: `acessos/catalog.py` (`GROUPS`). A coluna "Verificação hoje" diz onde o código **realmente** consulta a ação a partir do que a interface ainda alcança (seção 4.4). "Sem tela" = a verificação existe no serviço, mas a rota está desativada (410). Chaves gravadas: `demanda.*` (renomeadas de `atividade.*` pela migração `acessos/0003`).

Grupo (nº)	Ações	Verificação hoje
Demandas (14)	<code>demanda.visualizar</code>	Ficha, anexos, lista/Kanban/Calendário de Demandas, quadro da Demanda e Central de tarefas, mais a relação de leitura (dono, criador, responsável).
	<code>demanda.visualizar_todas</code>	Aba "Todas" e recortes de <code>filtered_activities_queryset</code> , Kanban legado, "outras pessoas/todas" na Central de tarefas.
	<code>demanda.criar</code> , <code>.editar</code> , <code>.alterar_dono</code>	<code>ActivityService</code> (<code>save_draft</code> / <code>publish_draft</code> / <code>create_activity</code> , <code>update_activity</code> , <code>change_owner</code>), editor de 4 etapas, edição inline e campos de quadro de domínio.
	<code>demanda.assumir</code> , <code>.marcar_pendente</code> , <code>.aprovar_pendencia</code> , <code>.concluir</code> , <code>.cancelar</code> , <code>.reabrir</code>	<code>ActivityService</code> (<code>claim</code> , <code>mark_pending</code> , <code>approve_pendency</code> , <code>finalize</code> / <code>complete_activity</code> , <code>cancel_activity</code> , <code>reopen_activity</code>). <code>resolve_pendency</code> também exige <code>marcar_pendente</code> , mas não tem rota (doc 13).
	<code>demanda.mover_estagio</code> , <code>.definir_etapa</code> , <code>.definir_condicao</code>	<code>set_stage</code> aceita <code>definir_etapa</code> ou a legada <code>mover_estagio</code> ; <code>set_condition</code> exige <code>definir_condicao</code> . Kanban de Demandas e edição inline.
Tarefas (21)	<code> tarefa.visualizar</code> , <code>.criar</code> , <code>.editar</code> , <code>.atribuir</code> , <code>.alterar_responsavel</code> , <code>.assumir</code> , <code>.aceitar</code> , <code>.recusar</code> , <code>.iniciar</code> , <code>.pausar</code> , <code>.retomar</code> , <code>.devolver</code> , <code>.concluir</code> , <code>.cancelar</code> , <code>.reabrir</code> , <code>.bloquear</code> , <code>.mover_setor</code> , <code>.mover_estagio</code> , <code>.definir_etapa</code> , <code>.definir_condicao</code> , <code>tempo.lancar_manual</code>	Sem tela (<code>TaskService</code> ; telas <code>/tarefas/<pk>/...</code> em 410). <code>tarefa.visualizar</code> só aparece na ramificação de tarefas do quadro de domínio, sem rota. Única via viva: <code>kanban/tarefas/<pk>/etapa condicao/ (definir_etapa / definir_condicao)</code> , sobre <code>Task</code> arquivada.
Filas (3)	<code>fila.visualizar_posicao_propria</code>	Nenhuma verificação em ponto algum.
	<code>fila.visualizar_completa</code> , <code>fila.reordenar</code>	Sem tela (<code>QueueView</code> , <code>QueueReorderView</code> , <code>QueueService.reorder</code> ; <code>/fila/</code> em 410).
Prazos (5)	<code>prazo.propor</code> , <code>.aceitar</code> , <code>.recusar</code> , <code>escalonamento.resolver</code>	Sem tela (<code>DeadlineService</code> ; <code>/prazos/...</code> , <code>/conflitos/...</code> em 410).
	<code>prazo.alterar_solicitado</code>	<code>DomainBoardMutationService.set_value</code> (campo "prazo solicitado" de quadro de domínio).
Comunicação (1)	<code>comunicacao.participar</code>	Mensagens da Demanda (<code>MessageService.post_activity_message</code>) e menções.
Cadastros (14)	<code>setor.editar</code> , <code>setor.inativar</code>	Telas de Setores (<code>core/views.py</code>), criar setor ao trocar o setor de uma Demanda; <code>setor.inativar</code> na ativação/inativação.
	<code>setor.criar</code>	Nenhuma (o cadastro usa <code>setor.editar</code>).
	<code>empresa.gerir</code> , <code>obra.gerir</code> , <code>centro_custo.gerir</code> , <code>cliente.gerir</code> , <code>motivo_devolucao.gerir</code> , <code>tag.gerir</code> , <code>cor_status.gerir</code> , <code>cor_prioridade.gerir</code>	Telas de Cadastros e de Prioridades (<code>core/views.py</code>); <code>cliente.gerir</code> / <code>empresa.gerir</code> / <code>obra.gerir</code> / <code>centro_custo.gerir</code> também mostram "criar" nos seletores da Demanda.
	<code>etapa.gerir</code> , <code>condicao.gerir</code>	"Etapas e status" (<code>core/views.py</code>), criar/editar opção no pop-over da lista e do Kanban.
	<code>estagio_tarefa.gerir</code>	Só menu: decide se "Cadastros" aparece; a tela de estágios de tarefa usa <code>etapa.gerir</code> .

Grupo (nº)	Ações	Verificação hoje
Processos (7)	<code>processo.criar</code> , <code>.editar_rascunho</code> , <code>.publicar</code> , <code>.criar-versao</code> , <code>.inativar</code> , <code>.aplicar</code>	Sem tela (<code>ProcessService</code> , <code>ProcessApplicationService</code> ; <code>/processos/...</code> e <code>.../processo/aplicar/</code> em 410).
	<code>processo.visualizar</code>	Só o menu.
Segurança (6)	<code>usuario.visualizar</code> , <code>usuario.editar</code> , <code>seguranca.gerir_perfis</code> , <code>seguranca.gerir_autorizacoes</code>	<code>ActionRequiredMixin</code> nas telas de <code>core</code> (seção 5); <code>usuario.editar</code> também protege <code>/usuarios/novo/</code> .
	<code>usuario.criar</code>	Só decide se o botão "criar pessoa" aparece nos seletores da Demanda.
	<code>usuario.inativar</code>	Nenhuma.
Auditoria (2)	<code>auditoria.visualizar</code>	Nenhuma: o Histórico (<code>/historico/</code>) só é limitado pela organização.
	<code>metricas.visualizar</code>	Menu "Visão do gestor" e <code>ManagementView</code> (<code>can_anywhere</code>).
Caixa de entrada (3)	<code>entrada.visualizar</code> , <code>.registrar</code> , <code>.triar</code>	<code>IntakeService</code> <code>/views</code> (seção 4.2), inativa por padrão (<code>INTAKE_ENABLED</code>).
Quadros (8)	<code>quadro.visualizar</code> , <code>.criar</code> , <code>.editar</code> , <code>.excluir</code> , <code>.gerir_colunas</code> , <code>.criar_item</code> , <code>.editar_item</code> , <code>.excluir_item</code>	<code>boards.services</code> e <code>boards/views.py</code> ; quadros de Demanda passam por <code>DemandBoardAccess</code> (seção 4.3).

06 — Atividades (Demandas) e tarefas

O núcleo operacional da LPS: as regras estão em `activities/` e os quadros em `boards/`. Uma **atividade** — **Demanda** na interface e no endereço (`/demandas/...` ; o código segue `Activity` e as rotas `activity-*`) — é um resultado a alcançar e tem exatamente um dono. Desde 02/10/2026 as **tarefas** de uma Demanda são itens do **Quadro da Demanda** (§2.0); o modelo operacional antigo — `Task` com fila, cronômetro, prazo negociado e processos — foi **desativado** (§0). Na interface, "Situação" virou **Etapa** e "Condição" virou **Status**. Atualizado em 03/10/2026, sobre o commit `add5acd`. As regras de produto estão em `Regras/02_ATIVIDADES_TAREFAS_E_FLUXOS.md` e `Regras/03_FILAS_PRAZOS_E_ESCALONAMENTO.md` (parte delas descreve o que foi desativado).

A regra de Demanda está em `activities/services.py` (`ActivityService` , `ActivityAttachmentService` , `MessageService`); o quadro de cada Demanda, em `boards/demand_services.py` (`BoardInstantiationService` , `DemandBoardAccess`). `TaskService` , `QueueService` e `DeadlineService` continuam no código, mas sem rota que os alcance. Cada método verifica a ação do catálogo, valida a transição, grava, audita e notifica.

0. O que está ativo e o que foi desativado em 02/10/2026

Em 02/10/2026 (commits `74fff88` a `add5acd` , migração `boards/0008`) a LPS centralizou o trabalho das Demandas em **quadros**. Rotas desativadas respondem **410 Gone** (texto simples) e seguem declaradas para links e favoritos antigos (lista completa em `09_ROTAS.md`).

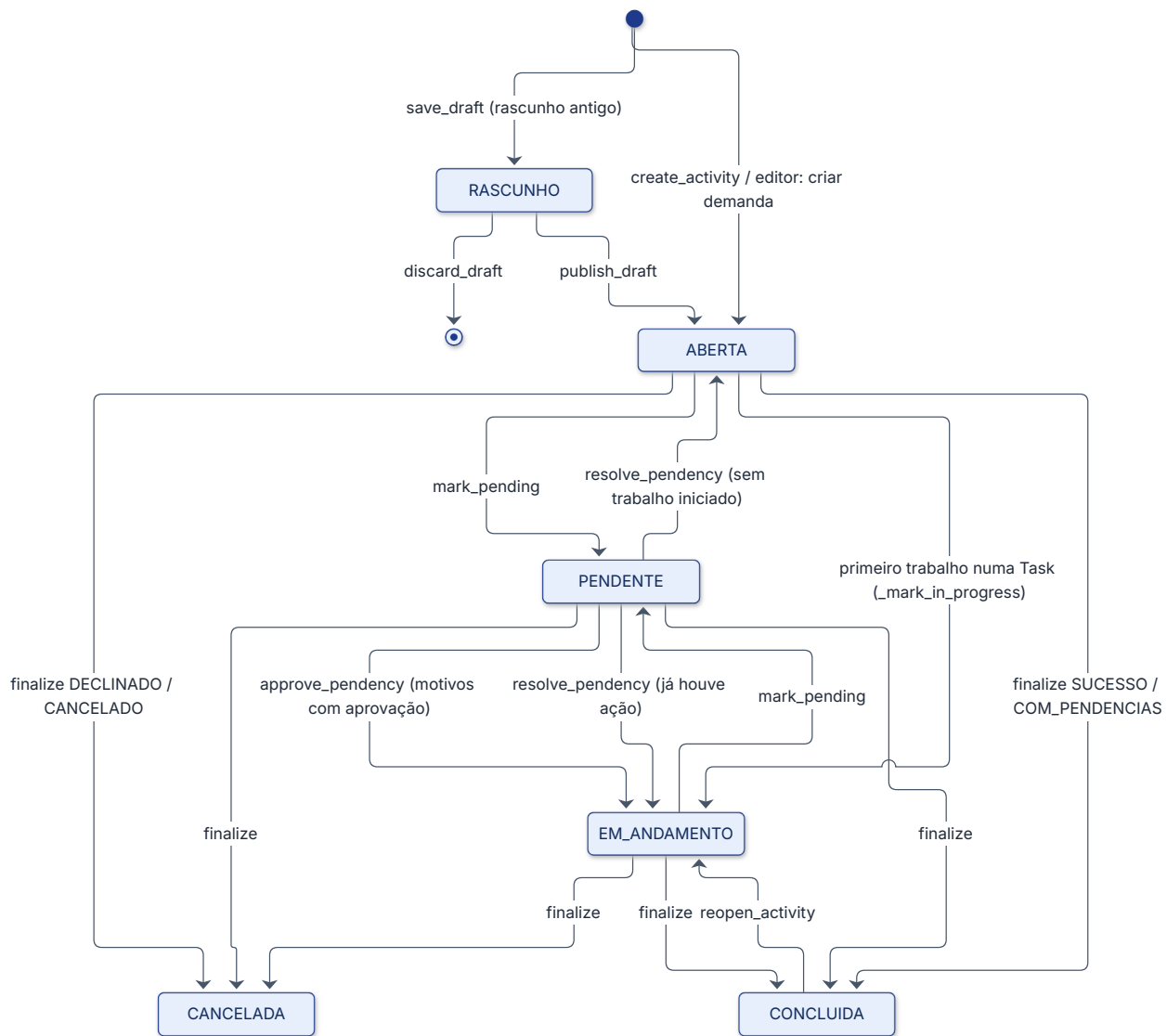
Capacidade	Estado	Onde
Criar, editar, finalizar, cancelar e reabrir Demanda; dono, prazo, pendência, mensagens, anexos	Ativo	§1
Lista, Quadro por etapas (Kanban), Calendário de prazos e edição inline das Demandas	Ativo	§1.6, §1.7
Quadro da Demanda (tarefas como itens, com visões Tabela, Kanban e Calendário)	Ativo	§2.0; 09_ROTAS.md §6.2
Painel Tarefas (<code>/tarefas/</code> , <code>/tarefas/kanban/</code> , <code>/tarefas/calendario/</code>), só leitura	Ativo	§2.0
Quadros modelo (<code>/quadros/</code>)	Ativo	§2.0
Kanban por setor (<code>/demandas/kanban-legado/</code> , ações <code>kanban/...</code>) e lista anterior (<code>/demandas/lista-legada/</code>)	Ativo, por compatibilidade	§1.6
<code>/atividades/...</code>	Redireciona (301/308) para <code>/demandas/...</code>	§1.2
Tarefa operacional (<code>Task</code>): ações, ficha, editor, dependência, "Já realizei este trabalho", tempo, checklist, participantes e atribuições, mensagens	Desativado — <code>/tarefas/<pk>/...</code> responde 410	§2.1–2.11
Fila do setor (<code>/fila/*</code>) e reordenação	Desativado (410)	§2.6
Prazo comprometido, propostas e conflitos (<code>tarefas/<pk>/prazo/propor/</code> , <code>/prazos/*</code> , <code>/conflitos/*</code>)	Desativado (410)	§2.7
Cronômetro e sessões de trabalho (iniciar, pausar, retomar)	Desativado (rotas em 410)	§2.4
Criar tarefa rápida (<code>demandas/<pk>/tarefas/rapida/</code> , <code>tarefas/nova-rapida/</code>)	Desativado (410)	§2.2

Capacidade	Estado	Onde
Processos (/processos/*) e "Aplicar processo" (demandas/<pk>/processo/*)	Desativado (410); dados preservados	§3
Caixa de Entrada (/entrada/)	Inativa por padrão (INTAKE_ENABLED): 404	§4

A migração `boards/0008-centralize_demand_board_items` converteu cada item de quadro ligado a uma `Task` (nome e responsável viraram valores do próprio item), **cancelou** toda `Task` ainda aberta (`CANCELADA` , auditoria `CANCEL` com o motivo "Tarefa operacional arquivada na centralização do quadro."), encerrou as sessões de trabalho e as passagens de fila abertas (renumerando as filas) e removeu `BoardItem.task` e `BoardColumn.binding` . As `Task` já concluídas ou canceladas continuam como histórico. A Demanda em si não mudou de estado.

1. Demanda (atividade)

1.1 Estados



- `ABERTA` → `EM_ANDAMENTO` só ocorre quando o trabalho começa numa `Task` (`start` , `log_manual_time` e `register_completed_work` chamam `ActivityService._mark_in_progress` , que audita `UPDATE` do status). Como as `Task` operacionais foram desativadas (§0), nenhuma rota dispara essa transição hoje: `EM_ANDAMENTO` é alcançado pela reabertura, pela aprovação de pendência e pela resolução de pendência simples.

- `BLOQUEADA` existe no enum, mas nenhum serviço coloca uma atividade nesse estado.
- `resolve_pendency` (ação `demanda.marcar_pendente` ; audita `PENDENCY_RESOLVED`) encerra a pendência que só avisava (`MATERIAL` , `INFORMACOES_CLIENTE`) e volta a `EM_ANDAMENTO` se já houve a primeira ação, ou a `ABERTA` . O serviço e a política existem, mas **nenhuma view o chama**: pela interface essas pendências só saem de `PENDENTE` pela finalização.
- Concluir exige que não haja `Task` aberta; os **itens do Quadro da Demanda não são verificados** (§1.4).

Os pontos acima estão em [13_PENDENCIAS_CONHECIDAS.md](#).

1.2 Editor único de Demandas

`activities/activity_editor.py` reúne criação, retomada de rascunho e edição. Todas usam `ActivityEditorForm` e `activities/activity_form.html` : **a mesma janela, em quatro etapas** (Informações principais, Cliente e obra, Quadro de tarefas, Descrição e arquivos), tanto para “Nova demanda” quanto para “Editar demanda”. Ao editar, a etapa do quadro mostra o quadro atual e deixa trocá-lo com confirmação (exclui as tarefas do quadro atual: ver `docs/13_F36`). A **Etapa** e o **Status** da Demanda (rótulos novos de “Situação” e “Condição”) são escolhidos na etapa 1.

Etapa	Campos (formulário → modelo)
1. Informações principais	Nome da demanda (<code>title</code> , *), Atribuído a (<code>owner</code> , *), Setor responsável (<code>sector</code> , *), Etapa (<code>stage</code>) e Status (<code>condition</code>) — as opções do setor escolhido; sem escolha valem os padrões do setor —, Prazo de vencimento (<code>requested_deadline</code> : data + hora opcional), Urgência (<code>urgency</code> , pílulas Baixa/Média/Alta), Organização (<code>company</code>)
2. Cliente e obra	Cliente (<code>client</code>), Obra (<code>site</code>), Centro de custo (<code>cost_center</code>), Solicitante (Externo) (<code>external_requester</code> , texto), Endereço complementar (<code>address</code>)
3. Quadro de tarefas	Quadro de tarefas (<code>board_setup_mode</code> : Começar em branco ou Usar quadro existente) e Modelo de quadro (<code>board_template</code> , obrigatório no segundo caso: um quadro modelo ativo da organização, de qualquer setor)
4. Descrição e arquivos	Observações (<code>description</code> , editor de texto) e Link / caminho dos arquivos (<code>files_location</code> , texto)

*Obrigatórios ao criar e ao salvar uma edição (a etapa 1 é validada antes de avançar e de novo no servidor, em `ActivityEditorForm.clean`). Sem escolha, o quadro nasce em branco.

- **Sem upload**: a etapa 4 só aponta onde os arquivos estão (link do Drive/OneDrive, caminho de rede ou de pasta). `files_location` aceita qualquer texto até 500 caracteres; na ficha da atividade só vira link clicável se começar por `http://` ou `https://` . Os anexos já enviados antes continuam na ficha e podem ser removidos pelo editor (lista “Anexos enviados antes”); o envio de anexos pela conversa da atividade não mudou.
- **Fora do formulário, de propósito**: marcadores, solicitante interno (`requested_by`), anotações internas (`internal_notes`) e o envio de arquivos. Os campos seguem no modelo e na ficha, mas, por não estarem no `ModelForm` , salvar uma edição **nunca os altera** nem os apaga.
- **Cliente → Obra → Centro de custo**: a busca de obras aceita `?client=<id>` e a de centros de custo `?site=<id>` ; ao mudar o campo de cima, o de baixo é zerado (`data-filter-field` , em `person-picker.js`). O servidor recusa obra de outro cliente e centro de custo de outra obra; obra ou centro de custo **sem vínculo** valem para qualquer um. Hoje só os centros de custo sem obra aparecem junto dos da obra escolhida; obras sem cliente somem da busca quando há cliente escolhido.
- **Organização** é o cadastro de empresas (`company`) — a mesma origem de dados de antes; só o rótulo mudou.
- O contador “0/2000” do editor de observações é só orientação: o servidor não corta o texto.
- Nada é gravado antes do botão final (`Criar demanda` / `Salvar alterações`): as quatro etapas são painéis do mesmo formulário e o backend de criação continua sendo `ActivityService.save_draft` + `publish_draft` (edição: `update_activity`); o quadro é criado em `publish_draft` (§2.0).

Rotas e respostas:

- `/demandas/nova/` : publica em uma única submissão, exigindo nome, responsável e setor. Aberta pelos botões (`data-activity-action` `data-activity-navigate`), responde JSON: sucesso `{redirect_url}` (a tela segue para a ficha), erro `400 {errors}` (a janela abre na etapa do primeiro erro). Sem JavaScript é uma página com as quatro etapas empilhadas.

- Rascunho antigo (`acao=rascunho` , sem botão na tela): aceita tudo vazio; `?pk=<id>` retoma apenas um rascunho da organização criado pela pessoa atual.
- `/demandas/<pk>/editar/` : mesmo formulário; exige `demanda.editar` (403 já ao abrir). `?passo=N` abre direto na etapa N (o clique em Cliente / Obra da lista abre na 2). Etapa e Status gravam pelos serviços próprios (`demanda.definir_etapa` ou `demanda.mover_estagio` ; `demanda.definir_condicao`); trocar o **setor** redefine Etapa e Status para o padrão do novo setor, exige `demanda.editar` também no setor de destino e é recusado enquanto houver pendência com aprovação aberta. A troca de responsável exige também `demanda.alterar_dono` e passa pelo serviço de transferência, com histórico. Sem essa ação, o responsável é somente leitura.
- `/demandas/nova-rapida/` : mesma janela dentro do seletor de atividades de uma tarefa; retorna `{id, name, summary}` em Ajax. O envio antigo só com título continua válido (sem exigir responsável nem setor).
- **Quadro de tarefas (etapa 3)**: ao criar, escolhe o quadro que a Demanda terá (§2.0). Ao editar, mostra o quadro **real** da Demanda e só deixa trocá-lo a quem gere a estrutura dele (dono, criador ou `quadro.gerir_colunas`); para as demais pessoas os campos ficam somente leitura. Trocar por outra escolha exige `confirm_board_replace` (sem ele: 400 {errors: {board_setup_mode}} e nada é gravado) e **exclui definitivamente** as tarefas, colunas, grupos e visões do quadro atual (`BoardInstantiationService.replace_for_activity` , na mesma transação da edição; audita `BOARD_UPDATED` com `items_deleted`). Pedido sem o campo do quadro nunca vira "em branco": o quadro fica como está.
- Auditoria: `update_activity` registra cada campo alterado (inclusive `external_requester` e `files_location`).

Criação usa `save_draft` e `publish_draft` na mesma transação. A publicação mantém as verificações de permissão, auditoria, notificações e menções existentes. `discard_draft` continua disponível somente para o criador.

Links antigos das etapas 2/3 redirecionam ao editor. Seus POSTs permanecem compatíveis para formulários já abertos antes da atualização.

O código da Demanda é `DEM-AAAA-NNNN` (antes `ATV-...` ; a busca ainda aceita o prefixo antigo). Os endereços `/atividades/<resto>` redirecionam (301; **308** em POST) para `/demandas/<resto>` , com a consulta, sem exigir login, e `/atividade-arquivos/...` responde 410.

Lista, Quadro e Calendário abrem a ficha completa da atividade. A antiga rota de painel lateral redireciona à ficha. O parâmetro `next` , validado em `navigation.py` , preserva o retorno à visualização e aos filtros de origem; URLs externas não são aceitas. Alterar prazo, transferir responsável, concluir, cancelar, pendência e reabertura usam janelas sobre a tela atual, com alternativa de página sem JavaScript.

1.3 Dono, edição e pendência

Operação	Serviço	Ação	Efeitos
Trocar dono	<code>change_owner</code>	<code>demanda.alterar_dono</code>	<code>OwnerChangeLog</code> , audita <code>OWNER_CHANGED</code> , notifica antigo e novo dono
Assumir do grupo	<code>claim</code>	<code>demanda.assumir</code>	atividade precisa ter setor designado
Editar	<code>update_activity</code>	<code>demanda.editar</code>	audita <code>UPDATE</code> por campo; bloqueado se concluída/cancelada; nova descrição reprocessa menções; trocar o setor redefine Etapa e Status do novo setor
Definir etapa	<code>set_stage</code>	<code>demanda.definir_etapa</code> (ou <code>demanda.mover_estagio</code>)	só etapa ativa do setor da demanda ; recusada em demanda concluída/cancelada; grava <code>stage_changed_at</code> e audita <code>UPDATE</code> "etapa"
Definir status	<code>set_condition</code>	<code>demanda.definir_condicao</code>	só status ativo do setor; recusado em demanda concluída/cancelada; audita <code>UPDATE</code> "status". Não muda o <code>status</code> operacional da Demanda
Marcar pendente	<code>mark_pending</code>	<code>demanda.marcar_pendente</code>	de <code>ABERTA</code> / <code>EM_ANDAMENTO</code> ; motivo e comentário obrigatórios; audita <code>PENDENCY_OPENED</code> e gera mensagem

Operação	Serviço	Ação	Efeitos
Aprovar pendência	approve_pendency	demanda.aprovar_pendencia	pendência APROVADA, atividade EM_ANDAMENTO, dono volta ao anterior, audita PENDING_APPROVED, notifica ACTIVITY_APPROVED

Pendência **com aprovação** (APROVACAO_GESTOR, AJUSTES_REVISOES): exige prazo de decisão e setor designado com gestor. O primeiro gestor do setor vira dono temporário (com OwnerChangeLog); todos os gestores recebem ACTIVITY_APPROVAL_NEEDED por notificação **e e-mail**; o dono anterior recebe ACTIVITY_PENDING .

Pendência **sem aprovação**: o dono recebe ACTIVITY_PENDING . Em INFORMACOES_CLIENTE com "avisar cliente", o cliente recebe e-mail (Client.email).

A pendência **sem aprovação** só tem a rota de abertura (demandas/<pk>/pendente/); o serviço resolve_pendency não está ligado a nenhuma tela (§1.1).

1.4 Finalização e reabertura

finalize(outcome, comment) é o popup único de encerramento; comentário obrigatório.

Resultado	Ação	Estado final	Pré-condição
SUCESSO, CONCLUIDO_COM_PENDENCIAS	demanda.concluir	CONCLUIDA	nenhuma Task aberta (as operacionais foram arquivadas em 02/10; os itens do Quadro da Demanda não impedem a conclusão)
DECLINADO, CANCELADO	demanda.cancelar	CANCELADA (comentário vira cancelled_reason)	—

Se a atividade tem processo aplicado, SUCESSO também exige os critérios de aceite obrigatórios atendidos (§3.4; processos foram desativados em 02/10, então vale só para demandas que já tinham processo aplicado).

Pendências abertas viram ENCERRADA; audita COMPLETE / CANCEL; notifica o dono. reopen_activity (demanda.reabrir) leva CONCLUIDA → EM_ANDAMENTO com motivo obrigatório (REOPEN, com o estado anterior em old_value). Só atividade **concluída** reabre; cancelada não (o serviço recusa e a tela não oferece). O botão "Reabrir demanda" aparece no aviso "Concluída em..." da ficha (e no menu : e no menu da lista, aba "concluídas") para quem tem demanda.reabrir — hoje só o perfil Administrador. Os caminhos antigos complete_activity / cancel_activity ainda existem (concluir/ e cancelar/ ; cancelar/ por GET redireciona ao popup de finalização).

1.5 Anexos, mensagens e menções

- **Anexos** (ActivityAttachmentService): adicionar exige demanda.editar (ou ser o criador do próprio rascunho); remover exige demanda.editar e apaga o arquivo. Auditado como UPDATE no campo "anexo". Local de gravação em 03_CONFIGURACAO.md. O download só sai de demandas/<pk>/anexos/<attachment_pk>/download/, que confere organização, demanda.visualizar e a visibilidade da mensagem a que o anexo pertence; não há rota pública de arquivos (o endereço antigo atividade-arquivos/... responde 410).
- **Mensagens** (MessageService.post_activity_message): exige comunicacao.participar. Notifica dono, criador, participantes e responsáveis das tarefas (menos o autor e os mencionados) com MESSAGE_POSTED. Participantes e responsáveis são os das Task históricas; os itens do quadro não entram nesta lista.
- **Tipos, visibilidade e respostas**: cada atualização tem um tipo (Mensagem, Decisão registrada, Risco, Compromisso, Impedimento — só DECISAO ganha destaque) e uma visibilidade (Todas as pessoas com acesso, Somente participantes, Somente meu setor — esta exige setor definido). A regra vale na leitura, não só no seletor. Uma resposta (parent) herda a visibilidade da mensagem de origem e não a amplia; demandas/<pk>/continuar/ envia comentário e/ou anexo num só envio.
- **Reações**: 👍 🙌 🎉 ✅ 👁, alternadas por pessoa (comunicacao.participar), só em mensagem que a pessoa pode ver.
- **@menções** (find_mentioned_users): @username em descrições, mensagens e comentários de finalização, pendência, bloqueio, devolução, recusa e conflito. Só notifica (MENTIONED) quem tem comunicacao.participar no recurso. Não há menção a setor.

1.6 Lista, Quadro por etapas e Calendário

`/demandas/` (Lista), `/demandas/kanban/` (**Quadro por etapas**) e `/demandas/calendario/` (**Calendário de prazos**) são a **mesma tela operacional**, `DemandWorkBoardView` (`boards/work_views.py`), montada sobre o **quadro de domínio** das Demandas (`DomainBoard`, um por organização, criado sob demanda por `ensure_domain_board`). Ele é uma configuração de *campos* e *visões* que **lê e grava o próprio Activity**; não usa `BoardItem` (isso é o Quadro da Demanda, §2.0).

- **Campos padrão:** Título, Responsável (dono), Setor, Prioridade (`urgency`), Prazo (`requested_deadline`), Etapa, Status (`condition`) e Tarefas. Quem tem `quadro.gerir_colunas` acrescenta campos próprios (Texto, Número, Moeda, Data e hora, Lista, Confirmação), guardados em `DomainCustomValue` e auditados como `quadro:<chave>`.
- **Visões padrão:** *Quadro principal* (a lista, agrupada por etapa), *Kanban* (`group_by=stage`, raias vazias visíveis — o fluxo aparece completo) e *Calendário* (por prazo). `?visao=` / `?view=` escolhe a visão. Com `quadro.gerir_colunas`: **+ Campo**, **Configurar cartões**, agrupamento (etapa, status, setor, responsável, prioridade), `show_empty`, nomes dos campos e largura/visibilidade das colunas.
- **Abas de escopo** (`?tab=`): *Sob minha responsabilidade* (`minhas`, padrão), *Do meu setor* (`grupo`), *Em que participo* (`participando`), *Concluídas* e, só com `demanda.visualizar_todas`, *Todas as demandas*. Rascunhos nunca aparecem; concluídas e canceladas só na aba *Concluídas* ou filtrando.
- **Filtros:** busca `q` (título, cliente, obra e código — aceita o prefixo antigo `ATV-`), setor, pessoa, Etapa, Status, cliente, obra, marcador, prazo (atrasadas, hoje, 7 dias, 30 dias, sem prazo), atalhos (*Atrasadas*, *Vencem hoje*) e ordenação (prazo, mais recentes, título). Os mesmos parâmetros valem nas três visões e os links entre elas os preservam. Valor inválido é ignorado.
- **O que cada gesto grava:** editar uma célula ou soltar o cartão grava **só** aquele campo (`workboard-value` → `DomainBoardMutationService.set_value` → `ActivityService`): título e prazo por `update_activity` (o prazo exige também `prazo.alterar_solicitado`), responsável por `change_owner`, setor por `update_activity`, Etapa por `set_stage`, Status por `set_condition`, prioridade por `update_activity`. Nenhum mexe no `status` operacional. A tela só deixa arrastar entre raias a quem tem a ação do campo agrupado (`demanda.mover_estagio`, `demanda.definir_condicao`, `demanda.editar` para setor e prioridade, `demanda.alterar_dono`); o serviço confere de novo. A tela manda a versão lida (`updated_at`): se a Demanda mudou desde então, a resposta é **409** e a célula é recarregada.
- **Calendário de prazos:** posiciona cada Demanda no dia do prazo solicitado.
- Lista, Kanban e Calendário abrem a ficha completa (`?next=` preserva a visão e os filtros).

Kanban por setor (anterior) — `/demandas/kanban-legado/` (`activities/kanban.py`), mantido por compatibilidade junto com as ações `kanban/...` e a gaveta `demandas/<pk>/gaveta/`: sempre de **um setor** por vez (`?setor=`); colunas = etapas ativas do setor (`ActivityStage`), mais "Sem etapa" quando sobra demanda sem etapa; quem participa do setor (ou tem `demanda.visualizar_todas` nele) vê todas as demandas, as demais só as em que atuam; arrastar grava **só** a etapa (`ActivityService.set_stage`) e escolher o Status grava **só** o status (`set_condition`); "+" na coluna cria a Demanda (setor do quadro, etapa da coluna, a pessoa como dona); o limite da coluna (`ActivityStage.column_limit`, `etapa.gerir`) é só informativo; a gaveta mostra dados, Etapa e Status, anexos, conversa e últimas atualizações. A versão para tarefas (`TaskBoard`) responde 410.

1.7 Edição inline da Lista

Na Lista de Demandas, título, responsável, setor, Etapa, Status e prazo editam no lugar (`demandas/<pk>/inline/`, só JSON; opções dos pop-overs em `.../inline/opcoes/`). É um adapter fino sobre o `ActivityService`: toda regra, autorização, auditoria e notificação vive no serviço (`update_activity`, `change_owner`, `set_stage`, `set_condition`), e a troca de setor confere a autorização também no setor de destino. Quem tem `etapa.gerir` / `condicao.gerir` no setor da demanda cria e edita Etapas e Status ali mesmo. O contrato de resposta e os códigos (403, 400, 404) estão em [09_ROTAS.md](#) §4.

2. Tarefa

2.0 Tarefas hoje: itens do Quadro da Demanda (ativo)

Desde 02/10/2026 uma tarefa de Demanda é um **item** (`BoardItem`) **do Quadro da Demanda**, não uma linha de `Task`.

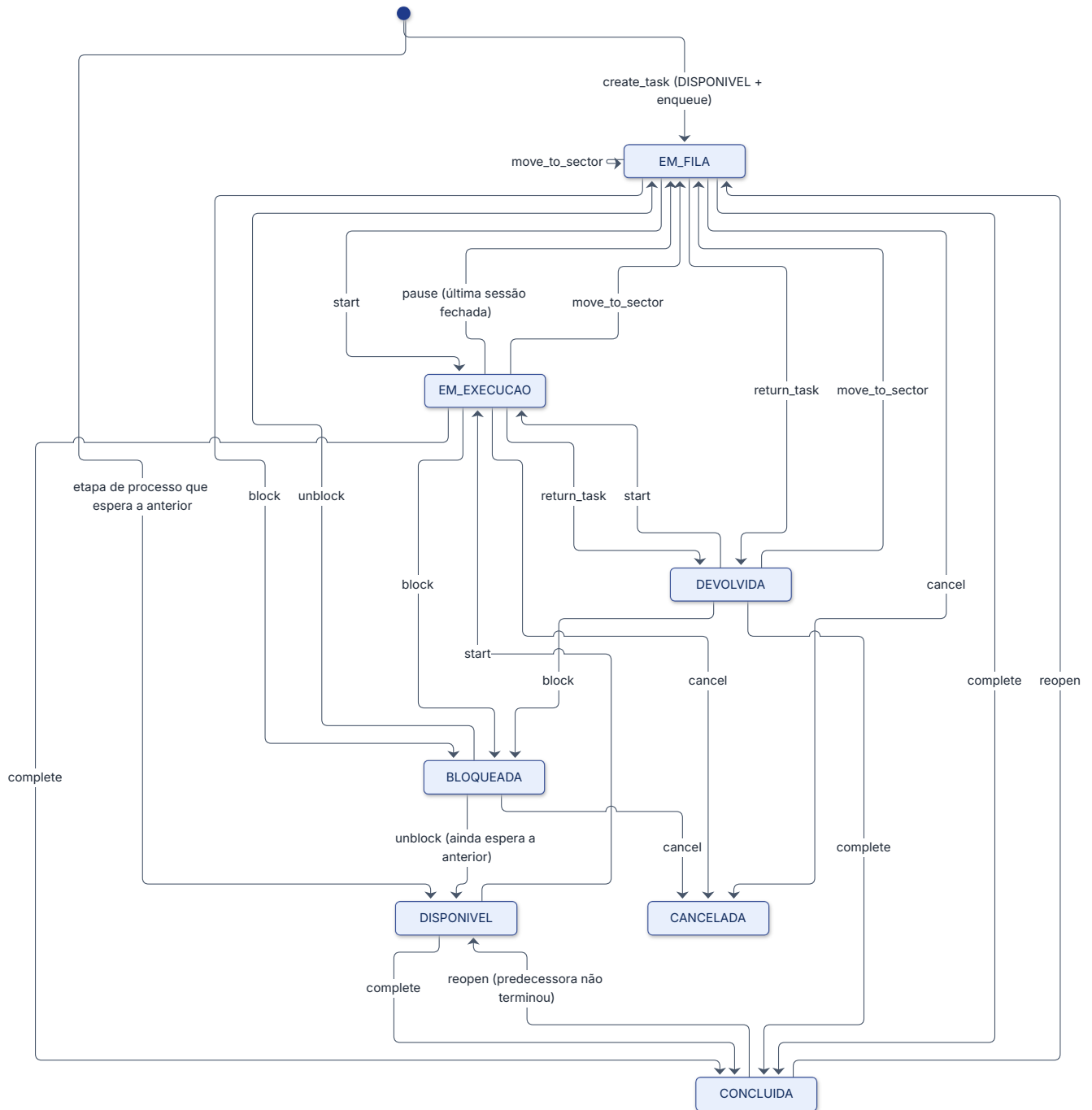
- **O quadro:** `Board` com `kind = DEMAND`, um por Demanda (`Activity.task_board`, relação 1:1), criado na mesma transação em que a Demanda é publicada — `publish_draft` (editor), `create_activity` (criação rápida, sempre em

branco) e a conversão da Caixa de Entrada — por `BoardInstantiationService.create_for_activity`. Se a Demanda já tem quadro, nada é recriado.

- **Em branco:** um grupo padrão e as colunas **Responsável** (Pessoa), **Status** (etiquetas *Não iniciado* — padrão —, *Em andamento* e *Concluído* — marca de conclusão) e **Data**, mais as visões Kanban e Calendário (a Tabela é a visão base). **Usando um quadro existente:** cópia **independente** de um quadro modelo (`kind = TEMPLATE` , de qualquer setor): grupos, colunas, etiquetas, visões e itens; `Board.source_template` guarda a origem, sem vínculo posterior. A estrutura mínima é recomposta se o modelo não a tiver.
- **Colunas:** Texto, Número, Moeda, Data, Pessoa, Status, Lista suspensa e Sinal de confirmação (os demais tipos estão reservados). A "tarefa" que o Painel Tarefas mostra é o item, com a célula **Pessoa** como responsável, o **Status** como situação e a coluna **Data** como prazo.
- **O que não existe mais para a tarefa:** fila, cronômetro e sessões, prazo solicitado × comprometido negociado, dependência, bloqueio, devolução, checklist, participantes e atribuições, processo e "Já realizei este trabalho". Tudo isso era da `Task` (§2.1–2.11) e agora, se for preciso, é uma coluna do quadro. O módulo `boards` não envia notificações: mexer num item não avisa ninguém (as menções da Demanda continuam valendo, §1.5).
- **Acesso** (`DemandBoardAccess`): vê o quadro quem participa da Demanda (dono, criador ou pessoa em célula Pessoa de um item ativo), quem tem `demanda.visualizar` ou `quadro.visualizar` ; cria e edita itens quem participa ou tem `quadro.editar_item` ; muda a estrutura (colunas, visões, etiquetas, grupos) o dono, o criador ou quem tem `quadro.gerir_colunas` . O quadro de uma Demanda não pode ser excluído pela tela. Detalhes das rotas em [09_ROTAS.md](#) §6.2.
- **Na ficha da Demanda**, o bloco **Tarefas** é uma prévia **somente leitura** do quadro real (total, concluídas, em andamento e menções novas, mais as primeiras linhas) com o botão **Abrir quadro**; criar, atribuir e organizar é no quadro (`/quadros/<id>/`).
- **Trocar o quadro** só no editor da Demanda (§1.2), com confirmação, pois exclui as tarefas do quadro atual. A Demanda antiga sem quadro ganha o dela ao ser editada.
- **Painel Tarefas** (`/tarefas/` , `/tarefas/kanban/` , `/tarefas/calendario/` ; `boards/task_center.py` , só leitura): o que está nos Quadros de Demanda em que a pessoa é **responsável** (célula Pessoa); Demandas em rascunho ou canceladas não aparecem. O estado vem do Status do item: etiqueta de conclusão = *Concluídas*; etiqueta padrão ou sem Status = *A fazer*; qualquer outra = *Em andamento*; "atrasada" é um selo (prazo anterior a hoje e não concluída). Lista (ordenada por prazo), Kanban de três colunas e Calendário semanal; filtros por busca, status, prazo e pessoa (a pessoa só se tem `quadro.visualizar` ou `demanda.visualizar_todas`). O endereço antigo `/tarefas/?demanda=<id>` redireciona ao quadro da Demanda.
- **Quadros modelo** (`/quadros/`): quadros de `kind = TEMPLATE` criados com `quadro.criar` (em branco ou a partir de um modelo inicial, sempre num setor). Servem de molde para o passo 3 do editor da Demanda. Visões Tabela, Kanban e Calendário; ver [09_ROTAS.md](#) §6.2.
- **A conclusão da Demanda não olha o quadro:** itens abertos não impedem finalizar (§1.4).

Task operacional — desativada em 02/10/2026. O restante desta seção (2.1 a 2.11) descreve o comportamento do código de serviço (`TaskService` , `QueueService` , `DeadlineService` , `WorkTimeService`), que continua no repositório e nos testes, mas **nenhuma rota o alcança** (410; ver [09_ROTAS.md](#) §4). Serve de referência histórica e para entender os dados antigos (`Task` , `QueueEntry` , `WorkSession` , `DeadlineProposal` ...).

2.1 Estados (Task , desativada)



O diagrama mostra os caminhos usuais; em código (`TaskTransitionPolicy`), `block` aceita qualquer estado exceto já bloqueada, concluída ou cancelada, `cancel` qualquer estado exceto concluída ou cancelada, e `move_to_sector` qualquer estado não terminal.

- Uma tarefa recém-criada termina em `EM_FILA` : `create_task` grava `DISPONIVEL` e `QueueService.enqueue` troca para `EM_FILA` . **Exceção:** a etapa de um processo que depende da anterior fica `DISPONIVEL` e fora da fila até a predecessora ser concluída (§3.2); só então `QueueService.enqueue` a leva a `EM_FILA` .
- `start` e `complete` recusam a tarefa cuja predecessora (`depends_on`) não está `CONCLUIDA` , e a tarefa gerada por processo enquanto houver input obrigatório não recebido (§3.3).
- `return_task` grava `DEVOLVIDA` e chama `move_to_sector(keep_status=True)` , que entra na fila do setor que recebeu **mantendo** `DEVOLVIDA` (o status diz a verdade até alguém agir); a tarefa sai de `DEVOLVIDA` ao ser iniciada, concluída, bloqueada, cancelada ou movida de novo. Uma etapa que ainda espera a anterior e nunca entrou na fila não pode ser devolvida.
- `NAO_INICIADA` é o default do model, mas nenhum fluxo usa.

- A `Task` ganhou **prioridade** (`priority` : Baixa/Média/Alta, `set_priority`) e Etapa/Status do setor (`set_stage` , `set_condition` ; ações `tarefa.definir_etapa` , `tarefa.definir_condicao`); `move_to_sector` redefine os dois para o padrão do novo setor.

2.2 Criação

`TaskService.create_task` — usado pela criação rápida dentro da atividade (`demandas/<pk>/tarefas/rapida/`) e pela criação avulsa com seletor de atividade (`tarefas/nova-rapida/`); **as duas rotas respondem 410** e nada mais cria `Task` .

- Exige `tarefa.criar` no endereço do **setor da tarefa**.
- Exige um `responsavel` da mesma organização (setor idem).
- Participantes iniciais entram por `add_executor` .
- Audita `TASK_CREATED` , notifica membros e gestores do setor (`TASK_ASSIGNED`) e processa menções.
- No título rápido, `TaskService.parse_quick_title` entende `#tag` (acha ou cria a tag) e `@username` (define o responsável se casar com exatamente uma pessoa da organização).

2.3 Responsável x participantes

- **Responsável:** um único usuário (`Task.responsavel`), responde pela conclusão.
- **Participantes:** `TaskExecutor` , podem ser vários; nunca incluem o responsável.

Operação	Serviço	Ação	Efeito
Assumir (entrar como participante)	<code>add_executor</code> em si mesmo	<code>tarefa.assumir</code>	participante imediato, <code>EXECUTOR_ADDED</code>
Atribuir a outra pessoa	<code>add_executor</code>	<code>tarefa.atribuir</code>	cria <code>TaskAssignment</code> <code>PENDENTE</code> , notifica <code>TASK_ASSIGNMENT_PENDING</code>
Aceitar atribuição	<code>accept_assignment</code>	<code>tarefa.aceitar</code>	vira participante (<code>ASSIGNMENT_ACCEPTED</code> + <code>EXECUTOR_ADDED</code>)
Recusar atribuição	<code>reject_assignment</code>	<code>tarefa.recusar</code>	motivo (<code>ReturnReason</code>) obrigatório; notifica quem atribuiu
Remover participante	<code>remove_executor</code>	<code>tarefa.assumir</code> (a si) / <code>tarefa.atribuir</code> (outro)	remoção lógica (<code>removed_at</code>)
Trocar responsável	<code>change_responsavel</code>	<code>tarefa.alterar_responsavel</code>	<code>TaskResponsavelChangeLog</code> , <code>RESPONSABLE_CHANGED</code> , notifica

2.4 Execução e tempo

- `start` (`tarefa.iniciar`): só responsável ou participante; abre uma `WorkSession` da pessoa (se ainda não houver) e vai para `EM_EXECUCAO` . Marca `first_action_at` na tarefa e na atividade e leva a atividade de `ABERTA` a `EM_ANDAMENTO` (`_mark_in_progress` , auditado). Uma pessoa **pode** ter sessões abertas em várias tarefas ao mesmo tempo (Regras 04 §115, revisada).
- `pause` (`tarefa.pausar`): fecha a sessão da pessoa; se não restar nenhuma sessão aberta, volta a `EM_FILA` .
- `resume` (`tarefa.retomar`): delega para `start` .
- `log_manual_time` (`tempo.lancar_manual` , “Adicionar tempo trabalhado”): cria `WorkSession` com `is_manual=True` e `logged_at` = agora; sem datas futuras ou invertidas. **A tela pede o motivo** (os mesmos de “Já realizei este trabalho”) e o comentário, com as mesmas regras (*Outro* e mais de 7 dias exigem comentário); em código `reason` continua opcional para não invalidar chamadores antigos. Só acrescenta tempo — não conclui a tarefa (para isso, ver §2.10.1).
- O cronômetro da barra superior lista as sessões abertas do usuário (context processor `my_active_sessions`).

2.5 Bloqueio, mudança de setor e devolução

Operação	Serviço	Ação	Efeito
Bloquear	<code>block</code>	<code> tarefa.bloquear </code>	motivo obrigatório; <code> TaskBlock </code> ; <code> BLOQUEADA </code> ; notifica dono (<code> TASK_BLOCKED </code>)
Desbloquear	<code>unlock</code>	<code> tarefa.bloquear </code>	fecha o bloqueio; <code> EM_FILA </code> ; <code> TASK_UNBLOCKED </code>
Enviar a outro setor	<code>move_to_sector</code>	<code> tarefa.mover_setor </code>	encerra as sessões abertas, sai da fila antiga (renumera), redefine Etapa e Status para o padrão do novo setor, <code> SectorTransfer </code> , entra na fila nova; notifica o novo setor
Devolver	<code>return_task</code>	<code> tarefa.devolver </code>	motivo (<code> ReturnReason </code>) obrigatório; <code> TaskReturn </code> ; <code> DEVOLVIDA </code> , depois <code> move_to_sector(keep_status=True) </code> (fica <code> DEVOLVIDA </code> na fila do setor que recebeu); notifica os dois setores e o dono

2.6 Fila

Desativada em 02/10/2026: `/fila/*` responde 410; `QueueService` e `QueueEntry` ficam como histórico.

- Cada setor tem uma fila; a posição é sempre mostrada como "N de M".
- `QueueService.enqueue` põe no fim; `renumber` fecha buracos na conclusão, cancelamento e saída de setor (motivo `AUTOMATICA_CONCLUSAO`).
- `QueueService.reorder ( fila.reordenar no setor )`: a tarefa movida recebe `MANUAL` , as deslocadas `AUTOMATICA_ENTRADA` ; cada mudança é auditada (`QUEUE_POSITION_CHANGED`) e o dono da atividade é notificado.
- `/fila/` (hoje 410) mostrava a fila completa só com `fila.visualizar_completa` no setor; sem ela, só as entradas em que a pessoa é dona, responsável ou participante.

2.7 Prazo

Desativado em 02/10/2026: `tarefas/<pk>/prazo/propor/` , `/prazos/*` e `/conflitos/*` respondem 410. O **prazo solicitado da Demanda** continua editável (`demanda.editar` ; na Lista/Quadro, também `prazo.alterar_solicitado`).

- **Solicitado** (`requested_deadline`): quem pede. **Comprometido** (`committed_deadline`): o que o executor assumiu.
- `DeadlineService.propose ( prazo.propor )` cria `DeadlineProposal` `PENDENTE` e notifica o dono.
- Só o **dono da atividade** aceita (`prazo.aceitar` , grava o prazo comprometido) ou recusa (`prazo.recusar`).
- Recusar abre um `DeadlineConflict` e notifica setor e dono (`DEADLINE_CONFLICT`). `resolve_conflict` exige `escalonamento.resolver` . O motor de escalonamento automático é D1 (Regras 10 §57) — hoje só registra e notifica.

2.8 Checklist

Desativado em 02/10/2026: as rotas de checklist respondem 410.

`TaskCheckListItem` , editado via Ajax (`static/js/checklist.js`). Adicionar e remover exigem `tarefa.editar` ; marcar/desmarcar também é permitido ao responsável ou participante. Guarda `done_by` / `done_at` . Não é auditado.

2.9 Atraso

`python manage.py check_overdue_tasks` → `TaskService.mark_overdue_tasks()` :

1. seleciona tarefas com `committed_deadline` no passado, `overdue_notified_at` nulo e status `DISPONIVEL` , `EM_FILA` , `EM_EXECUCAO` ou `BLOQUEADA` ;
2. notifica membros e gestores do setor, dono da atividade e responsável (`TASK_OVERDUE`) e envia e-mail (`notifications/email/task_overdue.txt`);
3. grava `overdue_notified_at` — cada tarefa é avisada uma única vez.

Precisa ser agendado externamente (não há Celery/worker). Desde 02/10/2026 não há mais `Task` aberta a avisar: a migração `boards/0008` cancelou as existentes e nenhuma rota cria novas.

2.10 Kanban de tarefas

Mesma estrutura do quadro de demandas (`TaskBoard` em `activities/kanban.py` ; hoje `/tarefas/kanban/` é o Painei Tarefas, §2.0, e `tarefas/kanban-legado/` responde 410): um setor por vez, colunas = `TaskStage` ativas do setor, arrastar grava só a etapa (`TaskService.set_stage` , exige `tarefa.definir_etapa`) e a condição grava só a condição (`tarefa.definir_condicao`). Diferenças:

- **Quem vê o quê:** quem participa do setor (ou tem `tarefa.visualizar` nele) vê todas as tarefas do setor; as demais pessoas, só as tarefas de que são responsáveis ou participantes. Concluídas e canceladas só com *Incluir concluídas e canceladas*.
- **Cartão:** título, "Demanda: ...", responsável, prazo comprometido (ou o solicitado), Status, "Bloqueada" quando for o caso.
- **Menu "...":** além de abrir, mover para etapa e copiar: **Concluir tarefa** (`task-complete-ajax`), **Mover para outro setor**, **Bloquear / Resolver bloqueio** e **Cancelar** — cada um só aparece se a pessoa pode e o estado permite; o servidor valida de novo (dependência, inputs do processo etc.) e a recusa aparece no aviso.
- **Criar:** uma tarefa precisa de uma demanda, então o "+" da coluna abre a janela *Nova tarefa* já com setor e etapa (`?sector=&stage=`), em vez do campo de nome na coluna.
- A gaveta da tarefa (`task-drawer`) ganhou os seletores de **Etapas** e **Status**, iguais aos do quadro; *Filtro* ganhou "Participante".

2.10.1 “Já realizei este trabalho”

Desativado em 02/10/2026: `tarefas/<pk>/ja-realizei/` responde 410. O que segue vale para o serviço e para os dados antigos.

`TaskService.register_completed_work(task, user, started_at, ended_at, reason, note="")` — para a tarefa que foi feita mas **ninguém iniciou**. Registra o período trabalhado e **conclui a tarefa agora**.

Princípio: a pessoa informa *quando trabalhou*; o sistema registra *quando soube*. Nunca se reescreve o passado operacional da fila.

Verdade	Onde fica	Exemplo (registrado às 14:00)
Quando o trabalho aconteceu	<code>WorkSession.started_at/ended_at</code> (informado)	09:00 → 10:30
Quando o sistema soube	<code>Task.completed_at</code> , <code>QueueEntry.left_at</code> , liberação da sucessora, <code>first_action_at</code> e <code>WorkSession.logged_at</code> = agora	14:00

Por isso `completed_at` **não** recebe o “terminei às” informado: não existe o histórico impossível “concluída às 10h, mas na fila até as 14h, com a sucessora liberada às 14h”.

Aspecto	Regra
Quem pode	responsável ou participante com <code>tarefa.concluir</code> (o Colaborador já tem). Não exige <code>tempo.lancar_manual</code> , que continua valendo só para “Adicionar tempo trabalhado”
Status	só <code>EM_FILA</code> ou <code>DISPONIVEL</code> (ainda não iniciada). <code>EM_EXECUCAO</code> → “use Concluir tarefa”; bloqueada, devolvida, concluída e cancelada são recusadas
Travas de <code>complete</code>	mesmas: etapa anterior concluída (<code>_assert_dependency_satisfied</code>) e inputs obrigatórios do processo recebidos (<code>_assert_process_inputs_ready</code>)

Aspecto	Regra
Período	fim depois do início; nada no futuro; o fim não pode ser anterior à criação da tarefa. Sobreposição com outras sessões da pessoa é permitida (Regras 04 §115)
Motivo	obrigatório: Esqueci de iniciar (padrão), Trabalhei fora da LPS , Ajuste do período , Outro
Comentário	até 255 caracteres; obrigatório para Outro e quando o início foi há mais de <code>RETROACTIVE_JUSTIFICATION_DAYS</code> (7 , constante em <code>activities/services.py</code>) dias
Governança	sem aprovação: mesmo dia é livre; dia anterior é permitido e destacado na ficha ("Dia anterior"); período antigo exige justificativa
Auditoria	<code>RETROACTIVE_LOGGED</code> ("Trabalho informado depois"): <code>new_value</code> = período, <code>reason</code> = "Informado em ... · motivo · comentário"; depois o <code>COMPLETE</code> normal
Atomicidade	sessão, primeira ação, auditoria e conclusão na mesma transação; se a conclusão falhar, nada fica para trás

A conclusão é `TaskService.complete`, sem alteração: saída da fila e renumeração, `TASK_COMPLETED` e `_release_dependents` acontecem no instante do registro.

Na ficha, o cartão **Tempo registrado** separa **Cronometrado** (`is_manual` falso) de **Informado pela pessoa** (`is_manual` verdadeiro) — o total da equipe continua sendo a soma — e mostra, em cada período informado, quem informou, quando, o motivo e o comentário. "Adicionar tempo trabalhado" (`TaskService.log_manual_time`, antes "Registrar tempo já trabalhado") só acrescenta tempo, não conclui a tarefa e também grava `logged_at`.

Ações da tarefa (ficha): barra com **Editar tarefa** à vista e o menu **Mais ações** — Devolver para correção, Enviar para outro setor, **Gerenciar dependência**, Registrar bloqueio, Propor novo prazo, Adicionar tempo trabalhado e, separado e em vermelho, Cancelar tarefa. Cada item só aparece para quem tem a ação (`can_return`, `can_move`, `can_manage_dependency`, `can_block`, `can_propose`, `can_log_time`, `can_cancel_task`); se nada é permitido a barra some. **Todas abrem em janela** sobre a ficha (`data-activity-action` + `LPSModal`) e, sem JavaScript, continuam sendo páginas: `TaskFormActionView` usa `ActivityActionResponseMixin` (sucesso → `{redirect_url}`, erro de campo ou de serviço → 400 com `{errors}`).

Editor da tarefa (`task-edit`)

Janela única no padrão do editor de atividade: **título, instruções, marcadores, prazo pedido pelo solicitante, responsável e participantes**, salvos por `TaskService.edit_task` numa transação só — se qualquer parte for recusada, nada é gravado. A ação `tarefa.editar` é exigida já ao **abrir** (403 no GET).

Parte	Ação exigida	Observação
Título, instruções, prazo pedido, marcadores	<code>tarefa.editar</code>	Cada mudança é auditada (<code>UPDATE</code> , com antes e depois — inclusive <code>tags</code>)
Responsável	<code>tarefa.alterar_responsavel</code>	Sem a ação, o campo sai do formulário e aparece como informação; quem vira responsável deixa de ser participante
Participantes	<code>tarefa.atribuir</code> (ou <code>tarefa.assumir</code> , para si)	Adicionar outra pessoa cria um convite (<code>TaskAssignment</code> pendente): aparece como "aguardando aceite" no editor e na ficha e só vira participante quando ela aceita. Adicionar a si mesmo é imediato

Fora do editor, de propósito: o **setor** (texto apontando "Enviar para outro setor") e a **dependência**, que mudam o fluxo. O prazo que a equipe se comprometeu a cumprir (`committed_deadline`) também não se edita ali — é combinado por "Propor novo prazo". Na tela os dois prazos têm nomes distintos: **Data/Hora do prazo** (o pedido pelo solicitante, em dois campos; sem hora vale até 23:59) × **Prazo que a equipe se comprometeu a cumprir**. O painel lateral ainda chama o primeiro de "Prazo pedido pelo solicitante". O prazo pedido continua editável por quem tem `tarefa.editar` (hoje, o Gestor de Setor): restringi-lo ao dono da atividade é uma decisão em aberto (F15).

Gerenciar dependência (task-dependency)

`TaskService.change_dependency(task, user, depends_on)` — exige `tarefa.editar` ; janela própria em Mais ações. O formulário só oferece tarefas que podem ser predecessoras (`TaskService.dependency_candidates` : mesma atividade, não canceladas, sem ciclo). O serviço recusa:

- **ciclo**, direto ou indireto ($A \rightarrow B \rightarrow C$ e $C \leftarrow A$) — também em `update_task` ;
- tarefa de outra atividade, cancelada ou a própria tarefa;
- **nova espera numa tarefa que já começou** (em execução, bloqueada, devolvida ou na fila com sessões de trabalho).

Consequências: tarefa `EM_FILA` que passa a esperar sai da fila (renumera) e volta a `DISPONIVEL` , com auditoria; tarefa que deixa de esperar é liberada para a fila. Auditoria `UPDATE` de `depends_on` com os títulos antes e depois.

2.11 Reabrir tarefa concluída

Desativado em 02/10/2026: `tarefas/<pk>/reabrir/` responde 410. A reabertura da **Demanda** (§1.4) segue ativa.

`TaskService.reopen(task, user, reason)` — ação `tarefa.reabrir` (sensível; perfis sugeridos Gestor de Setor e Administrador), motivo obrigatório, só para tarefa `CONCLUIDA` . As Regras eram omissas; as decisões estão em `Regras/02` (seção "Reabertura de tarefa concluída").

Aspecto	O que acontece
Estado	<code>CONCLUIDA</code> → <code>EM_FILA</code> . Se a própria predecessora ainda não terminou, fica <code>DISPONIVEL</code> fora da fila ("Aguardando etapa anterior")
Fila	nova passagem (<code>QueueEntry</code>) no fim da fila do setor; a passagem antiga fica como histórico
Preservado	sessões de trabalho e tempo, checklist, participantes, prazos, <code>first_action_at</code> (o histórico é fato); limpa <code>completed_at</code> / <code>completed_by</code>
Atividade <code>CONCLUIDA</code>	é reaberta junto , na mesma transação e com o mesmo motivo — exige também <code>demanda.reabrir</code> ; sem ela a reabertura é recusada e nada é gravado
Atividade <code>CANCELADA</code>	recusa
Tarefas que dependiam dela	se alguma já foi trabalhada (em execução, concluída, bloqueada, devolvida ou com sessão registrada), a reabertura é recusada listando-as; as que só esperam na fila voltam a <code>DISPONIVEL</code> (saem da fila, fila renumerada, auditoria <code>UPDATE</code> de situação) e são liberadas de novo quando esta for concluída
Auditoria / avisos	<code>REOPEN</code> da tarefa (<code>old_value=CONCLUIDA</code> , motivo); <code>TASK_ASSIGNED</code> "Tarefa reaberta" para setor, responsável e dono da atividade; menções no motivo

Onde aparece: botão **Reabrir tarefa** na ficha da tarefa e no painel lateral (bloco "O que fazer agora" de tarefa concluída) e item no menu : das linhas da lista com filtro "concluídas", sempre em popup de motivo (`activities/task_action_form.html` , rota `task-reopen`). Tarefa cancelada não reabre.

3. Processos (processos/)

Desativado em 02/10/2026. /processos/* e demandas/<pk>/processo/{aplicar,inputs,criterios} respondem **410 (09_ROTAS.md §5)**: não se cria, publica nem aplica processo. Os dados (`Process` , `ProcessVersion` , `ActivityInputValue` , `ActivityCriterionCheck` , `Task.process_step`) ficam preservados e os serviços (`ProcessService` , `ProcessApplicationService`) seguem no código, sem rota. O substituto é o Quadro da Demanda (§2.0), que pode partir de um quadro modelo. A ficha ainda mostra o cartão **Processo** das demandas que já tinham versão aplicada; como as rotas de inputs e critérios estão em 410, esses itens não podem mais ser marcados, mas a finalização continua lendo os critérios obrigatórios em aberto (§3.4): nessas demandas **SUCESSO fica bloqueado** e o caminho é *Concluído com pendências* (comportamento do código atual, a confirmar com o produto). O texto abaixo descreve o que o código ainda implementa.

Modelos reutilizáveis e versionados de "como fazer" um tipo de atividade, por empresa (Regras 11 / Regras/12_PROCESSOS_INPUTS_OUTPUTS_E_CRITERIOS_DE_ACEITE.md).

- `ProcessService.create` cria o processo e a **v1 em rascunho** (`processo.criar`).
- Rascunho é editável (`processo.editar_rascunho`): inputs, output com tipo de evidência, critérios de aceite e fluxo linear de etapas por setor.
- `publish` (`processo.publicar`) exige output e ao menos uma etapa; a versão publicada anterior vira `SUBSTITUIDO` . Versão publicada não muda mais.
- `create_new_version` (`processo.criar-versao`) copia a publicada para v(n+1) em rascunho; recusa se já houver rascunho.
- `set_active` (`processo.inativar`).
- Cada etapa pode ter um **responsável padrão** (`ProcessStep.default_responsavel` , opcional, só editável em rascunho, copiado por `create_new_version`).

3.1 Aplicar um processo a uma atividade

Um `ProcessVersion` publicado é um **molde imutável**. Aplicá-lo a uma atividade (`ProcessApplicationService.apply` , em `activities/process_application.py`) materializa, numa única transação:

versão publicada →	<code>Activity.process_version</code>	vínculo permanente
	└─ <code>ActivityInputValue × inputs</code>	um por <code>ProcessInput</code> , <code>is_received=False</code>
	└─ <code>ActivityCriterionCheck × crit.</code>	um por <code>ProcessCriterion</code> , <code>is_met=False</code>
	└─ <code>Task × etapas</code>	ordem, setor, título e <code>process_step</code> da etapa

Antes de gravar (qualquer falha aborta sem deixar nada): a pessoa está na organização da atividade e tem `processo.aplicar` nela; processo da mesma organização e **da mesma empresa da atividade**; processo ativo; versão `PUBLICADO` ; atividade sem processo, não `RASCUNHO` , `CONCLUIDA` nem `CANCELADA` ; todos os setores das etapas ativos e da organização; **toda etapa com responsável** (padrão da etapa ou escolhido agora, sempre pessoa ativa da organização — todas as etapas que faltam são listadas de uma vez); valores iniciais de input válidos.

Concorrência: a atividade é relida com `select_for_update` dentro da transação; uma segunda aplicação (clique duplo, duas abas com a página antiga) vê `process_version` já preenchido e é recusada. A constraint `unique_task_per_activity_process_step` é a rede de segurança no banco; um `IntegrityError` vira "Este processo já foi aplicado a esta atividade".

Não existe trocar, reaplicar, atualizar para a versão nova nem remover o processo de uma atividade: essas operações apagariam ou corromperiam o histórico.

Autorização: `processo.aplicar` na atividade autoriza materializar as tarefas de todos os setores do fluxo, sem exigir `tarefa.criar` em cada um (detalhes em [05_AUTORIZACAO.md §4.1](#)). `TaskService.create_task` e a aplicação compartilham `_create_task_core` , que não autoriza e é privado.

3.2 Dependência e liberação das etapas

`ProcessStep.depends_on_previous` vira `Task.depends_on` : a tarefa da etapa n depende da tarefa da etapa $n - 1$ (dependência linear; não há grafo).

Etapa	Estado ao aplicar
sem dependência (a primeira, ou <code>depends_on_previous=False</code>)	entra na fila do setor → <code>EM_FILA</code>
depende da anterior	criada e ligada, mas fora da fila , <code>DISPONIVEL</code> ; a tela diz "Aguardando etapa anterior"

Regras (no serviço, não só na tela):

- **iniciar** e **concluir** uma tarefa cuja predecessora não está `CONCLUIDA` levantam. Esta tarefa depende da conclusão de «X». O estado da predecessora é lido do banco, não do objeto em cache;
- **concluir a predecessora** libera as dependentes que estavam `DISPONIVEL`, fora da fila e em atividade ainda aberta: `QueueService.enqueue` (fila do setor certo), auditoria `TASK_RELEASED` e aviso ao setor e ao responsável. É idempotente;
- **só CONCLUIDA satisfaz a dependência**. Predecessora cancelada, bloqueada ou devolvida mantém a sucessora esperando (a sucessora nunca é liberada sozinha; quem gerencia pode remover a dependência em Mais ações → Gerenciar dependência, o que também a libera);
- `move_to_sector` de uma tarefa que espera muda o setor mas não a enfileira; `unlock` a devolve a `DISPONIVEL`; `return_task` é recusado (não há o que devolver).

3.3 Inputs, critérios e entrega esperada

- **Inputs** (`ActivityInputValue`): registrados na ficha da atividade por quem pode atualizá-los (`ActivityProcessService.can_update`, ver doc 05). Cada tipo é validado (`TEXTO`, `DATA`, `NUMERO`, `LINK`); `ARQUIVO` e `SELECAO` só confirmam o recebimento com uma observação. Cada mudança é auditada (`INPUT_UPDATED`, com o estado de antes e de depois).
- **Input obrigatório faltante não impede aplicar o processo** (a atividade pode nascer incompleta), **mas impede o início do fluxo**: `start` e `complete` de uma tarefa *gerada por processo* recusam enquanto houver input obrigatório não recebido (Antes de trabalhar nesta tarefa, registre o recebimento dos inputs obrigatórios...). Tarefas manuais não são afetadas.
- **Critérios de aceite** (`ActivityCriterionCheck`): marcar/desmarcar grava `met_by` / `met_at` e audita `CRITERION_UPDATED` (inclusive reabrir). O critério do molde não muda.
- **Entrega esperada** (`output_description`) e **tipo de evidência** aparecem no painel; ainda não há onde registrar a evidência (ver pendências).

3.4 Finalização de atividade com processo

Resultado	Critérios obrigatórios em aberto
<code>SUCESSO</code>	bloqueia , listando os que faltam (Falta atender: A; B)
<code>CONCLUIDO_COM_PENDENCIAS</code>	permite, com comentário obrigatório; os critérios abertos entram na auditoria e na conversa da atividade
<code>DECLINADO</code> / <code>CANCELADO</code>	não olha critérios

O caminho antigo `complete_activity` segue a regra do `SUCESSO`. Critérios opcionais nunca bloqueiam. Atividade sem processo não muda de comportamento.

3.5 Tela (desativada)

- Ficha da atividade → cartão **Processo** (âncora `#processo`): identificação da versão, resultado esperado, **Entradas** (x / y recebidas), **Etapas** (x / y concluídas), com o estado de cada uma e **Critérios de aceite** (x / y atendidos), com

caixa para marcar).

- Sem processo, quem tem `processo.aplicar` vê **Aplicar processo**: popup em 4 passos (Processo → Responsáveis → Entradas → Confirmar). O passo 1 lista só processos elegíveis (ativos, com versão publicada, da organização e da empresa da atividade); o passo 2 vem com o responsável padrão preenchido e sugere primeiro as pessoas do setor da etapa.

3.6 Notificações

- Ao aplicar: um aviso por tarefa que **já nasceu na fila** (setor + responsável, `TASK_ASSIGNED`); etapas que esperam não geram aviso; o dono da atividade, se não foi quem aplicou, recebe um resumo (`PROCESS_APPLIED`).
- Ao liberar uma etapa: aviso ao setor e ao responsável (`TASK_ASSIGNED` , "Tarefa liberada para a fila").

4. Caixa de Entrada (`intake/`) — inativa por padrão

Registra pedidos que chegam por fora (e-mail, Teams, conversa) e os transforma em Demandas. **Inativa desde 01/10/2026** (commit `0135cc0`): com `INTAKE_ENABLED` desligado (padrão; ver [03_CONFIGURACAO.md](#)) toda rota `entrada/*` responde **404**, a cada pedido, e o item "Entrada" some do menu. Ligar a chave no ambiente reativa tudo, sem mexer em rotas. As rotas e respostas estão em [09_ROTAS.md](#) §6.1.

- **Modelo**: `IntakeItem` (origem *E-mail, Teams, Pedido verbal ou conversa* ou *Formulário*, reservado; estado *Nova, Virou demanda* ou *Ignorada*; confiança *Alta, Média* ou *Baixa* das sugestões) e `IntakeEvent` (histórico).
- **Fluxo**: registrar (`entrada.registrar`) → triar (`entrada.triar`): editar as sugestões, **ignorar** (com motivo opcional), **restaurar** ou **criar demanda**.
- **Criar demanda** (`IntakeService.convert`): exige nome, dono (pessoa ativa da organização) e setor; confere organização, e obra × cliente. Cria a Demanda **pelo mesmo caminho do editor** (`ActivityService.save_draft` + `publish_draft` , o que cria também o Quadro da Demanda em branco) na mesma transação; se falhar, a solicitação continua nova. O item passa a *Virou demanda*, guarda a `activity` e o evento `CONVERTIDA` (com o código da Demanda). Solicitação já tratada não é tratada de novo.

07 — Contas e autenticação

Como uma pessoa passa a existir na LPS e como entra. Especificação das telas em `Telas/09_01_LOGIN.md` , `09_02_CRIAR_CONTA.md` , `09_03_CONFIRMAR_EMAIL.md` e `09_04_CADASTRO_NOVO_USUARIO_LPS.md` . Código em `accounts/` e nas telas de usuário de `core/` . Atualizado em 03/10/2026 (código no commit `add5acd` ; o app `accounts` não mudou nos commits de 01-02/10).

1. Duas formas de criar uma pessoa

	Autocadastro (<code>/accounts/criar-conta/</code>)	Cadastro pelo administrador (<code>/usuarios/novo/</code>)
Quem faz	a própria pessoa	quem tem <code>usuario.editar</code>
Conta nasce	inativa até confirmar o e-mail	ativa, com senha definida pelo admin
<code>username</code>	gerado (<code>conta-<20 hex></code>)	digitado pelo admin
Organização	nenhuma	a do admin (campo obrigatório, travado nela)
Setores	nenhum	"Setores em que atua" e "Setores que gerencia" no formulário (<code>AccessService.sync_user_sectors</code>)
Auditoria	não	<code>USER_CREATED</code> (+ <code>SECTORS_CHANGED</code>)

Em ambos os casos o signal `accounts.signals` cria o `Profile` vazio no `post_save` do `User` ; o `UserFormView` preenche a organização em seguida.

O cadastro pelo administrador (`UserForm` em `core/forms.py` , tela `core/user_form.html`) pede nome, e-mail, nome de usuário, organização, senha e confirmação (obrigatórias ao cadastrar: **não há convite por e-mail**, então o admin informa a senha inicial à pessoa) e setores. Na edição (`/usuarios/<id>/`), senha em branco mantém a atual; preencher grava `PASSWORD_RESET` no log, e desmarcar "Permitir que esta pessoa entre na LPS" inativa a conta sem apagar o histórico. Os acessos (perfis e concessões) são definidos depois, em `/usuarios/<id>/acessos/` (`05_AUTORIZACAO.md`).

Uma conta criada pelo autocadastro consegue logar depois de confirmar o e-mail, mas **toda tela da LPS a redireciona para `/accounts/me/`** até um administrador vinculá-la a uma organização (hoje, pelo Admin do Django — a tela `/usuarios/` só lista pessoas que já são da organização).

2. Autocadastro e confirmação de e-mail



- `SignUpForm` (`accounts/forms.py`): nome (gravado em `first_name`), e-mail único sem diferenciar maiúsculas (gravado em minúsculas), senha validada pelos validadores do Django (`AUTH_PASSWORD_VALIDATORS`), confirmação de senha e aceite dos termos obrigatório.
- `EmailVerification` (`accounts/models.py`):
 - código de 6 dígitos, válido por **15 minutos** (`CODE_TTL`);
 - no máximo **5 tentativas** (`MAX_ATTEMPTS`);
 - reenviar só depois de **60 segundos** (`RESEND_COOLDOWN`); o novo código substitui o anterior.

- A tela de confirmação mostra o e-mail mascarado (`ab***@dominio`) e usa a sessão (`PendingVerificationMixin`) para saber qual conta está pendente; sem ela, volta para o cadastro. Código expirado ou com 5 tentativas esgotadas pede um novo código; reenviar antes de 60 s é recusado com aviso.
- O e-mail do código (assunto "Confirme seu e-mail — LPS") é enviado por `send_mail` direto na view, sem tratamento de erro: sem SMTP configurado em produção, "Criar conta" dá erro 500 ([12_DEPLOY_RENDER.md](#)).
- Não existe tela para retomar a confirmação sem a sessão, nem limpeza de contas pendentes: uma conta abandonada continua ocupando o e-mail.
- `confirmar-email/alterar/` troca o e-mail da conta pendente (também único) e emite um novo código.
- Em dev, o e-mail com o código aparece no console do `runserver`.

URL	Nome	View
<code>/accounts/criar-conta/</code>	<code>signup</code>	<code>SignUpView</code>
<code>/accounts/confirmar-email/</code>	<code>verify-email</code>	<code>VerifyEmailView</code>
<code>/accounts/confirmar-email/reenviar/</code> (POST)	<code>verify-email-resend</code>	<code>ResendVerificationCodeView</code>
<code>/accounts/confirmar-email/alterar/</code>	<code>verify-email-change</code>	<code>ChangeVerificationEmailView</code>

3. Login

- `/accounts/login/` usa `LoginView` com `EmailAuthenticationForm` (o campo "username" vira um campo de e-mail).
- `accounts.auth_backends.EmailBackend` é o **único** backend: busca `User` por `email__iexact`, confere a senha e `user_can_authenticate` (contas inativas são recusadas). Se houver dois usuários com o mesmo e-mail, o login é recusado.
- O `username` continua existindo como identificador interno e para @menções.
- Após o login: `home` (`/`), ou o `next` informado. Logout: `/accounts/logout/` → login (`LOGIN_URL` , `LOGIN_REDIRECT_URL` e `LOGOUT_REDIRECT_URL` em [03_CONFIGURACAO.md](#)).

4. Recuperação de senha

Views nativas do Django, com templates próprios em `templates/accounts/` :

URL	Nome
<code>/accounts/esqueci-senha/</code>	<code>password_reset</code>
<code>/accounts/esqueci-senha/enviado/</code>	<code>password_reset_done</code>
<code>/accounts/redefinir-senha/<uidb64>/<token>/</code>	<code>password_reset_confirm</code>
<code>/accounts/redefinir-senha/concluido/</code>	<code>password_reset_complete</code>

Corpo e assunto do e-mail: `templates/registration/password_reset_email.html` e `password_reset_subject.txt`. A troca pelo próprio usuário **não** é auditada; `AuditLog.PASSWORD_RESET` só é gravado quando um admin define nova senha em `/usuarios/<id>/`.

5. Perfil e superusuário

- `/accounts/me/` (`profile`): a pessoa edita telefone e setor principal (`ProfileForm`). A organização aparece só para leitura.
- Superusuário: criado por `createsuperuser` (local) ou por `ensure_superuser` (deploy; lê `DJANGO_SUPERUSER_USERNAME` , `DJANGO_SUPERUSER_EMAIL` e `DJANGO_SUPERUSER_PASSWORD` , não faz nada se faltarem usuário ou senha e nunca altera um

usuário já existente). Passa por todas as verificações de autorização, mas precisa de organização no `Profile` para usar as telas da LPS — e de **e-mail** para conseguir logar.

6. Modelos e Admin

Modelo	Papel	No Admin
<code>Profile</code>	1:1 com <code>User</code> : organização (<code>PROTECT</code> , opcional), telefone, setor principal	sim (<code>ProfileAdmin</code>)
<code>UserSector</code>	Participação em setor (<code>MEMBRO</code> / <code>GESTOR</code>) com <code>joined_at</code> / <code>removed_at</code> ; única por usuário e setor enquanto ativa. Salvar ou apagar invalida o cache do motor de autorização (<code>accounts/signals.py</code>)	sim (<code>UserSectorAdmin</code>)
<code>EmailVerification</code>	1:1 com <code>User</code> : código, expiração, tentativas, <code>confirmed_at</code>	não

Detalhes de campos em [04_MODELOS_DE_DADOS.md](#). Migrations do app: `0001_initial` , `0002_alter_usersector_options_and_more` e `0003_emailverification` .

08 — Notificações e auditoria

O que cada evento gera: registro de auditoria (`audit.AuditLog`), notificação in-app (`notifications.Notification`) e e-mail. Regras de produto em `Regras/04_AUDITORIA_TEMPO_E_METRICAS.md` e `Regras/06_NOTIFICACOES_E_COMUNICACAO.md`. Atualizado em 03/10/2026 (commit `add5acd`): "atividade" é **Demanda** na interface (os identificadores seguem `ACTIVITY_*` / `activity`), e fila, processos, prazos e telas de tarefa foram desativados em 02/10 — ver "Quais eventos ainda são gerados" na seção 2.

1. Pontos de entrada

Para	Use	Onde
Audit	<code>AuditService.log(user=, action=, activity=, task=, target_user=, field_name=, old_value=, new_value=, reason=, target_type=, target_id=, metadata=)</code>	<code>audit/services.py</code> — único caminho de escrita; converte valores em texto. <code>target_type</code> / <code>target_id</code> / <code>metadata</code> servem aos eventos que não pertencem a uma demanda ou tarefa (quadros)
Notificar in-app	<code>NotificationService.notify(users=, event_type=, title=, message=, activity=, task=, url=, actor=)</code>	<code>notifications/services.py</code> — remove duplicados e grava em lote
Enviar e-mail	<code>EmailService.send_task_overdue</code> , <code>send_activity_approval_needed</code> , <code>send_client_information_request</code>	<code>notifications/services.py</code> — renderiza <code>templates/notifications/email/<nome>.txt</code> ; falha de envio é logada e não interrompe o fluxo
Resolver destinatários	<code>resolve_sector_members</code> , <code>resolve_sector_managers</code> , <code>resolve_sector_and_admins</code>	<code>notifications/recipients.py</code>

E-mails são **síncronos** (enviados durante a requisição) via `django.core.mail`. Não há fila.

2. Evento → auditoria → notificação → e-mail

Evento (serviço)	<code>AuditLog.Action</code>	<code>Notification.EventType</code> → destinatários	E-mail
Demanda criada/publicada	<code>CREATE</code>	<code>ACTIVITY_CREATED</code> → dono, criador	—
Campo da demanda editado (editor, lista inline, quadro de domínio)	<code>UPDATE</code> (um por campo)	menções, se a descrição mudou	—
Mudar etapa da demanda (Kanban, lista inline)	<code>UPDATE</code> com <code>field_name="etapa"</code> (nomes antes e depois, "Sem etapa" se vazio)	—	—
Mudar status (condição) da demanda	<code>UPDATE</code> com <code>field_name="status"</code> (nome do status; "Sem status")	—	—
Trocar o setor da demanda	<code>UPDATE</code> de setor mais o reinício de etapa e status do setor novo (três registros na mesma transação)	—	—
Primeira tarefa em andamento (<code>_mark_in_progress</code>)	<code>UPDATE</code> de <code>status</code> <code>ABERTA</code> → <code>EM_ANDAMENTO</code>	—	—

Evento (serviço)	AuditLog.Action	Notification.EventType → destinatários	E-mail
Troca de dono / assumir	OWNER_CHANGED	OWNER_CHANGED → antigo e novo dono	—
Finalizar como concluída	COMPLETE	ACTIVITY_COMPLETED → dono	—
Finalizar como cancelada	CANCEL	ACTIVITY_CANCELLED → dono	—
Reabrir	REOPEN	ACTIVITY_REOPENED → dono	—
Marcar pendente	PENDENCY_OPENED (+ OWNER_CHANGED se há aprovação)	ACTIVITY_APPROVAL_NEEDED → gestores do setor; ACTIVITY_PENDING → dono anterior (ou dono)	aprovação → gestores; pedido de informação → cliente, se marcado
Aprovar pendência	PENDENCY_APPROVED	ACTIVITY_APPROVED → dono	—
Resolver pendência sem aprovação (ActivityService.resolve_pendency ; sem rota)	grava AuditLog.Action.PENDENCY_RESOLVED , que não existe no enum (levantaria AttributeError ; doc 13)	ACTIVITY_APPROVED ("Pendência resolvida") → dono, se não foi ele	—
Anexo incluído/removido	UPDATE (campo "anexo")	—	—
Tarefa criada	TASK_CREATED	TASK_ASSIGNED → membros e gestores do setor	—
Tarefa editada (editor)	UPDATE (um por campo: título, descrição, prazo pedido e tags , com antes e depois); RESPONSABLE_CHANGED , EXECUTOR_ADDED/REMOVED e ASSIGNMENT_CREATED quando o editor mexe em responsável e participantes	menções; TASK_RESPONSABLE_CHANGED ; TASK_ASSIGNMENT_PENDING → convidado	—
Mudar a dependência	UPDATE de depends_on (títulos antes e depois); UPDATE de status (EM_FILA → DISPONIVEL) se a tarefa passou a esperar; TASK_RELEASED se deixou de esperar	TASK_ASSIGNED ("Tarefa liberada para a fila") quando é liberada	—
Assumir / aceitar atribuição	EXECUTOR_ADDED (+ ASSIGNMENT_ACCEPTED)	TASK_ASSIGNED → quem aceitou	—
Atribuir a outra pessoa	ASSIGNMENT_CREATED	TASK_ASSIGNMENT_PENDING → atribuído	—
Recusar atribuição	ASSIGNMENT_REJECTED	TASK_ASSIGNMENT_REJECTED → quem atribuiu	—
Remover participante	EXECUTOR_REMOVED	—	—
Trocar responsável	RESPONSABLE_CHANGED	TASK_RESPONSABLE_CHANGED → antigo e novo	—
Iniciar / retomar / tempo manual	SESSION_STARTED	—	—
Pausar	SESSION_PAUSED	—	—
Concluir tarefa	COMPLETE	TASK_COMPLETED → dono da atividade, responsável	—

Evento (serviço)	AuditLog.Action	Notification.EventType → destinatários	E-mail
"Já realizei este trabalho"	RETROACTIVE_LOGGED (new_value = período trabalhado; reason = "Informado em ... · motivo · comentário") e em seguida o COMPLETE normal; TASK_RELEASED nas sucessoras liberadas	TASK_COMPLETED → dono da atividade, responsável; TASK_ASSIGNED ("Tarefa liberada para a fila") nas sucessoras	—
Cancelar tarefa	CANCEL	—	—
Reabrir tarefa concluída	REOPEN da tarefa (old_value=CONCLUIDA , new_value=EM_FILA ou DISPONIVEL , motivo); REOPEN da atividade junto, se ela estava concluída; UPDATE de status (EM_FILA → DISPONIVEL) em cada tarefa seguinte que voltou a esperar	TASK_ASSIGNED ("Tarefa reaberta") → setor, responsável e dono da atividade; ACTIVITY_REOPENED → dono, se a atividade reabriu junto	—
Bloquear / desbloquear	BLOCK / UNBLOCK	TASK_BLOCKED / TASK_UNBLOCKED → dono da atividade	—
Enviar a outro setor	SECTOR_MOVED	TASK_ASSIGNED → novo setor	—
Devolver	RETURNED (+ SECTOR_MOVED)	TASK_RETURNED → os dois setores, dono	—
Aplicar processo	PROCESS_APPLIED (new_value = "Processo vN"; reason = contagens) + um TASK_CREATED por etapa + INPUT_UPDATED por input já informado	TASK_ASSIGNED → setor e responsável de cada tarefa que já nasceu na fila (etapas que esperam não avisam); PROCESS_APPLIED → dono da atividade, se não foi quem aplicou	—
Etapas liberadas (predecessora concluída)	TASK_RELEASED (new_value = setor da fila; reason = etapa que liberou)	TASK_ASSIGNED ("Tarefa liberada para a fila") → setor e responsável	—
Input do processo registrado/corrigido/reaberto	INPUT_UPDATED (field_name = nome do input; antes e depois: não recebido / recebido: valor)	—	—
Critério de aceite marcado/reaberto	CRITERION_UPDATED (field_name = critério; pendente ↔ atendido)	—	—
Reordenar fila	QUEUE_POSITION_CHANGED (por entrada afetada)	QUEUE_POSITION_CHANGED → dono da atividade	—
Propor prazo	DEADLINE_PROPOSED	DEADLINE_PROPOSED → dono	—
Aceitar prazo	DEADLINE_ACCEPTED	DEADLINE_ACCEPTED → quem propôs	—
Recusar prazo	DEADLINE_REJECTED + CONFLICT_OPENED	DEADLINE_CONFLICT → setor, dono	—
Resolver conflito	CONFLICT_RESOLVED	menções	—
Mensagem	—	MESSAGE_POSTED → envolvidos (menos autor e mencionados); MENTIONED → mencionados	—
Tarefa atrasada (check_overdue_tasks)	—	TASK_OVERDUE → setor, dono, responsável	sim

Evento (serviço)	AuditLog.Action	Notification.EventType → destinatários	E-mail
Segurança (AccessService , UserFormView)	PROFILE_CREATED , PROFILE_UPDATED , PROFILE_ACTIONS_CHANGED , PROFILE_ASSIGNED , PROFILE_REVOKED , ACTION_GRANTED , ACTION_REVOKED , SECTORS_CHANGED , USER_CREATED , PASSWORD_RESET (com target_user)	—	—

"Membros e gestores do setor" vem de `resolve_sector_and_admins` , que, apesar do nome, **não** inclui nenhum grupo de administradores.

Por que eventos de auditoria novos e não UPDATE + field_name : aplicar um processo, liberar uma etapa, registrar um input e atender um critério são eventos que as Regras (04 §178-214) pedem por nome e que a tela precisa ler como frases ("aplicou o processo Orçamento v3", "liberou a tarefa para a fila de Compras", "marcou o critério "Escopo revisado" como atendido"). Com UPDATE o histórico diria "alterou um campo". As frases estão em `activities/templatetags/lps.py` .

Ao **finalizar como "Concluído com pendências"** com critérios obrigatórios em aberto, o `reason` do COMPLETE e a mensagem "Finalização (...)" na conversa da atividade listam os critérios que ficaram de fora.

Quais eventos ainda são gerados (desde 02/10/2026)

A tabela acima descreve o que cada **serviço** grava. Parte dos serviços ficou sem tela: `/fila/` , `/processos/...` , `/prazos/...` , `/conflitos/...` e `/tarefas/<pk>/...` respondem 410, e a migração `boards/0008` cancelou as Task operacionais (as tarefas agora são itens de quadro, auditados como `BOARD_*` , abaixo).

Situação	Eventos
Gerados hoje	Todos os de Demanda (criação, edição, dono, etapa/status, concluir, cancelar, reabrir, pendência, aprovação, anexo), mensagens e menções, segurança (<code>PROFILE_*</code> , <code>ACTION_*</code> , <code>SECTORS_CHANGED</code> , <code>USER_CREATED</code> , <code>PASSWORD_RESET</code>) e quadros (<code>BOARD_*</code>).
Sem tela (dormentes)	<code>TASK_*</code> , <code>EXECUTOR_*</code> , <code>ASSIGNMENT_*</code> , <code>RESPONSIVEL_CHANGED</code> , <code>SESSION_*</code> , <code>RETROACTIVE_LOGGED</code> , <code>BLOCK / UNBLOCK</code> , <code>SECTOR_MOVED</code> , <code>RETURNED</code> , <code>QUEUE_POSITION_CHANGED</code> , <code>DEADLINE_*</code> , <code>CONFLICT_*</code> , <code>PROCESS_APPLIED</code> , <code>INPUT_UPDATED</code> , <code>CRITERION_UPDATED</code> , <code>TASK_RELEASED</code> e as notificações correspondentes. Os serviços e as frases do histórico continuam no código; o histórico antigo continua legível.
Em teoria ainda roda	<code>check_overdue_tasks</code> (<code>TaskService.mark_overdue_tasks</code> , <code>TASK_OVERDUE</code> + e-mail): olha só Task , e as operacionais foram canceladas — na prática não notifica mais nada.

Não auditado

Checklist, CRUD de cadastros, cores configuráveis, criação/edição de etapas e status do setor e o limite de cartões por coluna do Kanban (doc 13, F30), criação/publicação/versão de processo e alteração do responsável padrão de uma etapa (o **molde**), autocadastro e confirmação de e-mail, troca de senha pelo próprio usuário. (A mudança de etapa/status de uma **demanda** é auditada, inclusive pelo Kanban; antes de 02/10 o movimento de card não era.)

`AuditLog.Action.SESSION_RESUMED` existe mas nunca é gravado (`resume` delega para `start` , que grava `SESSION_STARTED` ; só a frase do histórico a conhece). `AuditLog.Action.PENDENCY_RESOLVED` , ao contrário, é **usado e não existe** (ver a linha "Resolver pendência" acima).

Os valores de `Notification.EventType` e `AuditLog.Action` são os do código (`notifications/models.py` , `audit/models.py`): 25 tipos de notificação e 63 ações de auditoria, das quais 20 são de quadros e 10 de segurança. Nenhum tipo de notificação é gerado pelos quadros nem pela Caixa de Entrada.

Caixa de Entrada (intake) — inativa por padrão (INTAKE_ENABLED=false , rotas em 404). Registrar, corrigir sugestões, ignorar, restaurar e converter uma solicitação **não** geram AuditLog nem Notification : a trilha é o `IntakeEvent` da própria solicitação (AuditLog só liga a atividade/tarefa, e a solicitação ainda não é nenhuma das duas). Converter chama

`ActivityService`, então a atividade criada tem os efeitos de sempre: `CREATE` e `ACTIVITY_CREATED` (dono e criador). O texto da solicitação vai para a descrição com as @menções neutralizadas, para um e-mail recebido de fora não notificar ninguém da equipe. O aviso de item novo é só o contador do menu (doc 13, F23).

Quadros (boards). Toda alteração do quadro grava `AuditLog` (sem `Notification`, sem e-mail), pelo mesmo `AuditService.log`, que agora aceita `target_type`, `target_id` e `metadata` (as colunas `activity / task` ficam vazias, porque o alvo não é atividade nem tarefa). Todo registro leva `metadata["board_id"]`; o histórico do quadro é `AuditLog.objects.filter(metadata__board_id=...)`.

Evento	<code>AuditLog.Action</code>	<code>target_type</code>	Campos
Criar / renomear / excluir quadro	<code>BOARD_CREATED / BOARD_UPDATED / BOARD_DELETED</code>	<code>board</code>	<code>field_name</code> , <code>old/new_value</code>
Criar / editar / mover / excluir grupo	<code>BOARD_GROUP_CREATED / _UPDATED / _MOVED / _DELETED</code>	<code>board_group</code>	<code>idem</code>
Criar / renomear ou configurar / excluir visualização (Kanban)	<code>BOARD_VIEW_CREATED / _UPDATED / _DELETED</code>	<code>board_view</code>	<code>field_name</code> = <code>name</code> ou a chave da configuração (<code>group_by</code> , <code>card_fields</code> ...), <code>old/new_value</code> ; um registro por chave que mudou
Criar (ou duplicar) / editar / mover / redimensionar / excluir coluna	<code>BOARD_COLUMN_CREATED / _UPDATED / _MOVED / _RESIZED / _DELETED</code>	<code>board_column</code>	<code>idem</code> ; troca de tipo grava <code>type</code> e <code>cleared_values</code> em <code>metadata</code> ; etiquetas gravam <code>etiqueta*</code> e <code>option_id</code>
Criar / renomear / mover / excluir item	<code>BOARD_ITEM_CREATED / _UPDATED / _MOVED / _DELETED</code>	<code>board_item</code>	<code>idem</code>
Preencher célula	<code>BOARD_CELL_UPDATED</code>	<code>board_cell</code>	<code>field_name</code> = nome da coluna, <code>old/new_value</code> = o texto que a tela mostra; <code>metadata</code> com <code>item_id</code> e <code>column_id</code> . Só quando o valor muda

Criar quadro a partir de um modelo grava também `metadata["template"]`.

Quadros de Demanda (boards/demand_services.py): criar o quadro que acompanha uma Demanda grava `BOARD_CREATED` com `metadata["activity_id"]`; **trocar o quadro** na edição da demanda grava `BOARD_UPDATED` (`field_name`="quadro de tarefas", `metadata` com `activity_id`, `items_deleted` e `replaced`). Os itens/células de uma "tarefa" saem como `BOARD_ITEM_*` e `BOARD_CELL_UPDATED`, como em qualquer quadro, sem `AuditLog.task`. Os quadros de domínio (lista de Demandas) gravam campos configuráveis como `UPDATE` com `field_name`="quadro:<chave>" ligado à demanda ou tarefa. O histórico do quadro está em `/quadros/<pk>/historico/` e **não** aparece no Histórico geral (`/historico/` só mostra registros com demanda).

3. Onde aparece

- **Histórico** (`/historico/`, `HistoryView`): `AuditLog` ligados a demandas da organização, com filtro `?event=` (tarefas, prazos, fila, devolucoes, tempo; as abas Fila, Devoluções e Tempo hoje só encontram histórico antigo — doc 13). Sem verificação de `auditoria.visualizar`. Eventos de segurança e de quadro, sem demanda, **não** aparecem ali. O texto de cada linha vem do filtro `{{ entry|audit_phrase }}` (`activities/templatetags/lps.py`).
- **Linha do tempo** da atividade/tarefa (`templates/activities/_timeline.html`): mesma fonte, filtrada pelo objeto.
- **Caixa de notificações** (`/notificacoes/`, `NotificationListView`):
 - abas `?filter=`: `para-mim` (padrão; o alias antigo `acao` continua valendo), `nao-lidas`, `mencoes`, `prazos` (propostas, aceite, recusa, conflito e atraso), `sistema` (os eventos informativos) e `todas`; `alertas` segue aceito na URL mas não tem aba; busca `q`, ordenação `ordem` (`recentes / antigas`), 30 por página;
 - cada evento é classificado em uma de seis categorias (`EVENT_CATEGORY`): delegação, prazo, menção, alerta, conclusão, informativo;
 - a aba **Ação necessária** reconsulta o estado atual (`_still_needs_action`): some da lista quando a proposta de prazo foi decidida, o conflito resolvido, o bloqueio encerrado, a atribuição respondida etc.;

- o abrir uma notificação (`<pk>/read/` , com `mode=open|read`) marca como lida e só redireciona para destinos internos validados. Notificação ligada a uma **tarefa** ainda leva a `/tarefas/<pk>/...` e a de conflito a `/conflitos/<pk>/resolver/` (ambos 410); as de demanda abrem `/demandas/<pk>/` (doc 13).
- o `markall` (`mark-all-read/`) marca todas as da pessoa como lidas.
- **Contador** do sino: context processor `unread_notifications_count` .

4. E-mail e configuração

Setting / env	Padrão	Efeito
<code>EMAIL_HOST</code> , <code>EMAIL_PORT</code> , <code>EMAIL_HOST_USER</code> , <code>EMAIL_HOST_PASSWORD</code> , <code>EMAIL_USE_TLS</code>	vazio, 587, vazio, vazio, <code>true</code>	Servidor SMTP.
<code>DEFAULT_FROM_EMAIL</code>	<code>no-reply@example.com</code>	Remetente de todos os e-mails do sistema.
<code>EMAIL_BACKEND</code>	dev: console · prod: SMTP	Em dev os e-mails saem no terminal.
<code>ADMIN_GROUP_NAME</code>	ADMIN	Lida em <code>config/settings/base.py</code> e não usada (doc 13, F2).

E-mails que o sistema envia (templates `templates/notifications/email/*.txt` , `EmailService`): `task_overdue` (tarefa atrasada; dormente, ver acima), `activity_approval_needed` (pendência aguardando aprovação, para os gestores do setor) e `activity_client_information_request` (pedido de informação ao cliente, só se marcado no pop-up de pendência e se o cliente tem e-mail). O e-mail de **confirmação de cadastro** (`accounts/views.py`) usa `send_mail` direto, sem `EmailService` . Não há preferência de notificação por usuário: todo destinatário recebe todo evento previsto.

09 — Rotas

Referência de todas as URLs. A coluna "Ação" indica a ação do catálogo verificada **na view** (`ActionRequiredMixin` ou `dispatch`); em `activities` , a maioria das verificações acontece dentro do serviço chamado — ver `06_ATIVIDADES_E_TAREFAS.md`. Toda view da LPS exige login e organização (`OrganizationRequiredMixin`), exceto login, cadastro, confirmação de e-mail e recuperação de senha.

Atualizado em 03/10/2026, sobre o commit `add5acd` . Em 02/10/2026 as tarefas operacionais (`Task`), a fila, o prazo negociado, o cronômetro e os processos foram **desativados**: as rotas continuam declaradas para links e favoritos antigos, mas respondem **410 Gone** (texto simples). As tarefas passaram a ser itens do Quadro de cada Demanda (`/quadros/...`).

Legenda e totais

- **410** — rota desativada: a view devolve `410 Gone` e não executa nada (`RetiredFeatureView` em `activities/views.py` , `RetiredProcessView` em `processes/views.py` , `MovedActivityFileView` em `core/legacy_redirects.py`). **Sombreada** — o padrão segue declarado, mas um `re_path` de captura declarado antes (`^tarefas/.$` , `^checkboxlist/.$` , `^prazos/.$` , `^conflitos/.$` , `^fila(?:/.*)?$` , `^.*$` em `processes`) responde primeiro: a view original não é mais alcançada.
- **301 / 308** — redireciona para o endereço novo (308 em POST, que mantém o método).
- **404** — Caixa de Entrada inativa (`INTAKE_ENABLED` desligado) ou objeto de outra organização.
- **Total: 247 padrões** `path()` / `re_path()` (fora 9 `include`), distribuídos assim:

Arquivo	Padrões	Observação
<code>config/urls.py</code>	9	+ 8 <code>include</code> (<code>accounts</code> , <code>notificacoes</code> , <code>processos</code> , <code>painel</code> , <code>entrada</code> , <code>quadros</code> , <code>core</code> , <code>activities</code>)
<code>accounts/urls.py</code>	5	
<code>core/urls.py</code>	52	
<code>activities/urls.py</code>	90	+ 1 <code>include</code> (<code>kanban_urls</code>); 50 respondem 410 (11 diretas + 39 sombreadas)
<code>activities/kanban_urls.py</code>	7	
<code>boards/urls.py</code>	41	36 <code>board-*</code> + 5 <code>workboard-*</code>
<code>intake/urls.py</code>	7	404 se a caixa está inativa
<code>notifications/urls.py</code>	3	
<code>painel/urls.py</code>	16	
<code>processes/urls.py</code>	17	todas respondem 410

Contando `legacy-activity-files` (410), **68 padrões** estão desativados (17 + 50 + 1).

1. Raiz (`config/urls.py`)

Caminho	Nome	Destino
<code>admin/</code>	—	Admin do Django

Caminho	Nome	Destino
accounts/login/	login	LoginView + EmailAuthenticationForm
accounts/logout/	logout	LogoutView
accounts/esqueci-senha/ , .../enviado/	password_reset , password_reset_done	views nativas
accounts/redefinir-senha/<uidb64>/<token>/ , .../concluido/	password_reset_confirm , password_reset_complete	views nativas
accounts/		accounts.urls
notificacoes/		notifications.urls
processos/		processes.urls — tudo responde 410 (seção 5)
painel/		painel.urls
entrada/		intake.urls (só com INTAKE_ENABLED ; desligado, 404 em todas as rotas)
quadros/		boards.urls (seção 6.2)
..		core.urls , depois activities.urls
atividade-arquivos/<path>	legacy-activity-files	410 (MovedActivityFileView): "Este link de arquivo expirou. Abra a demanda para baixar o anexo com segurança." Antes de 02/10/2026 redirecionava para demanda-arquivos/ ; hoje o anexo só sai por demandas/<pk>/anexos/<attachment_pk>/download/ (seção 4)
media/<path>		static.serve de MEDIA_ROOT , só com DEBUG (sem login)

Não existe rota para `ACTIVITY_FILES_URL` (`/demanda-arquivos/`): os anexos ficam em `ACTIVITY_FILES_ROOT` e são entregues pela view de download, que confere login, organização e acesso à demanda.

2. accounts (/accounts/)

Caminho	Nome	Função
me/	profile	Meu perfil (telefone, setor principal)
criar-conta/	signup	Autocadastro
confirmar-email/	verify-email	Digitar o código
confirmar-email/reenviar/	verify-email-resend	Reenviar código (POST)
confirmar-email/alterar/	verify-email-change	Trocar o e-mail pendente

3. core (/)

APIs de busca (JSON, filtradas pela organização, só login e organização): `api/pessoas/` (`person-search` , aceita `?sector=`), `api/clientes/` (`client-search`), `api/setores/` (`sector-search`), `api/empresas/` (`company-search`), `api/obras/` (`site-search` , aceita `?client=`), `api/centros-de-custo/` (`costcenter-search` , aceita `?site=`), `api/tags/` (`tag-search`) e as **opções do fluxo de um setor**: `api/setores/<sector_pk>/etapas/` (`sector-stage-options`) e

`api/setores/<sector_pk>/condicoes/ ( sector-condition-options )` — GET com `?dominio=demanda|tarefa` obrigatório (outro valor: 400) devolvem `{sector, items: [{id, name, color, is_default}]}` das etapas/status ativos do setor, para os seletores do editor de Demanda e dos quadros.

Cadastros — os nomes de rota seguem `<recurso>-create / <recurso>-edit ( sector , company , site , costcenter , client , returnreason , taskstage , tag ; profile em perfis/... )`:

Caminho	View	Ação
<code>cadastros/ (?tab=)</code>	<code>CadastroHomeView (cadastros)</code>	—
<code>cadastros/setores/novo/ , <pk>/</code>	<code>SectorFormView</code>	<code>setor.editar</code>
<code>cadastros/empresas/nova/ , <pk>/</code>	<code>CompanyFormView</code>	<code>empresa.gerir</code>
<code>cadastros/obras/nova/ , <pk>/</code>	<code>SiteFormView</code>	<code>obra.gerir</code>
<code>cadastros/centros-de-custo/novo/ , <pk>/</code>	<code>CostCenterFormView</code>	<code>centro_custo.gerir</code>
<code>cadastros/clientes/novo/ , <pk>/</code>	<code>ClientFormView (também em popup)</code>	<code>cliente.gerir</code>
<code>cadastros/motivos/novo/ , <pk>/</code>	<code>ReturnReasonFormView</code>	<code>motivo_devolucao.gerir</code>
<code>cadastros/estagios-de-tarefa/novo/ , <pk>/</code>	<code>TaskStageFormView</code>	<code>etapa.gerir</code>
<code>cadastros/estagios-de-tarefa/reordenar/</code>	<code>TaskStageReorderView (taskstage-reorder)</code>	<code>etapa.gerir</code>
<code>cadastros/tags/novo/ , <pk>/</code>	<code>TagFormView</code>	<code>tag.gerir</code>
<code>cadastros/<tab>/<pk>/situacao/</code>	<code>CadastroToggleActiveView (cadastro-toggle)</code>	por aba: <code>setor.inativar (setores)</code> , <code>empresa.gerir</code> , <code>obra.gerir</code> , <code>centro_custo.gerir</code> , <code>cliente.gerir</code> , <code>motivo_devolucao.gerir (motivos)</code> , <code>etapa.gerir (estagios-de-demanda / estagios-de-tarefa)</code> , <code>condicao.gerir (status-configuravel)</code> , <code>tag.gerir</code>
<code>cadastros/<tab>/<pk>/cor/</code>	<code>SwatchColorSaveView (cadastro-color-save)</code>	<code>tag.gerir (tags)</code> ou <code>etapa.gerir (estagios-de-demanda, estagios-de-tarefa)</code> ; outra aba: 404

Configurações

Caminho	View	Ação
<code>configuracoes/</code>	<code>SettingsView (settings)</code> — preferências de notificação (guardadas na sessão)	—
<code>configuracoes/estagios-de-demanda/novo/ , <pk>/</code>	<code>ActivityStageFormView (activitystage-create , activitystage-edit)</code>	<code>etapa.gerir (no setor da etapa)</code>
<code>configuracoes/estagios-de-atividade/<resto></code>	<code>Legacy-activity-stages</code>	Endereço antigo: redireciona (301; 308 em POST) para <code>estagios-de-demanda/<resto></code>

Caminho	View	Ação
<code>configuracoes/status/<domain>/novo/ , <pk>/</code>	<code>WorkflowStatusFormView (workflowstatus-create , workflowstatus-edit); domain = activity ou task (outro: 404)</code>	<code>condicao.gerir</code> (no setor do status)
<code>configuracoes/status/<domain>/<code>/editar/</code>	<code>EnumColorLabelFormView (enumcolor-label-edit); domain = activity_status ou task_status (outro, ou code desconhecido: 404) — nome, descrição e visibilidade de um status nativo (Activity.Status / Task.Status ; o code não muda)</code>	<code>cor_status.gerir</code>
<code>configuracoes/cores/<domain>/salvar/</code>	<code>EnumColorSaveView (enumcolor-save)</code>	<code>cor_status.gerir</code> (<code>activity_status</code> , <code>task_status</code>) ou <code>cor_prioridade.gerir</code> (<code>activity_urgency</code> , <code>task_priority</code>)
<code>configuracoes/cores/<domain>/restaurar/</code>	<code>EnumColorResetView (enumcolor-reset)</code>	a mesma de <code>salvar/</code> , por domínio
<code>configuracoes/fluxo/<kind>/<pk>/excluir/</code>	<code>FlowConfigDeleteView (flow-config-delete)</code>	<code>etapa.gerir</code> (<code>kind = activity-stage</code> , <code>task-stage</code>) ou <code>condicao.gerir</code> (<code>workflow-status</code>); outro <code>kind : 404</code>
<code>configuracoes/fluxo/<kind>/<pk>/situacao/</code>	<code>FlowConfigToggleActiveView (flow-config-toggle)</code>	ativa/inativa a opção sem apagar o histórico; mesma ação por <code>kind</code>
<code>configuracoes/fluxo/<kind>/reordenar/</code>	<code>FlowConfigReorderView (flow-config-reorder)</code>	POST <code>item_id</code> (lista na ordem final): persiste a ordem; mesma ação por <code>kind</code>

Usuários e segurança

Caminho	View	Ação
<code>usuarios/</code>	<code>UserListView (user-list)</code>	<code>usuario.visualizar</code>
<code>usuarios/novo/ , <pk>/</code>	<code>UserFormView (user-create , user-edit)</code>	<code>usuario.editar</code>
<code>usuarios/<pk>/acessos/</code>	<code>UserAccessView (user-access)</code>	<code>seguranca.gerir_autorizacoes</code>
<code>usuarios/<pk>/acessos/<assignment_pk>/remover/</code>	<code>UserAccessRemoveView (user-access-remove)</code>	<code>seguranca.gerir_autorizacoes</code>
<code>usuarios/<pk>/concessoes/</code>	<code>UserGrantActionView (user-grant)</code>	<code>seguranca.gerir_autorizacoes</code>
<code>usuarios/<pk>/concessoes/<grant_pk>/remover/</code>	<code>UserGrantRemoveView (user-grant-remove)</code>	<code>seguranca.gerir_autorizacoes</code>
<code>permissoes/</code>	<code>PermissionMatrixView (permissions)</code>	<code>seguranca.gerir_perfis</code>
<code>permissoes/<pk>/salvar/</code>	<code>PermissionUpdateView (permissions-update)</code>	<code>seguranca.gerir_autorizacoes</code>
<code>perfis/novo/ , <pk>/</code>	<code>ProfileFormView</code>	<code>seguranca.gerir_perfis</code>

4. activities (/)

Padrões em `activities/urls.py` (90) e `activities/kanban_urls.py` (7, via `include` no fim). A **ordem importa**: os `re_path` de captura de rotas desativadas vêm antes das rotas antigas e as sombreiam (ver a legenda). Nomes de rota (`activity-*`) seguem em inglês; o endereço é `/demandas/...` desde 01/10/2026.

Início e visões gerais

Caminho	Nome	Função
<code>''</code>	<code>home</code>	"O que precisa da minha atenção agora?"
<code>fila/ , fila/<sector_pk>/ , fila/entrada/<pk>/reordenar/</code>	<code>queue-legacy-retired</code> (captura <code>^fila(?:/.*)?\$</code>); sombreadas: <code>queue , queue-sector , queue-reorder</code>	410 — a Fila foi desativada em 02/10/2026
<code>gestao/</code>	<code>management</code>	Gestão por exceção (<code>metricas.visualizar</code> em algum escopo; 403 sem ela). Os blocos de fila e tarefas leem <code>Task / QueueEntry</code> históricos
<code>historico/</code>	<code>history</code>	Histórico de auditoria (<code>?event=</code>)

Demandas (`Activity` ; o endereço era `demandas/...` até 01/10/2026)

Caminho	Função
<code>atividades/<resto></code> (<code>legacy-activities</code>)	Redireciona (301; 308 em POST, que mantém o método) para <code>demandas/<resto></code> , com a consulta (<code>?tab=...</code>). Sem login. Cobre todas as rotas abaixo
<code>demandas/</code> (<code>activity-list</code>)	Superfície operacional de Demandas (<code>boards/work_views.py::DemandWorkBoardView</code> , sobre o quadro de domínio): visão Lista . Abas <code>?tab=</code> <code>minhas</code> (padrão) <code>grupo</code> <code>participando</code> <code>concluidas</code> <code>todas</code> (esta só com <code>demanda.visualizar_todas</code>). Filtros: <code>q</code> (título, cliente, obra; a busca por código aceita o antigo <code>ATV-2026-00007</code> para achar <code>DEM-2026-00007</code>), <code>setor</code> (alias <code>sector</code> , <code>grupo</code>), <code>pessoa</code> (alias <code>responsavel</code>), <code>estagio</code> , <code>condicao</code> , <code>cliente</code> , <code>obra</code> , <code>tag</code> , <code>prazo</code> (<code>atrasadas</code> , <code>hoje</code> , <code>7_dias</code> , <code>30_dias</code> , <code>sem_prazo</code>), <code>filtro</code> , <code>ordem / sort</code> (<code>prazo</code> , <code>recentes</code> , <code>titulo</code>) e <code>dir</code> ; <code>?visao= / ?view=</code> escolhe a visão salva. Valor inválido é ignorado. Só mostra demandas com <code>demanda.visualizar</code>
<code>demandas/lista-legada/</code> (<code>activity-list-legacy</code>)	Lista anterior (<code>ActivityListView</code>), mantida por compatibilidade
<code>demandas/nova/</code> (<code>activity-create</code>)	Janela de 4 etapas de criação (1 Informações principais, 2 Cliente e obra, 3 Quadro de tarefas — <code>board_setup_mode</code> <code>BLANK</code> <code>TEMPLATE</code> + <code>board_template</code> —, 4 Descrição e arquivos) e rascunho antigo; exige <code>demanda.criar</code> em algum escopo (403 sem ela). JSON no Ajax: <code>{redirect_url}</code> ou 400 <code>{errors}</code>
<code>demandas/<pk>/nova/contexto/</code> (<code>activity-wizard-contexto</code>), <code>.../detalhes/</code> (<code>activity-wizard-detalhes</code>), <code>.../descartar/</code> (<code>activity-wizard-discard</code>)	GET das etapas antigas redireciona ao editor; POST legado compatível; descartar rascunho
<code>demandas/nova-rapida/</code> (<code>activity-mini-create</code>)	Mesmo editor no seletor de Demanda; resposta JSON em Ajax (<code>{id, name, summary}</code>)
<code>demandas/busca/</code> (<code>activity-search</code>)	Busca de demandas abertas por título ou código (JSON <code>{results: [{id, name, summary}]}</code> , até 20), para o seletor de Demanda
<code>demandas/kanban/</code> (<code>activity-kanban</code>), <code>demandas/calendario/</code> (<code>activity-calendar</code>)	Mesma superfície (<code>DemandWorkBoardView</code>) com a visão forçada: Quadro por etapas (Kanban do quadro de domínio, raias pelo campo de agrupamento, padrão Etapa) e Calendário de prazos (pelo prazo solicitado). Mesmos filtros e abas de <code>demandas/</code> . Gravar um campo vem de <code>quadros/dominio/.../valor/</code> (seção 6.2)

Caminho	Função
<code>demandas/kanban-legado/</code> (<code>activity-kanban-legacy</code>)	Kanban por setor anterior (<code>activities/kanban.py::ActivityKanbanView</code> , <code>?setor=</code>), mantido por compatibilidade; usa as ações <code>kanban/...</code> abaixo
<code>demandas/<pk>/</code> (<code>activity-detail</code>)	Ficha da atividade (rascunho redireciona ao editor)
<code>demandas/<pk>/painel/</code> (<code>activity-drawer</code>)	Compatibilidade: redireciona à ficha completa
<code>demandas/<pk>/editar/</code> (<code>activity-edit</code>), <code>prazo/</code> (<code>activity-change-deadline</code>), <code>dono/</code> (<code>activity-change-owner</code>), <code>assumir/</code> (<code>activity-claim</code>)	Ações: <code>demanda.editar</code> (já no GET, 403 sem ela), <code>demanda.editar</code> (prazo, pelo serviço), <code>demanda.alterar_dono</code> , <code>demanda.assumir</code> . Editar (a mesma janela de 4 etapas; <code>?passo=2</code> abre direto em “Cliente e obra”, como o clique em Cliente / Obra da lista de Demandas; o passo 3 troca o quadro de tarefas e, se a escolha mudou, exige <code>confirm_board_replace</code> : sem ele <code>400 {errors: {board_setup_mode}}</code> e nada é gravado), prazo, trocar dono, assumir
<code>demandas/<pk>/inline/</code> (<code>activity-inline-update</code>)	Edição inline da lista de Demandas (só JSON, POST). <code>field = title</code> (+ <code>value</code>), <code>owner</code> (+ <code>value = id</code> da pessoa), <code>sector</code> (+ <code>value = id</code> do setor, ou <code>new_name</code> + <code>new_color</code> para criar o setor e aplicá-lo numa só transação; criar exige <code>setor.editar</code>), <code>stage</code> (+ <code>value</code> ; não limpável), <code>condition</code> (+ <code>value</code> ; vazio = sem condição) ou <code>requested_deadline</code> (+ <code>date YYYY-MM-DD</code> e <code>time HH:MM</code> opcional; sem data retira o prazo; sem hora vale 23:59). Resposta {ok, field, value, display, derived?} (dados estruturados, nunca HTML). 403 = permissão negada (<code>ActivityPermissionError</code>), 400 = regra de negócio/valor inválido (inclui <code>field</code> desconhecido ou <code>status</code>), 404 = outra organização ou rascunho. É um adapter fino: toda regra, autorização, auditoria e notificação vive no <code>ActivityService</code> (<code>update_activity</code> , <code>change_owner</code> , <code>set_stage</code> , <code>set_condition</code>); a troca de setor confere a autorização também no setor de destino (e o destino ativo), e estágio/status de demanda concluída ou cancelada é recusado em qualquer tela
<code>demandas/<pk>/inline/opcoes/</code> (<code>activity-inline-options</code>)	Opções dos pop-overs da lista de Demandas, sempre no contexto da demanda (o cliente nunca informa o setor). GET <code>?campo=sector stage condition</code> → {ok, current_id, allow_clear, can_create, items:[{id,name,color,text_color}]} (+ <code>can_manage</code> , <code>manage_url</code> , <code>sector_name</code> em estágio/status); só devolve a lista a quem poderia gravar. POST <code>campo=stage condition</code> , <code>acao=criar editar</code> , <code>name</code> , <code>color</code> (paleta oficial), <code>option_id</code> (editar) cria ou edita nome/cor de uma etapa ou condição do setor da demanda ; exige <code>etapa.gerir</code> / <code>condicao.gerir</code> nesse setor (201 ao criar). 403 permissão, 400 regra (nome vazio/repetido, cor fora da paleta, campo sem lista, sem setor, demanda concluída), 404 outra organização ou rascunho
<code>demandas/<pk>/finalizar/</code> (<code>activity-finalize</code>), <code>concluir/</code> (<code>activity-complete</code>), <code>cancelar/</code> (<code>activity-cancel</code>), <code>reabrir/</code> (<code>activity-reopen</code>)	Encerramento e reabertura. <code>finalizar/</code> (<code>?outcome=</code> pré-seleciona) é o popup único: <code>demanda.concluir</code> (Sucesso, Concluído com pendências) ou <code>demanda.cancelar</code> (Declinado, Cancelado), comentário obrigatório. <code>concluir/</code> (POST) e <code>cancelar/</code> (POST com motivo; o GET redireciona a <code>finalizar/?outcome=CANCELADO</code>) são os caminhos antigos. <code>reabrir/</code> : <code>demanda.reabrir</code> (só demanda concluída; motivo obrigatório; janela ou página, JSON no Ajax)
<code>demandas/<pk>/processo/aplicar/</code> (<code>activity-process-apply</code>)	410 — "Aplicar processo" desativado em 02/10/2026
<code>demandas/<pk>/processo/inputs/<input_pk>/</code> (<code>activity-input-update</code>)	410 — registrar/reabrir input do processo
<code>demandas/<pk>/processo/criterios/<check_pk>/</code> (<code>activity-criterion-update</code>)	410 — marcar/desmarcar critério de aceite
<code>demandas/<pk>/pendente/</code> (<code>activity-mark-pending</code>), <code>pendencia/aprovar/</code> (<code>activity-approve-pendency</code>)	Pendência: <code>demanda.marcar_pendente</code> (motivo e comentário; prazo de decisão nos motivos com aprovação) e <code>demanda.aprovar_pendencia</code> (só a pendência com aprovação; a simples não tem rota — o serviço <code>resolve_pendency</code> existe, mas nenhuma view o chama). Janela ou página, JSON no Ajax
<code>demandas/<pk>/mensagem/</code> (<code>activity-message</code>), <code>continuar/</code> (<code>activity-continue</code>)	Mensagem; comentário + anexo num envio (<code>body</code> , <code>file</code> , <code>parent</code> para responder, <code>kind</code> , <code>visibility</code>). <code>comunicacao.participar</code> no serviço

Caminho	Função
<code>demandas/<pk>/mensagens/<message_pk>/reagir/ (activity-message-reaction)</code>	POST <code>emoji</code> (👍👎🙌👉👎👉): alterna a reação, só em mensagem visível à pessoa; <code>comunicacao.participar</code>
<code>demandas/<pk>/anexos/ (activity-attachment-upload)</code>	POST <code>file</code> : anexa (<code>demanda.editar</code> , ou o criador no próprio rascunho; audita <code>UPDATE</code> "anexo")
<code>demandas/<pk>/anexos/<attachment_pk>/download/ (activity-attachment-download)</code>	GET: entrega o arquivo (<code>FileResponse</code> , como anexo) depois de conferir organização e <code>demanda.visualizar</code> , e a visibilidade da mensagem a que o anexo pertence; 403 sem acesso, 404 sem arquivo. É a única forma de baixar um anexo
<code>demandas/<pk>/anexos/<attachment_pk>/remove/ (activity-attachment-delete)</code>	POST: remove e apaga o arquivo (<code>demanda.editar</code>); JSON no Ajax
<code>demandas/<pk>/mover-estagio/ (activity-move-stage)</code> , <code>demandas/<pk>/condicao/ (activity-set-condition)</code>	POST: gravam só a Etapa (<code>stage</code> ; <code>demanda.definir_etapa</code> ou, na falta, <code>demanda.mover_estagio</code>) ou só o Status (<code>condition</code> ; <code>demanda.definir_condicao</code>) pelo <code>ActivityService</code> . Recusa em demanda concluída ou cancelada
<code>demandas/<pk>/gaveta/ (activity-kanban-drawer)</code>	Gaveta da demanda aberta a partir do Kanban por setor (fragmento; <code>demanda.editar</code> e <code>comunicacao.participar</code> só definem o que a gaveta oferece; rascunho e outra organização dão 404)
<code>demandas/<activity_pk>/tarefas/rapida/ (task-quick-create)</code>	410 — a tarefa nasce no Quadro da Demanda (<code>board-item-create</code>)

Tarefas — desde 02/10/2026 só existe o **Painel Tarefas** (leitura). Todo o resto de `tarefas/` responde 410.

Caminho	Função
<code>tarefas/ (task-list)</code> , <code>tarefas/kanban/ (task-kanban)</code> , <code>tarefas/calendario/ (task-calendar)</code>	Painel Tarefas (<code>boards/task_center_views.py::TaskCenterView</code> , só leitura; login e organização): o que está nos Quadros de Demanda em que a pessoa é responsável (célula Pessoa). Lista única (ordenada por prazo, concluídas ocultas por padrão), Kanban (A fazer / Em andamento / Concluídas, derivados da etiqueta de Status) e Calendário semanal (<code>?semana=YYYY-MM-DD</code>). Filtros <code>q</code> , <code>status</code> (<code>todo</code> / <code>doing</code> / <code>done</code>), <code>prazo</code> (<code>overdue</code> / <code>today</code> / <code>week</code> / <code>none</code>), <code>pessoa</code> (<code>todas</code> ou <code>id</code> ; ignorado sem <code>quadro.visualizar</code> / <code>demanda.visualizar_todas</code>). <code>?demanda=<id></code> (endereço antigo) redireciona ao Quadro da Demanda (404 sem acesso). "+ Adicionar" cria o item pelo <code>board-item-create</code>
<code>tarefas/lista-legada/ (task-list-legacy)</code> , <code>tarefas/nova-rapida/ (task-quick-create-standalone)</code>	410
<code>tarefas/<resto> (captura ^tarefas/.+\$, task-legacy-retired)</code>	410 para tudo o que não é uma das três telas acima
<code>tarefas/kanban-legado/</code>	410 (sombreada; <code>task-kanban-legacy</code>)
<code>tarefas/<pk>/ (task-detail)</code> , <code>painel/ (task-drawer)</code> , <code>editar/ (task-edit)</code> , <code>dependencia/ (task-dependency)</code> , <code>mover-estagio/ (task-move-stage)</code> , <code>condicao/ (task-set-condition)</code>	410 (sombreadas)

Caminho	Função
tarefas/<pk>/assumir/ , iniciar/ , pausar/ , retomar/ , concluir/ , desbloquear/ (task-assume , task-start , task-pause , task-resume , task-complete , task-unblock) e as variantes JSON iniciar/ajax/ , pausar/ajax/ , concluir/ajax/ (task-start-ajax , task-pause-ajax , task-complete-ajax)	410 (sombreadas)
tarefas/<pk>/bloquear/ (task-block) , devolver/ (task-return) , mover/ (task-move) , cancelar/ (task-cancel) , reabrir/ (task-reopen) , ja-realizei/ (task-retroactive) , tempo/ (task-manual-time) , mensagem/ (task-message)	410 (sombreadas)
tarefas/<pk>/executores/ , executores/<user_pk>/remove/ , alterar-responsavel/ , atribuicoes/<assignment_pk>/aceitar/ , .../recusar/ (task-executor-add , task-executor-remove , task-change-responsavel , task-assignment-accept , task-assignment-reject)	410 (sombreadas)
tarefas/<pk>/checklist/ (task-checklist-add) , checklist/<pk>/alternar/ (task-checklist-toggle) , checklist/<pk>/remove/ (task-checklist-remove); captura ^checklist/.*\$ (task-checklist-legacy-retired)	410 (sombreadas)
tarefas/<pk>/prazo/propor/ (deadline-propose) , prazos/<pk>/aceitar/ (deadline-accept) , prazos/<pk>/recusar/ (deadline-reject) , conflitos/<pk>/resolver/ (conflict-resolve); capturas ^prazos/.*\$ (deadline-legacy-retired) e ^conflitos/.*\$ (conflict-legacy-retired)	410 — o prazo negociado e o escalonamento foram desativados (sombreadas)

As views correspondentes (TaskDetailView , TaskActionView , DeadlineProposeView ...) continuam no código, sem rota alcançável; os serviços (TaskService , DeadlineService) também — ver a seção 2 de [06_ATIVIDADES_E_TAREFAS.md](#).

Quadro Kanban por setor — ações (activities/kanban_urls.py , incluído no fim de activities/urls.py ; <dom> é demandas ou tarefas , outro valor dá 404). Servem ao Kanban por setor (demandas/kanban-legado/ e à gaveta); <dom> ≥ tarefas só enxerga Task históricas. Todas exigem login e organização e respondem JSON; as que gravam passam pelos services e devolvem o cartão novo em card_html :

Caminho	Nome	Função
kanban/<dom>/<pk>/cartao/	kanban-card	GET: o cartão como está agora, ou {"visible": false} se saiu do quadro. Usa a consulta da URL (setor e filtros)
kanban/<dom>/<pk>/etapa/	kanban-set-stage	POST stage_id : grava só a etapa. 403 se o serviço recusa (outro setor, inativa, sem ação); 400 se vazio; 404 se a etapa é de outro tipo ou organização
kanban/<dom>/<pk>/condicao/	kanban-set-condition	POST condition_id (vazio remove a condição): grava só a condição

Caminho	Nome	Função
kanban/demandas/criar/	kanban-create-card	POST title , sector_id , stage_id : cria a demanda na coluna (201). Tarefa não se cria aqui (400)
kanban/<dom>/limite/	kanban-column-limit	POST stage_id , limit (1 a 999; vazio remove): exige etapa.gerir no setor
kanban/<dom>/opcoes/	kanban-create-option	POST kind (stage condition), name , color , sector_id : nova etapa/condição do setor; exige etapa.gerir / condicao.gerir

5. processos (/processos/)

Desativado em 02/10/2026. processes/urls.py abre com `re_path(r"^.*$", RetiredProcessView, name="process-retired")`, que captura **qualquer** caminho sob processos/ (inclusive a raiz) e responde 410 Gone com o texto "Processos foram desativados. Use o quadro da Demanda." Os dados (processos, versões, inputs, critérios, etapas) ficam preservados; não se cria nem se aplica processo. Os 16 padrões abaixo seguem declarados, **sombreados** pela captura, e as views (ProcessListView , ProcessCreateView ...) não são mais alcançadas:

```

( process-list ), novo/ ( process-create ), <pk>/ ( process-edit ), <pk>/informacoes/ ( process-basic-info ),
<pk>/output/ ( process-output ), <pk>/inputs/ ( process-input-add ;

• <input_pk>/remove/ , process-input-remove ), <pk>/critérios/ ( process-criterion-add ; +
  <criterion_pk>/remove/ , process-criterion-remove ), <pk>/fluxo/ ( process-step-add ; + reordenar/ process-
  step-reorder , <step_pk>/remove/ process-step-remove , <step_pk>/responsavel/ process-step-responsavel ),
  <pk>/publicar/ ( process-publish ), <pk>/nova-versao/ ( process-new-version ), <pk>/ativo/ ( process-toggle-
  active ). Autorização que seria exigida nos serviços ( processo.* ). Aplicar um processo era uma rota de demanda
  ( demandas/<pk>/processo/aplicar/ , seção 4), também 410.

```

6. notifications (/notificacoes/)

Caminho	Nome	Função
''	notification-list	Caixa com abas (?filter=), busca e paginação
<pk>/read/	notification-mark-read	Marca lida e redireciona para um destino interno validado
mark-all-read/	notification-mark-all-read	Marca todas

6.1 intake (/entrada/) — Caixa de Entrada

Inativa por padrão: com INTAKE_ENABLED desligado (ver 03_CONFIGURACAO.md) toda rota abaixo responde 404 (o decorador only_when_enabled , em intake/urls.py , olha a chave a cada pedido; vale até para quem tem entrada.*) e o item "Entrada" não aparece no menu. Foi inativada em 01/10/2026 (commit 0135cc0).

Nomes de rota são globais (o projeto não usa app_name), por isso o prefixo intake- . Todas exigem login e organização; item de outra organização dá 404, sem a ação dá 403 (seção 4.2 de 05_AUTORIZACAO.md). As telas de formulário abrem em janela (LPSModal) e também funcionam como página; com X-Requested-With: XMLHttpRequest respondem JSON.

Caminho	Nome	Método	Ação	Resposta
''	intake-list	GET	entrada.visualizar (em algum lugar)	Abas ? status=novos convertidos ignorados , busca ?q= , ?page= (25 por página)
registrar/	intake-capture	GET/POST	entrada.registrar (em algum lugar)	Sucesso: {"message", "redirect_url"}

Caminho	Nome	Método	Ação	Resposta
<pk>/	intake-detail	GET	entrada.visualizar no item	Texto original, o que a LPS entendeu e histórico
<pk>/editar/	intake-edit	GET/POST	entrada.triar no item	Sucesso: {"message", "html" (cartão novo), "target": "#intake-item-<pk>"}
<pk>/criar-demanda/	intake-convert	GET/POST	entrada.triar no item (+ demanda.criar no serviço)	Sucesso: {"message", "redirect_url": atividade}
<pk>/ignorar/	intake-ignore	GET/POST	entrada.triar no item	Sucesso: {"message", "remove": "#intake-item-<pk>"}
<pk>/restaurar/	intake-restore	só POST	entrada.triar no item	Sucesso: {"message", "remove"}; erro: 400 {"error"}

- Formulário inválido: 400 {"errors": {campo: [mensagens]}}; erro de regra aparece em __all__.
- Editar, criar demanda e ignorar numa solicitação que já foi tratada redireciona para o detalhe com a mensagem "Esta solicitação já foi tratada."
- Todos os nomes intake-* estão em _NAV_BY_URL_NAME (item "Entrada" ativo no menu); um teste confere.

6.2 boards (/quadros/) — Quadros dinâmicos

Nomes de rota globais com o prefixo board-. Todas exigem login e organização; objeto de outra organização dá **404**, sem a ação dá **403** (seção 4.3 de [05_AUTORIZACAO.md](#)). As gravações são **POST com corpo JSON** (CSRF no cabeçalho X-CSRFToken) e respondem {"ok": true, ...}; erro de regra: 400 {"ok": false, "error", "needs_confirmation"}; pede confirmação: **409** com needs_confirmation: true; sem permissão: 403; JSON inválido (ou que não é objeto): 400 "JSON inválido.". Os ids de grupo, coluna, etiqueta e item vão no caminho.

Caminho	Nome	Método	Ação	Corpo → resposta
''	board-list	GET	quadro.visualizar	Lista os quadros modelo da organização (kind=TEMPLATE; o quadro de cada Demanda não aparece aqui) e, com quadro.criar, o formulário "Criar um quadro"
novo/	board-create	POST (form)	quadro.criar	name, sector_id (obrigatório , setor ativo da organização), template (orcamentos, opcional) → cria um quadro modelo e redireciona a ele
<id>/	board-detail	GET	quadro.visualizar	A tabela. ?sort=<coluna>&dir=asc desc, ?q= (nome, texto, etiqueta, pessoa), ?pessoa=<id>
<id>/historico/	board-history	GET	quadro.visualizar	AuditLog do quadro, 50 por página
visoes/<id>/	board-view-detail	GET	quadro.visualizar	A visualização escolhida na aba: Kanban (as raias com os cartões) ou Calendário (a grade do mês, ?mes=AAAA-MM; mês inválido = o atual). ?q= e ?pessoa= como na tabela; ordem, agrupamento, coluna de Data e cor vêm da configuração salva

Caminho	Nome	Método	Ação	Corpo → resposta
<id>/visoes/novo/	board-view-create	POST	quadro.editar	{name, type: "KANBAN" "CALENDAR", settings} → {view, redirect_url}; Calendário num quadro sem coluna de Data: 400
visoes/<id>/editar/	board-view-update	POST	quadro.editar	{name?, settings?} (só as chaves enviadas mudam; valida e audita uma a uma) → {view, settings} completo
visoes/<id>/excluir/	board-view-delete	POST	quadro.editar	Exclusão lógica → {redirect_url} (a tabela). Os itens não são afetados
visoes/<id>/lanes/	board-view-lanes	POST	quadro.visualizar	Só de visualização Kanban (outra: 404). {q, pessoa} → {lanes_html, total_items, columns, settings, group_column_id, sum_column_id, card_column_ids}: o servidor redistribui os cartões e o navegador troca o HTML
visoes/<id>/calendario/	board-view-calendar	POST	quadro.visualizar	Só de visualização Calendário (outra: 404). {mes, q, pessoa} → {body_html, title, month, prev, next, today, columns, settings, date_column_id, color_kind, color_column_id, card_column_ids}: o servidor posiciona os cartões e o navegador troca o HTML (trocar de mês, depois de mover/criar/editar, ao configurar)
<id>/renomear/	board-rename	POST	quadro.editar	{name, description} → {name, description}
<id>/excluir/	board-delete	POST	quadro.excluir	→ {redirect_url}
<id>/grupos/novo/	board-group-create	POST	quadro.editar	{name, color} → {group, group_html}
grupos/<id>/editar/ · mover/ · excluir/	board-group-update · - reorder · - delete	POST	quadro.editar	{name, color} · {before_id, after_id} · recusa se houver itens
<id>/colunas/novo/	board-column-create	POST	quadro.gerir_colunas	{type, after_column_id} → {column, header_html, cells: {item_id: td}, after_column_id}
colunas/<id>/renomear/ · largura/ · mover/	board-column-rename · - resize · - reorder	POST	quadro.gerir_colunas	{name} → {column, header_html} · {width} → {width} (limitada a 96-640) · {before_id, after_id}
colunas/<id>/configuracoes/	board-column-settings	POST	quadro.gerir_colunas	{description, is_required, settings} → coluna + cabeçalho + células

Caminho	Nome	Método	Ação	Corpo → resposta
colunas/<id>/ocultar/ · duplicar/ · excluir/	board-column-hide · -duplicate · -delete	POST	quadro.gerir_colunas	{visible} · cópia ao lado · exclusão lógica
colunas/<id>/tipo/	board-column-type	POST	quadro.gerir_colunas	{type, preview} → {plan: {mode, filled, lost}}; sem confirm e fora de safe : 409 ; {type, confirm: true} converte
colunas/<id>/fragmento/	board-column-fragment	POST	quadro.visualizar	Redesenha cabeçalho e células de uma coluna (depois de editar etiquetas)
colunas/<id>/etiquetas/novo/ · etiquetas/<id>/editar/ · mover/ · excluir/	board-option-create · -update · -reorder · -delete	POST	quadro.gerir_colunas	{label, color} · {label, color, is_default, is_done} · {before_id, after_id} · {cleared_cells}
<id>/itens/novo/	board-item-create	POST	quadro.criar_item	{group_id, name, initial: {column_id, value} ou [{column_id, value}, ...]} → {item, row_html} (grupo de outro quadro: 404). initial preenche já uma ou várias colunas (até 30) na mesma transação (o Kanban cria o cartão dentro de uma raia; o Calendário, no dia clicado, com a data e os demais campos): valor inválido ou sem editar_item desfaz tudo
itens/<id>/renomear/ · mover/ · excluir/	board-item-rename · -move · -delete	POST	quadro.editar_item · quadro.editar_item · quadro.excluir_item	{name} · {group_id, before_id, after_id}
itens/<id>/detalhe/	board-item-detail	POST	quadro.visualizar	→ {drawer_html} : a gaveta do item (todos os campos, os ocultos também, editáveis no lugar pelos mesmos editores da tabela, e as últimas 8 mudanças do item). Só lê, então a permissão é conferida na view
itens/<id>/colunas/<id>/valor/	board-cell-update	POST	quadro.editar_item	{value} → {display, cell_html}; sem value limpa a célula

- A pesquisa de pessoas da célula reaproveita `person-search` (`/api/pessoas/`).
- **Quadro da Demanda** (`Board.kind = DEMAND` , um por Demanda, ligado por `Activity.task_board`): as mesmas rotas `board-*` valem, mas a autorização é relacional (`DemandBoardAccess` , em `boards/demand_services.py`): ver o quadro = participar da Demanda (dono, criador ou pessoa em célula Pessoa de um item ativo), `demanda.visualizar` ou `quadro.visualizar` ; criar e editar item (`quadro.criar_item` , `quadro.editar_item`) = participar ou `quadro.editar_item` ; excluir item (`quadro.excluir_item`) segue o catálogo; estrutura (`quadro.editar` , `quadro.gerir_colunas` , `quadro.excluir`) = dono, criador ou `quadro.gerir_colunas` . A tela de um Quadro de Demanda não oferece excluir o quadro. Chega-se a ele por `board-detail` (botão "Abrir quadro" da ficha da Demanda). O quadro nasce ao publicar a Demanda e só é trocado no editor da Demanda (seção 4).

Lentes sobre Demandas e Tarefas (`boards/work_views.py` ; a tela é `demandas/` , seção 4). Não usam `BoardItem` : o alvo é o objeto operacional (`Activity` ; `Task` só histórica). POST com corpo JSON ou formulário (CSRF); respondem `{success, message, errors, ...}` (400 com `errors` , 409 em conflito de edição, 403 sem permissão):

Caminho	Nome	Ação	Corpo → resposta
<code>dominio/<board_pk>/campos/<field_pk>/object_pk/<valor/></code>	<code>workboard-value</code>	<code>demanda.visualizar</code> (<code> tarefa.visualizar</code> no domínio Tarefa) na view; a regra do campo no serviço (<code>ActivityService.update_activity / change_owner / set_stage / set_condition</code> , <code>prazo.alterar_solicitado</code> no prazo, <code>demanda.editar</code> nos campos configuráveis)	<code>{value, updated_at?}</code> → <code>{success, html (célula nova), updated_at, target}</code> . <code>updated_at</code> é a versão que a tela leu: se a demanda mudou desde então, 409 "Esta informação foi alterada por outra pessoa"
<code>dominio/visualizacoes/<view_pk>/campos/<field_pk>/layout/</code>	<code>workboard-layout</code>	<code>quadro.gerir_colunas</code>	<code>{width (96-640), position, is_visible}</code> → <code>{success}</code>
<code>dominio/visualizacoes/<view_pk>/cartoes/</code>	<code>workboard-card-fields</code>	<code>quadro.gerir_colunas</code>	<code>{field_ids: [id, ...]}</code> → os campos exibidos nos cartões do Kanban, na ordem enviada; os demais ficam ocultos
<code>dominio/<board_pk>/campos/novo/</code>	<code>workboard-field-create</code>	<code>quadro.gerir_colunas</code>	<code>{label, type}</code> (<code>type</code> ∈ <code>TEXT</code> , <code>NUMBER</code> , <code>CURRENCY</code> , <code>DATETIME</code> , <code>SELECT</code> , <code>BOOLEAN</code> ; outro: 400) → <code>{success, field_id}</code> : novo campo configurável (valores em <code>DomainCustomValue</code> , auditados como <code>quadro:<chave></code>)
<code>dominio/visualizacoes/<view_pk>/configuracao/</code>	<code>workboard-view-settings</code>	<code>quadro.gerir_colunas</code>	<code>{group_by?, show_empty?, show_field_names?}</code> : <code>group_by</code> ∈ <code>stage</code> , <code>condition</code> , <code>sector</code> , <code>owner</code> , <code>responsavel</code> , <code>urgency</code> , <code>priority</code> e precisa existir no quadro (senão 400)

- Todos os nomes `board-*` estão em `_NAV_BY_URL_NAME` (item "Quadros" ativo no menu); um teste confere (36 rotas).
- O endereço dos endpoints chega ao JavaScript pelo `json_script #board-meta` , com um id fictício (`999999999` , e `999999998` para a coluna na rota da célula) que o script troca pelo real.

7. painel (/painel/)

Telas em branco ("em construção") que mantêm o menu completo: `equipe/pessoas/` (`equipe-pessoas`), `equipe/capacidade/` (`equipe-capacidade`), `equipe/carga-de-trabalho/` (`equipe-carga-trabalho`), `filas-e-gargalos/filas/` (`gargalos-filas`), `.../gargalos/` (`gargalos-gargalos`), `.../bloqueios/` (`gargalos-bloqueios`), `.../devolucoes/` (`gargalos-devolucoes`), `processos/modelos/` (`processos-modelos`), `insights/` (`insights`), `desenvolvimento/` (`desenvolvimento`), `resultados/` (`resultados`), `integracoes/` (`integracoes`), `configuracoes/motivos-de-bloqueio/` (`config-motivos-bloqueio`), `configuracoes/motivos-de-devolucao/` (`config-motivos-devolucao`) — 14 rotas, todas `TelaEmBrancoView` . Os endereços de filas, bloqueios, devoluções e modelos de processo são só espaços reservados do menu: a Fila e os Processos estão desativados (410 em `fila/` e `processos/`).

Telas reais, com views de `core` :

Caminho	Nome	Ação
<code>configuracoes/etapas-e-status/</code>	<code>config-etapas-status</code>	<code>etapa.gerir</code> ou <code>condicao.gerir</code> (a que a pessoa tiver no setor escolhido); a tela mantém Etapas de Demanda, Etapas de Tarefa e Status por setor

Caminho	Nome	Ação
configuracoes/prioridades/	config-prioridades	cor_prioridade.gerir

Para listar as rotas de verdade a qualquer momento:

```
python manage.py shell -c "from django.urls import get_resolver; [print(p) for p in get_resolver().reverse_dict.keys() if isinstance(p, str)]"
```

10 — Frontend

A interface é renderizada no servidor com templates do Django. CSS e JavaScript são arquivos estáticos simples — **não há bundler nem etapa de build**; o `package.json` da raiz existe só para os testes JavaScript com `jsdom` (`npm run test:js` , ver `11_TESTES.md`). As telas funcionam sem JavaScript sempre que possível; o JS melhora a experiência (arrastar, modais, busca), mas envia os mesmos formulários que já existem na página. Identidade visual em `Telas/Elementos/` e `Telas/MENU_E_SUBMENUS_LPS.md` .

Estado em 03/10/2026. Na interface "Atividade" virou **Demanda** (código segue `Activity`), "Situação" virou **Etap** e "Condição" virou **Status**. As telas operacionais passaram a ser **quadros dinâmicos** (`boards`): `/demandas/` (Lista, Kanban e Calendário) e `/tarefas/` (painel do que está nos quadros de Demanda) usam o motor de Quadros. Desde 02/10/2026 `/fila/` , `/processos/*` , `/conflitos/*` , `/prazos/*` e `/tarefas/<pk>/...` respondem **410** (`RetiredFeatureView`), e `/entrada/` fica inativa por padrão (`INTAKE_ENABLED=False`). Muitos templates e scripts dessas telas continuam no repositório; as tabelas abaixo marcam o que está **ativo** e o que é **legado** (arquivo presente, sem rota ou sem script que o carregue). Ver também `LPS_ELEMENTOS_SUBELEMENTOS_PROCESSOS.md` e `LPS_VISUALIZACAO_CALENDARIO.md` .

1. Templates (templates/)

Pasta / arquivo	Conteúdo
<code>base.html</code>	Casca da aplicação: menu lateral (recolhível), barra superior com cronômetro e sino, mensagens, blocos <code>{% block %}</code> (<code>content</code> , <code>scripts</code>). Carrega a fonte Inter (Google Fonts), todos os CSS globais e, para quem está autenticado, os scripts de uso geral (<code>shell</code> , <code>modal</code> , <code>activity-steps</code> , <code>person-picker</code> , <code>person-multi-picker</code> , <code>tag-picker</code> , <code>rich-text</code> , <code>mention</code> , <code>workflow-picker</code> , <code>color-utils</code> , <code>color-palette-picker</code> , <code>activity-workspace</code> , <code>workspace-filters</code>); os scripts de cada tela entram pelo bloco <code>scripts</code>
<code>_icon.html</code> , <code>_icons.html</code> , <code>_logo.html</code>	Ícones SVG (sprite em <code>_icons.html</code>) e logo, incluídos com <code>{% include %}</code>
<code>_sector_badge.html</code> , <code>_sector_picker.html</code> , <code>_sheet_progress.html</code> , <code>_sheet_status.html</code>	Componentes compartilhados na raiz: selo colorido de setor (variantes <code>table</code> , <code>compact</code> , <code>inline</code> ; cores em <code>--sector-color</code> / <code>--sector-text-color</code>), seletor de setor (<code>data-picker-kind="sector"</code> , usa <code>api/setores/</code>), progresso compacto "N de M" e selo de status das listas legadas (<code>lps-sheet-*</code>)
<code>accounts/</code>	Login, criar conta, confirmar e-mail, trocar e-mail, perfil, recuperação de senha
<code>activities/</code>	67 arquivos. Ativos : <code>home.html</code> (/), <code>activity_detail.html</code> (ficha da Demanda, com a conversa e a prévia do Quadro), <code>activity_form.html</code> (janela de 4 etapas, criar e editar), <code>activity_wizard_step2/3.html</code> , janelas de ação da Demanda (<code>activity_cancel/reopen/change_owner/change_deadline/finalize_form/pending_form/approve_dependency_form.html</code>), <code>management.html</code> (/gestao/), <code>history.html</code> (/historico/) e os parciais de campo (<code>_form_field(s)</code> , <code>_activity_field</code> , <code>_activity_modal_field</code> , <code>_task_field</code>). Legado (view existe, rota fixa desativada ou substituída pelo quadro) : lista e Kanban antigos (<code>activity_list.html</code> em <code>/demandas/lista-legada/</code> , <code>activity_kanban.html</code> em <code>/demandas/kanban-legado/</code> , <code>activity_calendar.html</code> , <code>_activity_row/_toolbar/_view_tabs/_scope_tabs</code> , <code>_kanban*.html</code>), tudo de tarefa (<code>task_detail/list/calendar/kanban/edit_form/quick_form(standalone)/action_form/retroactive_form/change_responsavel.html</code> , <code>_task*.html</code> , <code>_tasks_block.html</code> , <code>_timeline.html</code>), <code>queue.html</code> (/fila/), <code>conflict_resolve.html</code> e o processo aplicado (<code>activity_process_apply.html</code> , <code>_process_panel.html</code>). A barra de filtros compartilhada é <code>_workspace_filter_toolbar.html</code>
<code>core/</code>	13 arquivos: cadastros (<code>cadastros.html</code> , <code>cadastro_form.html</code>), configurações (<code>settings.html</code> , <code>prioridades.html</code> , <code>etapas_e_status.html</code> — Etapas e Status por setor, com <code>flow-config.js</code>), usuários (<code>user_list/user_form/user_access.html</code> , <code>_access_fields.html</code>), permissões (<code>permissions.html</code>), perfil (<code>profile_form.html</code>) e parciais <code>_field.html</code> , <code>_toggle_form.html</code>
<code>notifications/</code>	Caixa de notificações, textos de e-mail em <code>email/*.txt</code>

Pasta / arquivo	Conteúdo
<code>boards/</code>	Quadros: <code>board_list.html</code> , <code>board_detail.html</code> (a tabela), <code>board_history.html</code> , <code>board_kanban.html</code> (a visualização Kanban), <code>board_calendar.html</code> (a visualização Calendário; o corpo, que o servidor redesenha, é <code>_calendar_body.html</code> , com <code>_calendar_day.html</code> e <code>_calendar_card.html</code>), <code>_item_drawer.html</code> (a gaveta do item) e os parciais <code>_board_head.html</code> e <code>_board_view_tabs.html</code> (cabeçalho e abas, compartilhados pela tabela e pelo Kanban), <code>_kanban_lanes.html</code> e <code>_kanban_card.html</code> (o cartão reaproveita <code>_cell.html</code> : o campo do cartão é o mesmo desenho da célula da tabela), <code>_group.html</code> , <code>_item_row.html</code> , <code>_cell_td.html</code> (<code><td></code> com <code>data-cell</code> , <code>data-value</code> cru para o editor), <code>_cell.html</code> (o desenho por tipo) e <code>_column_header.html</code> . O servidor desenha tudo; o JS só liga comportamento e troca fragmentos que o servidor devolve (<code>header.html</code> , <code>cell.html</code> , <code>row.html</code> , <code>group.html</code>). Filtros e tags em <code>boards/templatetags/lps_board.py</code> (ver seção 3).
<code>boards/</code> (superfícies de domínio)	Telas de Demandas e Tarefas sobre o motor de Quadros: <code>demand_work_board.html</code> (<code>/demandas/</code> , <code>/demandas/kanban/</code> , <code>/demandas/calendario/</code> : a Lista usa a edição inline; o Kanban usa <code>work-board.js</code>), <code>work_board.html</code> (visualização genérica de domínio), <code>task_center.html</code> (<code>/tarefas/</code> , <code>/tarefas/kanban/</code> , <code>/tarefas/calendario/</code> : painel do que está nos quadros de Demanda em que a pessoa é responsável, com "+ Adicionar"), <code>task_board.html</code> (escolha de uma Demanda para abrir o respectivo quadro) e <code>_board_workspace.html</code> (corpo compartilhado de <code>board_detail.html</code> e <code>task_board.html</code>). Parciais <code>_work_toolbar.html</code> , <code>_work_table.html</code> , <code>_work_cell.html</code> , <code>_work_kanban.html</code> , <code>_work_kanban_card.html</code> , <code>_work_calendar.html</code> e <code>_demand_kanban_settings.html</code> (painel "Adicionar campo" / configurar cartões do Kanban de Demandas)
<code>intake/</code>	Caixa de Entrada: <code>intake_list.html</code> , <code>intake_detail.html</code> , as janelas <code>intake_capture/_edit/_convert/_ignore.html</code> e os parciais <code>_item_card</code> , <code>_facts</code> , <code>_actions</code> , <code>_source_icon</code> , <code>_modal_head</code> , <code>_modal_foot</code> , <code>_form_errors</code> . As janelas reaproveitam o casco <code>activity-modal</code> e os campos de <code>activities/_activity_modal_field.html</code>
<code>processes/</code>	<code>process_list.html</code> , <code>process_new.html</code> , <code>process_edit.html</code> — desativado : todas as rotas de <code>/processos/</code> respondem 410 e as telas não são alcançáveis; os arquivos ficam como referência
<code>painel/em_construcao.html</code>	Tela em branco dos itens de menu ainda não construídos
<code>registration/</code>	Assunto e corpo do e-mail de recuperação de senha

Convenção: arquivos que começam com `_` são **parciais** incluídos em outras telas ou devolvidos por Ajax — ex.: `boards/_item_drawer.html`, `boards/_calendar_body.html`, `activities/_task_drawer.html`, `_task_checklist.html`, `_timeline.html`, `_tasks_block.html`, os parciais `_kanban*.html` e `_workflow*.html` (`_workflow_controls.html`, `_workflow_picker_attrs.html`). No total há 154 arquivos em `templates/` (raiz 8, `accounts` 9, `activities` 67, `boards` 30, `core` 13, `intake` 13, `notifications` 5 (mais 3 de e-mail em `notifications/email/`), `painel` 1, `processes` 3, `registration` 2).

Processo aplicado (**legado**: `/demandas/<pk>/processo/aplicar/` responde 410): `activities/_process_panel.html` é o cartão **Processo** da ficha (entradas, etapas, critérios; sem processo, o convite "Aplicar processo") e `activities/activity_process_apply.html` é o popup de aplicação. Os estilos novos (`proc-panel`, `proc-block`, `proc-item`, `proc-step`, `apply-*`) ficam no fim de `static/css/app.css` e só usam variáveis, cards e tags que já existiam. `activities/_form_fields.html` renderiza qualquer formulário campo a campo e é reutilizado por várias telas.

As antigas pastas `templates/demands/` e `templates/workflows/` não existem mais.

2. Context processors

Disponíveis em todo template:

Processor	Variáveis	Uso
<code>notifications.con</code> <code>text_processors.un</code> <code>read_notifications</code> <code>_count</code>	contador de não lidas	sino
<code>acessos.context_p</code> <code>rocessors.navigati</code> <code>on</code>	<code>lps_nav</code> (flags management, cadastros, users, security, intake, intake_can_view, boards), <code>lps_org</code> , <code>lps_open_tasks</code> (contador do menu Tarefas), <code>lps_intake_new</code> (solicitações novas que a pessoa enxerga; uma avaliação de permissão e um COUNT por página), <code>nav_active</code> , <code>nav_active2</code> , <code>nav_cadastros_tab</code>	o que o menu mostra e qual item está ativo — só UX
<code>activities.context</code> <code>t_processors.my_ac</code> <code>tive_sessions</code>	<code>my_active_sessions</code> , <code>my_active_sessions_count</code>	cronômetro da barra superior (menu <code>[data-timer-menu]</code> tratado em <code>shell.js</code>)

Os três estão em `TEMPLATES[...]['OPTIONS']['context_processors']` de `config/settings/base.py`, depois dos padrões `request`, `auth` e `messages`. Com `INTAKE_ENABLED=False` a flag `lps_nav.intake` fica falsa e o menu não mostra "Entrada".

3. Template tags

Duas bibliotecas. `{% load lps %}` (`activities/templatetags/lps.py`) só tem **filtros**:

Filtro	O que faz
<code>text conversation_body</code>	(autoescape) Corpo de mensagem da conversa da Demanda: escapa o HTML e destaca <code>@menções</code>
<code>entry audit_phrase</code>	Converte um <code>AuditLog</code> em frase em português ("iniciou a tarefa")
<code>name field_label</code>	Nome amigável de um campo auditado
<code>timedelta duration_hm</code>	4h20, 35min
<code>user avatar_color</code>	Cor do avatar (uma de 6, por <code>pk</code>)

`{% load lps_board %}` (`boards/templatetags/lps_board.py`), usada pelos templates de `boards/`:

Item	Tipo	O que faz
<code>item cell_of:column_id</code>	filtro	A célula do item para uma coluna
<code>mapping get_item:key</code>	filtro	Acesso a dicionário por chave dinâmica
<code>color contrast</code>	filtro	Cor de texto (claro/escuro) por luminância do fundo
<code>user initials</code> , <code>user person_name</code>	filtros	Iniciais e nome de exibição da pessoa
<code>column_type type_icon</code>	filtro	Ícone do tipo de coluna
<code>{% raw_value cell column %}</code>	<code>simple_tag</code>	Valor cru da célula (para <code>data-value</code> , lido pelo editor do JS)
<code>{% is_overdue cell column %}</code>	<code>simple_tag</code>	Se a célula de prazo está vencida

---|---| `entry|audit_phrase` | Converte um `AuditLog` em frase em português ("iniciou a tarefa") || `name|field_label` | Nome amigável de um campo auditado || `timedelta|duration_hm` | 4h20, 35min || `user|avatar_color` | Cor do avatar (uma de 6, por `pk`) |

4. JavaScript (static/js/)

Arquivo	Papel
shell.js	Casca: menu no celular, recolher o menu lateral, menu do cronômetro da barra superior, janelas de ação
modal.js	Busca um formulário existente via <code>fetch</code> e abre por cima da tela
task-drawer.js	Legado (nenhum template carrega). Painel lateral da tarefa sobre a ficha da atividade
checklist.js	Legado (nenhum template carrega; coberto por <code>tests/checkList.test.cjs</code>). Checklist da tarefa (página e painel), sem recarregar
timer.js	Legado (nenhum template carrega). Cronômetro da sessão; conta localmente, o servidor é a referência
queue.js	Legado (<code>queue.html</code> ; <code>/fila/</code> responde 410). Arrastar para reordenar a fila (preenche o formulário de posição existente)
kanban.js	Kanban legado de Demandas e Tarefas por setor (carregado por <code>activity_kanban.html</code> e <code>task_kanban.html</code> ; só <code>/demandas/kanban-legado/</code> responde — <code>/tarefas/kanban-legado/</code> cai no 410; o Kanban atual é o de <code>boards.js</code> e <code>work-board.js</code>): filtros que se aplicam ao escolher, menus, arrastar entre etapas (grava só a etapa), criar demanda na coluna, limite da coluna, gaveta ao clicar no cartão e cartões/contadores sempre em dia
workflow-picker.js	Seletor de Etapa e de Condição (cartão, menu "Mover para etapa", gaveta; carregado em todas as telas): opções do setor, "+ Nova etapa/condição" para quem gere o catálogo; avisa o quadro com <code>lps:workfLow-changed</code>
task-stage-reorder.js	Legado (nenhum template carrega). Reordenar estágios de tarefa
person-picker.js , person-multi-picker.js	Seletor de pessoa (responsável) e de várias pessoas (participantes), usando <code>api/pessoas/</code>
tag-picker.js	Seletor de tags com chips
mention.js	Autocomplete de <code>@menção</code> em campos marcados
rich-text.js	Editor de descrição (<code>contenteditable</code>); o HTML é sanitizado no servidor por <code>core/sanitize.py</code>
form-summary.js	Legado (nenhum template carrega). Resumo ao lado do formulário, atualizado ao digitar
(ações da tarefa)	Os itens de Mais ações e o botão Editar tarefa usam <code>data-activity-action</code> (<code>activity-workspace.js</code> + <code>LPSModal</code>): abrem a janela, recarregam a ficha ao salvar e, se a janela falhar, navegam para a página. O menu <code><details class="menu-more"></code> fecha ao escolher um item, ao clicar fora e com Esc (script inline em <code>task_detail.html</code>)
activity-steps.js	Janela de demanda em 4 etapas — criar e editar são a mesma — e o diálogo de confirmação ao trocar o quadro (ver seção 6). Registra-se em <code>window.LPSWidgets</code> ; o teste <code>tests/activity-steps.test.cjs</code> exercita o fluxo com o <code>modal.js</code> real
retroactive-work.js	Legado (<code>task_retroactive_form.html</code> ; rota <code>/tarefas/<pk>/ja-realizei/</code> responde 410). Popup "Já realizei este trabalho": avisa quando a data é de um dia anterior e mostra o comentário só quando é necessário (motivo <i>Outro</i> ou trabalho de mais de 7 dias atrás). Nenhuma regra mora aqui — o servidor valida tudo; sem JavaScript o comentário fica sempre visível. Registra-se em <code>window.LPSWidgets</code>
process-apply.js	Legado (processos desativados; <code>/demandas/<pk>/processo/aplicar/</code> responde 410). Popup "Aplicar processo": 4 passos num só formulário (Processo → Responsáveis → Entradas → Confirmar). Mostra/habilita só os campos da versão escolhida; nenhuma regra de negócio (o servidor valida tudo). Registra-se em <code>window.LPSWidgets</code> para funcionar dentro do <code>LPSModal</code>
process-editor.js	Legado (<code>process_edit.html</code> ; <code>/processos/*</code> responde 410). Editor de processo: no "Adicionar etapa", sugere primeiro as pessoas do setor escolhido para o responsável padrão (as de outros setores ficam atrás de "Mostrar pessoas de outros setores")

Arquivo	Papel
<code>color-utils.js</code> , <code>color-palette-picker.js</code>	Paleta de 36 cores (espelha <code>core/colors.py</code>) e seletor
<code>task-modal.js</code>	Legado (nenhum template carrega). Janela de tarefa: troca o seletor de atividade pelo cartão da atividade escolhida e liga o botão “Alterar”. Registra-se em <code>window.LPSWidgets</code> . Teste: <code>tests/task-modal.test.cjs</code> (inclui o <code>person-picker.js</code> real)
<code>activity-inline-edit.js</code>	Edição inline da lista de Demandas (<code>/demandas/</code> , só em <code>boards/demand_work_board.html</code>): clicar no título, no responsável ou no prazo edita ali mesmo. Eventos delegados em <code>[data-activity-list]</code> ; endereços em <code>data-*</code> da tabela (id fictício trocado por <code>fillUrl</code>). Cada célula tem um modelo estruturado e é redesenhada a partir dele (nunca por cópia de <code>innerHTML</code> ; todo texto de usuário entra por <code>textContent</code>), uma requisição por vez por célula (<code>aria-busy</code>) e um número de revisão. Otimista : a célula muda na hora e volta ao último valor <i>confirmado</i> se o servidor recusar; sucesso é silencioso (só um realce <code>is-saved</code>), erro mostra um aviso curto (<code>.lps-toast.is-error</code> , <code>role="alert"</code>). Um único pop-over, na camada global <code>#activity-inline-overlay-root</code> (<code>position:fixed</code>), que acompanha scroll e resize e fecha por clique fora ou Esc; busca de pessoas em <code>api/pessoas/</code> (debounce 250 ms, descarta resposta velha). Título: Enter/blur salva, Esc cancela, vazio nunca salva. Prazo: data + hora opcional, Limpar/Aplicar. Só quem pode recebe os atributos <code>data-inline-field</code> (a lista decide por <code>inline_flags</code> ; o servidor confere de novo). Editores mais ricos entram por plugins (<code>LPSInlineEdit.use</code>), que recebem a mesma infraestrutura (estado por célula, <code>save</code> otimista com revisão, <code>getJSON</code> / <code>postJSON</code> , pop-over único, <code>onSaved</code> , células ligadas que ficam travadas até a resposta). Exporta <code>window.LPSInlineEdit</code> (<code>helpers</code> , <code>init</code> , <code>boot</code> , <code>use</code>) para os testes. O CSS (<code>.activity-inline-*</code>) fica em <code>demand-board.css</code> : em repouso a tabela é idêntica (verificado por geometria e pixels contra o HEAD).
<code>task-center.js</code>	Tela Tarefas (<code>/tarefas/</code>): “+ Adicionar” no fim da Lista e de cada coluna do Kanban. Abre um formulário inline (nome + Demanda, só as Demandas onde a pessoa pode criar item, vindas em <code>#task-center-add</code>) e cria o item pelo endpoint <code>board-item-create</code> (JSON, CSRF): responsável = a pessoa e Status = etiqueta da coluna (A fazer usa a padrão do quadro). Erro 403/400 e queda de conexão aparecem no formulário; sucesso recarrega mantendo os filtros. Todo texto entra por <code>textContent</code> . Exporta <code>window.LPSTaskCenter</code> (<code>buildBody</code> , <code>init</code>). CSS em <code>task-center.css</code> (inclui a prévia do Quadro na ficha da Demanda).
<code>activity-inline-options.js</code>	Plugin do anterior (só em <code>boards/demand_work_board.html</code> , modo Lista): as células coloridas Setor, Estágio e Status viram pop-over na tela, sem janela. Lista de botões coloridos de largura total (cor e cor do texto vêm do servidor; só <code>#RRGGBB</code> entra), opção atual com ✓ e anel, busca local (sem acento) acima de 8 opções, setas/Enter/Esc. Setor : trocar e criar (nome + 36 cores da paleta oficial, <code>LPSColors.PALETTE</code>) com “Criar e aplicar” numa só gravação; trocar redefine Estágio e Status (as duas células ficam travadas e são redesenhadas com o <code>derived</code> do servidor). Estágio/Status : escolher, limpar (só Status, “Sem condição”), criar (“Criar e aplicar” = cria a opção e depois a aplica) e, para quem gere as opções do setor, editar nome e cor ali mesmo, com propagação para todas as linhas da página que mostram a mesma opção; link “Gerenciar estágios e status” (só endereço do próprio site). Erros de criar/editar aparecem no próprio pop-over. Mantém a segunda linha “Vencida há N dias” do Status (derivada do prazo) em dia quando o Prazo é editado. O CSS fica em <code>demand-board.css</code> (<code>.activity-inline-color-*</code> , <code>.activity-inline-swatch*</code> , <code>.activity-inline-edit-*</code> , <code>.demand-board__inline-cell</code>).

Arquivo	Papel
<code>boards.js</code>	Quadros (<code>static/js/boards.js</code> , sem dependências): seletor de tipo de coluna, criar coluna/item/grupo sem recarregar, renomear no lugar (Enter salva, Esc cancela), redimensionar (arrastar a borda ou setas, 96-640), reordenar colunas e mover itens por arraste (envia os dois vizinhos), editores de célula dos 8 tipos (texto/número inline, data nativa, etiquetas, busca de pessoa, checkbox), gerenciador de etiquetas, configurações da coluna, conversão de tipo com confirmação, menu de coluna/item/grupo, grupos recolhíveis (guardado no <code>localStorage</code>), teclado nas células (setas, Enter/F2, Delete, Espaço). Autosave otimista: a célula muda na hora e volta ao valor anterior, com aviso, se o servidor recusar; o indicador "Salvando.../Salvo/Erro ao salvar" fica no topo. Pop-overs e janelas têm foco devolvido ao botão, Escape e (janelas) Tab preso. Como cada grupo repete os títulos, mexer numa coluna atualiza todos os títulos (<code>thAll</code>): criar insere em cada grupo, renomear/configurar troca em cada grupo, mover reordena em cada grupo e excluir remove de todos; redimensionar vale para todos (a largura é do <code><colgroup></code>). Exporta <code>window.LPSBoards</code> (<code>helpers</code> , <code>init</code>) para os testes. Modo Kanban (<code>[data-kanban-lanes]</code> na página): arrastar o cartão entre raia grava a etiqueta da coluna agrupadora pelo endpoint da célula (otimista; recusa devolve o cartão), e depois pede as raia de novo (<code>board-view-lanes</code>) para o servidor redistribuir e recalcular as somas; campos do cartão editam com os mesmos editores da tabela (<code>[data-cell]</code> serve a <code><td></code> e a <code><dd></code>); criar na raia manda <code>initial</code> junto; menu do cartão ("Mover para", Renomear, Excluir) como alternativa ao arrastar; renomear/recolorir raia e "Editar etiquetas" mexem na etiqueta; janela "Configurar cartões" (agrupar, raia em branco, soma, nomes dos campos, campos do cartão com <code>↑↓</code> e pré-visualização) salva sozinha a cada mudança; abas: "+" (nova visualização: Kanban ou Calendário; o Calendário num quadro sem coluna de Data oferece criar a coluna ali mesmo), renomear e excluir. Modo Calendário (<code>[data-cal-body]</code> na página): passar o mouse no dia mostra "+ Adicionar" (sempre visível sem mouse), que abre a janela de criar por cima do calendário com a data do dia (e título, grupo, horário opcional e os campos do cartão), mandando tudo em um <code>initial</code> só; arrastar o cartão para outro dia grava só a célula de data (mantém a hora; otimista, volta com aviso "O item voltou para dd/mm/aaaa" se o servidor recusar; dia que a coluna não aceita não é alvo); clicar no cartão abre a gaveta (<code>board-item-detail</code>) e clicar num campo do cartão edita o campo (com <code>[data-cell]</code> , os mesmos editores da tabela); a data se edita num pop-over próprio (data nativa, horário opcional por botões ou digitado, "Limpar horário", "Limpar data"; quem gere colunas liga "Exibir horário" ali mesmo); "+ N mais", "Sem data" e "Atrasados" abrem listas em pop-over; Hoje/anterior/próximo trocam o mês sem recarregar (<code>board-view-calendar</code> , a busca e a pessoa seguem e o endereço é atualizado com <code>?mes=</code>); "Configurar calendário" (data utilizada, colorir por, escala, fins de semana, concluídos e campos do cartão com <code>↑↓</code> e pré-visualização) salva sozinha a cada mudança. <code>request(endpoint, body, {quiet: true})</code> não mexe no indicador "Salvando..." (navegar e abrir a gaveta não são gravações). Escape fecha primeiro o pop-over e só depois a gaveta.
<code>activity-workspace.js</code>	Carregado em todas as telas autenticadas. Liga os links <code>data-activity-action</code> ao modal compartilhado, remove anexos sem descartar o texto não salvo do editor e expõe <code>window.LPSAjax</code> (<code>applyResult</code> : remove/troca cartão, segue <code>redirect_url</code> ou recarrega). Teste: <code>tests/activity-workspace.test.cjs</code>
<code>workspace-filters.js</code>	Barra de filtros compartilhada (<code>_workspace_filter_toolbar.html</code> , <code>data-workspace-filters</code>): um pop-over aberto por vez (<code>[data-workspace-popover]</code> , Esc e clique fora fecham), troca de setor reenvia o formulário (<code>data-workspace-sector</code>) e trava o envio duplo. Teste: <code>tests/workspace-filters.test.cjs</code>
<code>work-board.js</code>	Visualizações de domínio (<code>work_board.html</code> , Kanban de <code>demand_work_board.html</code>): arrastar o cartão entre raia grava pelo serviço do domínio (otimista; volta o cartão se o servidor recusar), editor inline de célula (<code>data-work-cell</code>), painel "Adicionar campo" (<code>workboard-field-create</code>), configuração do cartão e "agrupar por" (<code>workboard-view-settings</code>), aviso <code>[data-work-board-toast]</code> . Teste: <code>tests/work-board.test.cjs</code>
<code>demand-conversation.js</code>	Conversa da Demanda (<code>activity_detail.html</code>): rótulo do anexo escolhido e inserção no cursor (menções/respostas); publicar, reagir e responder são formulários HTML normais
<code>flow-config.js</code>	Tela Etapas e Status (<code>core/etapas_e_status.html</code>): arrastar e soltar para reordenar (<code>[data-flow-reorder]</code>); só move a linha depois que o servidor confirma a ordem (<code>flow-config-reorder</code>)
<code>demand-board.js</code>	Legado (nenhum template carrega). Criação rápida de tarefa no Quadro de Demanda (<code>[data-demand-task-composer]</code>); a criação passou ao <code>boards.js</code> e ao <code>task-center.js</code>
<code>intake.js</code>	(Caixa de Entrada inativa por padrão.) Caixa de Entrada: abre as ações (<code>a[data-intake-action]</code>) em <code>LPSModal</code> e aplica a resposta (<code>LPSAjax.applyResult</code> : remove o cartão, troca o cartão, segue <code>redirect_url</code> ou recarrega); o formulário <code>form[data-intake-restore]</code> envia por <code>fetch</code> , com trava de envio duplo. Depende só de <code>LPSModal.open</code> e <code>LPSAjax</code> ; sem eles, ou se a janela falhar, o link abre a página completa. Teste: <code>tests/intake.test.cjs</code>
<code>notifications.js</code>	Carregado por <code>notification_list.html</code> . Evita que o cache do navegador mostre contador/lista antigos

Arquivo	Papel
auth.js	Mostrar/ocultar senha nas telas de conta

Ao mudar a paleta, altere **os dois**: `core/colors.py` e `static/js/color-utils.js`.

Quem carrega o quê. `base.html` carrega o conjunto geral (seção 1). `boards.js` só entra em `board_detail.html`, `board_kanban.html`, `board_calendar.html` e `task_board.html`; `work-board.js` em `work_board.html` e, no modo Kanban, em `demand_work_board.html`; `activity-inline-edit.js` e `activity-inline-options.js` na Lista de `demand_work_board.html`; `task-center.js` em `task_center.html`; `demand-conversation.js` em `activity_detail.html`; `flow-config.js` em `etapas_e_status.html`; `auth.js` nas telas de conta. Namespaces globais usados entre scripts: `LPSModal`, `LPSWidgets`, `LPSAjax`, `LPSColors`, `LPSBoards`, `LPSInlineEdit`, `LPSTaskCenter`. Testes de cada script em `11_TESTES.md`. No total há 35 arquivos `.js` em `static/js/` (cerca de 8,7 mil linhas; `boards.js` sozinho tem ~2,6 mil).

5. CSS e imagens

- `static/css/app.css` (~4,5 mil linhas): tokens (`:root`), casca, formulários, listas, cartões, selos e componentes gerais; os blocos `proc-*` e `lps-sheet-*` são do processo e das listas legadas.
- `static/css/auth.css` : telas de conta (login, cadastro, confirmação).
- `static/css/activity-workspace.css` : janela de atividade e utilitários da área de atividades (classes `activity-*`); a Caixa de Entrada reaproveita o casco e os campos.
- `static/css/task-modal.css` : janela de tarefa (`.task-*` : seções numeradas, cartão da atividade, grade de prazo, participantes); o casco vem de `activity-workspace.css`.
- `static/css/kanban.css` : quadro Kanban (`.kanban-*`, `.condition-chip`) e o seletor de Etapa/Condição (`.workflow-*`); carregado em todas as telas porque a gaveta também usa o seletor. O CSS do quadro antigo (`.activities-kanban-*`, `.tasks-kanban-*`, `.kanban-*` antigos) saiu do `app.css`.
- `static/css/boards.css` : Quadros e Kanban (`board-*`, `boards-*`, `kanban-*`; a cor da raia é `--lane-color`, vinda da etiqueta, e o texto `--lane-text` por luminância): tabela de largura fixa por coluna (`table-layout: fixed` + `<colgroup>`); **cada grupo tem a sua linha de títulos das colunas** logo abaixo do nome do grupo, como no Monday: `tr.board-cols-row` dentro de cada `tbody[data-group]`, fixa no topo ao rolar; a primeira coluna também fica fixa), faixa de cor do grupo (`--group-color`), pop-overs, janelas, avisos. Etiqueta sempre com o **texto** escrito (nunca só cor) e o texto claro/escuro escolhido por luminância. O Calendário usa `cal-*` (grade, dia, cartão com a cor da etiqueta em `--card-color`, listas e janela de criar) e a gaveta `board-drawer*`; o "+ Adicionar" do dia só aparece no hover/foco (e sempre em tela sem mouse). Carregado em `base.html`.
- `static/css/intake.css` : Caixa de Entrada (`intake-*`, selos `tag--NOVO|CONVERTIDO|IGNORADO` e `intake-confidence--alta|media|baixa`). Estado nunca é só cor: todo selo traz texto.
- `static/css/work-board.css` : visualizações de domínio (`.work-board*`, `.work-toolbar*`, `.work-table*`, `.work-kanban*`, `.work-card*`, `.work-calendar*`, `.work-card-settings`, aviso `.work-board__toast`); o arquivo está minificado em uma linha.
- `static/css/demand-board.css` : Tela de Demandas (`.demand-board__*` : abas de visão e de escopo, filtros rápidos, tabela, cartão do calendário) e a edição inline (`.activity-inline-*`, `.activity-inline-color-*`, `.demand-board__inline-cell`).
- `static/css/task-center.css` : tela Tarefas (`.task-center__*` : abas, tabela, semana/dia) e a prévia do Quadro na ficha da Demanda (`.task-preview__*`).
- `static/css/demand-conversation.css` : conversa da Demanda (`.demand-conversation`, `.conversation-*` : mensagens, reações, respostas, anexos).
- `static/css/notification-page.css` : página de Notificações (`.notification-*`).
- Ícones novos no sprite (`_icons.html`): `mail` e `chat`.
- Todos os CSS acima, exceto `auth.css` (carregado só nas telas de conta), são carregados por `base.html` com `?v=` de versão manual.
- `static/img/` : `favicon.svg` e logos da LPS (`logo-lps-full.png`, `logo-lps-full-dark.png`, `logo-lps-symbol.png`, `logo-lps.jpeg`).

Identidade visual

Tokens em `:root` de `app.css`: azuis da marca (`--brand-navy #1b3a63`, `--brand-navy-deep #12294a`, `--brand-blue #1b62c4`, `--brand-blue-light #61a5f2`) e, desde 30/09/2026, o **verde da marca** (`--brand-green #18b981`, `-hover`, `-soft`, `-border`), usado em abas ativas, barras de progresso e destaques de sucesso. Menu lateral em navy (`--nav-bg`), ação principal em `--primary` (azul), superfícies `--bg/--surface/--border`, texto `--text/--text-2/--muted` (contraste mínimo 4,5:1) e estados `--danger/--warning/--success/--info` — **sempre com texto, nunca só cor**. Fonte **Inter** (Google Fonts). Raios `--radius 10px` e `--radius-sm 7px`, sombras `--shadow` e `--shadow-lg`. `auth.css`, `boards.css` e `kanban.css` redeclaram `:root` com os tokens que usam. Etiquetas e setores usam a paleta de 36 cores (`core/colors.py` ↔ `color-utils.js`) com o texto claro/escuro escolhido por luminância. Não há tema escuro.

Em produção os arquivos são coletados por `collectstatic` em `staticfiles/` e servidos pelo `whitenoise` com nome versionado por hash — ao editar um CSS/JS, basta fazer deploy; não é preciso trocar a URL no template (use sempre `{% static %}`).

6. Experiência unificada de atividades

O editor de criação/edição é `activities/activity_form.html`: uma janela de **quatro etapas** (Informações principais → Cliente e obra → Quadro de tarefas → Descrição e arquivos) com cabeçalho, indicador de progresso e rodapé fixos e só o miolo rolando. Os campos são agrupados em `ActivityEditorForm` (`STEP_FIELDS`, `steps`) e o estilo está em `static/css/activity-workspace.css` (classes `activity-*`, sobre os tokens de `app.css`).

`static/js/activity-steps.js` mostra uma etapa por vez, marca o progresso (✓ nas concluídas), valida nome/atribuído a/setor antes de sair da etapa 1 e leva o Enter dos campos de texto para “Continuar”. Todos os painéis são do **mesmo formulário** e só ficam `hidden`, então nada se perde ao avançar ou voltar. Escuta o envio em captura (antes do `LPSModal`) para que só o botão final envie e, quando o servidor devolve erros, abre a etapa do primeiro erro. Sem o script as quatro etapas aparecem empilhadas. **Criar e editar são a mesma janela**: as mesmas 4 etapas (Informações principais, Cliente e obra, **Quadro de tarefas** — começar em branco ou usar um quadro existente, e o campo “Modelo de quadro” só aparece e só é exigido no segundo caso —, Descrição e arquivos, sem upload), os mesmos botões (Continuar em toda etapa; “Criar demanda” / “Salvar alterações” só na última) e os mesmos textos. Erro ao montar o quadro (`BoardError`) volta como erro do formulário no passo 3. A janela de edição aceita `?passo=N` (o clique em Cliente / Obra da lista de Demandas abre a etapa 2).

Trocar o quadro ao editar. O passo 3 da edição mostra o quadro **real** da demanda (“Quadro atual: criado em branco / a partir do modelo X · N tarefas”, vindo de `Board.source_template`, não do campo `Activity.board_setup_mode`, que pode estar velho). Escolher outra opção mostra o aviso “Trocar o quadro exclui as tarefas do quadro atual” e, no envio final, o `activity-steps.js` **bloqueia** o envio e abre o diálogo `[data-board-confirm]` (`role="alertdialog"`, foco em “Cancelar”, Esc fecha só o diálogo, Tab preso nele, clicar no fundo cancela, o resto da janela fica `inert`): “As N tarefas do quadro atual serão excluídas definitivamente. Esta ação não pode ser desfeita.” + “Novo quadro: ...”. **Trocar quadro e excluir tarefas** marca a caixa `confirm_board_replace` e reenvia pelo botão final (`requestSubmit`, então passa pela validação e pelo `LPSModal` de sempre); **Cancelar** volta ao passo 3 sem enviar. Mudar a escolha de novo, ou o servidor recusar, zera a confirmação. Mesma escolha do quadro atual, criar demanda e demanda sem quadro **nunca** abrem o diálogo. Sem JavaScript a caixa “Confirmando a troca...” fica visível (o servidor exige a confirmação do mesmo jeito, `ConfirmationCheckbox`: só `1 / true / on`). Quem não pode gerir a estrutura do quadro (`DemandBoardAccess.can_manage_structure`) vê o passo 3 somente leitura, com a frase “Só o responsável, quem criou a demanda ou quem gerencia quadros pode trocar o quadro.” O editor de texto (`RichTextWidget`) ganhou tachado, lista numerada, marcador de vazio e contador (`limit`); seu HTML passa por `sanitize_description` (permite `s / strike`). A antiga ficha lateral foi retirada; lista, quadro e calendário abrem a mesma ficha.

`static/js/activity-workspace.js` liga os links `data-activity-action` ao modal compartilhado e remove anexos sem descartar o texto ainda não salvo no editor. `modal.js` preserva a janela anterior ao abrir um cadastro auxiliar, impede envio duplicado, mostra erros de validação/rede, controla foco por teclado e devolve foco ao elemento de origem ao fechar.

Janelas de tarefa: um padrão só

Legado (03/10/2026). `/tarefas/<pk>/...` e `/demandas/<pk>/tarefas/rapida/` respondem 410; as janelas abaixo (`task_quick_form*.html`, `task_edit_form.html`, `_task_drawer.html`) e os scripts `task-modal.js` e `task-drawer.js` permanecem no repositório e nos testes de serviço/formulário, mas não são alcançáveis. As tarefas hoje são itens do quadro da Demanda (`boards.js`, `task-center.js`).

“Nova tarefa” (dentro de uma atividade, `task_quick_form.html`, e fora dela, `task_quick_form_standalone.html`) e “Editar tarefa” (`task_edit_form.html`) incluem o **mesmo corpo**, `activities/_task_editor_fields.html`. É uma tela só, sem etapas (diferente de “Nova atividade”), em quatro seções numeradas, todas à vista:

1 Tarefa	atividade • o que fazer • setor • responsável
2 Prazo e organização	data do prazo • hora do prazo • marcadores
3 Participantes	(opcional)
4 Instruções	(opcional) – editor de texto de <code>rich-text.js</code>

O casco (cabeçalho, corpo que rola, rodapé fixo, sobreposição com desfoque) é o da janela de atividade (`.activity-modal*`, `_task_modal_head.html` e `_task_modal_foot.html`); os blocos próprios são `.task-*` em `static/css/task-modal.css`. Em telas até 520px a janela vira uma folha que sobe da base. Cada campo sai de `_task_field.html`, que mantém `.form-row` (é por ela que o `modal.js` pendura o erro do servidor).

- **Uma fonte de palavras:** `TASK_LABELS / TASK_HELP` em `activities/forms.py` alimentam `TaskQuickCreateForm` e `TaskEditorForm`; mudar um rótulo muda nas duas janelas. (O painel lateral tem texto próprio em modo leitura.)
- **Atividade:** em “Nova tarefa” fora de uma atividade há o seletor; ao escolher, ele dá lugar a um cartão (título e “Cliente • Setor: X • N tarefas”) com o botão **Alterar**, que volta ao seletor já aberto. Dentro de uma atividade e na edição o cartão é só informação (a tarefa já pertence a ela). O resumo vem de `activity_summary()` (`forms.py`): a busca (`ActivitySearchView`) e a criação rápida devolvem `summary` e o `person-picker.js` entrega a opção escolhida em `event.detail.item`; `static/js/task-modal.js` só troca o seletor pelo cartão. Ao reabrir com erro, `TaskQuickCreateStandaloneForm.selected_activity` desenha o cartão pelo servidor.
- **Prazo:** dois campos, “Data do prazo” e “Hora do prazo” (`TaskDeadlineField`, que usa o `SplitDateOptionalTimeWidget` da atividade; `form.deadline_inputs` entrega os dois `<input>` para o template dar um rótulo a cada). Sem hora vale até 23:59, como no prazo da atividade; hora sem data é erro (“Informe a data do prazo.”), em vez de ser ignorada.
- **Edição:** o setor e o prazo combinado aparecem como informação (`.field-static` e `.task-note`); responsável e participantes também, para quem não tem a ação. O convite pendente fica dentro de “Participantes”.
- **Nada recolhível:** as seções ficam sempre abertas; um erro de campo aparece onde o campo está. (O painel lateral continua com `<details class="fold">`.)
- **Rótulo dos seletores:** `PersonPickerWidget`, `ClientPickerWidget`, `ActivityPickerWidget` e os seletores simples (setor, empresa, obra, centro de custo) herdam de `VisibleHiddenInput` (`core/widgets.py`): guardam o valor num input `hidden`, mas `is_hidden = False`. Sem isso o Django os tratava como campo escondido — sem rótulo nem ajuda em `_form_field.html` e, onde havia um laço de `hidden_fields`, desenhados duas vezes (era o que acontecia em “Nova tarefa”). Use `_activity_field.html` (aceita `hide_label` e `compact`) para campos dentro de janelas.
- **Painel lateral da tarefa** (`_task_drawer.html`): mesmo padrão em modo leitura — cabeçalho “Atividade: ...” com os botões **Editar** e “Ver todos os detalhes” numa linha acima do título, Setor e Responsável em `.field-static`, e os blocos “Participantes” (com convites “aguardando aceite” e o link “Gerenciar participantes”) e “Prazo e detalhes” (prazo pedido, prazo combinado, instruções e marcadores). Checklist e comentários seguem como seções abaixo.

11 — Testes

Onde estão os testes, como rodá-los e como escrever novos seguindo o padrão do projeto. Atualizado em 03/10/2026 (HEAD add5acd); as contagens abaixo vêm de `grep` de `def test_` (Python) e de `test( / it(` (JavaScript), sem executar a suíte.

1. Onde estão

Testes do Django ficam em cada app (`tests.py` e, conforme o app, vários módulos `test_*.py`). Há **1667 testes Python** em 33 arquivos (contagem por `def test_`), dos quais 58 estão ignorados com `@unittest.skip` (55 em `test_process_views.py` , 1 em `test_task_editor.py` e 2 em `test_views.py`), além dos **378 testes JavaScript** descritos mais abaixo. Contagens "aprox." da tabela = `def test_` por arquivo:

Arquivo	Testes (<code>def test_</code>)	Foco
<code>activities/tests.py</code>	73	Serviços: dono único, executores e tempo, sessões, devolução, fila, prazos, pendências, checklist, isolamento entre organizações
<code>activities/test_views.py</code>	85	Views: permissões por escopo, telas, Ajax, redirecionamentos (2 testes de lista/Kanban/Calendário de tarefas estão com <code>@unittest.skip</code> : foram substituídos pelo quadro de cada Demanda)
<code>acessos/tests.py</code>	36	Motor de autorização: escopos, perfis, concessões, teto de tenant
<code>core/tests.py</code>	34	Cadastros, busca de pessoas, formulário de usuário
<code>notifications/tests.py</code>	42	Destinatários, categorias, "ação necessária"
<code>processes/tests.py</code>	22	Molde: responsável padrão (tenant, ativo, fora do setor), versão publicada imutável, cópia na nova versão, editor de etapas
<code>activities/test_process_application.py</code>	99	Aplicação de processo: materialização (inputs, critérios, tarefas), validações que não gravam nada, dependências e liberação, inputs e critérios, trava de inputs obrigatórios, atomicidade, reaplicação, versões, isolamento entre organizações, finalização e o cenário de aceite completo
<code>activities/test_process_views.py</code>	55 (todos ignorados)	Telas do processo — a classe-base <code>ProcessViewCase</code> tem <code>@unittest.skip</code> porque <code>processes</code> foi desativado e as rotas respondem 410: botão e popup "Aplicar processo", painel da ficha, progresso, atualização de inputs/critérios, finalização, tarefas que aguardam
<code>activities/test_reopen.py</code>	44	Reabrir tarefa e atividade concluídas: fila, histórico preservado, permissão por setor, motivo, tarefas seguintes, atividade concluída/cancelada, atomicidade, botões e popup
<code>activities/test_task_actions.py</code>	88	"Já realizei este trabalho": período informado × conclusão agora, fila e sucessoras, auditoria, permissões (responsável/participante, sem exigir tempo manual), status, limites de data e justificativa, travas de dependência e de inputs do processo, atomicidade; botão, barra "Ações da tarefa" filtrada por permissão, popup (JSON), cartão de tempo cronometrado × informado
<code>activities/test_task_editor.py</code>	78	Editor da tarefa (uma transação: dados, marcadores, responsável, participantes e convites pendentes), Gerenciar dependência (ciclo, fila, tarefa já iniciada), motivo no tempo manual, origem do tempo na gestão, ações da tarefa em janela (JSON) e destaque do menu lateral em toda rota de tarefa (<code>test_action_page_highlights_tasks</code> está ignorado)

Arquivo	Testes (def test_)	Foco
activities/test_task_popups.py	48	Janelas de tarefa (nova dentro da atividade, nova avulsa e editar): uma tela só, quatro seções numeradas na mesma ordem (Participantes antes de Instruções), sem etapas nem blocos recolhíveis, mesmas palavras e rótulos; data e hora do prazo em campos separados (data sem hora = 23:59, hora sem data é erro, valores preservados quando a página volta com erro); cartão da atividade (resumo, sem "Alterar" dentro da atividade e na edição, atividade de outra organização ignorada, resumo na busca sem uma consulta por atividade e na criação rápida); seletor desenhado uma só vez e com rótulo; salvar com pk de tarefa ≠ pk de atividade. Também confere o painel lateral (blocos recolhíveis, próprio)
activities/test_activity_modal.py	78	Nova/Editar demanda — a mesma janela em 4 etapas: estrutura, nomes e ajudas, obrigatórios, campos que saíram (upload, marcadores, solicitante interno, anotações), Cliente → Obra → Centro de custo (busca e validação), links e caminhos de arquivos, JSON da janela, edição igual à criação (mesmas etapas e botões), abrir direto numa etapa (?passo= : valores inválidos caem na 1ª, ignorado ao criar e com o formulário enviado) e trocar o quadro ao editar (EditBoardSwapTests : o passo mostra o quadro real e a contagem de tarefas, o diálogo só existe na edição, mesma escolha salva sem perguntar, outra escolha sem confirmação é erro no passo 3 e não grava nada (nem os outros campos; 0 / false não confirmam), com confirmação as tarefas somem e o quadro nasce no mesmo pk , campo do quadro ausente nunca troca, modelo inativo/de outra organização recusado, falha no quadro desfaz a edição inteira, demanda sem quadro cria sem confirmar, quem não gere o quadro vê o passo somente leitura e um pedido forjado não toca nele, outra organização 404)
boards/test_demand_boards.py	21	DemandBoardTests (7): Quadro da Demanda com coluna Pessoa e Status padrão, colaborador citado numa célula de Pessoa, contrato da linha em branco, só o controle padrão de adicionar item, itens do modelo copiados sem materializar tarefas operacionais, rotas antigas de tarefa e processo respondem 410 (/tarefas/<pk>/ , /tarefas/nova-rapida/ , /demandas/<pk>/tarefas/rapida/ , /demandas/<pk>/processo/aplicar/ , /processos/) e Nova demanda com modelo da organização. ReplaceDemandBoardTests (14): trocar o quadro da Demanda (BoardInstantiationService.replace_for_activity): recusa sem confirmação sem mexer em nada, em branco → modelo e modelo → em branco no mesmo Board (itens, colunas, grupos e visões antigos somem; o novo nasce completo), cópia independente do modelo nos dois sentidos, mesma escolha não faz nada (mesmo com o modelo desativado depois), auditoria BOARD_UPDATED com items_deleted , demanda sem quadro, só quem gere a estrutura troca, modelo inativo/de fora recusado sem apagar, atomicidade (falha no meio da recriação mantém o quadro antigo), outros quadros intactos, contagem só de tarefas ativas, escolha atual vinda do quadro real
boards/tests.py	100	Quadros, serviços: posição decimal (ponto médio, rebalanceamento, vizinhos que já não são adjacentes), quadro/grupo/coluna (tipos ativos e reservados, nome repetido, largura 40→96 e 900→640, settings por tipo), conversão de tipo (segura, com texto → número, destrutiva, confirmação), etiquetas (duplicada, padrão única, excluir limpa células), itens (etiqueta padrão, mover, excluir), células de cada tipo (formato brasileiro, arredondamento, limites, fim de semana, pessoa de outra organização, etiqueta de outra coluna, obrigatória, valor antigo preservado em recusa), auditoria só quando muda, ordenação no banco por tipo, busca, filtro por pessoa, sem N+1, modelo Orçamentos, permissão e isolamento entre organizações
boards/test_kanban.py	63	Visualização Kanban: Kanban padrão em todo quadro e no modelo, migração de dados, validação da configuração (agrupar só por Status/Lista do quadro , soma só de Número/Moeda, campos do cartão, ordenação, chaves desconhecidas), coluna agrupadora e campos padrão do cartão, raias (ordem das etiquetas, sem duplicar nem perder item, raia "Em branco" nos três modos, raias vazias, soma formatada, filtros e ordem dentro da raia, renomear/recolorir etiqueta reflete), ViewService (permissão, nome único, auditoria por chave, valores inválidos preservam o antigo), página (acesso, 404 entre organizações e de visualização excluída, abas na tabela e no Kanban, controles por permissão, campos/ordem do cartão, soma, XSS, número de consultas igual com 3 ou 15 cartões), API (contratos, 403, 404 em todo endpoint, fragmento das raias) e criar o cartão na raia (valor inicial, tudo ou nada)

Arquivo	Testes (def test_)	Foco
<code>boards/test_task_center.py</code>	35	Painel Tarefas e prévia do Quadro na ficha: escopo por responsável (e item sem pessoa fora), coluna de Pessoa inativa, Demanda cancelada/rascunho fora e bloqueada dentro, isolamento entre organizações, estado derivado da etiqueta, atrasada/vence hoje pela data local (sem inventar prazo), cards, menções (só não lidas da pessoa), busca/filtros/ordem, filtro de pessoa autorizado no backend , Kanban, semana do calendário, <code>?demanda=</code> redireciona, "Abrir quadro" = Quadro da Demanda (nunca o modelo), consultas constantes com 3 ou 15 itens, selo do menu, "+ Adicionar" (demandas oferecidas, corpo do endpoint, recusa pelo serviço), prévia (≤5 linhas, abertas por prazo, sem misturar quadros, menções da Demanda, Demanda sem quadro, progresso e "pronta para concluir" vindos do quadro)
<code>activities/test_inline_edit.py</code>	104	Edição inline da lista de Demandas (Entrega 1: título, responsável, prazo; Entrega 2: cliente/obra, setor, estágio, status): funções puras do prazo (23:59 como "sem hora", exibição, fuso, datas inválidas, atraso), invariantes que ficam no Service (título normalizado/vazio/>200 recusado também fora do inline; dono ativo da mesma organização; <code>ActivityPermissionError</code> é subclasse de <code>ActivityError</code>), contrato JSON e códigos (200/400/403/404, sem <code>messages.*</code> , CSRF, só POST, rascunho 404), auditoria, <code>OwnerChangeLog</code> e notificação, valor igual = no-op, demanda concluída/cancelada travada, <code>field=status</code> recusado, flags por linha (quem edita, quem só troca título, leitor), página (marcadores só para quem pode, leitor vê o link de sempre), consultas constantes com 3 ou 15 linhas e uma só consulta de autorização para a lista; Entrega 2: invariantes do Service (autorização no setor de destino, destino ativo, atomicidade com falha no meio, cliente × obra, estado terminal em estágio/status também no Kanban), setor (trocar, criar e aplicar, nome repetido/cor fora da paleta, nada fica criado se a troca falha), listas de opções por campo, estágio/status (limpar, outro setor/inativo/outro domínio recusados, ação legada), criar/editar opções (permissão no setor da demanda), flags e marcadores por perfil, a linha "Vencida há N dias" e consultas constantes
<code>boards/test_calendar.py</code>	101	Visualização Calendário: validação da configuração (coluna de Data só do quadro, escala só Mês, cor por Status/Lista/grupo/nenhuma, campos do cartão, chaves do Kanban descartadas), coluna de Data/cor/campos padrão, mês e grade (segunda a domingo, 5 semanas em outubro/2026, ano bissexto), cada item no seu dia uma vez , dias de outro mês, só o intervalo da grade é carregado , sem data (contador e lista, limpar a data não exclui), dia inteiro × hora (sem hora inventada, fuso local, coluna sem horário), "+ N mais", fins de semana ocultos (nada some do banco, aviso, dias bloqueados), concluídos (filtro visual, só o Status decide) e atrasados (só em coluna de prazo, independente do mês), cor (renomear/recolorir reflete, nunca só a cor), campos do cartão, XSS, busca e pessoa nos links de mês, <code>ViewService</code> (congela Data e cor, exige coluna de Data, dois calendários com Datas diferentes sobre os mesmos itens, auditoria por chave, permissão, excluir a visão não apaga itens), página (acesso, abas, controles por permissão, metadados do JS, consultas iguais com 1 ou 13 cartões), API (fragmento, 404 entre tipos e organizações, criar no dia tudo ou nada , mover só a data com auditoria da data anterior e da nova, manter a hora, recusas, gaveta com histórico só do item)
<code>boards/test_views.py</code>	71	Quadros, telas e API: login, sem organização, 403, 404 entre organizações em cada endpoint de escrita (e nada muda), controles só para quem pode, conteúdo da tabela (valores, "vencido", contraste da etiqueta, XSS em nome/texto/título), <code>?sort / ?q / ?pessoa</code> , número de consultas igual com 3 ou 15 itens , contratos JSON (400/403/409, JSON inválido, CSRF, só POST), menu e as 36 rotas mapeadas
<code>boards/test_domain_workboards.py</code>	19	Superfícies de domínio sobre o motor de Quadros (<code>/demandas/</code> , <code>/demandas/kanban/</code> , <code>/demandas/calendario/</code>): quadros padrão idempotentes, título inline pelo serviço com auditoria, prioridade da tarefa, rotas principais, Kanban de Demandas com estágios vazios, configuração do Kanban (raias vazias, nomes dos campos), arrastar pelo serviço de Etapa ou de Status, agrupar por Status, agrupamento inválido e outra organização recusados; <code>DemandFilterToolbarTests</code> : barra de filtros na mesma ordem sem controles aposentados, atalhos de prazo exclusivos, setor delimita etapas e status em todas as visões, várias pessoas e "sem responsável" como OU, URLs antigas de pessoa compatíveis, limpar no calendário mantém o mês

Arquivo	Testes (def test_)	Foco
boards/test_centralization_migration.py	1	Migração que desliga Task de BoardItem (centralização das tarefas em quadros por Demanda): preserva os dados (TransactionTestCase , parte de audit 0011)
activities/test_activity_workspace.py	36	Editor de demanda, anexos e navegação: criar e editar usam o mesmo editor, publicação numa só submissão, rascunho vazio e retomada, responsável obrigatório preserva o formulário
intake/tests.py	75	Caixa de Entrada, serviços: ações no catálogo e perfis sugeridos, endereço do item para o motor de autorização (setor/obra sugeridos), política de estados, registrar (validações, duplicata em 7 dias, external_id único por origem, escopo de setor), corrigir sugestões (só em nova, outra organização, obra x cliente, não perder o acesso), ignorar/restaurar, converter (atividade publicada, solicitante interno x externo, descrição com origem e link, HTML escapado, @menção neutralizada , rollback total, segunda conversão, demanda.criar ainda exigido, escopo de setor), visibilidade e contador
intake/test_textparse.py	36	Parser sem banco: normalização, assunto, domínio, primeira linha útil e prazo em português (dia da semana, "amanhã", dia 15, 15/10, 15 de outubro, hora, pista de prazo, datas inválidas e passadas)
intake/test_suggestions.py	27	Sugestões: cliente por nome e por domínio (desempates, provedores gratuitos), obra limitada ao cliente, setor pelo assunto e pelo histórico, prazo, pontuação e faixas de confiança
intake/test_views.py	62	Acesso (login, sem organização, 403, 404 entre organizações), lista (abas, busca, paginação, escopo de setor, XSS, número de consultas não cresce com os cartões), registrar, detalhe, editar, criar demanda, ignorar, restaurar (contratos JSON, aviso que acompanha a navegação) e menu (item, contador, mapa _NAV_BY_URL_NAME)
intake/test_inativa.py	6	Caixa de Entrada desligada (INTAKE_ENABLED=False , o padrão): toda rota dá 404 mesmo para quem tem todas as permissões (página e Ajax, GET e POST), nada é gravado, o menu não mostra "Entrada" nem conta itens (nem consulta new_count), e ligar a chave traz tudo de volta. Os demais testes do app rodam com a chave ligada (IntakeTestCase)
acessos/test_chaves_demanda.py	5	Migração acessos/0003_chaves_demanda ("Atividade" → "Demanda"): o catálogo usa as chaves novas, a renomeação mantém os mesmos registros e todas as concessões (diretas e por perfil), reverte, é idempotente e funde quando seed_acoes novo rodou antes da migração
notifications/test_demanda_mensagens.py	6	Migração notifications/0008 : troca só o texto fixo que o sistema gravou (títulos conhecidos e trechos ancorados da mensagem), nunca o título digitado, o motivo, comentários ou menções; idempotente
activities/test_demanda_endereco_e_codigo.py	27	Endereço e código da Demanda: as páginas em /demandas/... ; toda rota com o endereço antigo /atividades/... redirecionando (301; 308 em POST; consulta mantida; sem login; nunca leva para fora do site); anexos (/atividade-arquivos/ → /demanda-arquivos/ , pasta DEM- ou ATV- , caminho que sobe de pasta); o código DEM-AAAA-NNNN e a busca com o prefixo antigo (lista e seletor de tarefa); migração activities/0020 (códigos, pastas e nomes dos anexos; texto digitado intacto; idempotente; reversível; código já ocupado; arquivo ausente ou que não pôde ser movido)
activities/test_kanban.py	87	Quadro Kanban de demandas e de tarefas: setor obrigatório e lembrado, setor que a pessoa não pode abrir ignorado, visibilidade (participante do setor x quem só atua), colunas só do setor, "Sem etapa" só quando preciso, limite que só avisa, filtros e ordenação (prazo vazio por último), valores inválidos ignorados, consultas que não crescem com os cartões , arrastar/seletor gravam só etapa ou condição (status, fila e cronômetro intactos), recusas (outro setor, inativa, sem ação, outra organização, tipo errado), criar na coluna, limite e novas opções só para quem gere, endpoint do cartão, gaveta da demanda e da tarefa, janela de nova tarefa já com setor e etapa

Arquivo	Testes (def test_)	Foco
intake/test_codigo_demand a.py	3	Migração intake/0003 : só a nota que é exatamente um código (evento "Virou demanda") troca ATV- por DEM- ; texto em volta e outros eventos ficam; idempotente e reversível
accounts , audit	0	tests.py vazio

Base compartilhada dos quadros: boards/testing.py (BoardTestCase , montado uma vez por classe em setUpTestData : a Biasi com o quadro Orçamentos de dois grupos, uma coluna de cada tipo ativo, três itens e pessoas com cada nível de acesso: tudo, só preenche itens, só vê, sem acesso e de outra organização; set_cell(item, "texto", valor)).

tests/boards-kanban.test.cjs (29 testes, jsdom) cobre o modo Kanban do boards.js sobre o HTML real de board_kanban.html (raias montadas por boards.kanban.build_lanes): arrastar cartão (otimista, contagens, volta se recusado, raia em branco, mesma raia não envia nada), menu do cartão, renomear título, editar campo no cartão (inclusive o campo que **não é <td>**) e redistribuir quando muda o agrupador, criar na raia com valor inicial, renomear/recolorir/editar etiquetas pela raia, agrupar/ordenar pela barra, janela de configuração com autosave e pré-visualização, abas (criar, renomear, excluir) e perfil só de leitura.

tests/activity-inline-edit.test.cjs (57 testes, jsdom) cobre static/js/activity-inline-edit.js e activity-inline-options.js sobre o HTML real de boards/demand_work_board.html (bloco content renderizado com objetos em memória): marcadores só onde a linha permite, nada acrescentado em repouso (sem seta, lápis ou botão), título (Enter/blur/Esc, vazio nunca salva, CSRF, otimista, recusa e queda de conexão restauram, texto como texto e não HTML), **volta ao último valor confirmado** e não ao HTML inicial, **uma requisição por célula**, responsável (lista, busca com debounce, resposta velha ignorada, mesma pessoa não envia), prazo (data+hora opcional, Limpar, atraso vindo do servidor, recusa restaura), pop-over único, clique fora/Esc e foco de volta, acompanha scroll/resize e teclado. **Entrega 2:** link de Cliente / Obra (só onde a linha permite, sem data-activity-navigate), setor (lista colorida, atual marcado, trocar com Estágio/Status travados e redesenhados, recusa volta ao confirmado, busca sem acento, setas/Esc, criar e aplicar com nome + 36 cores, erros no próprio pop-over), estágio e status (escolher, limpar, criar e aplicar, editar nome/cor com propagação a outras linhas, link de gerir só do próprio site) e a linha "Vencida há N dias" (vinda do servidor e atualizada pelo prazo).

tests/boards-calendar.test.cjs (59 testes, jsdom) cobre o modo Calendário do boards.js sobre o HTML real de board_calendar.html (grade montada por boards.calendar_view.build_month , sem banco): funções puras da hora (parseTimeInput etc.), grade e cartões (cor, texto da etiqueta, "Atrasada", "+ N mais", dias bloqueados), controles por permissão, **criar no dia** (data preenchida, tudo em um pedido, título e hora validados antes de enviar, cancelar não manda nada, recusa mantém a janela, busca de pessoa, grupo), **arrastar entre dias** (otimista, só a data, mantém a hora, volta e avisa se recusado, dia bloqueado e mesmo dia não enviam, dia cheio recontado, leitor não arrasta), **navegar** (sem recarregar, só o novo mês, busca e pessoa mantidas, ctrl-clique é do navegador, falha mantém o mês), listas em pop-over, gaveta (abrir, fechar, Escape em duas etapas, editar campo, renomear, excluir com confirmação, leitor só lê), pop-over de data e hora (rápidos, digitado, limpar horário/data, ativar horário), janela de configuração com autosave e recusa, aviso de fim de semana oculto, quadro sem coluna de Data e a aba "+" criando um Calendário.

tests/boards.test.cjs (61 testes, jsdom) cobre boards.js sobre o **HTML dos templates reais** (board_detail.html renderizado com objetos em memória, sem banco; mesmo método do activity-steps.test.cjs) e um fetch falso: o que a tela manda (URL, corpo, cabeçalhos, CSRF) e o que faz com a resposta, otimismo com reversão, mensagens de erro, permissões visuais, arraste de item e de coluna (inclusive desfazer quando o servidor recusa), teclado, janelas e pop-overs, a **linha de títulos de cada grupo** (criar, renomear, mover e excluir coluna atuam em todos os grupos; o campo de renomear abre no grupo onde se clicou) e a camada da paleta de cores (z-index do pop-over acima da janela, lido do CSS). Roda como os demais (node --test tests/boards.test.cjs , com NODE_PATH apontando para o jsdom).

Base compartilhada da Caixa de Entrada: intake/testing.py (IntakeTestCase , com a Biasi, os setores Comercial e Compras, o cliente Convivy com a obra Residencial Aurora e pessoas com cada nível de acesso: triagem geral, só registra, gestor de um setor, sem acesso e de outra organização; new_item(**overrides) cria a solicitação direto no banco).

Base compartilhada dos testes de processo: activities/testing.py (make_user(username, organization, actions=()) e ProcessTestCase , com o cenário Orçamento v3: três setores/pessoas, três etapas em sequência, três inputs e quatro critérios) e processes/testing.py (build_process , que monta um molde direto pelo ORM, e publish_version(version, user) , que publica uma versão). Os testes que ainda usam ProcessTestCase (test_reopen.py , test_task_actions.py ,

`test_process_application.py`) exercitam serviços; os que chamam rotas de tarefa, fila ou processo pelo `client` esbarram nas respostas 410 desde 02/10/2026 e precisam ser revistos (não foram executados nesta revisão: o Django não estava instalado no ambiente).

Além disso, `tests/checklist.test.cjs` testa o componente JavaScript do checklist, `tests/process-apply.test.cjs` , o assistente de 4 passos do popup "Aplicar processo", `tests/retroactive-work.test.cjs` , o popup "Já realizei este trabalho" (aviso de dia anterior e comentário só quando necessário), e `tests/activity-steps.test.cjs` (51 testes), a janela de demanda em 4 etapas — criar e editar (inclui o diálogo de troca de quadro: só abre ao editar com escolha diferente, Confirmar reenvia com `confirm_board_replace=1` , Cancelar/Esc/clique fora voltam ao passo 3 sem enviar, foco preso, texto como texto — e a integração com o `modal.js` real) (seção 4). `tests/task-modal.test.cjs` (8 testes) cobre o cartão da atividade da janela de tarefa (trocar o seletor pelo cartão, "Alterar", seleção limpa, início repetido e janela injetada depois), inclusive com o `person-picker.js` real. `tests/kanban.test.cjs` (41 testes) cobre `kanban.js` e `workflow-picker.js` num fixture que espelha os atributos dos templates: arrastar (grava só a etapa, volta se o servidor recusa, "Sem etapa" nunca é destino), contadores e limite, criar na coluna, limite, gaveta e releitura do cartão ao fechar, menus, ações, filtros e o seletor (opções do setor, escolher, limpar condição, criar opção, erros). Uma exceção dentro de um tratador de evento falha o teste. `tests/intake.test.cjs` (12 testes) cobre `static/js/intake.js` com o `LPSAjax` real: abrir em janela, teclas modificadoras, queda para a página completa, remover/trocar cartão, redirecionar, recarregar e o envio de "Restaurar" (CSRF, envio duplo e erros).

Testes JavaScript (`tests/*.test.cjs`)

Executados com `node --test` (Node 18+) e `jsdom 26.1.0`: **378 testes em 18 arquivos**. Os marcados com `*` chamam o Python do projeto para renderizar o template Django real.

Arquivo	Testes	Cobre
<code>tests/activity-inline-edit.test.cjs *</code>	57	<code>activity-inline-edit.js</code> e <code>activity-inline-options.js</code> (lista de Demandas)
<code>tests/activity-steps.test.cjs *</code>	51	<code>activity-steps.js</code> : janela de demanda em 4 etapas e diálogo de troca de quadro
<code>tests/activity-workspace.test.cjs</code>	8	<code>activity-workspace.js</code> (<code>LPSAjax</code> , <code>data-activity-action</code>)
<code>tests/boards-calendar.test.cjs *</code>	59	modo Calendário do <code>boards.js</code>
<code>tests/boards-kanban.test.cjs *</code>	29	modo Kanban do <code>boards.js</code>
<code>tests/boards.test.cjs *</code>	61	tabela de Quadros do <code>boards.js</code>
<code>tests/checklist.test.cjs *</code>	10	<code>checklist.js</code> (template <code>_task_checklist.html</code> ; nenhuma tela ativa carrega o script hoje)
<code>tests/intake.test.cjs</code>	12	<code>intake.js</code>
<code>tests/kanban.test.cjs</code>	41	<code>kanban.js</code> e <code>workflow-picker.js</code>
<code>tests/modal.test.cjs</code>	5	<code>modal.js</code>
<code>tests/notifications.test.cjs</code>	1	<code>notifications.js</code>
<code>tests/process-apply.test.cjs *</code>	14	<code>process-apply.js</code> (fluxo desativado)
<code>tests/retroactive-work.test.cjs *</code>	8	<code>retroactive-work.js</code>
<code>tests/sector-picker.test.cjs</code>	1	seletor de setor (<code>person-picker.js</code> , <code>data-picker-kind="sector"</code>)
<code>tests/task-center.test.cjs</code>	7	<code>task-center.js</code> (tela Tarefas)
<code>tests/task-modal.test.cjs</code>	8	<code>task-modal.js</code> com o <code>person-picker.js</code> real
<code>tests/work-board.test.cjs</code>	2	<code>work-board.js</code> (visualizações de domínio)
<code>tests/workspace-filters.test.cjs</code>	4	<code>workspace-filters.js</code> (barra de filtros compartilhada)

2. Rodando

```
python manage.py test                                # tudo
python manage.py test activities                      # um app
python manage.py test activities.test_views          # um módulo
python manage.py test activities.test_views.TaskCheckListViewTests # uma classe
python manage.py test core.tests.UserFormAjaxTests.test_non_ajax_request_still_redirects # um teste
```

Testes JavaScript (Node 18+; o jsdom está em `devDependencies` do `package.json`):

```
npm install                # uma vez: instala o jsdom em node_modules/
npm run test:js            # = node --test tests/*.test.cjs
node --test tests/boards.test.cjs # um arquivo
```

Rode a partir da raiz (`workflow`). Os testes marcados com `*` na tabela chamam o Python do `.venv` (ou o da variável `PYTHON`) para renderizar o template real; os demais montam o HTML à mão. O `package.json` existe só para os testes — o deploy no Render continua sem etapa Node.

Opções úteis: `-v 2` (nome de cada teste), `--parallel` (mais rápido), `--keepdb` (reaproveita o banco de teste entre execuções).

Os testes usam `config.settings.dev` (via `manage.py`) com um banco SQLite em memória criado e destruído a cada execução — o seu `db.sqlite3` não é tocado. A suíte completa leva alguns minutos, porque todas as migrations rodam no início. Grande parte do resto é o hash de senha: cada `create_user(..., password=...)` usa PBKDF2 (centenas de milissegundos). Para iterar mais rápido **localmente**, rode com um módulo de settings que herda de `config.settings.dev` e troca só `PASSWORD_HASHERS` por `["django.contrib.auth.hashers.MD5PasswordHasher"]` (`--settings=meu_settings_de_teste`); testes novos que não precisam de senha podem criar o usuário sem ela (`create_user(username, email=...)`) e usar `client.force_login` . O `--parallel` não funciona no Windows deste projeto (erro `cannot pickle 'traceback' object` ao reportar uma falha).

Formulário de usuário

Os testes de criação devem enviar organização, `password1` e `password2` , assim como a interface atual. Os testes do editor de demanda estão também em `activities/test_activity_workspace.py` (editor, anexos e navegação).

3. Escrevendo testes

O padrão é criar organização, usuários e permissões explicitamente no `setUp` :

```
from django.contrib.auth import get_user_model
from django.test import TestCase

from acessos import catalog
from acessos.testing import grant_action
from core.models import Organization, Sector

User = get_user_model()

class MinhaFeatureTests(TestCase):
    def setUp(self):
        self.org = Organization.objects.create(name="Biasi")
        self.sector = Sector.objects.create(organization=self.org, name="Compras")
        self.user = User.objects.create_user("ana", email="ana@example.com", password="x")
        self.user.profile.organization = self.org
        self.user.profile.save(update_fields=["organization"])
        grant_action(self.user, catalog.TAREFA_CRIAR, organization=self.org)
```

Helpers de `acessos/testing.py` :

Helper	Uso
<code>ensure_catalog()</code>	Grava o catálogo de ações no banco de teste
<code>grant_action(user, key, organization=None, scope=None, sector=None, relation=None)</code>	Concessão direta; escopo da organização por padrão, ou de setor/relação
<code>grant_actions(user, keys, **kwargs)</code>	Várias de uma vez
<code>make_profile(organization, name, keys)</code>	Cria um perfil de acesso com as ações
<code>assign_profile(user, profile, sector=None, relation=None)</code>	Atribui o perfil com escopo

Outras bases por app: `activities/testing.py` (`make_user` , `ProcessTestCase`), `boards/testing.py` (`BoardTestCase`), `intake/testing.py` (`IntakeTestCase`) e `processes/testing.py` (`build_process` , `publish_version`).

Dicas:

- Para testar **negação**, crie o usuário sem conceder a ação e confira que o serviço levanta `ActivityError` (ou a view responde 403).
- Para testar **isolamento**, crie uma segunda organização e confira que o recurso dela não aparece / dá 404.
- Para views com login: `self.client.force_login(self.user)` .
- Views Ajax: envie `HTTP_X_REQUESTED_WITH="XMLHttpRequest"` .
- Um usuário sem organização (`User.objects.create_user(...)` sem ajustar o perfil) serve para testar o redirecionamento do `OrganizationRequiredMixin` .

4. Teste JavaScript do checklist

`tests/checklist.test.cjs` renderiza o template real `activities/_task_checklist.html` (chamando o Python do `.venv` , ou o da variável `PYTHON`) e executa `static/js/checklist.js` no jsdom. Com o jsdom instalado pelo `npm install` (seção 2), basta, a partir da raiz do repositório:

```
node --test tests/checklist.test.cjs
```

Sem `npm install`, ainda é possível instalar o jsdom numa pasta temporária e apontar o `NODE_PATH` para ela (método descrito em `tests/README.md`). O jsdom não valida layout nem o envio por Enter do navegador — confira também manualmente (ver `tests/README.md`). Observação: `checklist.js` e o parcial `_task_checklist.html` continuam no repositório, mas nenhuma tela ativa os carrega desde que as tarefas passaram a viver nos quadros por Demanda; o teste cobre o componente isolado.

12 — Deploy no Render

A aplicação é publicada no Render pelo Blueprint `render.yaml`, no plano free: um Web Service (`lps-web`) e um PostgreSQL (`lps-db`). Este documento explica o arquivo, o primeiro deploy, as limitações do plano gratuito e o histórico de correções de deploy. Atualizado em 03/10/2026 (código no commit `add5acd`).

1. render.yaml

Chave	Valor	Por quê
<code>databases[0]</code>	<code>lps-db</code> , free, banco <code>lps</code> , usuário <code>lps</code>	Postgres gerenciado
<code>services[0].type / runtime / plan</code>	<code>web / python / free</code>	
<code>buildCommand</code>	<code>pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate && python manage.py seed_acoes && python manage.py ensure_superuser</code>	Instala, coleta estáticos para o whitenoise, aplica migrations, sincroniza o catálogo de ações (idempotente: uma ação nova do código passa a existir em produção sem precisar de Shell) e cria o superusuário (se configurado)
<code>startCommand</code>	<code>gunicorn config.wsgi:application</code>	<code>wsgi.py</code> já usa <code>config.settings.prod</code>
<code>DJANGO_SETTINGS_MODULE</code>	<code>config.settings.prod</code>	Necessário porque o build usa <code>manage.py</code> , que sozinho usaria dev
<code>SECRET_KEY</code>	<code>generateValue: true</code>	O Render gera um valor aleatório no primeiro deploy
<code>ALLOWED_HOSTS</code>	<code>.onrender.com</code>	Qualquer subdomínio do Render
<code>CSRF_TRUSTED_ORIGINS</code>	<code>https://*.onrender.com</code>	Formulários sob HTTPS
<code>DATABASE_URL</code>	<code>fromDatabase: lps-db</code>	String de conexão injetada pelo Render
<code>PYTHON_VERSION</code>	<code>3.12.7</code>	Versão do Python no build
<code>DJANGO_SUPERUSER_USERNAME /EMAIL/PASSWORD</code>	<code>sync: false</code>	Preenchidos à mão no dashboard; nunca vão para o Git

O Blueprint **não** define `INTAKE_ENABLED` (a Caixa de Entrada fica inativa; para ligar, crie a variável em Environment — ver [03_CONFIGURACAO.md](#)) nem as variáveis `EMAIL_*` (seção 3). O Render injeta `RENDER_EXTERNAL_HOSTNAME`, que `prod.py` acrescenta a `CSRF_TRUSTED_ORIGINS`. O Blueprint também não tem Cron Job (ver seção 4).

As configurações de produção que tornam isso possível estão em `config/settings/prod.py`: `whitenoise` com `CompressedManifestStaticFilesStorage` e `SECURE_PROXY_SSL_HEADER` (o Render termina o TLS no proxy; sem essa linha, `SECURE_SSL_REDIRECT` entra em loop).

2. Primeiro deploy

- Render → **New +** → **Blueprint** → conecte o repositório `Lucasrafael703/workflow`. O Render lê o `render.yaml` e cria banco e serviço.
- No serviço `lps-web` → **Environment**, preencha `DJANGO_SUPERUSER_USERNAME`, `DJANGO_SUPERUSER_EMAIL` (obrigatório na prática: o login é por e-mail) e `DJANGO_SUPERUSER_PASSWORD`. Salvar dispara novo deploy, e o build cria o usuário.
- Entre em `https://<serviço>.onrender.com/admin/` com esse e-mail e senha.
- O banco de produção começa **vazio de dados de negócio**. Pelo Admin:

1. crie a **Organização**;
2. em **Accounts → Perfis**, vincule seu usuário a ela. Os quadros padrão de Demandas e Tarefas são criados na primeira vez que a tela é aberta; não há `seed_default_boards` no build (ver [02_AMBIENTE_LOCAL.md](#), seção 6).
5. O build roda `seed_acoes`, então o catálogo de ações é criado/atualizado a cada deploy. Ele **não** cria perfis de acesso nem os atribui: para usuários comuns, monte os perfis em **/permissões/** (o superusuário funciona porque ignora o motor) ou, só numa organização de demonstração, use `seed_lps_demo --org-name "..."`. Ação nova do catálogo (ex.: `tarefa.reabrir`) aparece em `/permissões/`, mas **não** entra sozinha em perfis já existentes: marque-a nos perfis que devem ter.

Deploys seguintes acontecem a cada push na branch `main`.

Cada deploy roda `migrate` no PostgreSQL. As migrations de dados mais sensíveis do histórico recente: `core/0012_sector_scoped_stages_conditions` (etapas e status por setor), `activities/0020_codigo_da_demanda` (renomeia `ATV-...` para `DEM-...` e as pastas de anexos) e `boards/0008_centralize_demand_board_items` (converte as tarefas operacionais em itens de quadro e **cancela** as `Task` operacionais; o histórico é preservado). Teste migrations de dados num PostgreSQL com dados antes de publicar: o SQLite é mais tolerante (seção 6).

3. E-mail em produção

Nenhuma variável `EMAIL_*` está no `render.yaml`, então produção usa o backend SMTP com host vazio. Enquanto não configurar (`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `DEFAULT_FROM_EMAIL` no dashboard):

- **Criar conta** dá erro 500: o código de confirmação é enviado por `send_mail` sem tratamento de erro (`accounts/views.py`), depois que o usuário inativo já foi gravado;
- **Esqueci minha senha** mostra a tela de "enviado", mas o e-mail não sai (o Django só registra a falha no log);
- os e-mails de notificação (`EmailService`) também só registram a falha no log, sem quebrar a tela.

4. Limitações do plano free

Limitação	Efeito na LPS
Serviço hiberna após ~15 min sem acesso	Primeira requisição depois disso demora para responder
Postgres free expira (é apagado) após o período de gratuidade	Será preciso recriar o banco e rodar as migrations de novo
Disco não persistente	Anexos (<code>ACTIVITY_FILES_ROOT</code>) e mídia enviados somem a cada deploy/reinício
Sem Shell (recurso pago)	Não dá para rodar <code>manage.py</code> manualmente — por isso o superusuário é criado no build
Sem Cron Job gratuito	<code>check_overdue_tasks</code> não roda sozinho; avisos de atraso não são disparados (com o fluxo operacional de Tarefas desativado, o comando hoje tende a não encontrar nada)

5. Python local × Render

O desenvolvimento local usa Python 3.13 ou 3.14 (o `.venv` atual é 3.13); o Render, 3.12.7 (fixado em `PYTHON_VERSION`). As versões em `requirements.txt` (Django 5.2.17, `django-environ`, `nh3`, `gunicorn`, `whitenoise`, `psycopyg[binary]=3.2.13` etc.) foram escolhidas para ter wheels nas duas (a `psycopyg` 3.2.3 não tinha wheel para 3.14). Ao atualizar dependências, confira se a versão nova tem wheel para as duas versões de Python — ou alinhe `PYTHON_VERSION` com a versão local.

6. Histórico de correções de deploy

Data	Commit	Problema → correção
26/09/2026	862338e	Preparação do deploy gratuito: <code>gunicorn</code> , <code>whitenoise</code> e <code>psycopg</code> no <code>requirements.txt</code> , storage de manifest só em produção, <code>render.yaml</code> (Web Service + Postgres; a versão inicial trazia um Cron Job).
26/09/2026	79d7ff7	Loop infinito de redirecionamento atrás do proxy TLS → <code>SECURE_PROXY_SSL_HEADER</code> ; formulários rejeitados por CSRF → <code>CSRF_TRUSTED_ORIGINS</code> por variável (<code>https://*.onrender.com</code>); <code>/media/</code> e anexos davam 404 com <code>DEBUG=False</code> (o helper <code>static()</code> não gera rota nesse caso) → rotas explícitas; <code>ACTIVITY_FILES_ROOT</code> passou a ficar dentro do projeto; <code>psycopg</code> 3.2.3 → 3.2.13 (wheel para Python 3.14). Depois, em 01/10, os anexos deixaram de ter rota pública (ver 03_CONFIGURACAO.md §4).
26/09/2026	30dc7f2	Blueprint recusado: o Render não tem Cron Job no plano free → removido (<code>check_overdue_tasks</code> fica para execução manual/agendada fora do Render).
26/09/2026	a65a49d	Sem Shell no plano free para <code>createsuperuser</code> → comando <code>ensure_superuser</code> no fim do <code>buildCommand</code> , idempotente, com as três variáveis <code>DJANGO_SUPERUSER_*</code> .
30/09/2026	0c55d22	<code>seed_acoes</code> entra no <code>buildCommand</code> : ações novas do catálogo passam a existir em produção sem Shell.
01/10/2026	901d67e	<code>prod.py</code> passa a incluir <code>RENDER_EXTERNAL_HOSTNAME</code> em <code>CSRF_TRUSTED_ORIGINS</code> , para o login não ser rejeitado se a variável do Blueprint estiver dessincronizada com o hostname do serviço.
02/10/2026	cdff812	Todo deploy desde <code>a756be2</code> falhava no <code>migrate</code>: <code>core/0012</code> gravava linhas com FK e, na mesma transação, fazia <code>AlterField</code> / <code>AddConstraint</code> nas mesmas tabelas; no PostgreSQL com dados isso dava "cannot ALTER TABLE core_taskstage because it has pending trigger events" (no SQLite e em banco vazio passa). Correção: ao fim da função de dados, só no PostgreSQL, <code>SET CONSTRAINTS ALL IMMEDIATE</code> e depois <code>DEFERRED</code> . Bancos que já tinham aplicado a 0012 não rodam de novo. Até a correção, o Kanban por setor, os quadros dinâmicos, o Calendário e a tela de Demandas não tinham subido em produção.

13 — Pendências conhecidas

Defeitos, lacunas e dívidas técnicas encontrados ao documentar o código (29/09/2026) e revisados em 03/10/2026 contra o commit `add5acd`. Cada item foi conferido no código-fonte. A lista serve para priorizar. Ao resolver um item, remova-o daqui (os já resolvidos nesta revisão ficam riscados, com o número original, para as referências cruzadas continuarem valendo). Depois de 02/10 fila, processos e telas de tarefa estão desativados (410) e as tarefas vivem em quadros por Demanda: ver [05_AUTORIZACAO.md](#) §4.4.

Legenda de severidade: **Alta** (segurança ou quebra em produção) · **Média** (comportamento errado para o usuário) · **Baixa** (limpeza).

1. Segurança

#	Sev.	Onde	Problema
S1	Alta resolvida	<code>activities/views.py</code> (<code>ActivityAttachmentDownloadView</code>), <code>config/urls.py</code>	Anexos só saem por <code>/demandas/<pk>/anexos/<id>/download/</code> , que exige login, organização, <code>demanda.visualizar</code> e a visibilidade da mensagem. <code>/atividade-arquivos/...</code> responde 410 e <code>/media/</code> só é servido com <code>DEBUG</code> .
S2	Média	<code>acessos/catalog.py</code> × <code>views</code>	Ações de visualização ainda sem verificação: <code>auditoria.visualizar</code> (o Histórico <code>/historico/</code> é limitado só pela organização: qualquer pessoa da organização lê a auditoria de todas as demandas), <code>fila.visualizar_posicao_propria</code> e <code>tarefa.visualizar</code> (sem tela). Corrigido em parte: <code>demanda.visualizar</code> passou a ser exigida na ficha, nos anexos, na lista e na Central de tarefas, e o dono, o criador e o responsável de tarefa leem a própria demanda por relação (doc 05 §3 e §8).
S3	Média	<code>acessos/catalog.py</code> × <code>views</code>	Outras ações sem verificação: <code>usuario.criar</code> (só decide se o botão "criar pessoa" aparece; <code>/usuarios/novo/</code> exige <code>usuario.editar</code>), <code>usuario.inativar</code> , <code>setor.criar</code> (o cadastro e a criação de setor na lista de Demandas usam <code>setor.editar</code>) e <code>estagio_tarefa.gerir</code> (só liga o menu Cadastros). Outras só são verificadas em serviços sem tela, desde 02/10: <code>tarefa.*</code> , <code>fila.*</code> , <code>prazo.*</code> , <code>processo.*</code> , <code>tempo.lancar_manual</code> ; conceder essas ações não muda nada para o usuário. Lista completa em doc 05 §8.
S4	Média resolvida	<code>activities/views.py</code> , <code>activities/kanban.py</code>	Mover card no Kanban agora passa por <code>ActivityService.set_stage</code> / <code>TaskService.set_stage</code> : exige <code>demanda.definir_etapa</code> / <code>tarefa.definir_etapa</code> , valida setor e organização e audita a mudança. (As rotas antigas <code>mover-estagio/</code> usam o mesmo serviço.)

2. Defeitos

#	Sev.	Onde	Problema
D1	Alta resolvida	<code>acessos/services.py</code> , <code>ResourceContext.of</code>	O ramo <code>site</code> agora usa só <code>organization_id</code> e <code>site_id</code> (a obra não tem mais <code>company</code>); verificar uma obra como recurso não levanta mais <code>AttributeError</code> .
D2	Alta (prod)	<code>accounts/views.py</code> , <code>_send_verification_email</code>	<code>send_mail</code> sem tratamento de erro. Sem SMTP configurado, "Criar conta" dá 500 depois de já ter gravado o usuário inativo.
D3	Média resolvida	<code>activities/services.py</code> , <code>return_task</code>	<code>move_to_sector(..., keep_status=True)</code> mantém DEVOLVIDA na fila do setor de destino. (Sem tela desde 02/10: devolução é rota <code>/tarefas/<pk>/devolver/</code> , em 410.)
D4	Média resolvida	<code>activities/policies.py</code> , <code>TaskTransitionPolicy._check_block</code>	Recusa bloquear tarefa concluída ou cancelada.

#	Sev.	Onde	Problema
D5	Média	activities/services.py	<code>_mark_in_progress</code> já passa a atividade de <code>ABERTA</code> para <code>EM_ANDAMENTO</code> (auditado) quando o trabalho numa tarefa começa (<code>start</code> , tempo manual, "já realizei"), mas essas rotas estão em 410: trabalhando só nos quadros a Demanda continua <code>ABERTA</code> . <code>EM_ANDAMENTO</code> só vem hoje de reabertura ou aprovação de pendência.
D6	Média	activities/services.py , resolve_pendency	Pendência sem aprovação (<code>MATERIAL</code> , <code>INFORMACOES_CLIENTE</code>) não tem como ser resolvida pela interface: <code>ActivityService.resolve_pendency</code> existe e a política o prevê, mas nenhuma rota o chama . Além disso ele grava <code>AuditLog.Action.PENDENCY_RESOLVED</code> , valor que não existe em <code>audit/models.py</code> (levantaria <code>AttributeError</code> na primeira chamada; falta o valor e uma migração em <code>audit</code>). A atividade fica <code>PENDENTE</code> até ser finalizada.
D7	Baixa	activities/models.py:908 e :911 , TaskResponsavelChangeLog	Dois <code>__str__</code> ; o segundo (vence) usa <code>self.user</code> , que não existe — <code>str()</code> levanta <code>AttributeError</code> (aparece no Admin, na listagem e no histórico de mudanças). Parece ter sido escrito para <code>TaskExecutor</code> , que ficou sem <code>__str__</code> . Confirmado em teste de 03/10. Correção: apagar o segundo método.
D8	Baixa	activities/models.py , serviços	<code>Activity.Status.BLOQUEADA</code> (confirmado em teste de 03/10) e <code>Task.Status.NAO_INICIADA</code> existem, mas nenhum serviço os grava: o status Bloqueada de demanda não é atingível , e o filtro/atalho "Bloqueadas" da lista e do Kanban de Demandas (<code>activities/filtering.py</code> , <code>activities/kanban.py</code> , <code>forms.py</code>) fica sempre vazio. <code>AuditLog.Action.SESSION_RESUMED</code> nunca é gravado.
D9	Média	activities/views.py (<code>filtered_activities_queryset</code> , <code>decorate_activity_cards</code> , <code>HomeView</code>), boards/domain_services.py (<code>build_cells</code>)	Coluna Tarefas da lista de Demandas mostra "0 de 0" e o Início mostra 0 tarefas (em aberto, atrasadas, bloqueadas, "Em execução agora") embora os quadros tenham tarefas: tudo isso conta <code>Task (Count("tasks"))</code> , que a <code>boards/0008</code> cancelou e que não recebe mais trabalho, e não os itens do quadro da Demanda. A ficha da Demanda e o selo "Tarefas" do menu (<code>boards.task_center.open_task_count</code>) já leem o quadro, então os números divergem entre telas.
D10	Média	templates/base.html , core/cadastros.html , activities/home.html , activities/management.html , activities/task_list.html , activities/_task_page_head.html , core/settings.html	Links para telas desativadas: o grupo Processos do menu lateral ("Meus processos", "Criar processo", <code>/processos/</code>) e o atalho de Cadastros abrem a mensagem 410; "Ver fila" na Visão do gestor , os setores de Início e Configurações e "Ver fila do setor" levam a <code>/fila/...</code> (410). O item "Filas e gargalos" do menu abre telas em branco do <code>painel (F4)</code> .
D11	Média	notifications/views.py (<code>_notification_destination</code>)	Abrir uma notificação ligada a uma tarefa redireciona para <code>/tarefas/<pk>/</code> (e <code>#prazo</code> , <code>#conversa</code> , <code>#historico</code>) e a de conflito de prazo para <code>/conflitos/<pk>/resolver/</code> : ambos 410 desde 02/10. Só as ligadas apenas à demanda abrem <code>/demandas/<pk>/</code> . 8 testes de <code>notifications</code> já falham por isso (F35).
D12	Baixa	templates/activities/history.html , HistoryView	As abas Fila , Devoluções e Tempo trabalhado do Histórico (<code>?event=fila</code> , <code>?event=devolucoes</code> , <code>?event=tempo</code>) filtram eventos que deixaram de ser gerados (<code>QUEUE_POSITION_CHANGED</code> , <code>SECTOR_MOVED</code> , <code>RETURNED</code> , <code>SESSION_*</code>): para dados novos ficam vazias (não dão 410).

#	Sev.	Onde	Problema
D13	Média	activities/services.py , templates/activities/_timeline.html	O histórico mostra códigos internos do enum: ABERTA → PENDENTE , EM_ANDAMENTO ... Os serviços gravam <code>old_value / new_value</code> com o código (COMPLETE , REOPEN , PENDENCY_* , _mark_in_progress) e a linha do tempo imprime o texto cru, sem <code>get_status_display</code> nem rótulo configurável de status.
D14	Baixa	templates/core/cadastrros.html (7 tabelas), templates/core/user_list.html:31	Cabeçalho " Etapa " onde deveria ser " Situação " (a coluna mostra Ativa/Inativa). A troca em massa do commit 33e5f25 (Situação→Etapa, Condição→Status) pegou também o rótulo de ativo/inativo.
D15	Baixa	static/css/demand-board.css (.demand-board__filters)	A barra de filtros de Demandas passa da largura a 1440 px: a grade tem 7 colunas com mínimos (260 px + 2 colunas <code>max-content</code> + 4 × 130 px), o filtro de pessoas fixa <code>min-width: 235px</code> e a quebra para menos colunas só entra em <code>max-width: 1420px</code> , abaixo de 1440. Observado em teste de 03/10; a causa no CSS é provável e não foi medida no navegador.
D16	Baixa	static/css/notification-page.css , templates/notifications/_notification_table.html	Em Notificações o texto se sobrepõe nas colunas Demanda e De / para : a grade tem 8 colunas (<code>min-width: 1150px</code>) e só os textos principais têm reticências (<code>overflow: hidden; white-space: nowrap</code>); as células de Demanda e De/para (<code>.notification-grid__person</code> , com nome e "para você") não cortam o excesso. Observado em teste de 03/10; causa provável, não medida.
D17	Baixa	templates/boards/_board_head.html , static/css/boards.css (.board-title)	O título do quadro da demanda aparece cortado: é um <code><input></code> de 24 px com <code>max-width: 640px</code> e sem quebra, então títulos longos (o quadro herda o nome da Demanda) não cabem. Observado em teste de 03/10.

3. Funcionalidades incompletas

Nota (03/10/2026): F6–F10 (processos), F12–F17 e a parte de tarefa de F11 descrevem processos, tarefas, fila e prazos **desativados** em 02/10 (rotas em 410). Valem para o código, que continua lá, mas não são alcançáveis pela interface; reavalie se algum fluxo voltar. F22–F27 só importam com a Caixa de Entrada ligada (F29).

#	Onde	Situação
F2	notifications/recipients.py , ADMIN_GROUP_NAME	A setting é lida mas não usada; <code>resolve_sector_and_admins</code> devolve só membros e gestores do setor. O help de <code>check_overdue_tasks</code> ainda fala em "administradores".
F3	Autocadastro	Conta criada em "Criar conta" nunca recebe organização pelo app; é preciso o Admin do Django. A tela <code>/usuarios/</code> só lista quem já está na organização.
F4	painel/	Vários itens do menu são telas em branco (equipe, filas e gargalos, insights, resultados, integrações...).
F5	Escalonamento de prazo	Conflitos só são registrados e notificados; o motor de escalonamento é D1 (Regras 10 §57).
F6	Processo aplicado — inputs ARQUIVO e SELECAO	O modelo não tem onde guardar o conteúdo com segurança: não há vínculo entre <code>ActivityInputValue</code> e <code>ActivityAttachment</code> (Regras 08:1280 pede que o valor referencie o mecanismo de anexos) nem lista de opções em <code>ProcessInput</code> . Hoje só se registra a confirmação de recebimento e uma observação livre. Solução: FK opcional de <code>ActivityInputValue</code> para <code>ActivityAttachment</code> (dentro de <code>activities</code> , sem ciclo de models) e <code>options</code> em <code>ProcessInput</code> .

#	Onde	Situação
F7	Processo aplicado — evidência do output	A entrega esperada e o tipo de evidência (ARQUIVO / LINK / CHECKLIST / CONFIRMACAO) aparecem na ficha, mas não há onde registrar a evidência (Regras 08:1298 prevê atividade_outputs 1:1). A conclusão não confere "output registrado" (Regras 12 §32), só os critérios obrigatórios.
F8	Processo aplicado — etapa cuja predecessora foi cancelada	Só CONCLUIDA libera a sucessora (decisão consciente; as Regras não definem o caso). Se a predecessora é cancelada, a sucessora fica DISPONIVEL fora da fila até alguém remover a dependência em "Editar tarefa" ou cancelá-la; não há aviso automático ao dono.
F9	Processo aplicado — Activity.company	Nas Regras a empresa da atividade é obrigatória (08:1204); no código é opcional. Atividade sem empresa não recebe processo (a tela explica). Tornar a empresa obrigatória ao publicar o rascunho resolveria.
F10	Processo aplicado — momento da aplicação	As Regras se contradizem: "ao criar a atividade" (02:1313, 08:2583) × "aplicar em uma atividade" (05:990, 10:309). Foi implementado o segundo (botão na ficha). O editor de "Nova atividade" ainda não oferece o processo.
F11	Reabertura	Só tarefa e atividade concluídas reabrem; cancelada não (as Regras não definem). Ao reabrir a atividade, completion_outcome continua com o valor antigo (ex.: SUCESSO) até a próxima finalização. Tarefas seguintes que voltam a esperar por causa de uma reabertura só ficam na auditoria — os responsáveis delas não são avisados.
F12	Reabertura — permissões em organizações já implantadas	seed_acoes cria tarefa.reabrir mas não a acrescenta a perfis existentes; é preciso marcá-la em /permissoes/ (doc 05 §6). demanda.reabrir continua só no Administrador.
F13	"Já realizei este trabalho" — trabalho anterior à criação da tarefa	O fim informado não pode ser anterior à criação da tarefa (a mensagem explica). Quem fez o trabalho antes de a tarefa existir não consegue registrá-lo assim; se isso atrapalhar na prática, a trava pode ser relaxada. Por desenho, first_action_at , a saída da fila e o tempo de espera na fila usam o momento do registro, não o trabalho informado.
F14	Tempo informado — correção	O cartão da tarefa separa cronometrado × informado e a gestão mostra a origem das horas por motivo (F17), mas não há política de correção de um período já registrado (Regras 04 §114).
F15	Editor da tarefa — prazo pedido	O prazo pedido pelo solicitante é editável por quem tem tarefa.editar (hoje o Gestor de Setor; o Colaborador não tem). Restringi-lo ao dono da atividade / demanda.editar evitaria que a própria equipe mexa no prazo contra o qual é medida (Regras 04 §64), mas tiraria do gestor do setor a correção de um erro de digitação. Decisão de produto em aberto.
F16	Convites de participante	Um convite pendente (TaskAssignment) não pode ser cancelado por quem convidou — só aceito ou recusado pela pessoa convidada. O editor mostra o convite como "aguardando aceite" e não o duplica.
F17	Origem do tempo — alcance	O quadro "Origem do tempo registrado" da gestão segue os filtros da tela (período da atividade, cliente, responsável...), como os demais indicadores, e conta só sessões encerradas. Não há ainda tela de auditoria do tempo informado por pessoa nem correção de um período já registrado (Regras 04 §114).
F18	Editor de atividade — campos que saíram	A janela (hoje de 4 etapas, F36) não tem marcadores, solicitante interno (requested_by), anotações internas (internal_notes) nem envio de arquivos. Os dados antigos continuam no banco e na ficha, mas não há mais onde editar marcadores e anotações internas de uma atividade. Se isso fizer falta, o caminho é um bloco opcional na etapa 2 ou 3.
F19	Editor de atividade — "Organização"	O campo "Organização" grava Activity.company (cadastro de empresas). Se a intenção era outra coisa (ex.: um agrupamento parecido com setor), o rótulo e a origem dos dados precisam ser revistos.
F20	Editor de atividade — limite do texto	O contador "0/2000" das observações é só orientação: não há corte nem erro acima de 2000 caracteres (descrições antigas maiores continuam editáveis).
F21	Cliente → Obra → Centro de custo	Obra sem cliente e centro de custo sem obra valem para qualquer escolha no servidor; na busca, ao escolher um cliente somem as obras sem cliente. Não há cadastro que force o vínculo.

#	Onde	Situação
F22	Caixa de Entrada — canais	A v1 só recebe solicitações registradas à mão (colar o texto do e-mail/Teams). <code>IntakeItem.source</code> (<code>EMAIL</code> , <code>TEAMS</code> , <code>FORMULARIO</code>) e <code>external_id</code> (único por origem) já existem para um canal automático (ex.: Power Automate/Outlook por um endpoint com token) entrar sem migração; esse endpoint, o formulário público e qualquer leitura de caixa de correio ainda não existem.
F23	Caixa de Entrada — aviso	Não há notificação nem e-mail quando chega uma solicitação: só o contador "Entrada" no menu. Um <code>Notification.EventType</code> próprio exigiria migração em <code>notifications</code> , ajustes em <code>EVENT_CATEGORY</code> e <code>_still_needs_action</code> , e um "quem tem a ação X no setor" que ainda não existe em <code>recipients.py</code> . Itens novos parados há dias também ainda não geram alerta.
F24	Caixa de Entrada — histórico da atividade	A atividade criada mostra a origem só na descrição (texto + link "Ver solicitação original") e a trilha fica em <code>IntakeEvent</code> . O histórico (<code>AuditLog</code>) da atividade não tem um evento "criada a partir de uma solicitação": exigiria um valor novo em <code>AuditLog.Action</code> (migração em <code>audit</code>) e uma frase em <code>ACTION_PHRASES</code> . Quem lê a descrição sem <code>entrada.visualizar</code> recebe 403 ao abrir o link.
F25	Caixa de Entrada — sugestões	As sugestões são regras locais: nomes de cliente/obra/setor cadastrados, domínio do e-mail e datas em português. Não entende "semana que vem", "fim do mês" nem "urgente", não lê anexos e depende de o cadastro de clientes e obras estar completo. Trocar por IA é reescrever <code>intake/suggestions.py::suggest</code> , o único ponto de chamada.
F26	Caixa de Entrada — visibilidade	Solicitação sem setor sugerido só é vista por quem tem escopo de organização; o gestor de um setor só vê as do seu. A lista só considera escopos de organização e de setor (<code>accessible_sector_ids</code> ignora empresa, obra e centro de custo).
F27	Caixa de Entrada — permissões	(Só importa com a Caixa ligada, F29.) <code>seed_acoes</code> cria <code>entrada.*</code> mas não as acrescenta a perfis já existentes (nem ao Administrador): marque-as em <code>/permissoes/</code> em cada organização (doc 05 §6).
F28	"Atividade" → "Demanda" — o que ainda diz "atividade"	A interface, o endereço das páginas e o código gerado foram trocados (glossário no começo do doc 01). Continuam com o nome antigo, por serem identificadores internos e não o que a pessoa vê: o código (<code>Activity</code> , <code>activities/</code> , constantes <code>ATIVIDADE_*</code> , rotas <code>activity-*</code>), as tabelas, a pasta em disco <code>atividade_arquivos/</code> (variável <code>ACTIVITY_FILES_ROOT</code> ; renomear exigiria mover os anexos a cada deploy), os domínios <code>activity / activity_status</code> nas URLs de status e os valores internos (<code>MINHAS_ATIVIDADES</code>). Redirecionamentos que mantêm links antigos (podem ser removidos quando ninguém mais os usar): <code>/atividades/...</code> , <code>/configuracoes/estagios-de-atividade/...</code> e os valores <code>?tab=status-atividade</code> , <code>?tab=estagios-atividade</code> , <code>?visao=atividade</code> , <code>?sort=atividade</code> (ver <code>core/legacy_redirects.py</code>). <code>/atividade-arquivos/...</code> já não redireciona : responde 410 ("este link de arquivo expirou"; o anexo se baixa pela Demanda). A busca por código aceita o prefixo antigo <code>ATV-</code> (<code>activities.models.code_search_term</code>). Texto digitado (título, descrição, comentários) que cite um código <code>ATV-AAAA-NNNN</code> não foi reescrito. Os documentos <code>Regras/</code> e <code>Telas/</code> (especificação de produto) também seguem com o termo antigo.
F29	Caixa de Entrada — inativa	Desde 01/10/2026 a Caixa de Entrada vem desligada (<code>INTAKE_ENABLED</code> , padrão <code>false</code>): o item "Entrada" não aparece no menu e <code>/entrada/...</code> dá 404, até a funcionalidade ganhar mais estrutura. Nada foi apagado (app, tabelas, migrações, permissões <code>entrada.*</code> , que continuam aparecendo em <code>/permissoes/</code> sem efeito). Para reativar: <code>INTAKE_ENABLED=true</code> no ambiente (Render: Environment do serviço web) e marcar <code>entrada.*</code> nos perfis (F27).
F30	Quadro Kanban por setor — decisões e limites	(1) A ordem dos cartões dentro da coluna vem do filtro <i>Ordenar</i> ; arrastar para cima ou para baixo na mesma coluna não é guardado. (2) Etapa e condição criadas, renomeadas ou limitadas (<code>column_limit</code>) não são auditadas : <code>AuditLog</code> só liga a demanda/tarefa. A mudança de etapa e de condição de um item é auditada. (3) "Sem etapa" junta demandas sem etapa e as de etapa inativada. (4) Concluídas e canceladas ficam fora do quadro até <i>Incluir concluídas e canceladas</i> . (5) Tarefa não é criada na coluna: precisa de uma demanda, então o "+" abre a janela <i>Nova tarefa</i> já com setor e etapa. (6) Quem não participa do setor e não tem <code>demanda.visualizar_todas / tarefa.visualizar</code> nele só vê o que atua. (7) As abas de escopo (minhas, do setor, participando) saem do quadro: o recorte vem do setor + <i>Pessoa</i> . (8) O Kanban de Tarefas (<code>/tarefas/kanban/</code>) foi substituído em 02/10 pela Central de tarefas sobre itens de quadro (F35); o código antigo vive em <code>/tarefas/kanban-legado/</code> , que responde 410. O Kanban de Demandas segue valendo.

#	Onde	Situação
F31	Quadros dinâmicos — o que ficou de fora da primeira versão	(1) Filtros por valor de coluna e "Agrupar por" aparecem desabilitados ("em breve"); hoje há busca, filtro por pessoa e ordenar por coluna (todas pelo servidor). (2) Uma célula de Pessoa guarda uma pessoa (<code>multiple</code> recusado); a tabela <code>BoardCellUser</code> já comporta várias. (3) Tipos reservados no enum e ainda sem tela: <code>FILE</code> (upload exige o armazenamento de anexos, hoje efêmero no plano free, P3), <code>TIMELINE</code>, <code>PRIORITY</code>, <code>CONFIRMATION</code>, <code>RELATION</code>, <code>FORMULA</code>, <code>AI_EXTRACT</code>; sem automações nem totais por coluna. (4) As permissões <code>quadro.*</code> valem só para a organização inteira (o endereço do quadro não tem setor): um escopo de setor não casa (doc 05 §4.3). Exceção: nos quadros de Demanda, dono, criador e pessoas nas células Pessoa acessam por relação (<code>DemandBoardAccess</code>). (5) Quem tem <code>quadro.editar_item</code> pode mover itens entre grupos; não há permissão separada para isso. (6) Reordenar etiquetas é por setas (sem arrastar) e o histórico do quadro é uma página simples (50 por página, sem filtro). (7) Excluir coluna/item/grupo/etiqueta é lógico e não há tela de "lixeira" para restaurar. (8) A migração <code>acessos/0005</code> concede por mapeamento aos perfis existentes (ver doc 04 §7.2): confira em <code>/permissoes/</code> se o perfil de cada gestor ficou como esperado. (9) Edição simultânea: vale a última gravação por célula; quem tem a tela aberta só vê a mudança de outra pessoa ao recarregar (sem tempo real). (11) Kanban: agrupa só por Status e Lista suspensa (Pessoa, Prioridade etc. ficam para depois); arrastar dentro da mesma raia não guarda a ordem (a ordem vem da configuração: ordem do quadro, título, criação ou uma coluna); sem gaveta de detalhes (clicar no título renomeia no lugar); as visualizações são compartilhadas (configurar exige <code>quadro.editar</code>, não há visão pessoal nem filtros salvos por visualização); a lista de tipos do "+" mostra Linha do tempo e Dashboard como "em breve" (o Calendário já existe: ver F32); o resumo da raia é só a soma de uma coluna (sem média nem contagem por campo); sem tempo real entre pessoas. (10) Os itens de exemplo do modelo Orçamentos só entram por <code>python manage.py seed_board_templates --exemplos</code>; não rodam no deploy.
F32	Calendário dos quadros — decisões e limites	(1) Só a escala Mês: Semana, Dia e Agenda aparecem desabilitadas ("em breve"); a configuração já guarda <code>period</code>. (2) Filtros por valor de coluna continuam "em breve" (como na tabela e no Kanban): o motor de filtros do quadro ainda não existe e o Calendário não terá um próprio; hoje há busca e Pessoa. (3) Um item ocupa um dia (sem intervalo, recorrência, dependências, Gantt nem sincronização com Google/Outlook: fora da primeira versão pela especificação). (4) Arrastar da lista "Sem data" para um dia não existe: abre-se o item e define-se a data (aparece na hora). (5) A data se edita num pop-over próprio (data nativa + hora opcional); a hora só vale se a coluna tiver "Exibir horário" (quem gere colunas liga no próprio pop-over). (6) As visualizações são compartilhadas (configurar exige <code>quadro.editar</code>; sem visão pessoal). (7) A gaveta do item mostra todos os campos e as últimas 8 mudanças; comentários, anexos e checklist não existem nos quadros ainda. (8) "Concluído" é decidido pela primeira coluna de Status; "Atrasado" só em coluna marcada como prazo. (9) Edição simultânea: vale a última gravação por célula; o cartão só volta (com aviso) se o servidor recusar, e não há tempo real. (10) Não há alternativa de teclado para arrastar: Enter abre a gaveta, onde a data se muda no pop-over. (11) O calendário de Demandas e Tarefas (<code>work_board.html</code>, lente do app <code>activities</code>) é outra tela e não foi alterado; o Calendário dos quadros só existe em <code>/quadros/</code>.
F33	Edição inline da lista de Demandas (Entrega 1: título, responsável, prazo) — decisões e dívidas	(1) A tela ativa é <code>boards/demand_work_board.html</code>; a lista antiga só existe em <code>/demandas/lista-legada/</code>. (2) O inline é um adapter (<code>activities/inline_edit.py</code>, <code>inline_views.py</code>): a regra vive no <code>ActivityService</code>. Novos invariantes do Service (valem para todas as telas): título normalizado (strip), não vazio e ≤ 200; dono ativo da mesma organização. (3) <code>ActivityPermissionError</code> (subclasse de <code>ActivityError</code>) distingue permissão (403) de regra (400) sem mudar quem já captura <code>ActivityError</code>. (4) Prazo só com data é gravado 23:59 (como o formulário) e a lista mostra só a data quando a hora é 23:59; dívida técnica: sem migração não dá para diferenciar o 23:59 automático de um 23:59 escolhido de propósito (hoje os dois aparecem como "sem hora" e o editor abre com a hora vazia). Vale só para a tabela de Demandas. (5) Demanda concluída/cancelada fica travada na lista (o Service já recusa edição e troca de dono). (6) Sucesso é silencioso; erro mostra aviso curto. Última gravação vence (sem tempo real nem controle de versão por célula). (7) A condição manual chamada "Vencida" (criada à mão no setor SST) contradiz o prazo e deve ser aposentada: a Entrega 2 passa a mostrar o atraso derivado do prazo ("Vencida há N dias") na coluna STATUS. (8) Cliente/Obra, Setor, Estágio e Status (Condição) foram feitos na Entrega 2 (ver F34); ficam para a Entrega 3 as Tarefas expansíveis.

#	Onde	Situação
F36	Editar demanda = Nova demanda, com troca de quadro e confirmação — decisões e limites	(1) Editar e criar são a mesma janela : 4 etapas, mesmos botões e textos (o “Salvar alterações em qualquer etapa” da edição foi retirado). (2) Trocar o quadro exclui de vez as tarefas, colunas, grupos e visões do quadro atual e recria o conteúdo no mesmo Board (mesmo endereço, mesmo pk): <code>BoardInstantiationService.replace_for_activity</code> , na mesma transação da edição (falhou, nada muda). Não há “desfazer”; o histórico guarda só a contagem (<code>AuditLog BOARD_UPDATED</code> , <code>items_deleted</code>). Itens já “excluídos” (<code>is_active=False</code>) também são apagados e não entram na contagem mostrada. (3) A confirmação existe no navegador (diálogo) e no servidor (<code>confirm_board_replace</code> , estrito; sem ele, erro no passo 3 e nada é gravado). Pedido que omite a escolha do quadro mantém o quadro atual. (4) Mesma escolha = nada; demanda sem quadro (as 4 antigas do banco local) ganha um sem confirmação. (5) Só troca quem gere a estrutura (<code>can_manage_structure</code> : responsável, criador ou <code>quadro.gerir_colunas</code>); os demais veem o passo 3 somente leitura. (6) Trocar para um modelo copia também as tarefas do modelo (decisão anterior da Nova demanda). (7) Sem migração.
F35	Painel Tarefas e prévia do Quadro na ficha da Demanda — decisões e limites	(1) A “tarefa” é o item do Quadro da Demanda (<code>BoardItem</code>); <code>Task</code> virou histórico. O documento original usava <code>Task.status / committed_deadline</code> : foi traduzido para as células do quadro (Pessoa, Status, Data). (2) O painel é um filtro do que está dentro dos quadros em que a pessoa é responsável; não agrupa a lista e não há “Meu dia”/“Concluídas hoje” (itens não têm <code>completed_at</code> ; decisão do usuário). (3) Estado derivado da etiqueta de Status: concluída (<code>is_done</code>) → Concluídas; padrão (<code>is_default</code>) ou sem Status → A fazer; as demais → Em andamento (não existe “Bloqueadas”). Atrasada = prazo < hoje (data local) e não concluída; a coluna “Data” padrão não tem <code>is_deadline</code> , então vale a primeira coluna de Data. (4) Kanban sem arrastar; “+ Adicionar” (Lista e Kanban) cria o item pelo <code>board-item-create</code> com a pessoa como responsável (diverge do documento, pedido do usuário). (5) Menções = notificações MENTIONED não lidas da pessoa; não há badge por tarefa (a notificação só liga à Demanda). (6) Ver outra pessoa/todas exige <code>quadro.visualizar</code> ou <code>demanda.visualizar_todas</code> ; acesso aos quadros calculado em lote (<code>can_many</code>). (7) Demandas antigas sem Quadro (4 no banco local) não aparecem no painel e a ficha mostra “Nenhum quadro configurado” (o <code>backfill_demand_boards</code> está desligado). (8) A ficha passou a ler progresso/“pronta para concluir” do Quadro (antes lia <code>Task</code> , sempre vazio). (9) O selo “Tarefas” do menu agora conta itens abertos do usuário nos quadros. (10) Teto de 2000 itens por consulta e 50 por coluna do Kanban. (11) 8 testes de <code>notifications</code> (rotas <code>/tarefas/<id>/</code> agora 410) já falhavam antes.
F34	Edição inline da lista de Demandas (Entrega 2: cliente/obra, setor, estágio e status) — decisões e dívidas	(1) Cliente / Obra não é editor inline: o clique abre a janela “Editar demanda” direto na etapa 2 (? passo=2, só no GET e só na edição); ao salvar a lista recarrega (filtros e aba ficam na URL). Na edição o “Salvar alterações” aparece em qualquer etapa (vale para todas as telas que editam demanda). (2) Setor (pop-over na tela): lista os setores ativos da organização; a autorização é conferida no setor de destino dentro do <code>ActivityService.update_activity</code> (e destino ativo), antes de qualquer gravação; trocar o setor redefine etapa e condição para os padrões do novo setor e audita as três mudanças numa transação. (3) Criar setor ali mesmo exige <code>setor.editar</code> (a ação que o cadastro já usa; <code>setor.criar</code> continua sem uso — S3; só Administrador a tem nos perfis sugeridos) e é “criar e aplicar” numa transação: se a troca falhar o setor não fica criado. O setor nasce vazio : sem etapas, condições nem membros/gestor (nada disso é criado sozinho); a demanda mostra “Sem estágio”/“Sem condição” até alguém criar as opções (o pop-over já oferece “+ Novo estágio/status” a quem gere). (4) Estágio e Status : opções coloridas do setor da própria demanda (o cliente nunca informa o setor); estágio não é limpável (como no Kanban), status sim (“Sem condição”). Demanda concluída/cancelada não troca estágio nem status em nenhuma tela (regra nova no <code>ActivityTransitionPolicy</code> : <code>set_stage / set_condition</code> ; o Kanban passa a recusar também). A ação antiga <code>demanda.mover_estagio</code> ainda vale para o estágio. (5) Quem tem <code>etapa.gerir / condicao.gerir</code> no setor da demanda cria (“criar e aplicar” = duas chamadas: a opção fica criada mesmo se aplicá-la falhar) e edita nome e cor ali mesmo; inativar, ordenar e padrão ficam na tela de configuração (link no rodapé). Editar uma opção atualiza todas as linhas da página que a mostram; última gravação vence. (6) Atraso derivado : a célula STATUS ganha a 2ª linha “Vencida há N dias” (para todos, quando aberta e vencida; <code>overdue_days</code> é calculado em UTC e pode diferir de 1 dia perto da meia-noite); nunca é marcação manual; a condição manual “Vencida” criada à mão no SST continua sendo só uma condição e deve ser aposentada. (7) Cliente × obra passou a ser invariante do <code>ActivityService</code> (obra de outro cliente é recusada ao mudar cliente/obra; obra sem cliente vale para qualquer um; dado antigo inconsistente continua salvável se cliente e obra não mudam). (8) <code>SectorService.create</code> converte a corrida de nome em erro de negócio e limita o nome a 150 caracteres. (9) A linha não some quando a pessoa perde o acesso por mudar setor/obra (some ao recarregar); as ações seguintes devolvem 403/404 com aviso. (10) O texto “Sem estagio” da lista ganhou acento (“Sem estágio”). (11) <code>_sector_configuration_message</code> continua sem uso; Tarefas expansíveis ficam para a Entrega 3.

4. Deploy

#	Onde	Situação
P2	<code>render.yaml</code>	Sem variáveis de e-mail: ver D2 e 12_DEPLOY_RENDER.md .
P3	Plano free	Anexos se perdem a cada deploy; <code>check_overdue_tasks</code> não roda (sem cron nem Shell).

5. Limpeza do repositório

#	Item
L1	<code>demands/</code> , <code>workflows/</code> , <code>templates/demands/</code> , <code>templates/workflows/</code> : não existem mais nesta árvore (resolvido).
L2	<code>.claude/launch.json</code> ainda usa <code>venv/Scripts/python.exe</code> , e o ambiente virtual atual é <code>.venv</code> (a pasta <code>venv/</code> quebrada de outra máquina não existe mais aqui).
L3	<code>tests/README.md</code> e o cabeçalho de <code>tests/checklist.test.cjs</code> mandam rodar "a partir da pasta <code>workflow</code> " (nome antigo); o certo é a raiz do repositório.
L4	<code>Regras/12_...</code> e <code>Regras/13_...</code> têm "11 —" no título, inconsistente com o nome do arquivo.
L5	Docstring de <code>ActivityAttachmentService</code> cita <code>MEDIA_ROOT</code> ; o storage real é <code>ACTIVITY_FILES_ROOT</code> .
L6	<code>Activity</code> , <code>Task</code> e <code>DeadlineConflict</code> ainda declaram <code>Meta.permissions</code> do Django (<code>can_change_owner</code> etc.), sem uso — a autorização é o motor do <code>acessos</code> .

Apêndices

Especificações complementares referenciadas pelos capítulos técnicos.

Apêndice	Documento
Apêndice A	Elementos, Subelementos e Integração com Processos
Apêndice B	Especificação Completa das Colunas Dinâmicas
Apêndice C	Especificação da Visualização por Calendário
Apêndice D	Especificação da Visualização por Kanban
Apêndice E	Especificação da Visualização por Tabela (Quadro)

Apêndice A — Elementos, Subelementos e Integração com Processos

Status de implementação (03/10/2026)

Este documento é uma **especificação**; o código (HEAD `add5acd`) implementa só parte dela, e de forma diferente da proposta. Na interface, Atividade virou **Demanda** (o código segue `Activity`), "Situação" virou **Etapa** e "Condição" virou **Status**; "Etapa" neste documento continua significando a **etapa do Processo** (`ProcessStep`), que não é a Etapa de fluxo da Demanda. Em vez de Elemento/Subelemento num mesmo Quadro, cada Demanda ganhou o **seu próprio Quadro** (`Board` com `kind=DEMAND`, `Activity.task_board`, criado em branco ou copiado de um modelo `kind=TEMPLATE`), e as tarefas são **itens planos** (`BoardItem`) desse quadro, sem `parent`. O app `processes` está **desativado**: desde 02/10/2026 `/processos/*` e `/demandas/<pk>/processo/aplicar/` respondem 410, e a `Task` antiga ficou só como histórico (`/tarefas/<pk>/...` também responde 410). Os modelos e o serviço de aplicação de processo continuam no código, sem tela. Legenda: **Implementado, Parcial, Não implementado, Divergente**.

Requisito / bloco	Status	Onde está no código
Demanda como Elemento (§3, §6.1)	Implementado	<code>activities.Activity</code> ; lista em <code>/demandas/</code> sobre <code>boards.DomainBoard</code> (<code>domain=DEMAND</code>) com campos próprios (<code>DomainBoardField</code> , <code>DomainCustomValue</code>)
Esquema de colunas próprio por nível (§3.2, §4.3, §5)	Divergente	Cada Demanda tem um Quadro com colunas próprias (<code>BoardColumn</code> , 8 tipos ativos: Texto, Número, Moeda, Data, Pessoa, Status, Lista suspensa, Sinal de confirmação); o esquema da lista de Demandas é outro (<code>DomainBoardField</code>)
Tarefa como Subelemento ligado ao pai (§4, §8, §40, §193)	Divergente	Tarefa = <code>BoardItem</code> do Quadro da Demanda (<code>boards/models.py</code>); não há relação pai × filho entre itens nem <code>Task</code> por item; o vínculo é Quadro ↔ <code>Activity</code> (<code>OneToOneField</code>)
Profundidade de um nível (§7)	Parcial	Ocorre por construção (Demanda → itens do seu quadro); itens não têm filhos
Exclusão do pai e mover filho (§8.1, §8.2)	Parcial	Excluir a Demanda remove o Quadro (<code>on_delete=CASCADE</code>); mover item só entre grupos do mesmo quadro (<code>ItemMoveView</code>)
Tabela com Subelementos expansíveis, cabeçalhos próprios do filho (§9–§12, §170–§172)	Não implementado	Existe a Tabela do quadro de cada Demanda (<code>boards/board_detail.html</code> , <code>boards.js</code>) e a prévia de até 5 tarefas na ficha (<code>boards/demand_preview.py</code>)
Kanban de Demandas com filhos expansíveis (§13–§17)	Não implementado	Há Kanban da lista de Demandas (<code>work-board.js</code>) e Kanban dentro do Quadro de cada Demanda (<code>board_kanban.html</code>), mas separados e sem expandir filhos
Calendário de Demandas e de Tarefas (§19)	Parcial	Calendário mensal por quadro (<code>board_calendar.html</code>); <code>/demandas/calendario/</code> é uma lista por prazo; <code>/tarefas/calendario/</code> é semanal (<code>boards/task_center.py</code>)
Status independentes e Status do filho não move o pai (§5.2, §16)	Implementado	Demanda: Etapa + Status do setor (<code>workflow-picker.js</code>); tarefa: coluna Status do quadro; nenhuma regra automática move o pai
Painel Gestão × Operação (§18, §165–§167)	Parcial	<code>/demandas/</code> (gestão) e <code>/tarefas/</code> (o que está nos quadros em que a pessoa é responsável: <code>boards/task_center.py</code> , <code>task_center_views.py</code>)
Processo como blueprint versionado: <code>Process</code> , <code>ProcessVersion</code> , <code>ProcessInput</code> , <code>ProcessCriterion</code> , <code>ProcessStep</code> (§20–§23, §31–§33)	Implementado (desativado)	<code>processes/models.py</code> ; versão publicada imutável (testes em <code>processes/tests.py</code>); telas <code>process_*.html</code> inacessíveis (410)

Requisito / bloco	Status	Onde está no código
Aplicar Processo à Demanda, gravar <code>_process_version</code> , fixar versão (§28, §69, §71, §193 itens 5–6)	Implementado (desativado)	<code>activities/process_application.py</code> (<code>ProcessApplicationService.apply</code>), <code>Activity.process_version</code> ; rota <code>activity-process-apply</code> responde 410
Gerar Subelementos a partir das etapas (§41–§52, Fase 3)	Divergente	O serviço cria objetos <code>Task</code> (com <code>process_step</code> , <code>depends_on</code> e <code>fila</code>), não itens do quadro; congelado
Dependências e disponibilidade (§47–§50)	Parcial	Uma dependência por tarefa (<code>Task.depends_on</code> , <code>TaskService.pending_dependency</code>); só no modelo legado
Inputs e critérios de aceite do Processo (§35–§39, §58–§59, Fase 4)	Implementado (desativado)	<code>processes.ActivityInputValue</code> , <code>ActivityCriterionCheck</code> ; <code>ActivityProcessService.update_input/set_criterion</code> ; sem tela
Output e evidência (§56–§57)	Parcial	Campos de output/evidência na versão do Processo (<code>ProcessVersion</code>); sem tela ativa
Trocar ou remover Processo (§72, §73)	Não implementado	Não há serviço nem rota
Subelemento manual e origem manual × processo (§52–§55)	Parcial	Itens manuais são o caso normal do quadro; a origem não é registrada
Template de Elemento e de Subelemento (§76–§77)	Divergente	Substituído por modelos de quadro (<code>Board.kind=TEMPLATE</code> , <code>boards/starter_templates.py</code> , <code>BoardInstantiationService</code> em <code>boards/demand_services.py</code> , escolha no passo 3 do Nova demanda; trocar o quadro apaga as tarefas, com confirmação)
Progresso e conclusão da Demanda a partir dos filhos (§60–§65, Fase 5)	Parcial	Ficha: total/concluídas, "pronta para concluir" lidos do quadro (<code>ActivityDetailView</code> , <code>DemandBoardPreviewQuery</code>); sem <code>rollup</code> , fórmula ou coluna Relação (tipos reservados)
Rollups, Fórmula, indicadores, Dashboard (§66–§68, §136–§138, §162)	Não implementado	Tipos <code>FORMULA</code> , <code>RELATION</code> , <code>AI_EXTRACT</code> existem só como reservados em <code>BoardColumn.Type</code>
Permissões (§86–§89, §180–§182)	Parcial	Ações <code>quadro.*</code> em <code>acessos/catalog.py</code> ; <code>DemandBoardAccess</code> decide quem gere a estrutura e os itens de cada Quadro de Demanda
Auditoria e histórico (§90–§92)	Implementado	<code>audit.AuditLog</code> com <code>BOARD_*</code> (itens, células, colunas, visões); histórico do quadro em <code>board_history.html</code>
Multi-organização (§94)	Implementado	Todo <code>Board / DomainBoard</code> tem <code>organization</code> ; testes de isolamento em <code>boards/test_views.py</code>
Aprendizado do Processo, Processo condicional e com caminhos, automações (§132–§135, §155–§159, Fase 6)	Não implementado	—
Linguagem da interface (§197)	Parcial	Interface e URLs dizem Demanda/Etapa/Status (<code>/demandas/</code>); o código mantém <code>Activity</code> , <code>Task</code>

Divergência: este documento propõe Subelementos **dentro** do Quadro do pai. A implementação atual separa os níveis: a Demanda é uma linha de `/demandas/` e o Quadro dela contém as tarefas como itens planos. Qualquer requisito de expansão, `rollup` ou herança de contexto (§9–§17, §66, §96, §141–§147) continua válido como proposta, mas precisa de decisão de produto antes de ser construído.

1. Objetivo

Este documento define o conceito oficial de:

- **Elemento;**

- **Subelemento;**
- relação pai × filho;
- comportamento nas visualizações;
- relação com Demandas e Tarefas;
- integração com o motor de Processos da LPS;
- regras de criação automática e manual;
- versionamento;
- inputs, etapas, outputs e critérios de aceite;
- permissões;
- auditoria;
- agregações;
- critérios de aceite.

O objetivo é permitir que a LPS seja flexível como um Quadro dinâmico, sem perder o conceito de **processo padronizado**.

2. Estado atual da LPS

2.1 Atividade (hoje Demanda) e Tarefa

Na arquitetura atual:

```
Activity
└─ Task
```

Uma `Activity` pode possuir várias `Task`.

Na experiência futura da LPS:

```
Activity = Demanda
Task     = Tarefa
```

Portanto, no Quadro de Demandas:

```
Elemento    = Demanda
Subelemento = Tarefa
```

Divergência (03/10/2026): a interface já usa Demanda, mas a Tarefa deixou de ser a `Task` filha da `Activity`: as tarefas viram itens (`BoardItem`) do Quadro de cada Demanda e a `Task` ficou como histórico (rotas `/tarefas/<pk>/...` respondem 410).

2.2 Processo atual

A LPS já possui o app `processes`.

Hoje um Processo é um modelo reutilizável e versionado de **como fazer um tipo de atividade**.

A estrutura existente contempla:

```
Process
├── ProcessVersion
│   ├── ProcessInput
│   ├── ProcessCriterion
│   └── ProcessStep
```

Uma versão de Processo possui:

- inputs;
- output esperado;
- tipo de evidência;
- critérios de aceite;
- etapas ordenadas;
- setor responsável por cada etapa;
- dependência da etapa anterior.

2.3 Lacuna registrada na época (histórico)

A documentação atual registra uma lacuna importante:

Processos podem ser criados, versionados e publicados, mas ainda não são aplicados às Atividades.

Atualmente existem estruturas como:

```
Activity.process_version
ActivityInputValue
ActivityCriterionCheck
```

porém o sistema ainda não:

- aplica automaticamente uma versão de Processo à Activity;
- gera Tarefas a partir das etapas;
- preenche inputs do Processo na Activity;
- registra os critérios de aceite durante a execução.

Portanto, a integração descrita neste documento é uma **evolução arquitetural proposta**, não um comportamento já implementado.

Divergência (03/10/2026): a lacuna acima foi fechada no backend e depois desativada.

`ProcessApplicationService.apply` aplica uma versão, gera tarefas e registra inputs e critérios (`activities/process_application.py`), mas as rotas e telas de Processo respondem 410 desde 02/10/2026 e o fluxo novo de tarefas passa pelos Quadros por Demanda.

3. Conceito de Elemento

3.1 Definição

Um **Elemento** é o registro principal de um Quadro.

Exemplos:

Quadro: Demandas
Elemento: Arena Center Norte – Elaborar orçamento

Quadro: Contratos
Elemento: Contrato 0B-077 – Alfalog

Quadro: Compras
Elemento: Pedido de compra 00382

3.2 O Elemento possui seu próprio esquema de colunas

Exemplo de Demanda:

Título
Cliente
Obra
Solicitante
Responsável
Setor
Status
Prioridade
Prazo
Valor

O conjunto de colunas pertence ao esquema do **Elemento** daquele Quadro.

3.3 Elemento é a unidade principal de gestão

O Elemento deve representar algo que faça sentido acompanhar sozinho.

Exemplo:

Demanda

A pergunta gerencial deve ser:

O que precisa ser entregue?

e não:

Quais pequenos passos estão sendo executados?

Os pequenos passos pertencem ao nível inferior.

4. Conceito de Subelemento

4.1 Definição

Um **Subelemento** é um registro filho vinculado diretamente a um Elemento.

Exemplo:

```
Demanda: Elaborar orçamento Arena Center Norte
```

```
Tarefas:
```

```
|— Conferir projetos
|— Levantar quantitativos
|— Solicitar cotações
|— Revisar proposta
```

4.2 Subelemento não é apenas uma linha visual menor

Ele é um registro real.

Possui:

- ID próprio;
- título;
- valores próprios;
- status próprio;
- responsável próprio;
- prazo próprio;
- histórico próprio;
- permissões;
- auditoria;
- possibilidade de conclusão.

4.3 Subelemento possui esquema próprio de colunas

Esse é um requisito central.

O Subelemento **não herda automaticamente todas as colunas do Elemento**.

Exemplo:

Elemento — Demanda

```
Cliente
Obra
Solicitante
Responsável
Status comercial
Prazo
Valor
```

Subelemento — Tarefa

```
Responsável
Setor
Status da tarefa
Prioridade
Prazo solicitado
Prazo comprometido
Checklist
```

Os dois níveis usam o mesmo motor de tipos de coluna, mas seus esquemas são independentes.

5. Mesmo motor, schemas diferentes

A arquitetura deve ser:

```
Board
├── ElementSchema
│   └── Columns
└── SubitemSchema
    └── Columns
```

Não:

```
BoardColumn
└── copiar automaticamente para subelementos
```

5.1 Reutilização de tipos

Os mesmos componentes de coluna devem ser reutilizados.

Exemplo:

```
PeopleColumn
StatusColumn
DateColumn
MoneyColumn
FormulaColumn
RelationColumn
```

Eles podem existir no Elemento e no Subelemento com configurações diferentes.

5.2 Status independentes

Exemplo:

Status da Demanda

```
Entrada
Levantamento
Cotação
Revisão
Enviado
Declinado
```

Status da Tarefa

```
A fazer
Em execução
Bloqueada
Concluída
```

Mesmo tipo técnico:

```
Status
```

mas duas colunas e duas configurações independentes.

6. Elemento e Subelemento na LPS

6.1 Demandas

Para o primeiro caso principal:

```
Elemento = Demanda
Subelemento = Tarefa
```

Isso conversa diretamente com a estrutura atual:

```
Activity
└─ Task
```

6.2 Motor genérico

Apesar disso, a arquitetura não deve chamar todo Subelemento de Tarefa internamente.

Porque outros Quadros podem usar:

```
Contrato
└─ Aditivos
```

```
Pedido de compra
└─ Itens
```

```
Obra
└─ Entregáveis
```

```
Inspeção
└─ Não conformidades
```

Portanto:

Tarefa é um caso de uso de Subelemento.

7. Profundidade

Na primeira versão, permitir apenas:

```
Elemento
└─ Subelemento
```

Não permitir:

```
Elemento
└─ Subelemento
   └─ Sub-sub-elemento
      └─ ...
```

7.1 Motivo

Profundidade ilimitada gera:

- navegação complexa;
- Kanban difícil;
- filtros ambíguos;
- permissões confusas;
- fórmulas recursivas;
- relatórios difíceis de interpretar.

Se um terceiro nível surgir como necessidade real, deve ser tratado como outro domínio ou relação explícita.

8. Relação pai × filho

Todo Subelemento deve possuir:

```
parent_item_id
```

ou equivalente.

Regra:

Subelemento não existe solto.

Ele pertence a exatamente um Elemento pai.

8.1 Exclusão do pai

Não excluir filhos silenciosamente.

Ao excluir/inativar um Elemento:

- verificar Subelementos;
- manter histórico;
- usar exclusão lógica quando houver dados operacionais.

8.2 Mover Subelemento para outro pai

Não deve ser uma edição trivial.

Mover uma Tarefa de uma Demanda para outra altera contexto de negócio.

Deve exigir:

- permissão;
- confirmação;
- auditoria;
- validação de compatibilidade.

9. Apresentação na Tabela

Na Tabela, o Elemento aparece como linha principal.

Exemplo:

▼ Arena Center Norte | Ryan | Em andamento | 15/10 | R\$ 2,5 mi

Ao expandir:

Tarefa	Responsável	Status	Prazo
Conferir projetos	Jennifer	Concluída	05/10
Levantar quantitativos	Ryan	Em execução	08/10
Solicitar cotações	Luan	A fazer	10/10

+ Adicionar tarefa

10. Cabeçalhos próprios do Subelemento

Ao expandir um Elemento, o bloco filho deve possuir seus próprios cabeçalhos.

Exemplo:

Tarefa | Responsável | Status | Prazo | Prioridade | +

Esses cabeçalhos não precisam coincidir com os cabeçalhos da linha pai.

11. Configurar colunas do Subelemento

Dentro do cabeçalho dos Subelementos:



abre o mesmo seletor de tipos:



A nova coluna é adicionada somente ao schema do Subelemento.

12. Redimensionamento e movimentação

As colunas do Subelemento devem poder:

- ser movidas;
- redimensionadas;
- renomeadas;
- filtradas;
- ordenadas;
- ocultas.

Independentemente das colunas do Elemento.

13. Apresentação no Kanban

No Kanban de Elementos, o card principal representa o Elemento.

Exemplo:



O indicador:



mostra a quantidade de Subelementos.

14. Expandir Subelementos no Kanban

Ao clicar no indicador:



Os filhos aparecem abaixo do card pai.

15. Coluna do Kanban do pai

Regra fundamental:

No Kanban de Elementos, a coluna é determinada pelo campo do Elemento.

Exemplo:

Agrupar por: Status da Demanda

A Demanda está em:

Em andamento

Todas as Tarefas expandidas permanecem visualmente sob essa Demanda.

16. Status do Subelemento não move o pai

Se:

Demanda = Em andamento

e:

Tarefa A = Concluída
Tarefa B = Em execução

a Demanda continua na coluna:

Em andamento

Não mover a Demanda automaticamente.

17. Kanban próprio dos Subelementos

Para operar Tarefas:

Kanban de Tarefas

Agrupar por:

Status da Tarefa

Resultado:

A fazer | Em execução | Bloqueada | Concluída

Cada Tarefa passa a ser card principal.

18. Duas perspectivas complementares

Gestão

Kanban de Demandas

Responde:

- onde estão as Demandas;
- quais estão travadas;
- quais têm muitas Tarefas;
- quais possuem risco.

Operação

Kanban de Tarefas

Responde:

- o que precisa ser feito;
 - quem é responsável;
 - o que está bloqueado;
 - o que vence.
-

19. Apresentação no Calendário

Elemento e Subelemento podem possuir calendários diferentes.

Exemplo:

Calendário de Demandas

Data = prazo da Demanda

Calendário de Tarefas

Data = prazo comprometido da Tarefa

Não misturar os dois níveis automaticamente.

20. O que é Processo

Processo não é Elemento.

Processo não é Subelemento.

Processo é:

um modelo versionado de como um tipo de trabalho deve ser executado.

Analogia:

Processo = receita
Elemento = pedido que será produzido
Subelemento = etapas concretas de execução daquele pedido

21. Processo como blueprint

Blueprint significa um modelo estruturado usado para criar instâncias reais.

Exemplo:

Processo:
Elaboração de orçamento

Define:

Inputs
Etapas
Output
Critérios de aceite
Responsabilidades

Quando aplicado:

Demanda:
Orçamento Arena Center Norte

gera a execução concreta.

22. Processo não deve armazenar trabalho operacional

O Processo não deve conter:

```
Ryan está executando  
prazo atual 15/10  
comentário do cliente  
arquivo recebido hoje
```

Isso pertence ao Elemento e aos Subelementos reais.

O Processo guarda:

```
como deve funcionar
```

e não:

```
o que está acontecendo agora
```

23. Estrutura conceitual

```
PROCESSO  
|  
├── Inputs necessários  
├── Etapas  
├── Output esperado  
└── Critérios de aceite  
  
    ↓ aplicar  
  
ELEMENTO  
|  
├── Dados reais  
├── Valores dos inputs  
├── Resultado  
├── Critérios verificados  
└── SUBELEMENTOS  
    ├── Etapa/Tarefa A  
    ├── Etapa/Tarefa B  
    └── Etapa/Tarefa C
```

24. Processo x Quadro

O Quadro define:

```
quais dados podem existir
```

O Processo define:

```
como um determinado tipo de trabalho usa esses dados e deve ser executado
```


25. Board Schema não deve ser recriado pelo Processo

Evitar:

```
cada Processo cria uma nova coluna duplicada chamada Prazo
```

O Processo deve referenciar colunas existentes por ID.

Exemplo:

```
ProcessInput:
"Projeto recebido"
→ board_column_id = 84
```

26. Vantagem

Renomear:

```
Projeto recebido
```

para:

```
Projetos recebidos
```

não quebra o Processo.

A referência continua pelo ID.

27. Tipos de Processo por Quadro

Um Quadro pode possuir vários Processos.

Exemplo:

```
Quadro: Demandas

Processos:
- Elaborar orçamento
- Revisar orçamento
- Visita técnica
- Solicitação interna
- Aprovação de documento
```

Todos geram Demandas dentro do mesmo motor, mas com fluxos diferentes.

28. Escolha do Processo

Ao criar uma Demanda:

```
Criar demanda

Tipo/processo:
[ Elaborar orçamento v ]
```

Ou:

Sem processo

A organização pode decidir se Processo é obrigatório para determinados Quadros.

29. Elemento sem Processo

A LPS deve permitir, em Quadros autorizados:

Elemento sem Processo

Isso atende trabalhos:

- ad hoc;
- exceções;
- atividades administrativas;
- registros simples.

Ad hoc significa algo criado para uma necessidade específica, fora de um fluxo padronizado.

30. Processo obrigatório

Para processos críticos, a organização pode configurar:

Exigir Processo: Sim

Exemplo:

Pedido de compra

pode exigir um Processo publicado.

31. Versão do Processo

Ao aplicar um Processo, o Elemento deve guardar:

process_version_id

e não apenas:

process_id

32. Motivo

Processos evoluem.

Exemplo:

```
Elaborar orçamento v3
```

é aplicado em 01/10.

Em 15/10 é publicada:

```
v4
```

A Demanda iniciada em v3 continua sabendo exatamente quais regras recebeu.

33. Imutabilidade da versão publicada

Uma versão publicada não deve ser alterada retroativamente.

Fluxo:

```
v3 PUBLICADA
→ criar v4 RASCUNHO
→ editar
→ publicar v4
→ v3 SUBSTITUÍDA
```

Elementos existentes em v3 permanecem vinculados à v3.

34. Atualização de Elemento em andamento

Não migrar automaticamente:

```
v3 → v4
```

Isso pode mudar tarefas e critérios durante a execução.

Se houver necessidade de migração:

```
Atualizar para v4
```

deve ser uma ação explícita e auditada.

35. Inputs do Processo

Input é uma informação necessária para executar o trabalho.

Exemplos:

```
Projeto
Memorial
Prazo solicitado
Escopo
Contato do cliente
```

36. Input deve mapear para Coluna quando possível

Exemplo:

```
ProcessInput:
Prazo solicitado
```

mapeado para:

```
ElementoColumn:
Prazo
```

Não armazenar o mesmo valor em dois lugares sem necessidade.

37. Inputs especiais

Alguns inputs podem não ser colunas normais.

Exemplo:

```
Confirmação de recebimento
```

Ainda assim, a preferência é usar o motor de colunas sempre que possível.

38. Input obrigatório

Um Processo pode declarar:

```
Projeto = obrigatório
Prazo = obrigatório
Cliente = obrigatório
```

A Demanda não avança para execução sem os inputs necessários.

39. Origem do Input

A estrutura atual contempla fontes como:

```
Solicitante
Executor
Terceiro
```

A interface deve deixar isso claro.

Exemplo:

```
Projeto
Responsável por fornecer: Solicitante
```

40. Processo e Subelementos

As etapas do Processo devem poder gerar Subelementos.

Exemplo:

```
Processo: Elaborar orçamento  
  
Etapa 1: Conferir documentação  
Etapa 2: Levantar quantitativos  
Etapa 3: Solicitar cotações  
Etapa 4: Revisar proposta  
Etapa 5: Enviar proposta
```

Ao aplicar:

```
Demanda: Arena Center Norte
```

pode gerar:

```
Tarefas:  
- Conferir documentação  
- Levantar quantitativos  
- Solicitar cotações  
- Revisar proposta  
- Enviar proposta
```

41. ProcessStep como template

`ProcessStep` é definição.

A Tarefa é instância.

```
ProcessStep  
"Solicitar cotações"
```

gera:

```
Task  
"Solicitar cotações"
```

42. Não usar a própria etapa como Tarefa

Porque a etapa pertence à versão do Processo e é reutilizada por muitas Demandas.

Exemplo:

```
ProcessStep #17
```

pode gerar centenas de Tarefas ao longo do tempo.

43. Campos padrão por etapa

Uma etapa pode definir valores sugeridos para o Subelemento.

Exemplo:

```
Nome: Solicitar cotações
Setor padrão: Suprimentos
Prioridade padrão: Média
Prazo relativo: D+2
```

44. Responsável padrão

Uma etapa pode definir:

```
Setor responsável
```

mas evitar fixar pessoa específica como regra principal.

Preferir:

```
Setor = Suprimentos
```

e aplicar regra de atribuição.

45. Atribuição de pessoa

Possibilidades:

```
Manual
Gestor do setor
Usuário padrão
Balanceamento futuro
Quem criou
Responsável do Elemento
```

Essa política deve ser configurável.

46. Prazo relativo

O Processo pode possuir regras como:

```
D+1
D+2
3 dias úteis
```

D significa o dia de referência.

Exemplo:

```
Data da Demanda = 05/10
Etapa = D+2
→ prazo sugerido 07/10
```

Calendário útil/feriados exigem um motor específico.

47. Dependências

Uma etapa pode depender de outra.

Exemplo:

```
Conferir projetos
  ↓
Levantar quantitativos
  ↓
Solicitar cotações
```

48. Dependência não precisa impedir criação

As Tarefas podem ser criadas todas de uma vez, mas com dependências.

Exemplo:

```
Tarefa B.depends_on = Tarefa A
```

49. Disponibilidade da Tarefa

Regra configurável:

```
Criar todas
```

mas:

```
Tarefa B = aguardando dependência
```

até A ser concluída.

50. Etapas paralelas

O Processo precisa permitir futuramente:

```
Entrada ┌ Cotação elétrica
        ├── Cotação hidráulica
        └── Cotação mecânica
```

A estrutura atual é essencialmente linear.

Paralelismo é uma evolução.

51. Etapa não é obrigatoriamente uma Tarefa única

Em alguns casos:

```
ProcessStep  
Cotação
```

pode gerar:

```
Cotação fornecedor A  
Cotação fornecedor B  
Cotação fornecedor C
```

Portanto, arquiteturalmente, manter relação:

```
ProcessStep 1 → N Subelementos
```

quando necessário.

52. Subelemento manual em Processo

Mesmo que o Processo gere Tarefas, o usuário autorizado pode precisar criar:

```
Tarefa adicional
```

Exemplo:

```
Revisar incompatibilidade encontrada
```

53. Origem do Subelemento

Guardar:

```
origin = PROCESS | MANUAL | AUTOMATION
```

e, se houver:

```
process_step_id
```

54. Vantagem

Permite distinguir:

```
Tarefa prevista pelo Processo
```

de:

```
Tarefa criada durante execução
```


Isso gera informação valiosa sobre retrabalho e exceções.

55. Não transformar origem em avaliação de pessoa

Tarefas extras podem indicar:

- projeto incompleto;
- mudança do cliente;
- risco;
- oportunidade;
- falha de processo.

Não concluir automaticamente que houve improdutividade.

56. Output do Processo

O Processo deve definir:

```
qual resultado final é esperado
```

Exemplo:

```
Proposta comercial validada e enviada ao cliente.
```

57. Evidência

O Processo atual já prevê tipo de evidência como:

```
Arquivo  
Link  
Checklist  
Confirmação
```

A execução deve vincular a evidência ao Elemento.

58. Critérios de aceite

Critérios definem:

```
como saber que a entrega está aceitável
```

Exemplo:

```
Escopo revisado  
Valores conferidos  
Inclusões/exclusões revisadas  
Condição de pagamento validada  
Arquivo salvo no local correto
```

59. Critérios pertencem ao Elemento

A definição está no Processo.

O cumprimento pertence à execução.

```
ProcessCriterion  
→ ActivityCriterionCheck
```

60. Conclusão do Elemento

Um Elemento vinculado a Processo pode exigir:

```
todos os critérios obrigatórios atendidos
```

antes de conclusão.

61. Subelementos pendentes

Regra configurável:

```
Não permitir concluir Elemento com Subelementos obrigatórios abertos.
```

62. Subelementos opcionais

Alguns podem ser:

```
opcionais
```

Sua ausência não bloqueia conclusão.

63. Regra de conclusão

Exemplo:

```
Pode concluir Demanda se:  
  
- inputs obrigatórios recebidos;  
- tarefas obrigatórias concluídas;  
- critérios obrigatórios atendidos;  
- output/evidência registrado.
```

64. Progresso

O Elemento pode exibir progresso derivado dos Subelementos.

Exemplo:

```
7 de 10 concluídas  
70%
```

65. Progresso não deve alterar Status automaticamente sem regra

Ter:

```
100% tarefas concluídas
```

não significa necessariamente:

```
Demanda concluída
```

Pode faltar:

- aceite;
- envio;
- evidência;
- aprovação.

66. Rollup

Rollup significa agregação de dados dos registros filhos.

A LPS deve prever agregações.

Exemplos:

```
CONTAR(Tarefas)
CONTAR_CONCLUÍDAS(Tarefas)
MÍNIMO(Tarefas.Prazo)
MÁXIMO(Tarefas.Prazo)
SOMA(Tarefas.Horas)
```

67. Exemplos gerenciais

No Elemento:

```
Tarefas abertas: 4
Tarefas atrasadas: 2
Próximo prazo: 08/10
Última conclusão: 05/10
```

68. Fórmula x Rollup

Fórmula trabalha com campos do próprio registro.

Exemplo:

```
Margem = (Venda - Custo) / Venda
```

Rollup trabalha com registros relacionados.

Exemplo:

```
Tarefas abertas = COUNT(Subitems where Status ≠ Concluída)
```

São conceitos diferentes.

69. Fluxo completo de aplicação de Processo

Exemplo:

```
1. Usuário cria Demanda
2. Escolhe Processo "Elaborar orçamento"
3. Sistema fixa ProcessVersion publicada
4. Inputs obrigatórios são apresentados
5. Usuário preenche contexto
6. Sistema cria Elemento
7. Etapas geram Subelementos/Tarefas
8. Tarefas recebem setor/prazo/regras padrão
9. Execução acontece
10. Inputs adicionais chegam
11. Critérios são verificados
12. Evidência/output é registrado
13. Elemento é concluído
```

70. Criação rápida

O usuário pode criar rapidamente:

```
Título
```

e escolher o Processo depois, se a regra do Quadro permitir.

71. Aplicar Processo depois

Ação:

```
Aplicar processo
```

Só permitida se:

- Elemento não possuir outro Processo incompatível;
- usuário possuir permissão;
- versão estiver publicada;
- conflitos de colunas forem validados.

72. Trocar Processo

Troca de Processo durante execução é operação de alto impacto.

Não tratar como dropdown comum.

Exigir:

- revisão;
- impacto nas Tarefas;
- critérios;
- inputs;
- histórico;
- confirmação.

73. Remover Processo

Não remover Processo de Elemento em execução silenciosamente.

Se permitido:

- manter histórico da versão utilizada;
- não apagar Tarefas;
- auditar motivo.

74. Processo e schema de Subelemento

O Processo pode dizer quais colunas do schema de Subelemento são relevantes.

Exemplo:

```
Etapa "Cotação"
```

```
Campos principais:
```

- ```
- Responsável
- Prazo
- Fornecedor
- Status
```

Mas não criar um schema novo para cada execução.

---

## 75. Campos específicos de Processo

Se um Processo realmente precisar de um campo inexistente:

```
+ Adicionar coluna ao Quadro
```

Essa mudança é estrutural no Quadro.

Deve ser explícita.

---

## 76. Template de Elemento

O Processo pode definir:

```
campos recomendados
valores padrão
campos obrigatórios
```

sem duplicar os tipos de coluna.

---

## 77. Template de Subelemento

Cada ProcessStep pode definir:

```
Título sugerido
Setor
Prioridade
Prazo relativo
Campos obrigatórios
Dependência
Critério de conclusão
```

---

## 78. Processo não controla layout global

O Processo não deve decidir:

```
largura da coluna
posição da coluna
cor da view
zoom
```

Isso pertence à Visualização/Quadro.

---

## 79. Processo pode sugerir visualização

Futuro:

```
Ao aplicar este Processo:
sugerir Kanban "Fluxo de orçamento"
```

Mas não é requisito do núcleo.

---

## 80. Grupo

Grupo é organização visual dentro do Quadro.

Não confundir:

```
Grupo
```

com:

```
Processo
```

Grupo pode representar:

```
Em aberto
Outubro
Cliente A
Obra X
```

Processo define método.

---

## 81. Status

Não confundir:

```
Status do Elemento
Status do Subelemento
Etapa do Processo
```

São conceitos relacionados, mas diferentes.

---

## 82. Etapa do Processo × Status da Tarefa

ProcessStep:

```
o que deve ser feito
```

Status da Tarefa:

```
qual o estado daquela execução
```

Exemplo:

```
ProcessStep = Solicitar cotações
Task.status = Em execução
```

---

## 83. Processo × Kanban

O Kanban não precisa refletir literalmente ProcessStep.

Pode agrupar por:

```
Status
Setor
Responsável
Prioridade
```

Processo é estrutura de execução.

Kanban é visualização.

---

## 84. Processo x Calendário

O Calendário pode usar:

```
prazo dos Subelementos
```

gerados pelo Processo.

Não criar calendário próprio do Processo.

## 85. Processo x Tabela

Na Tabela, expandir o Elemento mostra os Subelementos derivados ou manuais.

Pode haver indicador:

```
4/7 tarefas
```

## 86. Permissões

A autorização deve respeitar o motor atual:

```
SUJEITO + AÇÃO + ESCOPO + ORIGEM
```

## 87. Ver Elemento não significa editar Subelemento

Permissões podem diferir.

Exemplo:

```
Gestor:
ver Demanda
editar Demanda
ver Tarefas

Executor:
ver Demanda
editar apenas Tarefas do seu escopo
```

## 88. Criar Subelemento

Exigir ação correspondente.

Exemplo:

```
tarefa.criar
```

no caso de Tarefas.

## 89. Processo

Aplicar Processo deve possuir ação própria.



A documentação atual já prevê:

```
processo.aplicar
```

mas a implementação ainda precisa ser conectada.

---

## 90. Auditoria

Registrar:

- Processo aplicado;
- versão;
- Tarefas geradas;
- Subelemento manual criado;
- mudança de pai;
- alteração de campos;
- conclusão;
- critérios;
- evidências;
- migração de versão.

---

## 91. Histórico do Elemento

O drawer do Elemento deve mostrar uma linha do tempo integrada.

Exemplo:

```
02/10 Demanda criada
02/10 Processo "Elaborar orçamento v3" aplicado
02/10 5 tarefas geradas
03/10 Projeto recebido
04/10 Tarefa "Conferir projetos" concluída
...
```

---

## 92. Histórico do Subelemento

Cada filho mantém histórico próprio.

Exemplo:

```
Responsável alterado
Prazo negociado
Bloqueado
Desbloqueado
Concluído
```

---

## 93. Notificações

Notificar com base em eventos de negócio.

Exemplo:

- Tarefa atribuída;

- prazo alterado;
- bloqueio;
- conclusão;
- aprovação necessária.

Não notificar porque o card foi expandido.

---

## 94. Multi-organização

Processo, Elemento e Subelemento precisam pertencer à mesma organização.

Não permitir:

```
Processo de Organização A
→ Elemento da Organização B
```

---

## 95. Relações de escopo

O Subelemento herda parte do contexto do pai.

Exemplo:

```
Organização
Cliente
Obra
Centro de custo
```

mas pode possuir:

```
Setor operacional próprio
Responsável próprio
```

---

## 96. Herança de contexto

Diferenciar:

```
contexto herdado
```

de:

```
valor copiado
```

Preferir referência ao contexto do pai quando o dado é estrutural.

---

## 97. Cliente e Obra

Uma Tarefa vinculada à Demanda não precisa duplicar Cliente e Obra se essas informações puderem ser obtidas do pai.

Isso reduz inconsistência.

## 98. Campos sobrescrevíveis

Se houver necessidade real:

Tarefa possui obra diferente do pai

a regra deve ser explícita.

Não permitir divergência silenciosa.

## 99. Item principal e resultado

Elemento deve representar um resultado relevante.

Exemplo bom:

Elaborar orçamento Arena Center Norte

Exemplo ruim:

Abrir e-mail

O segundo é melhor como Tarefa.

## 100. Critério para decidir Elemento ou Subelemento

Pergunta:

Isso precisa ser acompanhado independentemente como entrega principal?

Se sim:

Elemento

Se é uma parte da entrega:

Subelemento

## 101. Outra pergunta

Esse registro continua fazendo sentido sem o pai?

Se não:

Subelemento

## 102. Não transformar tudo em Tarefa

Evitar decomposição excessiva.

Exemplo:

```
Abrir arquivo
Ler página
Enviar mensagem
```

Isso aumenta microgerenciamento.

---

### 103. Subelemento deve representar unidade real de trabalho

Exemplo:

```
Levantar quantitativos
Solicitar cotação
Revisar orçamento
```

São entregas operacionais verificáveis.

---

### 104. Processo deve reduzir necessidade de cobrança

Se o Processo gera as Tarefas corretas, responsáveis e prazos ficam visíveis.

O gestor não precisa perguntar:

```
"Em que pé está?"
```

---

### 105. Foco em macrogestão

Elemento:

```
visão gerencial
```

Subelementos:

```
execução operacional
```

Processo:

```
padronização
```

Essa divisão ajuda a evitar microgerenciamento.

---

### 106. Exemplo — Orçamento

```
PROCESSO
Elaborar orçamento
```

Inputs:

```
Projetos
Escopo
Prazo do cliente
Cliente
Obra
```

Etapas:

```
1. Conferir documentação
2. Levantar quantitativos
3. Solicitar cotações
4. Compôr orçamento
5. Revisar
6. Enviar
```

Output:

```
Proposta comercial enviada
```

Critérios:

```
Escopo revisado
Premissas registradas
Valores validados
Exclusões registradas
```

---

## 107. Execução — Orçamento

```
ELEMENTO
Arena Center Norte — Orçamento
```

Colunas:

```
Cliente: Center Norte
Obra: Arena Center Norte
Responsável: Ryan
Prazo: 15/10
Status: Em andamento
Valor: R$ 2.500.000
```

## 108. Subelementos — Orçamento

### TAREFAS

Conferir projetos  
Responsável: Jennifer  
Status: Concluída

Levantar quantitativos  
Responsável: Ryan  
Status: Em execução

Solicitar cotações  
Responsável: Luan  
Status: A fazer

Revisar proposta  
Responsável: Paulo  
Status: A fazer

## 109. Exemplo — Compra

Processo:

Realizar compra

Elemento:

Pedido de compra — Cabos Arena

Subelementos:

Validar especificação  
Solicitar três cotações  
Comparar propostas  
Aprovar compra  
Emitir pedido  
Acompanhar entrega

## 110. Exemplo — Contrato

Elemento:

Contrato GEHAKA

Subelementos podem ser:

Aditivo 01  
Aditivo 02  
Aditivo 03

Neste caso, os filhos não precisam ser Tarefas.

Isso reforça que o motor deve ser genérico.

---

## 111. Exemplo — Obra

Elemento:

```
Entrega de documentação elétrica
```

Subelementos:

```
As built
ART
Laudo
Manual
Certificados
```

ART significa **Anotação de Responsabilidade Técnica**.

---

## 112. Modelagem proposta

Conceitualmente:

```
Board
├── element_schema
├── subitem_schema
└── views

BoardItem
├── board
├── parent = null
└── values

BoardSubitem
├── board
├── parent_item
├── origin
├── process_step
└── values
```

---

## 113. Alternativa de modelagem

Também é possível usar uma única tabela:

```
BoardItem
├── parent_id nullable
├── level
└── values
```

---

## 114. Recomendação

Preferir modelo que preserve explicitamente:

- Elemento;
- Subelemento;
- schema de cada nível.

Mesmo que use uma única tabela física.

A regra de domínio deve permanecer clara.

## 115. Processo proposto

```
Process
├── ProcessVersion
│ ├── Inputs
│ ├── Steps
│ ├── Output
│ └── Criteria
```

## 116. Ligação proposta

```
ProcessVersion
├──
└── BoardItem (Elemento)
 ├── ProcessInputValues
 ├── CriterionChecks
 └── BoardSubitems
 └── source_step
```

## 117. Compatibilidade com models atuais

Durante migração:

```
BoardItem(Demanda)
↔ Activity
```

```
BoardSubitem(Tarefa)
↔ Task
```

Não criar dados duplicados sem estratégia de sincronização.

## 118. Estratégia de transição

Fase inicial:

```
Activity continua fonte de verdade da Demanda
Task continua fonte de verdade da Tarefa
```

A camada de Boards fornece:

- schemas;
- visualizações;
- colunas customizadas.



## 119. Aplicação de Processo

A primeira integração real deve fazer:

```
Activity.process_version = versão publicada
```

e gerar Tarefas reais via:

```
TaskService.create_task
```

Não inserir diretamente no banco ignorando Service.

---

## 120. Services

Manter arquitetura:

```
URL
→ View fina
→ Service
→ Model
```

A aplicação de Processo deve ser um Service transacional.

---

## 121. Transação

Aplicar Processo deve ser atômico.

Atômico significa:

ou todas as alterações são concluídas, ou nenhuma é gravada.

Se falhar ao gerar a Tarefa 4:

```
não deixar apenas 3 de 5 criadas
```

sem tratamento explícito.

---

## 122. Idempotência

Aplicar a mesma versão duas vezes por acidente não pode duplicar todas as Tarefas.

Idempotência significa que repetir a mesma operação não gera efeito duplicado indevido.

---

## 123. Identificação de geração

Guardar vínculo:

```
process_application_id
```

ou chave equivalente.

---

## 124. Processo republicado

Nova versão afeta somente novas aplicações por padrão.

---

## 125. Mudança estrutural do Quadro

Se uma nova versão exige uma coluna que não existe:

```
Processo requer "Fornecedor homologado"
```

a publicação deve alertar:

```
Coluna necessária não existe no Quadro.
```

---

## 126. Validação na publicação

Antes de publicar ProcessVersion:

- output definido;
  - ao menos uma etapa;
  - referências de coluna válidas;
  - setores válidos;
  - campos obrigatórios existentes;
  - dependências sem ciclos;
  - templates de Subelemento válidos.
- 

## 127. Ciclo em etapas

Bloquear:

```
A depende de B
B depende de A
```

---

## 128. Alteração de coluna usada em Processo

Se uma coluna é usada por versão publicada:

- renomear é permitido;
  - excluir exige análise;
  - alterar tipo pode ser bloqueado.
- 

## 129. Renomear coluna

Seguro porque Processo referencia ID.

---

## 130. Alterar tipo

Exemplo:

```
Prazo: Data
```

não pode virar:

```
Arquivo
```

se versões publicadas dependem do tipo Data.

---

## 131. Processo como padrão, não prisão

Permitir execução real lidar com exceções.

Exemplo:

```
Cliente pediu revisão adicional.
```

Criar Tarefa manual.

O histórico registra a exceção.

---

## 132. Aprendizado do Processo

Depois, a LPS pode mostrar:

```
Em 68% das execuções deste Processo,
foi criada manualmente uma tarefa "Revisar projeto".
```

Isso pode indicar que ela deveria virar etapa padrão.

---

## 133. Inteligência futura

Exemplo:

```
Sugestão:
Adicionar "Revisar projeto" ao Processo v5?
```

Essa é uma aplicação de Inteligência Artificial baseada em histórico.

---

## 134. Processo e melhoria contínua

Fluxo:

```
Executar
→ observar exceções
→ aprender
→ atualizar Processo
→ publicar nova versão
```

---

## 135. Processo e capital intelectual

Quando uma boa sequência de trabalho é incorporada ao Processo, o conhecimento deixa de depender apenas de uma pessoa.

Exemplo:

```
Jennifer executa uma atividade de forma eficiente
→ método é entendido
→ Processo é melhorado
→ próximas Demandas recebem o padrão
```

Sem transformar desempenho individual em ranking automático.

---

## 136. Indicadores úteis

Por Processo:

- quantidade de Elementos executados;
  - prazo médio;
  - taxa de conclusão;
  - etapas que mais atrasam;
  - etapas extras manuais;
  - bloqueios recorrentes;
  - setores com fila;
  - critérios frequentemente não atendidos.
- 

## 137. Indicadores de Elemento

- Subelementos totais;
  - concluídos;
  - atrasados;
  - bloqueados;
  - próximo prazo;
  - percentual de critérios atendidos.
- 

## 138. Evitar métrica enganosa

Não usar:

```
quantidade de Tarefas concluídas
```

isoladamente para dizer quem é mais produtivo.

Uma Tarefa pode levar 5 minutos ou 5 dias.

---

## 139. Visualização por Processo

Futuro:

```
Filtrar:
Processo = Elaborar orçamento v4
```

Isso permite comparar execuções do mesmo método.

---

## 140. Visualização dos Subelementos

Filtros devem permitir:

```
Mostrar Tarefas de Demandas
onde Cliente = X
```

sem duplicar Cliente na Tarefa.

---

## 141. Contexto do pai em filtros

O motor deve compreender campos herdados.

Exemplo:

```
Tarefa
→ parent.Cliente
```

## 142. Busca

Buscar:

```
Arena Center Norte
```

pode encontrar:

- Elemento;
- Subelementos;
- contexto relacionado.

A UI deve indicar o nível do resultado.

---

## 143. Drawer do Elemento

Deve possuir seção:

```
Tarefas 4/7
```

com:

- lista;
- adicionar;

- abrir;
- filtrar.

---

## 144. Drawer do Subelemento

Deve possuir:

```
Demanda vinculada
Arena Center Norte
```

clicável.

---

## 145. Colunas do pai no card filho

Por padrão, não repetir.

Mas uma visualização de Tarefas pode mostrar:

```
Demanda vinculada
Cliente
Obra
```

como campos derivados do pai.

---

## 146. Campos derivados

Marcar como:

```
Somente leitura
```

se vierem do pai.

---

## 147. Agrupamento de Subelementos

Pode agrupar Tarefas por:

```
Status
Responsável
Setor
Prioridade
Demanda pai
Cliente do pai
Obra do pai
```

---

## 148. Processo e Grupo visual

Um Processo pode gerar Tarefas em diferentes setores.

O Kanban pode agrupar por:

```
Setor
```

independentemente da ordem do Processo.

---

## 149. Ordem do Processo

Guardar ordem da etapa separadamente da ordem visual.

---

## 150. Reordenar Kanban não muda Processo

Se o usuário muda a ordem visual das colunas:

A fazer | Bloqueada | Em execução | Concluída

não altera a sequência do Processo.

---

## 151. Concluir Subelemento

Conclusão deve obedecer regra da Tarefa e, se derivada de Processo, validar critério específico da etapa quando houver.

---

## 152. Reabrir Subelemento

Se permitido:

- auditar;
- recalcular progresso do pai;
- atualizar agregações.

---

## 153. Cancelar Subelemento

Não considerar automaticamente como concluído.

O Processo precisa decidir se uma etapa cancelada satisfaz a execução.

---

## 154. Etapa dispensada

É diferente de cancelada.

Futuro status de execução:

Dispensada

com motivo.

---

## 155. Processo condicional

Futuro:

Se Valor > R\$ 1.000.000  
→ adicionar etapa Aprovação da Diretoria

Não faz parte da primeira integração obrigatória.

---

## 156. Processo com caminhos

Futuro:

```
Se projeto completo
→ Levantamento

Se incompleto
→ Solicitar complementação
```

Exige motor de condições.

---

## 157. Primeira versão do Processo integrado

Manter simples:

```
fluxo linear
etapas
setor
ordem
dependência
criação de Tarefas
inputs
output
critérios
```

---

## 158. O que não fazer agora

Não implementar simultaneamente:

- BPMN completo;
- fluxos recursivos;
- decisões complexas;
- dezenas de gatilhos;
- subprocessos ilimitados;
- múltiplos níveis de Subelementos.

BPMN significa **Business Process Model and Notation**, uma notação formal para modelagem de processos.

---

## 159. Processo × Automação

Processo:

```
define método
```

Automação:

```
reage a evento
```

Exemplo:



Processo:

```
Etapa = Revisar proposta
```

Automação:

```
Quando tarefa "Revisar proposta" concluir
→ notificar responsável da Demanda
```

---

## 160. Processo x Fórmula

Fórmula:

```
calcula
```

Processo:

```
organiza execução
```

---

## 161. Processo x Rollup

Rollup permite ao Elemento enxergar execução dos Subelementos.

Exemplo:

```
Tarefas atrasadas = 2
```

---

## 162. Processo x Dashboard

Dashboard pode consumir:

- Elementos;
- Subelementos;
- Processos;
- agregações.

Não é responsabilidade do Processo desenhar o Dashboard.

---

## 163. Regra central de produto

O Processo define o padrão; o Elemento representa a entrega; o Subelemento representa a execução.

## 164. Outra forma de enxergar

PROCESSO

Como fazemos?

ELEMENTO

O que precisamos entregar?

SUBELEMENTO

O que precisa ser feito para entregar?

## 165. Papel gerencial

Gestor olha principalmente:

Elementos

e exceções dos Subelementos.

## 166. Papel operacional

Executor trabalha principalmente:

Subelementos

com contexto do Elemento pai.

## 167. Gestão por exceção

Em vez de acompanhar toda Tarefa:

mostrar ao gestor:

- atrasadas;
- bloqueadas;
- sem responsável;
- risco de prazo;
- excesso de carga.

## 168. Processo como prevenção de microgerenciamento

Um Processo bem definido reduz a necessidade de perguntar:

Você já fez?

Quem está fazendo?

Qual é o próximo passo?

Onde está o arquivo?

Essas respostas ficam no fluxo.

## 169. Regra para card pai

Card de Elemento deve mostrar indicador de filhos.

Exemplo:

Um card de fundo escuro com o texto "3 tarefas" em uma fonte clara.

## 170. Estado expandido

Ao expandir:

- não sair do Kanban;
- filhos aparecem abaixo;
- edição contextual;
- `+ Adicionar tarefa`.

## 171. Persistência da expansão

Pode ser sessão/local.

Não precisa ser propriedade de negócio do Elemento.

## 172. Muitos Subelementos no card

Não renderizar 50 filhos indiscriminadamente.

Exemplo:

Um card de fundo escuro com o texto "Mostrar 5" e "+ 17 tarefas" em uma fonte clara.

ou usar scroll interno controlado.

## 173. Performance

Carregar filhos sob demanda quando o card for expandido.

Evitar carregar todos os Subelementos de todas as Demandas no primeiro render.

## 174. Contador

O contador deve refletir quantidade de filhos visíveis conforme permissão.

Não revelar:

Um card de fundo escuro com o texto "10 tarefas" em uma fonte clara.

se o usuário só pode acessar 2 e a quantidade total for sensível.

## 175. Criação contextual no Kanban

Clicar:

```
+ Adicionar tarefa
```

já preenche:

```
parent = Demanda atual
```

## 176. Criação contextual na Tabela

Mesma regra.

## 177. Criação no drawer

Dentro da Demanda:

```
+ Nova tarefa
```

cria vinculada.

## 178. Processo e criação manual

Se uma Tarefa manual é criada:

```
origin = MANUAL
```

Não tentar vinculá-la artificialmente a uma ProcessStep.

## 179. Vincular manual a etapa

Pode existir ação futura:

```
Vincular à etapa
```

mas deve ser explícita.

## 180. Segurança na aplicação do Processo

Antes de gerar filhos, verificar:

- organização;
- permissão de aplicar Processo;
- permissão para criar Subelementos nos setores envolvidos;
- validade da versão.

## 181. Falha de permissão em uma etapa

Não gerar Processo parcialmente sem explicação.

Exemplo:

Você não possui permissão para criar Tarefa no setor Suprimentos.

---

## 182. Aplicação por gestor

Um perfil autorizado pode aplicar Processo com etapas em vários setores.

A autorização deve ser desenhada deliberadamente.

---

## 183. Critérios de aceite — Elemento

### ELS-001

Elemento possui ID próprio.

### ELS-002

Elemento utiliza schema próprio de colunas.

### ELS-003

Pode existir sem filhos.

### ELS-004

Pode ser visualizado em Tabela, Kanban e Calendário.

### ELS-005

Mudanças refletem nas demais visualizações.

---

## 184. Critérios de aceite — Subelemento

### SUB-001

Subelemento possui ID próprio.

### SUB-002

Pertence exatamente a um pai.

### SUB-003

Possui schema de colunas independente do pai.

## **SUB-004**

Pode ter Status, Pessoa, Data e demais campos próprios.

## **SUB-005**

Pode ser criado sem sair da visualização atual.

## **SUB-006**

Pode ser expandido dentro da Tabela.

## **SUB-007**

Pode ser expandido abaixo do card pai no Kanban.

---

## **185. Critérios de aceite — Kanban**

### **KSB-001**

Card do pai mostra contador de Subelementos.

### **KSB-002**

Clicar expande filhos sem mudar de tela.

### **KSB-003**

Status do filho não move o card pai.

### **KSB-004**

Kanban próprio de Subelementos pode usar Status do filho.

### **KSB-005**

Adicionar Subelemento contextualiza o pai automaticamente.

---

## **186. Critérios de aceite — Processo**

### **PRO-001**

Apenas versão publicada pode ser aplicada.

### **PRO-002**

Elemento guarda versão aplicada.

## PRO-003

Nova versão não altera Elementos em execução automaticamente.

## PRO-004

Etapas podem gerar Subelementos.

## PRO-005

Subelementos gerados mantêm referência à etapa de origem.

## PRO-006

Aplicação não pode duplicar filhos ao ser repetida acidentalmente.

## PRO-007

Aplicação deve ser atômica.

---

## 187. Critérios de aceite — Inputs

### INP-001

Inputs obrigatórios são identificados.

### INP-002

Quando possível, Input referencia coluna existente.

### INP-003

Renomear a coluna não quebra o Processo.

### INP-004

Excluir coluna usada por versão publicada exige tratamento.

---

## 188. Critérios de aceite — Output

### OUT-001

Processo possui descrição do resultado esperado.

### OUT-002

Execução pode registrar evidência.

## **OUT-003**

Conclusão pode exigir evidência quando configurado.

---

## **189. Critérios de aceite — Critérios**

### **CRT-001**

Critério pertence à versão do Processo.

### **CRT-002**

Verificação pertence ao Elemento real.

### **CRT-003**

Critério obrigatório pode bloquear conclusão.

### **CRT-004**

Nova versão não reescreve checks históricos.

---

## **190. Critérios de aceite — Auditoria**

### **AUD-001**

Aplicação de Processo é auditada.

### **AUD-002**

Versão aplicada é registrada.

### **AUD-003**

Criação automática de filhos é rastreável.

### **AUD-004**

Filho manual possui origem identificável.

### **AUD-005**

Migração de Processo exige histórico.

---

## **191. Critérios de aceite — Gestão**

### **GST-001**

Elemento mostra quantidade de filhos.

---



## GST-002

Pode exibir progresso agregado.

## GST-003

Pode exibir quantidade de atrasados/bloqueados.

## GST-004

Indicadores não alteram Status automaticamente sem regra.

---

## 192. Implementação em fases

### Fase 1 — Relação Elemento/Subelemento

- schema de Elemento;
  - schema de Subelemento;
  - expandir/recolher;
  - criação contextual;
  - contador;
  - colunas independentes.
- 

### Fase 2 — Kanban

- filhos expansíveis;
  - card configurável do filho;
  - Kanban próprio de Subelementos.
- 

### Fase 3 — Processo aplicado

- selecionar versão;
  - gravar `process_version` ;
  - gerar Tarefas;
  - vínculo Task ↔ ProcessStep;
  - origem manual/processo.
- 

### Fase 4 — Inputs e critérios

- mapear inputs;
  - registrar valores;
  - critérios de aceite;
  - validação de conclusão.
- 

### Fase 5 — Rollups

- total;
  - concluídas;
-

- atrasadas;
- bloqueadas;
- próximo prazo.

---

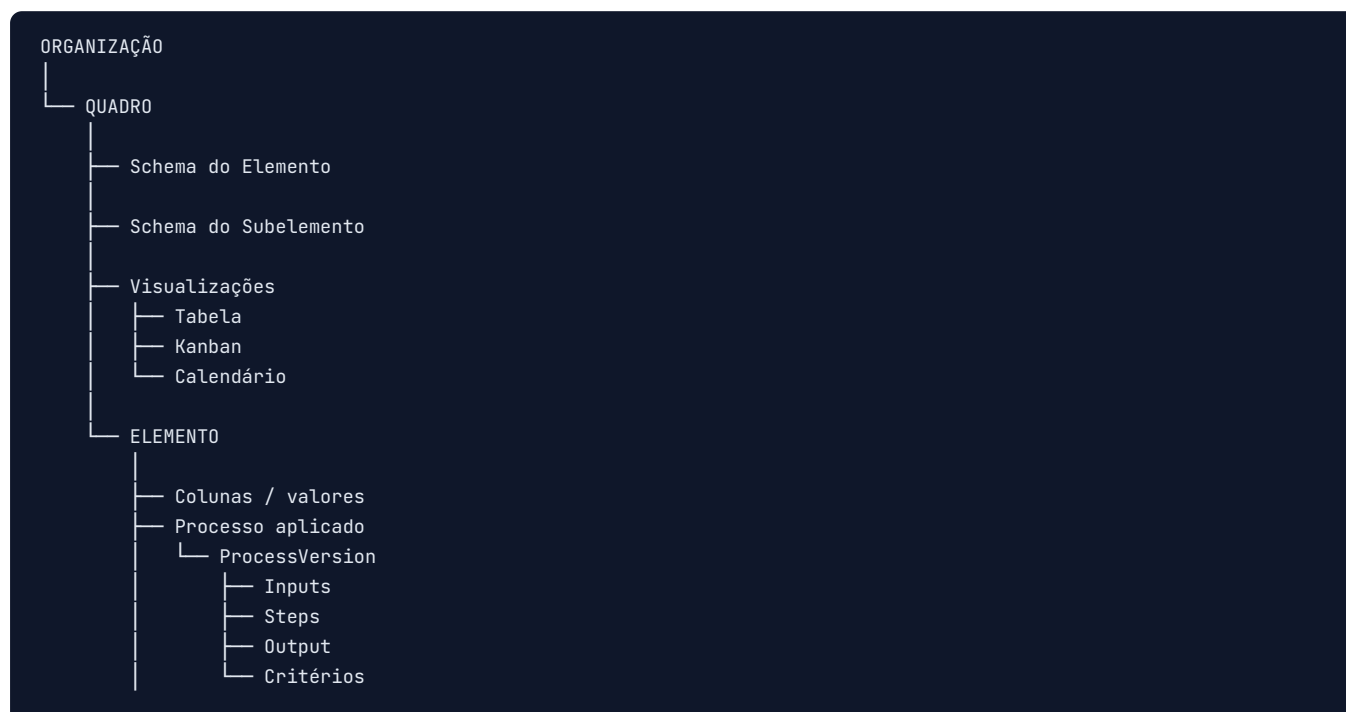
## Fase 6 — Aprendizado

- exceções recorrentes;
  - sugestões de melhoria do Processo;
  - indicadores por versão.
- 

### 193. Decisões arquiteturais obrigatórias

1. Elemento e Subelemento são registros diferentes.
  2. Subelemento possui schema independente.
  3. Um nível de filhos na primeira versão.
  4. Processo é blueprint, não execução.
  5. Processo aplicado deve fixar uma versão.
  6. ProcessStep é template; Subelemento é instância.
  7. Um Step pode gerar um ou mais filhos quando necessário.
  8. Filhos manuais continuam permitidos.
  9. Origem do filho deve ser registrada.
  10. Processo não deve duplicar os dados do Quadro.
  11. Inputs devem mapear para colunas por ID sempre que possível.
  12. Mudança de versão não altera execução antiga automaticamente.
  13. Kanban do pai e Kanban dos filhos são visualizações distintas.
  14. Status do pai e do filho são independentes.
  15. Processo deve reduzir microgerenciamento, não aumentar.
- 

### 194. Diagrama final



#### SUBELEMENTOS

- Subelemento A ← ProcessStep
- Subelemento B ← ProcessStep
- Subelemento C ← Manual

## 195. Aplicação específica em Demandas

QUADRO  
Demandas

ELEMENTO  
Demanda

SUBELEMENTO  
Tarefa

PROCESSO  
Como aquela Demanda deve ser executada

## 196. Exemplo resumido

Processo:  
Elaborar orçamento v4

gera execução:

Demanda:  
Arena Center Norte

com filhos:

Conferir documentação  
Levantar quantitativos  
Solicitar cotações  
Revisar proposta  
Enviar proposta

Cada filho possui:

Responsável  
Setor  
Status  
Prazo  
Prioridade

A Demanda possui:

Cliente  
Obra  
Solicitante  
Responsável  
Status  
Prazo  
Valor

## 197. Regra de linguagem da interface

O motor interno pode utilizar:

Elemento  
Subelemento

Mas a interface deve usar a linguagem do domínio.

Exemplo:

Demandas / Tarefas

e não:

Elementos / Subelementos

se isso for mais natural para o usuário.

Outro Quadro pode usar:

Pedido / Itens

## 198. Definição final — Elemento

Elemento é a unidade principal de trabalho ou registro dentro de um Quadro, com identidade, colunas e ciclo de vida próprios.

## 199. Definição final — Subelemento

Subelemento é uma unidade filha pertencente a um Elemento, com identidade e esquema próprios, usada para decompor a entrega principal em partes operacionais ou componentes relacionados.

## 200. Definição final — Processo

Processo é um modelo versionado que define como um determinado tipo de Elemento deve ser executado, incluindo inputs, etapas, output e critérios de aceite.

## 201. Relação final

PROCESSO = padrão  
ELEMENTO = entrega  
SUBELEMENTO = execução

Essa divisão deve ser uma das bases centrais da arquitetura da LPS.

Ela permite:

- flexibilidade de Quadro;
- padronização de Processo;
- execução operacional;
- visão gerencial;
- rastreabilidade;
- evolução contínua;
- gestão por exceção;
- menor dependência de microgerenciamento.

# Apêndice B — Especificação Completa das Colunas Dinâmicas

## Status de implementação (03/10/2026)

**Resumo.** O motor de colunas existe no app `boards` (quadros dinâmicos: `Board`, `BoardColumn`, `BoardColumnOption`, `BoardItem`, `BoardCell`), com visões Tabela ("Quadro principal", implícita), Kanban e Calendário sobre os mesmos itens e valores. Das 13 colunas-base desta especificação, **8 estão implementadas e podem ser criadas**: Texto, Número, Moeda, Data, Pessoas (somente uma pessoa por célula), Status, Lista suspensa (seleção única) e Confirmação (tipo técnico `CHECKBOX`, rótulo "Sinal de confirmação"). **Prioridade, Arquivo, Relação, Cronograma e Fórmula não existem como colunas de quadro**: `PRIORITY`, `FILE`, `RELATION`, `TIMELINE`, `FORMULA` (além de `CONFIRMATION` e `AI_EXTRACT`) estão apenas reservados no catálogo `BoardColumn.Type` e não se criam nem se editam. As operações comuns (criar, renomear inline, mover, redimensionar, ocultar, duplicar, alterar tipo com plano de conversão, excluir logicamente, autosave por célula), a auditoria e as permissões no servidor estão implementadas. **Não existem** filtros por coluna, autoajuste de largura, linha de resumo, verificação de dependências na exclusão, restrição de visualização por coluna, cópia/cola e exportação. Em paralelo, Demandas e Tarefas usam o "quadro de domínio" (`DomainBoard*`) sobre `Activity` e `Task`, com campos de sistema fixos e campos customizados simples (`DomainCustomValue`). Estimativa: cerca de 45% da especificação está implementada (revisão do código no commit `add5acd`, de 02/10/2026).

Vocabulário atual da interface: "Atividade" virou **Demanda** (o código segue `Activity`), "Situação" virou **Etap**a (`stage`) e "Condição" virou **Status** (`condition`). No quadro dinâmico, o item aparece como "Tarefa" (`Board.item_label`, padrão "Nome da Tarefa").

| Requisito / bloco                                                     | Status                                    | Onde está no código                                                                                                                                                                           |
|-----------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modelo de coluna (§5)                                                 | Divergente (nomes diferentes)             | <code>boards/models.py</code> ( <code>BoardColumn : is_required</code> , <code>board</code> , <code>settings</code> , <code>position</code> <code>decimal</code> , <code>width</code> 96–640) |
| Tipos-base (§4, §61, §62)                                             | Parcial (8 de 13)                         | <code>boards/models.py</code> ( <code>BoardColumn.Type</code> , <code>ACTIVE_TYPES</code> ); <code>boards/presentation.py</code> ( <code>TYPE_INFO</code> )                                   |
| Operações comuns: criar, renomear, mover, redimensionar (§6.1–6.4)    | Implementado                              | <code>boards/services.py</code> ( <code>ColumnService</code> , <code>PositionService</code> ); <code>boards/views.py</code> ; <code>static/js/boards.js</code>                                |
| Autoajuste, filtro por coluna (§6.5, §6.7)                            | Não implementado                          | — (só busca textual e filtro por pessoa em <code>boards/queries.py</code> )                                                                                                                   |
| Ordenar, agrupar, ocultar, excluir (§6.6, §6.8, §6.12, §6.13)         | Parcial                                   | <code>boards/queries.py</code> ( <code>sort_expression</code> ); <code>boards/kanban.py</code> ; <code>ColumnService.set_visible</code> , <code>soft_delete</code>                            |
| Duplicar coluna, alterar tipo (§6.9, §6.11)                           | Divergente                                | <code>ColumnService.duplicate</code> , <code>conversion_plan</code> , <code>change_type</code>                                                                                                |
| Autosave (§6.14)                                                      | Parcial                                   | gravação por célula em <code>CellService.set_value</code> ; diálogo "Configurações da coluna" tem botão Salvar                                                                                |
| Visões: Tabela, Kanban, Calendário, gaveta de detalhes (§7, §28, §29) | Parcial                                   | <code>boards/kanban.py</code> , <code>boards/calendar_view.py</code> , <code>templates/boards/*</code> ; formulário de criação não existe                                                     |
| Colunas Texto, Número, Moeda, Data (§8–§11)                           | Parcial                                   | <code>boards/validators.py</code> , <code>boards/services.py</code> ( <code>CellService</code> ), <code>templates/boards/_cell.html</code>                                                    |
| Coluna Pessoas (§12)                                                  | Parcial (uma pessoa)                      | <code>BoardCellUser</code> ; <code>core/views.py</code> ( <code>PersonSearchView</code> ); <code>boards/validators.py</code>                                                                  |
| Colunas Status e Lista suspensa (§13, §14)                            | Implementado (Lista só com seleção única) | <code>BoardColumnOption</code> , <code>OptionService</code> ; <code>static/js/boards.js</code> ( <code>openLabels</code> , <code>editOption</code> )                                          |

| Requisito / bloco                                                 | Status                                     | Onde está no código                                                                                                                                                                             |
|-------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Coluna Confirmação (§16)                                          | Implementado (como <code>CHECKBOX</code> ) | <code>BoardCell.value_boolean</code> ; <code>static/js/boards.js</code>                                                                                                                         |
| Prioridade, Arquivo, Relação, Cronograma, Fórmula (§15, §17–§20)  | Não implementado em quadros dinâmicos      | apenas valores reservados em <code>BoardColumn.Type</code> ; no quadro de domínio, Prioridade e "Demanda vinculada" existem como campos de sistema                                              |
| Campos de negócio e conjuntos iniciais (§21–§23)                  | Parcial                                    | <code>boards/domain_defaults.py</code> , <code>boards/domain_services.py</code> , <code>boards/work_views.py</code> , <code>boards/starter_templates.py</code> (modelo "Orçamentos")            |
| Obrigatória, valor padrão, validações (§24–§26)                   | Parcial                                    | <code>normalize_cell_value</code> ; <code>ItemService.create</code> (etiqueta padrão); obrigatoriedade não é checada na criação do item                                                         |
| Resumo de coluna (§27)                                            | Parcial                                    | só soma de Número/Moeda no cabeçalho da raia do Kanban ( <code>boards/kanban.py</code> , <code>build_lanes</code> )                                                                             |
| Edição contextual, popovers, gaveta (§30–§32, §58, §59)           | Implementado                               | <code>static/js/boards.js</code> ( <code>openCellEditor</code> , <code>editDate</code> , <code>editOption</code> , <code>editPerson</code> ); <code>templates/boards/_item_drawer.html</code>   |
| Segurança, multi-organização, auditoria (§33, §36–§38)            | Implementado                               | <code>boards/services.py</code> ( <code>_require</code> , <code>_audit</code> ); <code>boards/views.py</code> ( <code>get_*</code> por organização); <code>boards/presentation.py</code>        |
| Restrição por coluna, notificações de coluna (§34, §35, §39, §40) | Não implementado                           | —                                                                                                                                                                                               |
| Valores vazios, pesquisa (§41, §45)                               | Parcial                                    | <code>templates/boards/_cell.html</code> ; <code>BoardQueryService.search_filter</code>                                                                                                         |
| Cópia e cola, exportação (§43, §44)                               | Não implementado                           | —                                                                                                                                                                                               |
| Campos não editáveis, de sistema, futuros, IA (§46–§50)           | Parcial (só no quadro de domínio)          | <code>DomainBoardField.is_system</code> ; <code>BoardColumn.Type.AI_EXTRACT</code> reservado                                                                                                    |
| Nomes, soft delete, performance, acessibilidade (§51–§57)         | Parcial                                    | <code>ColumnService._unique_name</code> ; <code>is_active</code> ; <code>BoardQueryService.with_cells</code> ; <code>static/css/boards.css</code>                                               |
| Contrato por tipo e arquitetura de classes (§61, §62)             | Divergente                                 | lógica por tipo em ramos <code>if</code> ( <code>validators</code> , <code>services</code> , <code>queries</code> , <code>templates</code> , <code>JS</code> ), sem <code>BaseColumnType</code> |
| Critérios de aceite (§63–§69)                                     | Parcial                                    | tabelas de status em cada seção                                                                                                                                                                 |

## Nomes na especificação x nomes no código

| Especificação                                                                   | Código                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>board_id</code> , <code>required</code>                                   | <code>board</code> (chave estrangeira), <code>is_required</code>                                                                                                                                                                         |
| <code>MONEY</code> , <code>PEOPLE</code> , <code>CONFIRMATION</code>            | <code>CURRENCY</code> , <code>PERSON</code> , <code>CHECKBOX</code> ( <code>CONFIRMATION</code> fica reservado)                                                                                                                          |
| <code>date_format</code> (configuração de Data)                                 | <code>format</code> ( <code>DD/MM/YYYY</code> , <code>DD/MM/YY</code> , <code>YYYY-MM-DD</code> )                                                                                                                                        |
| opção de Status: <code>name</code> , <code>order</code> , <code>behavior</code> | <code>BoardColumnOption</code> : <code>label</code> , <code>position</code> , <code>is_default</code> , <code>is_done</code> , <code>is_active</code>                                                                                    |
| valor da célula                                                                 | <code>BoardCell</code> tipada: <code>value_text</code> , <code>value_number</code> , <code>value_date</code> , <code>value_datetime</code> , <code>value_boolean</code> , mais <code>BoardCellUser</code> e <code>BoardCellOption</code> |

## 1. Objetivo

Este documento define o padrão oficial de **colunas dinâmicas da LPS**.

A coluna é uma das peças centrais do novo motor de Quadros.

A ideia é simples:

```
Quadro
├── Colunas
├── Itens
└── Visualizações
```

Cada coluna define:

- qual informação existe;
- qual é o tipo da informação;
- como o usuário preenche;
- como o valor aparece;
- como o valor é validado;
- como pode ser filtrado;
- como pode ser ordenado;
- como pode ser agrupado;
- como aparece no Kanban;
- como aparece no Calendário;
- como pode participar de Fórmulas;
- quais permissões controlam sua edição e visualização.

A coluna não pertence exclusivamente à Tabela.

Ela pertence ao **Quadro**.

Tabela, Kanban e Calendário usam a mesma coluna e o mesmo valor.

## 2. Regra principal

Um dado deve existir uma única vez e ser apresentado de formas diferentes pelas visualizações.

Exemplo:

```
Coluna: Status
Valor: Em andamento
```

Na Tabela:

```
| Em andamento |
```

No Kanban:

```
coluna "Em andamento"
```



No Calendário:

```
card com etiqueta "Em andamento"
```

No filtro:

```
Status = Em andamento
```

Não criar uma versão do Status para cada tela.

**Status (03/10/2026): Implementado nos quadros dinâmicos.** O valor existe uma única vez em `BoardCell` (por item e coluna) e as visões só o leem: a Tabela ("Quadro principal", sem registro próprio), o Kanban ( `BoardView` tipo `KANBAN` , `boards/kanban.py` ) e o Calendário ( `BoardView` tipo `CALENDAR` , `boards/calendar_view.py` ). Mover um cartão grava a mesma célula pelo mesmo serviço da Tabela ( `CellService.set_value` ).

### 3. Situação atual da LPS x nova arquitetura

Na LPS atual, `Activity` e `Task` ainda possuem vários campos definidos diretamente no código e no banco de dados.

Exemplos atuais:

```
Activity.owner
Activity.requested_by
Activity.sector
Activity.client
Activity.site
Activity.status
Activity.urgency
Activity.requested_deadline

Task.responsavel
Task.sector
Task.status
Task.requested_deadline
Task.committed_deadline
```

Na nova arquitetura, esses conceitos passam a ser apresentados por um **motor comum de colunas**.

Exemplo:

|                   |              |
|-------------------|--------------|
| Responsável       | → Pessoas    |
| Solicitante       | → Pessoas    |
| Setor             | → Relação    |
| Cliente           | → Relação    |
| Obra              | → Relação    |
| Prazo             | → Data       |
| Valor da proposta | → Moeda      |
| Status            | → Status     |
| Prioridade        | → Prioridade |
| Margem            | → Fórmula    |

Isso permite adicionar novos campos sem criar nova tela e sem alterar o banco para cada pequena necessidade de negócio.

**Divergência (03/10/2026):** o motor comum ainda não substituiu os campos de Activity e Task. Eles continuam no modelo e no banco ( Activity : client , site , cost\_center , sector , stage , condition , status , urgency , owner , requested\_by , requested\_deadline ; Task : sector , stage , condition , status , priority , requested\_deadline , committed\_deadline , responsavel ). O quadro de domínio ( DomainBoard , DomainBoardField , boards/domain\_defaults.py ) apresenta um subconjunto deles como **campos de sistema** ( is\_system=True ) e grava sempre pelos services de activities ( ActivityService , TaskService ), que mantêm regras e auditoria. Só os campos customizados simples ( DomainCustomValue ) usam o mecanismo novo. Termos atuais: Situação = **Etapa** ( stage ), Condição = **Status** ( condition ), Atividade = **Demanda**. O mapeamento da lista acima (Setor, Cliente e Obra como Relação; Margem como Fórmula) é **futuro**.

## 4. Tipos-base oficiais da LPS

A primeira arquitetura deve trabalhar com 13 tipos-base:

1. Texto
2. Número
3. Moeda
4. Data
5. Pessoas
6. Status
7. Lista suspensa
8. Prioridade
9. Confirmação
10. Arquivo
11. Relação
12. Cronograma
13. Fórmula

Esses são os tipos fundamentais.

Colunas de negócio são construídas sobre eles.

Exemplo:

```
"Responsável" não é um tipo.
É uma coluna do tipo Pessoas.

"Cliente" não é um tipo.
É uma coluna do tipo Relação.

"Valor da proposta" não é um tipo.
É uma coluna do tipo Moeda.
```

**Divergência (03/10/2026):** em quadros dinâmicos estão implementados **8 dos 13 tipos**: Texto ( TEXT ), Número ( NUMBER ), Moeda ( CURRENCY ), Data ( DATE ), Pessoas ( PERSON ), Status ( STATUS ), Lista suspensa ( DROPDOWN ) e Confirmação ( CHECKBOX , rótulo "Sinal de confirmação"). O seletor os oferece nesta ordem: Status, Lista suspensa, Texto, Data, Pessoa, Número, Moeda, Sinal de confirmação ( BoardColumn.ACTIVE\_TYPES ). **Prioridade, Arquivo, Relação, Cronograma e Fórmula não foram implementados** como tipos de quadro: PRIORITY , FILE , RELATION , TIMELINE , FORMULA , CONFIRMATION e AI\_EXTRACT existem no catálogo apenas como reserva ("fases futuras") e ColumnService.create os recusa ("Tipo de coluna inválido"). O quadro de domínio de Demandas e Tarefas tem um catálogo separado ( DomainBoardField.Type : TEXT , NUMBER , CURRENCY , DATETIME , PERSON , STAGE , SECTOR , PRIORITY , SELECT , BOOLEAN , CHECKLIST , RELATION ), usado para os campos de sistema.

## 5. Estrutura conceitual de uma coluna

Toda coluna deve possuir propriedades comuns.

Exemplo conceitual:

```
Column

id
board_id
name
type
position
width
required
description
is_visible
is_active
settings
created_at
updated_at
```

### 5.1 id

Identificador interno e imutável.

O nome visível nunca deve ser usado como identidade técnica.

### 5.2 board\_id

Quadro ao qual a coluna pertence.

### 5.3 name

Nome mostrado ao usuário.

Exemplos:

```
Responsável
Prazo
Cliente
Valor
Status
```

Pode ser renomeado sem quebrar dados.

### 5.4 type

Define o comportamento da coluna.

Exemplos:

```
TEXT
NUMBER
MONEY
DATE
PEOPLE
STATUS
DROPDOWN
PRIORITY
CONFIRMATION
FILE
RELATION
TIMELINE
FORMULA
```

**Divergência:** os identificadores técnicos são `TEXT`, `NUMBER`, `CURRENCY` (e não `MONEY`), `DATE`, `PERSON` (e não `PEOPLE`), `STATUS`, `DROPDOWN` e `CHECKBOX` (a spec chama de `CONFIRMATION`, que ficou reservado). Os demais (`PRIORITY`, `FILE`, `RELATION`, `TIMELINE`, `FORMULA`) estão reservados e não são criáveis.

## 5.5 position

Define a ordem da coluna.

A ordem deve ser alterável por arrastar e soltar.

## 5.6 width

Largura visual da coluna na Tabela.

Deve ser persistida.

## 5.7 required

Indica se o valor é obrigatório em criação/edição.

## 5.8 description

Ajuda contextual da coluna.

Exemplo:

```
Prazo comprometido de entrega ao solicitante.
```

## 5.9 is\_visible

Controla se a coluna aparece na visualização atual.

Não significa exclusão.

**Divergência:** `BoardColumn.is_visible` vale para o quadro inteiro (a Tabela), não por visualização. Kanban e Calendário escolhem seus campos de cartão à parte, em `BoardView.settings["card_fields"]`. Só o quadro de domínio guarda visibilidade, ordem e largura por visualização (`DomainBoardViewColumn`).

## 5.10 is\_active

Permite desativação lógica.

## 5.11 settings

Guarda configurações específicas de cada tipo.

Exemplo de Data:

```
{
 "show_time": true,
 "date_format": "DD/MM/YYYY"
}
```

Exemplo de Pessoas:

```
{
 "multiple": false
}
```

**Divergência:** o JSON de configuração é validado por `clean_column_settings` ( `boards/validators.py` ) e só aceita chaves conhecidas por tipo: Data ( `show_time` , `allow_weekends` , `is_deadline` , `format` ), Número ( `decimal_places` , `unit` , `minimum` , `maximum` ), Moeda ( `currency` , `decimal_places` , `minimum` , `maximum` ) e Pessoa ( `multiple` , que só aceita `false` ). Na Data a chave é `format` e não `date_format` . Texto, Status, Lista e Confirmação não têm configuração própria.

Mapeamento das propriedades comuns para `BoardColumn` :

| Propriedade da spec                                  | Campo no código                                      | Observação                                                                                                                                                                 |
|------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                                      | <code>id</code>                                      | imutável; o nome nunca é identidade                                                                                                                                        |
| <code>board_id</code>                                | <code>board</code>                                   | chave estrangeira para <code>Board</code>                                                                                                                                  |
| <code>name</code>                                    | <code>name</code>                                    | até 120 caracteres                                                                                                                                                         |
| <code>type</code>                                    | <code>type</code>                                    | ver §5.4                                                                                                                                                                   |
| <code>position</code>                                | <code>position</code>                                | decimal (20,6), passo 1000; o cliente envia os vizinhos ( <code>before_id</code> , <code>after_id</code> ) e o servidor calcula a posição ( <code>PositionService</code> ) |
| <code>width</code>                                   | <code>width</code>                                   | padrão 160, mínimo 96, máximo 640                                                                                                                                          |
| <code>required</code>                                | <code>is_required</code>                             | ver §24                                                                                                                                                                    |
| <code>description</code>                             | <code>description</code>                             | o serviço grava até 500 caracteres                                                                                                                                         |
| <code>is_visible</code>                              | <code>is_visible</code>                              | por quadro, ver §5.9                                                                                                                                                       |
| <code>is_active</code>                               | <code>is_active</code>                               | exclusão lógica                                                                                                                                                            |
| <code>settings</code>                                | <code>settings</code>                                | ver acima                                                                                                                                                                  |
| <code>created_at</code> ,<br><code>updated_at</code> | <code>created_at</code> ,<br><code>updated_at</code> | além de <code>created_by</code>                                                                                                                                            |

## 6. Regras comuns a todas as colunas

Toda coluna deve seguir as mesmas regras de experiência, exceto onde o tipo exigir comportamento diferente.

### 6.1 Criar coluna

Fluxo:

[ + ]

Pesquisar ou descrever sua coluna

Status

Texto

Pessoas

Lista suspensa

Data

Números

...

O usuário escolhe o tipo.

A coluna aparece imediatamente.

Não abrir uma página separada.

**Status (03/10/2026): Implementado, com diferenças.** O botão "+" do cabeçalho abre um popover com a grade de tipos ("Escolha o tipo da coluna"); a coluna entra na hora, sem trocar de página ( `ColumnService.create` , `static/js/boards.js` , `openTypePicker` ).

**Divergência:** o seletor não tem campo de pesquisa nem "descrever sua coluna", e oferece 8 tipos (ver §4). Nomes iniciais: Texto, Números, Valor, Data, Pessoa, Status, Lista suspensa, Confirmação (com sufixo "2", "3"... se já existir). Uma coluna de Status nasce com três etiquetas: "Não iniciado" (padrão), "Em andamento" e "Concluído" (conta como concluído).

### 6.2 Renomear

O nome deve ser editável inline.

Fluxo:

Status

clicou:

[ Status ]

digitou:

Etapa comercial

Enter:

Etapa comercial

Autosave.

**Status (03/10/2026): Implementado.** Edição inline no cabeçalho (Enter confirma, Esc cancela) e gravação imediata ( `ColumnService.rename` , endpoint `board-column-rename` ). O serviço não impede nome repetido (ver §52).

## 6.3 Mover

A coluna deve poder ser arrastada horizontalmente.

Exemplo:

Cliente | Responsável | Status | Prazo

vira:

Cliente | Status | Responsável | Prazo

A ordem é salva automaticamente.

**Status (03/10/2026): Implementado.** Arrastar o cabeçalho; a nova posição é gravada entre as vizinhas ( `ColumnService.reorder` , endpoint `board-column-reorder` ) e desfeita na tela se o servidor recusar.

## 6.4 Redimensionar

O usuário arrasta a divisória lateral do cabeçalho.

Exemplo:

| Responsável |

pode virar:

| Responsável |

A largura é persistida.

**Status (03/10/2026): Implementado.** Arrastar a divisória do cabeçalho ou usar as setas do teclado (16 px; 48 px com Shift); limites de 96 a 640 px, padrão 160 ( `ColumnService.resize` , endpoint `board-column-resize` ). No quadro dinâmico a largura é da coluna e vale para todas as pessoas.

## 6.5 Autoajuste

Duplo clique na divisória pode ajustar automaticamente a largura conforme o conteúdo.

**Status (03/10/2026): Não implementado.** Não há tratamento de duplo clique na divisória ( `static/js/boards.js` ). Continua como requisito futuro.

## 6.6 Ordenar

Toda coluna ordenável deve oferecer:

```
Ordenar crescente
Ordenar decrescente
Remover ordenação
```

Cada tipo define sua lógica de comparação.

**Status (03/10/2026): Parcial.** O menu da coluna oferece "Ordenar ascendente", "Ordenar descendente" e "Remover ordenação". A ordenação é por uma coluna de cada vez, via `?sort=<id>&dir=asc|desc` (a página recarrega) e é feita no banco ( `BoardQueryService.sort_expression` ), dentro de cada grupo: Texto sem diferenciar maiúsculas, Número e Moeda numericamente, Data cronologicamente (considera a hora quando `show_time` está ligado), Status e Lista pela posição das etiquetas, Pessoa pelo nome, Confirmação pelo valor; vazios por último. O Kanban tem ordenação própria (ordem do quadro, título, criação ou coluna), salva na visualização.

## 6.7 Filtrar

Toda coluna filtrável deve abrir um filtro coerente com seu tipo.

Exemplos:

Texto:

```
contém
não contém
é igual a
está vazio
```

Número:

```
maior que
menor que
entre
igual a
```

Data:

```
antes de
depois de
entre
hoje
esta semana
atrasado
```

Pessoas:

```
contém Ryan
não contém Ryan
está vazio
```



**Status (03/10/2026): Não implementado (filtro por coluna).** Os operadores listados acima (contém, maior que, entre, atrasado, é Eu etc.) não existem. O que há: busca textual do quadro ( `q` , até 120 caracteres: nome do item, textos, rótulos de etiqueta e nomes de pessoa) e um filtro por pessoa ( `pessoa` ), repetidos no Kanban e no Calendário ( `BoardQueryService._filtered_items` ). No quadro de domínio, as telas de Demandas e Tarefas usam os filtros do app `activities` (prazo, etapa, status, pessoa).

## 6.8 Agrupar por

Tipos categóricos podem ser usados para agrupamento.

Exemplos:

```
Status
Prioridade
Pessoa
Setor
Cliente
Lista suspensa
```

Tipos como Texto livre não devem ser a primeira opção para agrupamento.

**Status (03/10/2026): Parcial.** Na Tabela, o agrupamento é por **grupos manuais** do quadro ( `BoardGroup` ), que não são um valor de coluna. Só o Kanban agrupa por coluna, e apenas por **Status ou Lista suspensa** ( `GROUPABLE_TYPES` em `boards/kanban.py` ); Pessoa, Data e os demais tipos ficam para uma próxima versão. No quadro de domínio, o agrupamento aceita `stage` , `condition` , `sector` , `owner / responsavel` e `urgency / priority` ( `WorkBoardViewSettingsView` ).

## 6.9 Duplicar coluna

Duplica:

- tipo;
- configurações;
- opções;
- largura;
- validações.

Por padrão, não duplica os valores dos itens.

Uma opção futura pode permitir duplicar também valores.

**Divergência (03/10/2026):** a duplicação copia tipo, configurações, descrição, obrigatoriedade, largura e etiquetas e **também os valores das células** (pessoas e etiquetas incluídas); a spec pede que, por padrão, os valores não sejam copiados. A cópia nasce ao lado da original com o nome "<nome> (cópia)" ( `ColumnService.duplicate` ).

## 6.10 Adicionar coluna à direita

A partir do menu:

```
Adicionar coluna à direita
```

Abre o seletor de tipos sem mudar de tela.

**Status (03/10/2026): Implementado.** Item "Adicionar coluna à direita" no menu da coluna; abre o seletor de tipos e cria a coluna logo após a escolhida ( `after_column_id` ).

## 6.11 Alterar tipo

Só deve ser permitido entre tipos compatíveis ou após validação de conversão.

Exemplos seguros:

```
Número → Moeda
Texto → Número, somente se todos os valores forem válidos
Data → Data com hora
```

Exemplos perigosos:

```
Pessoas → Número
Arquivo → Data
```

Nesses casos, bloquear ou exigir uma migração explícita.

**Divergência (03/10/2026):** `ColumnService.conversion_plan` classifica a troca em três modos. **Segura** (nada se perde): Número↔Moeda, Status↔Lista suspensa e qualquer tipo → Texto (o valor vira o texto exibido). **Com leitura** ( `parsed` ): Texto → Número, Moeda ou Data; o que não converte é apagado, e a tela informa quantos valores se perdem. **Destrutiva:** qualquer outro par, com todos os valores da coluna apagados e as etiquetas desativadas. Fora do modo seguro, o servidor responde 409 e só converte com `confirm` ; a tela mostra uma pré-visualização ( `preview` ). Diferenças em relação ao texto: Pessoas → Número **não é bloqueado**, é permitido com confirmação e perda dos valores; Texto → Número não exige que todos os valores sejam válidos; "Data → Data com hora" não é troca de tipo, é a opção `show_time` . Ao trocar, as configurações voltam ao padrão do novo tipo.

## 6.12 Ocultar

Ocultar não exclui a coluna.

A coluna continua existindo e pode alimentar:

- filtros;
- Fórmulas;
- Kanban;
- Calendário;
- automações.

**Status (03/10/2026): Parcial.** "Ocultar coluna" grava `is_visible=False` para o quadro todo; a barra mostra "Colunas ocultas (N)" com o botão "Mostrar". A coluna continua existindo e pode ser usada no Kanban (agrupamento, soma, cartão) e no Calendário. Filtros por coluna, Fórmulas e automações ainda não existem.

## 6.13 Excluir

Exclusão deve exigir confirmação.

Se houver dados históricos, Fórmulas ou automações usando a coluna:

Esta coluna é usada por:

- 2 Fórmulas
- 1 visualização
- 3 filtros salvos

O sistema deve impedir exclusão insegura ou solicitar tratamento das dependências.

**Status (03/10/2026): Parcial.** A exclusão pede confirmação e é lógica ( `is_active=False` , `ColumnService.soft_delete` ). **Não há verificação de dependências:** as visualizações que apontavam para a coluna deixam de usá-la e recaem no padrão ( `resolve_group_column` , `resolve_date_column` ), sem aviso ao usuário. Fórmulas, filtros salvos e automações não existem.

## 6.14 Autosave

Não criar botão geral:

Salvar coluna

Pequenas mudanças devem ser persistidas automaticamente.

**Divergência (03/10/2026):** renomear, mover, redimensionar, ocultar, editar células e editar etiquetas gravam na hora. Já o diálogo "Configurações da coluna" (descrição, obrigatória, casas decimais, moeda, horário etc.) tem botão **Salvar**, e a ocultação recarrega a página.

## 7. Modo de apresentação geral

A mesma coluna possui diferentes formas de apresentação.

### 7.1 Tabela

Exibe uma célula por item.

Exemplo:

|              |  |
|--------------|--|
| Status       |  |
| Em andamento |  |
| Concluído    |  |

**Status (03/10/2026): Implementado.** A Tabela é o "Quadro principal", implícito em todo quadro ( `templates/boards/board_detail.html` , `_board_workspace.html` ), com grupos, colunas e uma célula por item.

### 7.2 Kanban

Pode:

- ser usada para agrupar;
- aparecer dentro do cartão;
- ser editada pelo próprio cartão.

**Status (03/10/2026): Implementado.** Raias = etiquetas da coluna agrupadora (Status ou Lista); cartões com campos configuráveis e editáveis no próprio cartão; arrastar o cartão grava a etiqueta da coluna agrupadora ( `boards/kanban.py` , `templates/boards/_kanban_lanes.html` ).

## 7.3 Calendário

Colunas de Data podem posicionar o item.

Outras colunas aparecem como informações do cartão.

**Status (03/10/2026): Implementado (escala mensal).** A coluna de Data escolhida ( `date_field` ) posiciona o item; arrastar o cartão para outro dia grava a data. Só existe a escala "Mês" ( `PERIODS = ("month",)` ); há painéis "Sem data" e "Atrasados" (este quando a coluna é prazo), cor por Status/Lista/grupo e campos de cartão configuráveis ( `boards/calendar_view.py` ).

## 7.4 Formulário de criação

O tipo determina o componente de entrada.

Exemplo:

Pessoas:

Pesquisar pessoa...

Data:

Calendário

Status:

Lista de etiquetas

**Status (03/10/2026): Não implementado como formulário completo.** O item nasce só com o nome ( `ItemService.create` ); o Kanban cria o cartão já dentro da raias (preenche a etiqueta) e o Calendário cria no dia clicado (preenche a data), via valores iniciais opcionais ( `initial` , até 30). A criação de Demandas é um fluxo próprio, em 4 passos (fora do motor de colunas).

## 7.5 Drawer de detalhes

A coluna aparece como campo editável no detalhe do item.

**Status (03/10/2026): Implementado.** A gaveta do item ( `board-item-detail` , `templates/boards/_item_drawer.html` ) mostra todas as colunas, editáveis no lugar, o grupo e as últimas 8 mudanças, com link para o histórico completo do quadro.

## 8. Coluna Texto

**Status (03/10/2026): Implementado (parcial nos filtros e no resumo).** Valor em `BoardCell.value_text`, com limite de 5000 caracteres ( `MAX_TEXT_LENGTH` ), edição inline (clicar, digitar, Enter; Esc cancela), ordenação alfabética sem diferenciar maiúsculas e participação na busca do quadro. Onde: `boards/validators.py`, `templates/boards/_cell.html`.

**Divergência:** o texto longo é cortado com reticências por CSS, mas a célula não mostra tooltip com o valor completo. Os filtros de §8.7 e a contagem de preenchidos de §8.9 não existem. O limite de 5000 caracteres não é configurável (a spec cita "máximo 100 caracteres" como exemplo de validação).

### 8.1 Objetivo

Guardar uma informação textual curta e livre.

Exemplos:

```
Código externo
Número do pedido
Objeto resumido
Observação curta
Nome complementar
```

### 8.2 Armazenamento

String.

### 8.3 Edição

Diretamente na célula.

Fluxo:

```
clicar → digitar → Enter
```

### 8.4 Apresentação na Tabela

Texto simples.

Se for muito longo:

```
Instalação elétrica do ...
```

Tooltip mostra o valor completo.

### 8.5 Apresentação no Kanban

Pode aparecer como linha textual.

Evitar mais de duas linhas no cartão.

## 8.6 Apresentação no Calendário

Pode aparecer como informação secundária.

Não deve dominar o cartão.

## 8.7 Filtros

```
contém
não contém
é igual a
começa com
termina com
está vazio
não está vazio
```

## 8.8 Ordenação

Alfabética.

## 8.9 Resumo

Não possui soma ou média.

Pode oferecer contagem de preenchidos.

## 8.10 Regra

Texto não deve ser usado para armazenar números que precisam ser calculados.

Errado:

```
"1500000"
```

para Valor.

Correto:

```
Moeda = 1.500.000
```

## 9. Coluna Número

**Status (03/10/2026): Parcial.** Valor em `BoardCell.value_number` (decimal 24,6; valor absoluto abaixo de  $10^{15}$ ), edição inline que aceita o formato brasileiro ("1.234,56"), arredondamento `ROUND_HALF_UP` e ordenação numérica feita no banco. Configurações implementadas: `decimal_places` (0 a 6, padrão 2), `unit` (sufixo de até 12 caracteres, ex.: "%"), `minimum` e `maximum`.

**Divergência:** não existem as configurações "Separador" e "Prefixo". Filtros (§9.6) e Fórmulas (§9.9) não existem. O único resumo é a soma opcional no cabeçalho de cada raia do Kanban ( `sum_column` ); soma, média, mínimo, máximo e contagem na Tabela são futuros.

## 9.1 Objetivo

Guardar valores numéricos calculáveis.

Exemplos:

```
Quantidade
Probabilidade
Número de lotes
Horas previstas
Pontos
Percentual
```

## 9.2 Armazenamento

Número decimal.

## 9.3 Edição

Inline.

Aceitar apenas números válidos.

## 9.4 Configurações

```
Casas decimais
Separador
Prefixo
Sufixo
Valor mínimo
Valor máximo
```

Exemplo:

```
85 %
```

## 9.5 Apresentação na Tabela

Formatada conforme configuração.

## 9.6 Filtros

```
igual
diferente
maior que
menor que
maior ou igual
menor ou igual
entre
está vazio
```

## 9.7 Ordenação

Numérica.

Nunca lexicográfica.

Correto:

```
2
10
100
```

e não:

```
10
100
2
```

## 9.8 Resumo

Pode oferecer:

```
Soma
Média
Mínimo
Máximo
Contagem
```

## 9.9 Fórmulas

Pode ser entrada e resultado de Fórmulas.

## 10. Coluna Moeda

**Status (03/10/2026): Parcial.** Tipo explícito `CURRENCY` que compartilha o motor de Número ( `value_number` ). Configurações: `currency` ( `BRL` , `USD` ou `EUR` , com símbolos "R\$", "US\$" e "€"), `decimal_places` , `minimum` , `maximum` . Exibição como "R\$ 1.250.000,00" ( `CellService.display_value` ). Soma por raia no Kanban.

**Divergência:** "Exibir símbolo" e "Formato" não são configuráveis (o símbolo sempre aparece). Filtros, média, mínimo, máximo e Fórmulas não existem.

### 10.1 Objetivo

Representar valores financeiros.

Exemplos:

```
Valor da proposta
Custo
Contrato
Saldo
Valor medido
Orçamento
```



---

## 10.2 Base técnica

Pode compartilhar motor com Número, mas deve ser um tipo explícito para o usuário.

---

## 10.3 Configurações

```
Moeda
Casas decimais
Exibir símbolo
Formato
```

Inicial:

```
BRL – Real brasileiro
```

BRL significa **Brazilian Real**, código internacional do Real brasileiro.

---

## 10.4 Apresentação

```
R$ 1.250.000,00
```

---

## 10.5 Filtros

Mesmos de Número.

---

## 10.6 Resumo

```
Soma
Média
Mínimo
Máximo
```

---

## 10.7 Fórmulas

Pode participar de:

```
Saldo = Contrato - Medido
```

---

## 11. Coluna Data

**Status (03/10/2026): Parcial.** Data e Data com hora são a mesma coluna: `value_date` mais `value_datetime` quando `show_time` está ligado. Configurações: `show_time`, `allow_weekends` (recusa sábado e domingo no servidor), `is_deadline` (destaca "vencido" quando a data já passou e alimenta o painel "Atrasados" do Calendário) e `format` ( `DD/MM/YYYY`, `DD/MM/YY` ou `YYYY-MM-DD` ). Aceita entrada ISO e `dd/mm/aaaa`. Ordenação cronológica e uso como coluna principal do Calendário estão implementados.

**Divergência:** a edição usa um popover com o campo de data nativo do navegador ( `date` ou `datetime-local` ) e os botões "Hoje" e "Limpar"; não há ícone de relógio para adicionar horário (o horário aparece quando a coluna está configurada com `show_time` ). A chave de configuração é `format`, não `date_format`. Os filtros de §11.11 e as Fórmulas de §11.13 não existem. Desligar `show_time` mantém a data (a hora deixa de ser exibida).

### 11.1 Objetivo

Guardar Data e, opcionalmente, horário.

Exemplos:

```
Prazo
Data de envio
Visita técnica
Data de entrega
Próximo feedback
```

### 11.2 Regra central

Data e Data + hora são a mesma família de campo.

Não exigir uma coluna separada para Hora.

### 11.3 Configuração

```
Exibir horário: Sim/Não
Formato da data
Permitir finais de semana
Definir como prazo
```

### 11.4 Sem horário

```
15/10/2026
```

### 11.5 Com horário

```
15/10/2026 14:00
```

### 11.6 Edição

Popover contextual com calendário.

O usuário não sai da tela.

---

## 11.7 Adicionar horário

Ícone de relógio.

Ao clicar:

Horário opcional

---

## 11.8 Apresentação na Tabela

Compacta.

15/10/2026

ou:

15/10/2026 14:00

---

## 11.9 Kanban

Pode aparecer como chip:

 15/10/2026

---

## 11.10 Calendário

Pode ser a coluna que posiciona o item no dia.

---

## 11.11 Filtros

antes de  
depois de  
entre  
hoje  
amanhã  
esta semana  
este mês  
atrasado  
está vazio

---

## 11.12 Ordenação

Cronológica.

## 11.13 Fórmulas

Pode participar de cálculos de Data.

Exemplo:

```
Dias restantes = Prazo - Hoje
```

## 12. Coluna Pessoas

**Status (03/10/2026): Parcial (uma pessoa por célula).** Tipo `PERSON` ; o vínculo guarda o usuário ( `BoardCellUser.user` ), não o nome digitado, e o servidor só aceita usuário ativo da mesma organização ( `CellService.set_value` ). Seleção por popover com busca ( `PersonSearchView` , `/api/pessoas/` , no máximo 20 resultados); a Tabela mostra avatar com iniciais e nome.

**Divergência:** a configuração `multiple` só aceita `false` ("Várias pessoas ficam para uma versão futura"), embora a tabela de vínculos já comporte várias. O filtro é um único seletor "Todas as pessoas" no topo do quadro (pessoas que aparecem em alguma célula de Pessoa), sem os operadores "é Eu", "contém", "não contém" e "está vazio". O agrupamento por Pessoa não existe no Kanban dos quadros dinâmicos (só Status e Lista); existe nos quadros de domínio ( `owner` , `responsavel` ). A busca não é restrita por setor, apenas por organização.

### 12.1 Objetivo

Relacionar um item a usuários reais da organização.

Não é texto livre.

### 12.2 Exemplos

```
Responsável
Solicitante
Aprovador
Participantes
Supervisor
Engenheiro responsável
```

### 12.3 Configuração

```
Permitir:
- uma pessoa
- várias pessoas
```

### 12.4 Uma pessoa

Exemplo:

```
Responsável
[PB Paulo Biasi]
```

## 12.5 Várias pessoas

Exemplo:

```
Participantes
[Ryan] [Luan] [Jennifer]
```

## 12.6 Seleção

Popover:

```
Pesquisar pessoas...

Ryan
Luan
Jennifer
Guilherme
```

## 12.7 Regra de identidade

Salvar `user_id`, não o nome digitado.

Se Ryan mudar o nome exibido, o vínculo continua correto.

## 12.8 Apresentação na Tabela

Avatar + nome.

Em pouco espaço:

```
PB
```

ou:

```
PB Paulo
```

## 12.9 Kanban

Pode aparecer como:

```
PB Paulo
```

ou apenas avatar.

## 12.10 Calendário

Pode aparecer de forma compacta.

## 12.11 Filtros

```
é Paulo
contém Ryan
não contém Ryan
é Eu
está vazio
```

## 12.12 Agrupamento

Pode agrupar por Pessoa.

Exemplo:

```
Ryan
Luan
Jennifer
```

## 12.13 Permissões

O seletor só deve mostrar pessoas dentro do escopo permitido.

## 12.14 Importante

Uma coluna Pessoas não significa automaticamente "Responsável".

O significado é definido pelo nome da coluna e pela regra de negócio.

## 13. Coluna Status

**Status (03/10/2026): Implementado.** As etiquetas ficam em `BoardColumnOption` (`id` estável, `label`, `color`, `position`, `is_default`, `is_done`, `is_active`), de modo que renomear, recolorir ou reordenar não regrava as células. Edição por popover (lista de etiquetas, "Limpar", "Editar etiquetas"); o diálogo "Editar etiquetas" cria, renomeia, recolora (paleta de 16 cores), reordena (botões ↑ e ↓) e exclui etiquetas, e marca a padrão e a que "conta como concluído". Ordenação pela posição das etiquetas, agrupamento do Kanban e todas as visões refletem a mudança.

**Divergência:** os campos `name`, `order` e `behavior` da spec são `label`, `position` e as marcas `is_default` e `is_done`. O popover da célula não tem "+ Nova etiqueta" (a criação é no diálogo de etiquetas). Etiqueta repetida na mesma coluna é recusada (sem diferenciar maiúsculas). Excluir uma etiqueta esvazia as células que a usavam, após confirmação. Os filtros de §13.9 não existem. **Cuidado arquitetural (§13.12):** o Status de quadro é configurável e independente de `Activity.condition` / `Activity.stage` e de `Task.status`; `is_done` só influencia o Calendário (ocultar concluídos e calcular atrasados) e não altera nenhum estado operacional.

## 13.1 Objetivo

Representar estado ou etapa de fluxo.

Exemplos:

```
Não iniciado
Em andamento
Aguardando cliente
Em revisão
Concluído
```

---

## 13.2 Estrutura

Cada opção deve possuir:

```
id
name
color
order
is_active
behavior opcional
```

---

## 13.3 Identidade estável

O ID nunca muda ao renomear.

Exemplo:

```
id = 42
nome = Working on it
```

usuário altera:

```
nome = Trabalhando nisso
```

continua:

```
id = 42
```

---

## 13.4 Cor

A cor pertence à opção.

Alterar a cor deve atualizar todas as visualizações.

---

## 13.5 Edição

Clique na célula.

Popover:

```
Trabalhando nisso
Parado
Concluído
+ Nova etiqueta
Editar etiquetas
```

---

## 13.6 Renomear

Pode ser feito sem sair da tela.

---

## 13.7 Alterar cor

Paleta visual inline.

---

## 13.8 Ordenação

Por ordem configurada das opções.

Não alfabeticamente.

---

## 13.9 Filtro

```
é
não é
está em
não está em
está vazio
```

---

## 13.10 Agrupamento

É um dos principais tipos para agrupar Tabela e Kanban.

---

## 13.11 Kanban

Pode ser usado como coluna de agrupamento.

Mover um cartão entre grupos altera o valor da coluna Status configurada para aquele Kanban.

---

## 13.12 Cuidado arquitetural

Status visual configurável e estado de negócio crítico não devem ser confundidos sem uma regra explícita.

---



## 14. Coluna Lista suspensa

**Status (03/10/2026): Parcial (seleção única).** Tipo `DROPDOWN`, com as mesmas etiquetas de Status ( `BoardColumnOption` ), edição por popover, ordenação pela posição, agrupamento no Kanban e uso para colorir o Calendário.

**Divergência:** a seleção múltipla (§14.3) não existe; a célula guarda uma única etiqueta. Na Tabela a opção aparece como etiqueta colorida (pílula), a mesma de Status, e não como vários chips. Uma Lista nova nasce sem etiquetas (só Status recebe as três etiquetas iniciais). Os filtros de §14.6 não existem.

### 14.1 Objetivo

Classificar itens em opções configuráveis que não necessariamente representam fluxo.

Exemplos:

```
Disciplina
Tipo de orçamento
Região
Categoria
Modalidade
Tipo de contrato
```

### 14.2 Diferença para Status

Status:

```
Não iniciado → Em andamento → Concluído
```

Lista:

```
Elétrica
Hidráulica
Mecânica
Civil
```

A Lista não exige uma sequência de progresso.

### 14.3 Configuração

Pode permitir:

```
seleção única
seleção múltipla
```

### 14.4 Edição

Popover com opções.

## 14.5 Apresentação

Chips.

Exemplo:

```
[Elétrica]
```

OU:

```
[Elétrica] [Dados] [SDAI]
```

SDAI significa **Sistema de Detecção e Alarme de Incêndio**.

## 14.6 Filtros

```
contém
não contém
é
não é
está vazio
```

## 14.7 Agrupamento

Permitido.

## 15. Coluna Prioridade

**Status (03/10/2026): Não implementado como coluna de quadro dinâmico.** `PRIORITY` está reservado em `BoardColumn.Type` e não pode ser criado.

**Divergência:** Prioridade existe apenas como **campo de sistema** do quadro de domínio: `urgency` na Demanda ( `Activity.Urgency` ) e `priority` na Tarefa ( `Task.Priority` ), ambos com os valores Baixa, Média e Alta (não há "Crítica"), cores configuráveis por organização ( `core.colors.EnumColorResolver` ), edição inline pelos services e agrupamento. Prioridade padrão, ordem semântica por coluna e filtros próprios não existem no motor de colunas.

### 15.1 Objetivo

Representar importância relativa para decisão e organização do trabalho.

### 15.2 Valores iniciais

```
Alta
Média
Baixa
```

Pode existir:

```
Crítica
```

se houver necessidade real.

Evitar dez níveis.

---

## 15.3 Apresentação

Chip com ícone e cor.

- Alta
- Média
- Baixa

A interface não deve depender só da cor.

---

## 15.4 Ordenação

Ordem semântica:

```
Crítica
Alta
Média
Baixa
```

---

## 15.5 Filtros

```
é
não é
está em
está vazio
```

---

## 15.6 Agrupamento

Permitido.

---

## 15.7 Regra

Prioridade não deve virar mecanismo de pressão indiscriminada.

Se tudo for Alta, a coluna perde valor.

---

## 16. Coluna Confirmação

**Status (03/10/2026): Implementado (como `CHECKBOX` ).** Rótulo "Sinal de confirmação"; nome inicial "Confirmação". Valor em `BoardCell.value_boolean` com três estados (sem resposta, sim, não); um clique ou a barra de espaço alterna; ordenação implementada.

**Divergência:** o identificador é `CHECKBOX` , não `CONFIRMATION` (reservado). Os filtros "marcado" e "não marcado" e o uso em Fórmula não existem.

## 16.1 Objetivo

Guardar estado binário.

Exemplo:

```
Sim / Não
Feito / Não feito
Confirmado / Não confirmado
```

## 16.2 Apresentação

Checkbox ou ícone.



## 16.3 Exemplos

```
Visita realizada
Projeto recebido
Cliente respondeu
Contrato assinado
```

## 16.4 Filtro

```
marcado
não marcado
```

## 16.5 Ordenação

Marcados e não marcados.

## 16.6 Fórmula

Pode ser usada como condição booleana.

Booleana significa uma informação com dois estados lógicos: verdadeiro ou falso.

## 16.7 Regra

Não usar Confirmação para substituir um fluxo complexo.

Se existem cinco estados, usar Status.

## 17. Coluna Arquivo

**Status (03/10/2026): Não implementado.** FILE está reservado em BoardColumn.Type , sem criação, upload, painel de anexos nem download. Todo o conteúdo abaixo é requisito futuro.

## 17.1 Objetivo

Associar documentos ao item.

---

## 17.2 Exemplos

```
Projeto
Memorial
Proposta
Foto
Planilha
PDF
Contrato
```

## 17.3 Apresentação na Tabela

```
 4 arquivos
```

ou chips compactos.

---

## 17.4 Ao clicar

Abrir painel contextual.

Não redirecionar para uma página genérica de arquivos.

---

## 17.5 Configuração

Pode aceitar:

```
um arquivo
vários arquivos
tipos permitidos
tamanho máximo
```

## 17.6 Segurança

Download deve respeitar:

- login;
- organização;
- autorização do recurso.

Não servir arquivos por URL pública sem validação.

---

## 17.7 Filtro

Inicialmente:

```
tem arquivo
não tem arquivo
```

## 17.8 Ordenação

Pode ordenar por quantidade ou presença.

## 18. Coluna Relação

**Status (03/10/2026): Não implementado em quadros dinâmicos.** `RELATION` está reservado em `BoardColumn.Type`. Todo o conteúdo abaixo é requisito futuro, com uma exceção no quadro de domínio: o campo de sistema `activity` ("Demanda vinculada") do quadro de Tarefas é do tipo `RELATION`, guarda o vínculo real (`Task.activity`) e é **somente leitura**. `sector` é um campo de sistema do tipo `SECTOR`, editável pelos services. Cliente, Obra e Centro de custo existem em `Activity` (`client`, `site`, `cost_center`), mas não são campos do quadro.

## 18.1 Objetivo

Relacionar um item a outro registro real.

## 18.2 Exemplos

```
Setor
Cliente
Obra
Centro de custo
Demanda vinculada
Fornecedor
Contrato
Pedido
```

## 18.3 Regra central

Não armazenar apenas o texto mostrado.

Armazenar o ID do objeto relacionado.

## 18.4 Exemplo

```
Demanda vinculada:
Arena Center Norte
```

internamente:

```
activity_id = 381
```

## 18.5 Tipos de relação

Primeira versão:

```
1 para 1 lógico
1 para vários
vários para vários
```

A escolha depende da configuração.

---

## 18.6 Seleção

Popover com busca.

Exemplo:

```
Pesquisar obra...

Arena Center Norte
Ascenty Sumaré
Santa Isabel
```

## 18.7 Apresentação

Nome do objeto relacionado.

Pode possuir ícone do tipo.

---

## 18.8 Clique

Abre o drawer do objeto relacionado.

Evitar navegação completa.

---

## 18.9 Filtro

Por objeto relacionado.

---

## 18.10 Agrupamento

Permitido para relações categóricas.

Exemplo:

```
agrupar por Setor
```

## 18.11 Colunas especializadas construídas sobre Relação

```
Setor
Cliente
Obra
Centro de custo
Demanda vinculada
Processo
```

Não criar um tipo primitivo diferente para cada uma.

## 19. Coluna Cronograma

**Status (03/10/2026): Não implementado.** `TIMELINE` está reservado em `BoardColumn.Type`. O Calendário usa apenas colunas de Data ( `DATE_TYPES = (DATE,)` ). Todo o conteúdo abaixo é requisito futuro.

### 19.1 Objetivo

Guardar um intervalo temporal com início e fim.

### 19.2 Exemplo

```
10/10/2026 → 18/10/2026
```

### 19.3 Diferença para Data

Data:

```
Entrega: 18/10
```

Cronograma:

```
Execução: 10/10 → 18/10
```

### 19.4 Apresentação na Tabela

Barra ou texto compacto.

```
10/10 → 18/10
```

### 19.5 Linha do tempo

Pode alimentar visualização futura de Linha do tempo.

### 19.6 Gantt

Pode alimentar visualização futura de Gantt.



Gantt é uma visualização de planejamento em barras horizontais distribuídas ao longo do tempo.

## 19.7 Filtros

```
inicia antes de
inicia depois de
termina antes de
termina depois de
sobrepõe período
```

## 19.8 Regra

Cronograma de processo da LPS não deve tentar substituir ferramentas completas de planejamento de obra, como Microsoft Project, sem um objetivo claro.

## 20. Coluna Fórmula

**Status (03/10/2026): Não implementado.** FORMULA está reservado em BoardColumn.Type . Não há motor de cálculo, referência por id, detecção de ciclos nem tratamento de erros. O mais próximo é o campo de sistema somente leitura "Tarefas"/"Checklist" ( CHECKLIST ) do quadro de domínio, que exibe "feitas/total" com barra de progresso. Todo o conteúdo abaixo é requisito futuro.

### 20.1 Objetivo

Calcular um valor automaticamente a partir de outras colunas.

O usuário não digita o resultado.

### 20.2 Exemplos

Margem:

```
(Valor - Custo) / Valor
```

Saldo:

```
Contrato - Medido
```

Valor total:

```
Quantidade * Preço unitário
```

Dias restantes:

```
Prazo - Hoje
```

Percentual:

```
Concluídas / Total
```

## 20.3 Regra principal

Fórmula é somente leitura.

O usuário altera as entradas.

O resultado é recalculado.

## 20.4 Não é automação

Fórmula:

```
calcula valor
```

Automação:

```
executa ação
```

Fórmula não deve:

- criar tarefa;
- alterar Status;
- enviar notificação;
- trocar responsável;
- apagar registro.

## 20.5 Configuração

Exemplo:

```
Fórmula:
[Quantidade] * [Preço unitário]
```

```
Resultado:
[Moeda]
```

## 20.6 Tipos de resultado

```
Número
Moeda
Percentual
Texto
Data
Duração
Booleano
```

Booleano significa verdadeiro ou falso.

---

## 20.7 Apresentação na Tabela

Mesmo padrão do tipo resultante.

---

## 20.8 Kanban

Pode aparecer como informação do cartão.

---

## 20.9 Calendário

Pode aparecer no cartão, mas não deve posicionar o item se o resultado não for Data.

---

## 20.10 Filtros

Dependem do tipo resultante.

---

## 20.11 Ordenação

Depende do tipo resultante.

---

## 20.12 Agrupamento

Somente se o resultado for categórico e a operação for suportada.

---

## 20.13 Dependências

Se a Fórmula usa:

```
Valor
Custo
```

essas colunas não podem ser excluídas silenciosamente.

---

## 20.14 Referência por ID

Internamente:

```
column_id_41 - column_id_52
```

Não:

```
"Valor" - "Custo"
```

Assim renomear uma coluna não quebra a Fórmula.

---

## 20.15 Ciclos

Bloquear:

```
A = B + 1
B = A + 1
```

Isso é uma dependência circular.

## 20.16 Erros

Exibir erro compreensível:

```
#DIV/0
```

pode ser apresentado ao usuário como:

```
Não foi possível calcular: divisão por zero.
```

## 21. Campos de negócio da LPS

Abaixo, como os principais campos da LPS atual devem ser representados no novo motor.

| Campo de negócio    | Tipo-base  |
|---------------------|------------|
| Título              | Texto      |
| Responsável         | Pessoas    |
| Solicitante         | Pessoas    |
| Participantes       | Pessoas    |
| Status              | Status     |
| Prioridade/Urgência | Prioridade |
| Setor               | Relação    |
| Cliente             | Relação    |
| Obra                | Relação    |
| Centro de custo     | Relação    |
| Demanda vinculada   | Relação    |
| Prazo               | Data       |
| Prazo solicitado    | Data       |
| Prazo comprometido  | Data       |
| Valor               | Moeda      |
| Quantidade          | Número     |
| Probabilidade       | Número     |

| Campo de negócio    | Tipo-base                                |
|---------------------|------------------------------------------|
| Disciplina          | Lista suspensa                           |
| Tags                | Lista suspensa ou tipo Tag especializado |
| Projeto recebido    | Confirmação                              |
| Arquivos            | Arquivo                                  |
| Período de execução | Cronograma                               |
| Margem              | Fórmula                                  |

**Divergência (03/10/2026):** os campos de sistema do quadro de domínio têm tipos próprios, não os tipos-base acima. Estado atual:

- Título: `title` ( `TEXT` ).
- Responsável: `owner` na Demanda e `responsavel` na Tarefa ( `PERSON` , editável).
- Setor: `sector` ( `SECTOR` , editável).
- Etapa: `stage` ( `STAGE` ); Status: `condition` ( `SELECT` ); ambos por setor.
- Prioridade/Urgência: `urgency` (Demanda) e `priority` (Tarefa), ver §15.
- Prazo solicitado: `requested_deadline` , do tipo `DATETIME` (data e hora), e não `Data`.
- Demanda vinculada: `activity` ( `RELATION` , somente leitura).
- Tarefas/Checklist: `tasks` e `checklist` ( `CHECKLIST` , somente leitura).
- **Não expostos como campos do quadro:** Solicitante ( `Activity.requested_by` ), Participantes, Cliente, Obra, Centro de custo, Prazo comprometido ( `Task.committed_deadline` ), Tags, Arquivos, Período de execução e Margem.

Valor, Quantidade, Probabilidade, Disciplina e Projeto recebido podem existir como **colunas de um quadro dinâmico** (por exemplo, o modelo "Orçamentos") ou como campos customizados do quadro de domínio, criados em "Novo campo" ( `WorkBoardFieldCreateView` : `TEXT` , `NUMBER` , `CURRENCY` , `DATETIME` , `SELECT` , `BOOLEAN` ). No quadro de domínio, o valor customizado fica em `DomainCustomValue` e é editado como texto ou data e hora; o cadastro de opções para campos `SELECT` não tem tela ( `DomainBoardChoice` só aparece no admin, somente leitura).

## 22. Demandas — conjunto inicial sugerido

Template inicial:

```
Título
Solicitante
Responsável
Setor
Cliente
Obra
Status
Prioridade
Prazo
```

Extras:

```
Valor
Probabilidade
Disciplina
Visita técnica
Arquivos
```

**Status (03/10/2026): Parcial.** O quadro de domínio de Demandas nasce ( `ensure_domain_board` , sob demanda e idempotente) com: Título da demanda, Responsável, Setor, Prioridade, Prazo ( `requested_deadline` ), Status ( `stage` ), Status ( `condition` ) e Tarefas. Não há Solicitante, Cliente nem Obra, e os "extras" sugeridos (Valor, Probabilidade, Disciplina, Visita técnica, Arquivos) só existem se forem criados à mão ou vierem do modelo de quadro "Orçamentos" ( `boards/starter_templates.py` : Cliente, Responsável, Setor, Prazo, Status com 9 etiquetas, Valor, Probabilidade de 0 a 100 %, Pendência e Visita técnica; 4 grupos e o Kanban "Kanban por Status").

**Divergência:** em `boards/domain_defaults.py` o rótulo do campo `stage` ainda é "Status", o mesmo de `condition` , embora pelo vocabulário atual `stage` seja **Etapa**. Isso produz duas colunas chamadas "Status" (ver §52). A confirmar se é intencional.

## 23. Tarefas — conjunto inicial sugerido

Template inicial:

```
Título
Demanda vinculada
Responsável
Setor
Status
Prioridade
Prazo solicitado
Prazo comprometido
```

Extras:

```
Participantes
Checklist
Arquivos
Cronograma
```

Checklist pode permanecer inicialmente como recurso especializado de Tarefa e não como tipo de coluna-base.

**Status (03/10/2026): Parcial.** O quadro de domínio de Tarefas nasce com Título da tarefa, Demanda vinculada, Responsável, Setor, Prioridade, Prazo ( `requested_deadline` ), Status ( `stage` ) e Checklist. Faltam Prazo comprometido, Participantes, Arquivos e Cronograma. O Checklist permanece como recurso especializado de Tarefa, como a spec sugere.

**Divergência:** além do quadro de domínio, **cada Demanda possui o seu próprio quadro de tarefas** ( `Board` com `kind="DEMAND"` , ligado a `Activity` , criado em branco ou copiado de um modelo por `BoardInstantiationService` ). Em branco, ele recebe as colunas Responsável (Pessoa), Status (3 etiquetas) e Data, além das visões Kanban e Calendário. Esse quadro usa as colunas dinâmicas da §5 em diante; o acesso segue `DemandBoardAccess` (dono, criador ou participante da Demanda).

## 24. Coluna obrigatória

Uma coluna pode ser configurada como obrigatória.

Exemplo:

```
Responsável: obrigatório
```

Na criação:

```
Não é possível criar a tarefa sem Responsável.
```

Não permitir item inválido silenciosamente.

**Divergência (03/10/2026): Parcial.** `BoardColumn.is_required` é configurável ("Valor obrigatório" nas configurações da coluna). O servidor recusa **limpar** uma célula obrigatória ("<coluna>" é obrigatória) e a tela esconde o botão "Limpar". Mas o item é criado só com o nome ( `ItemService.create` ), sem checar colunas obrigatórias; portanto "Não é possível criar a tarefa sem Responsável" **não** é atendido hoje.

## 25. Valor padrão

Alguns tipos podem ter valor padrão.

Exemplo:

```
Status padrão = Não iniciado
Prioridade padrão = Média
```

Não preencher valores como Responsável automaticamente sem regra explícita.

**Status (03/10/2026): Parcial.** Só existe a etiqueta padrão ( `BoardColumnOption.is_default` ) de Status e Lista, aplicada automaticamente a cada item novo ( `ItemService.create` ); o Status inicial é "Não iniciado". Os demais tipos não têm valor padrão, e Prioridade padrão não se aplica (tipo não implementado). Nenhum valor de Pessoa é preenchido sem regra explícita.

## 26. Validações

Toda coluna deve aceitar regras de validação compatíveis.

Exemplos:

Número:

```
mínimo 0
máximo 100
```

Data:

```
não permitir data anterior a hoje
```

Texto:

máximo 100 caracteres

Relação:

somente Obras ativas

**Status (03/10/2026): Parcial.** Validação sempre no servidor ( `normalize_cell_value` ): mínimo e máximo em Número e Moeda, casas decimais, bloqueio de fins de semana em Data, etiqueta ativa da própria coluna, pessoa da organização. Divergências: o limite de Texto é fixo (5000), "não permitir data anterior a hoje" não existe e a validação de Relação ("somente Obras ativas") é futura.

## 27. Resumo de coluna

A Tabela pode mostrar resumo na parte inferior.

Número:

Soma: 1.250

Moeda:

Total: R\$ 4.820.000

Status:

3 Em andamento  
5 Concluídos

Pessoas:

5 pessoas

Data:

Mais próxima: 05/10

Não forçar resumo onde não fizer sentido.

**Status (03/10/2026): Parcial.** Não há linha de resumo no rodapé da Tabela. Existem apenas: contagem de itens por grupo e por raia, e a soma de uma coluna de Número ou Moeda no cabeçalho de cada raia do Kanban ( `build_lanes` , `format_total` ). Distribuição de Status, "Mais próxima" em Data e demais resumos são futuros.

## 28. Colunas em visualizações

A configuração do Quadro define quais colunas existem.

Cada visualização define quais aparecem.



Exemplo:

Quadro:

15 colunas

Tabela Comercial:

9 visíveis

Kanban Gerencial:

5 campos no cartão

Calendário:

4 campos no cartão

Não duplicar coluna.

**Divergência (03/10/2026):** no quadro dinâmico, a existência, a ordem, a largura e a visibilidade das colunas são do Quadro e valem para a Tabela; Kanban e Calendário escolhem seus campos de cartão por visualização ( `BoardView.settings["card_fields"]` ), sem duplicar coluna. Não há visões de Tabela com subconjuntos próprios (como "Tabela Comercial"). No quadro de domínio, largura, ordem e visibilidade são por visualização ( `DomainBoardViewColumn` , `DomainBoardCardField` ).

## 29. Configuração do cartão

Kanban e Calendário devem permitir:

Clique ou arraste os campos que deseja mostrar.

Exemplo:

Status  
Responsável  
Prazo  
Valor

A ordem deve ser alterável por arrastar.

**Status (03/10/2026): Implementado, com diferenças.** O diálogo "Configurar cartões" permite marcar os campos e ordená-los, com pré-visualização do cartão. Divergência: a ordem é alterada pelos botões ↑ e ↓, não por arrastar. Limites: Kanban com até 12 campos (padrão 4, `MAX_CARD_FIELDS` ); Calendário com até 6 (padrão 3), além da opção "Mostrar o nome de cada campo" no Kanban. No quadro de domínio: `WorkBoardCardFieldsView` .

## 30. Edição contextual

Regra global:

Se o valor puder ser editado no local em que está sendo visto, não abrir outra página.

Exemplos:

Status:

clicar → escolher etiqueta

Pessoa:

clicar → pesquisar pessoa

Data:

clicar → calendário

Prioridade:

clicar → escolher prioridade

Texto:

clicar → digitar

**Status (03/10/2026): Implementado.** Os 8 tipos são editados no lugar (Tabela, cartão do Kanban, cartão do Calendário e gaveta) pelo mesmo editor ( `openCellEditor` ); a gravação é otimista, com desfazer visual se o servidor recusar. Prioridade e Relação não se aplicam (tipos não implementados).

## 31. Popovers

Tipos indicados para popover:

Pessoas  
Status  
Lista suspensa  
Prioridade  
Data  
Relação  
Confirmação

**Status (03/10/2026): Parcial.** Usam popover: Pessoa, Status, Lista suspensa e Data. Confirmação alterna ao clique, sem popover. Prioridade e Relação não existem.

## 32. Modal ou drawer

Usar para:

- configuração avançada da coluna;
- criação completa de item;

- detalhes completos;
- arquivos;
- Fórmula complexa;
- permissões;
- histórico.

**Status (03/10/2026): Parcial.** Existem o diálogo de configuração da coluna, o diálogo de etiquetas, a gaveta de detalhes do item e o histórico completo do quadro ( board-history , 50 por página). Arquivos, Fórmula complexa e permissões por coluna são futuros.

## 33. Segurança

Colunas não podem ignorar a autorização da LPS.

Exemplo:

Se o usuário pode ver a Demanda, mas não editar:

```
célula visível
edição bloqueada
```

**Status (03/10/2026): Implementado.** Toda gravação passa por `_require` em `boards/services.py`, com as ações do catálogo `QUADRO_VISUALIZAR`, `QUADRO_EDITAR`, `QUADRO_EXCLUIR`, `QUADRO_GERIR_COLUNAS`, `QUADRO_CRIAR_ITEM`, `QUADRO_EDITAR_ITEM` e `QUADRO_EXCLUIR_ITEM` (negação = 403). Quem não pode editar vê a célula sem o comportamento de edição ( `permissions.edit_item` ), e o servidor confere de novo. Quadros de Demanda ( `kind="DEMAND"` ) somam a regra relacional de `DemandBoardAccess`. No quadro de domínio, a edição de cada campo exige a permissão do serviço de `activities` correspondente.

## 34. Restrição de visualização

Futuro:

```
Esta coluna pode ser vista por:
- gestores
- financeiro
```

Isso exige cuidado porque filtros, Fórmulas e exportações também precisam respeitar a restrição.

Não implementar apenas escondendo no HTML.

**Status (03/10/2026): Não implementado.** Não há restrição de visualização por coluna; é requisito futuro.

## 35. Restrição de edição

Exemplo:

```
Valor final
Visualização: Comercial + Diretoria
Edição: Diretoria
```

A validação real deve ocorrer no servidor.

**Status (03/10/2026): Não implementado.** A edição é controlada por quadro (permissões `QUADRO_*`), não por coluna.

## 36. Multi-organização

Pessoas, Relações, Status e opções devem sempre respeitar a organização do Quadro.

Não permitir selecionar:

- pessoa de outra empresa;
- obra de outra organização;
- cliente de outro tenant;
- status pertencente a outro tenant.

Tenant significa a organização isolada dentro de uma aplicação multiempresa.

**Status (03/10/2026): Implementado.** Quadro, grupo, coluna, etiqueta, visão e item são carregados sempre pela organização da pessoa ( `get_board` , `get_column` etc. em `boards/views.py` ; outra organização = 404). A pessoa de uma célula precisa ser usuária ativa da organização do quadro; a etiqueta precisa ser ativa e da própria coluna. Relações entre organizações não se aplicam (tipo não implementado).

## 37. Auditoria

Mudanças relevantes em colunas de negócio devem ser auditáveis.

Exemplos:

```
Responsável alterado
Prazo alterado
Status alterado
Prioridade alterada
Cliente alterado
Obra alterada
```

**Status (03/10/2026): Implementado.** `_audit` grava em `AuditLog` com `metadata["board_id"]` : criação, renomeação, movimento, largura, ocultação, troca de tipo, alterações de etiqueta e cada mudança de célula (valor antigo e novo, como a tela mostra). O histórico fica em `board-history` e na gaveta do item ( `BoardQueryService.history` , `item_history` ).

## 38. Alterações visuais

Mudanças exclusivamente visuais podem possuir auditoria simplificada.

Exemplos:

```
largura
posição
visibilidade na view
```

Não precisam gerar notificação de negócio.

**Divergência (03/10/2026):** não há uma categoria de "auditoria simplificada": largura, posição e visibilidade de coluna também geram registros no histórico do quadro ( `BOARD_COLUMN_RESIZED` , `BOARD_COLUMN_MOVED` , `BOARD_COLUMN_UPDATED` ), sem notificação.

## 39. Notificações

Editar uma coluna não significa automaticamente notificar alguém.

Notificação é regra de negócio.

Exemplos que podem notificar:

```
troca de Responsável
prazo alterado
Status crítico
```

Exemplos que não devem:

```
largura aumentada
coluna movida
coluna oculta
```

**Status (03/10/2026): Não implementado para colunas de quadro.** `boards/services.py` não dispara notificações ao editar célula, e mudanças visuais nunca notificam (conforme a spec). No quadro de domínio, mudanças de responsável e prazo passam pelos services de `activities` , cuja notificação não foi verificada aqui.

## 40. Fórmulas e permissões

Uma Fórmula não deve revelar informação de uma coluna que o usuário não possui autorização para enxergar.

Esse ponto exige desenho técnico antes de habilitar restrição por coluna.

**Status (03/10/2026): Não aplicável hoje.** Sem Fórmulas e sem restrição de visualização por coluna.

## 41. Valores vazios

Cada tipo precisa possuir representação visual limpa.

Exemplo:

```
-
```

ou célula vazia.

Não usar:

```
null
None
undefined
```

na interface.

**Status (03/10/2026): Parcial.** Os templates nunca exibem `None`, `null` ou `undefined`. Status vazio mostra "—"; Lista, Texto, Número, Data e Moeda vazios mostram a célula em branco; Pessoa vazia mostra um avatar vazio; Confirmação sem resposta mostra a caixa vazia ( `templates/boards/_cell.html` ).

## 42. Campo vazio e filtro

Todos os tipos devem suportar:

```
está vazio
não está vazio
```

quando fizer sentido.

**Status (03/10/2026): Não implementado**, pois não há filtros por coluna (ver §6.7).

## 43. Cópia e cola

Evolução desejável para a Tabela:

```
Ctrl+C
Ctrl+V
```

com validação por tipo.

Não permitir colar texto inválido em Número sem aviso.

**Status (03/10/2026): Não implementado** (Ctrl+C e Ctrl+V). Existe navegação por setas, edição com Enter ou F2 e limpeza com Delete ou Backspace.

## 44. Exportação

Valores exportados devem manter o significado.

Exemplo:

Pessoas:

```
Paulo Biasi
```

e não:

```
user_id=42
```

Moeda:

```
1250000.00
```

mais formatação conforme formato de exportação.

**Status (03/10/2026): Não implementado.** Não há endpoint de exportação nos quadros.

---

## 45. Pesquisa global

A pesquisa pode indexar:

```
Texto
Status
Lista
Pessoas
Relações
```

Número, Data e Fórmula podem entrar de forma específica.

**Status (03/10/2026): Parcial.** A busca do quadro ( `BoardQueryService.search_filter` ) cobre o nome do item, o texto das colunas de Texto, os rótulos das etiquetas (Status e Lista) e o nome e usuário das pessoas. Número, Data e Moeda não entram na busca. Não existe pesquisa global entre quadros.

---

## 46. Campos não editáveis

Algumas colunas podem ser somente leitura.

Exemplos:

```
Código da Demanda
Criado em
Criado por
Última atualização
Margem calculada
Dias em atraso
```

Esses campos podem usar tipos-base, mas com `editable = false` .

**Status (03/10/2026): Parcial.** Nos quadros dinâmicos não há colunas somente leitura. No quadro de domínio, `activity` (Demanda vinculada) e `tasks / checklist` são somente leitura ( `editable=False` em `build_cells` ), e campos de sistema sem regra de gravação são recusados ("Este campo não pode ser editado diretamente").

---

## 47. Colunas de sistema

Além das 13 colunas-base, o sistema pode expor campos técnicos:

```
ID
Código
Criado em
Criado por
Atualizado em
```

Eles não devem aparecer no seletor principal de criação como colunas comuns, salvo em uma área:

```
Campos do sistema
```

**Status (03/10/2026): Parcial.** `DomainBoardField.is_system` marca os campos de sistema do quadro de domínio. ID, Código, Criado em, Criado por e Atualizado em não são oferecidos como colunas, e não existe a área "Campos do sistema" no seletor.

## 48. Colunas futuras

Não fazem parte da primeira versão obrigatória:

```
E-mail
Telefone
Localização
Link
Avaliação
Dependência
Progresso
Espelhamento
Documento rico
Extração por IA
```

IA significa **Inteligência Artificial**.

Esses tipos podem ser adicionados depois sem quebrar o motor central.

**Status (03/10/2026): Nenhuma implementada**, como previsto. Só `AI_EXTRACT` está reservado em `BoardColumn.Type`.

## 49. Extração por IA

A opção "Extract info" vista no Monday não deve ser copiada como tipo-base agora.

Na LPS, isso deve ser uma capacidade futura:

```
Extrair valor de documento
Extrair CNPJ
Extrair prazo
Extrair valor contratual
```

Pode preencher outras colunas.

Não deve ser confundida com Fórmula.

**Status (03/10/2026): Não implementado.** Apenas o valor reservado `AI_EXTRACT`; sem extração de documentos.

## 50. Documento

A opção "monday Doc" também não é coluna-base da primeira versão da LPS.

Documento rico pode ser um recurso futuro.

**Status (03/10/2026): Não implementado**, como previsto.



## 51. Regras de nomes

Evitar nomes genéricos:

```
Campo 1
Campo 2
Info
Outro
```

Preferir:

```
Prazo de entrega
Responsável
Valor da proposta
Setor responsável
```

**Status (03/10/2026): Parcial.** Cada tipo nasce com um nome descritivo (§6.1), mas o sistema não valida nomes genéricos.

## 52. Duplicidade de nomes

O sistema pode permitir duas colunas visualmente iguais, mas isso gera confusão.

Recomendação:

```
Status comercial
Status operacional
```

em vez de:

```
Status
Status
```

**Divergência (03/10/2026):** a criação e a duplicação de coluna evitam repetição ("Status 2", "<nome> (cópia)", em `ColumnService._unique_name`), mas **renomear aceita nome repetido** (`ColumnService.rename` não valida). Além disso, no quadro de domínio de Demandas os campos `stage` e `condition` têm hoje o mesmo rótulo "Status" (ver §22).

## 53. Dependências de exclusão

Antes de excluir uma coluna, verificar:

- Fórmulas;
- filtros;
- agrupamentos;
- automações;
- visualizações;
- relatórios;
- integrações.

**Status (03/10/2026): Não implementado.** `ColumnService.soft_delete` não consulta Fórmulas, filtros, visualizações, relatórios ou integrações (ver §6.13).

## 54. Soft delete

Soft delete significa exclusão lógica, mantendo o registro técnico e histórico.

Para colunas já utilizadas, preferir:

```
is_active = false
```

em vez de apagamento físico imediato.

**Status (03/10/2026): Implementado.** Colunas, etiquetas, grupos, itens, visões e quadros usam `is_active=False` ; `BoardCell.column` é `PROTECT` , então o banco impede apagar fisicamente uma coluna com valores.

## 55. Performance

A Tabela pode possuir muitas colunas.

Regras:

- carregar apenas valores necessários;
- evitar consulta por célula;
- usar pré-carregamento de Pessoas e Relações;
- calcular Fórmulas de forma eficiente;
- indexar campos de busca e filtros mais usados.

**Status (03/10/2026): Implementado (sem Fórmulas).** Leitura sem consulta por célula ( `BoardQueryService.with_cells` e `attach_cells` ), ordenação em subconsulta no banco e índices em `BoardCell` por coluna e valor numérico, de data e booleano. O Calendário busca só o intervalo da grade do mês.

## 56. Responsividade

No desktop:

- arrastar;
- redimensionar;
- edição inline.

No mobile:

- priorizar as colunas selecionadas;
- permitir scroll horizontal;
- abrir popovers como drawers;
- evitar células minúsculas.

**Status (03/10/2026): Parcial.** A Tabela tem rolagem horizontal e há regras `@media` para telas de até 720 e 760 px em `static/css/boards.css` ; arrastar, redimensionar e editar inline funcionam no desktop. A abertura de popovers como gavetas no celular não foi verificada.

## 57. Acessibilidade

Nunca usar apenas cor.

Status deve mostrar:

● Em andamento

e não apenas um quadrado azul.

Pessoas devem ter nome acessível mesmo com avatar.

**Status (03/10/2026): Parcial.** Etiquetas sempre mostram o texto (nunca só cor); a Confirmação tem texto para leitor de tela ("Sim", "Não", "Sem resposta"); Pessoa mostra nome ao lado do avatar; cabeçalhos ordenados têm `aria-sort`; menus e diálogos têm `aria-label`. O marcador "●" na etiqueta de Status não existe (a cor vai no fundo da etiqueta).

## 58. Teclado

Comportamentos desejados:

Enter = editar/confirmar  
Esc = cancelar/fechar  
Tab = próxima célula  
Shift+Tab = célula anterior

**Status (03/10/2026): Parcial.** Enter ou F2 abrem a edição, Esc cancela e fecha, Tab confirma a edição inline, as setas movem o foco entre células, Delete e Backspace limpam, e a barra de espaço alterna a Confirmação. O avanço para a próxima célula com Tab e Shift+Tab não foi verificado (a edição inline apenas confirma). A divisória de coluna é redimensionável por teclado.

## 59. Regra de UX

UX significa **User Experience**, ou experiência do usuário.

A regra central da LPS:

O usuário clica na informação que quer mudar e altera ali.

Não:

clicar  
abrir tela  
procurar formulário  
editar  
salvar  
voltar

Mas:

clicar  
alterar  
pronto

**Status (03/10/2026): Implementado** para os 8 tipos de coluna e para colunas, etiquetas e grupos.

## 60. Regra de produto

As colunas são blocos reutilizáveis.

Não desenvolver novamente:

```
seletor de Pessoa para Demandas
seletor de Pessoa para Tarefas
seletor de Pessoa para Compras
```

Desenvolver:

```
PeopleColumn
```

e reutilizar.

**Status (03/10/2026): Parcial.** Os editores por tipo são únicos e reaproveitados entre Tabela, Kanban, Calendário e gaveta ( `openCellEditor` ), e a busca de pessoas ( `personSearchNode` ) é um só componente. Não existe uma classe `PeopleColumn` ; os quadros de domínio (Demandas e Tarefas) têm editores próprios em `static/js/work-board.js` .

## 61. Contrato conceitual por tipo

Cada tipo de coluna deve implementar:

```
render()
editor()
validate()
serialize()
deserialize()
sort()
filter()
summarize()
format()
```

Alguns também:

```
group()
calculate()
```

**Divergência (03/10/2026):** o contrato existe como comportamento, mas distribuído em ramos por tipo, não em classes: `validate` e `serialize` em `normalize_cell_value` ( `boards/validators.py` ); `format` em `CellService.display_value` ( `boards/services.py` ); `sort` em `BoardQueryService.sort_expression` ( `boards/queries.py` ); `render` em `templates/boards/_cell.html` ; `editor` em `openCellEditor` ( `static/js/boards.js` ). `filter` e `summarize` existem só de forma parcial (busca global e soma no Kanban); `calculate` não existe.

## 62. Exemplo de arquitetura



**Divergência (03/10/2026):** a hierarquia `BaseColumnType` e suas subclasses **não foram implementadas**; a arquitetura real é um catálogo ( `BoardColumn.Type` ) com despacho por tipo nos módulos citados em §61. Nenhum tipo `Priority` , `File` , `Relation` , `Timeline` ou `Formula` existe.

## 63. Critérios de aceite gerais

| Status (03/10/2026): |                  |                                                                                                         |
|----------------------|------------------|---------------------------------------------------------------------------------------------------------|
| Critério             | Status           | Observação                                                                                              |
| COL-GER-001          | Implementado     | criar pelo "+" do cabeçalho, sem sair do quadro                                                         |
| COL-GER-002          | Implementado     | renomear inline                                                                                         |
| COL-GER-003          | Implementado     | arrastar o cabeçalho                                                                                    |
| COL-GER-004          | Implementado     | arrastar a divisória ou setas; 96 a 640 px                                                              |
| COL-GER-005          | Implementado     | posição e largura gravadas em <code>BoardColumn</code>                                                  |
| COL-GER-006          | Parcial          | células, nome, posição, largura e etiquetas gravam na hora; o diálogo de configurações tem botão Salvar |
| COL-GER-007          | Não implementado | não há filtros por coluna                                                                               |
| COL-GER-008          | Implementado     | ordenação por tipo, no banco                                                                            |
| COL-GER-009          | Implementado     | ocultar só muda <code>is_visible</code>                                                                 |
| COL-GER-010          | Não implementado | a exclusão não verifica dependências                                                                    |

## **COL-GER-001**

O usuário consegue criar uma coluna sem sair do Quadro.

## **COL-GER-002**

O usuário consegue renomear inline.

## **COL-GER-003**

O usuário consegue mover por arrastar e soltar.

## **COL-GER-004**

O usuário consegue redimensionar.

## **COL-GER-005**

A posição e largura permanecem após recarregar.

## **COL-GER-006**

Toda alteração simples usa autosave.

## **COL-GER-007**

Filtros respeitam o tipo.

## **COL-GER-008**

Ordenação respeita o tipo.

## **COL-GER-009**

Ocultar não exclui dados.

## **COL-GER-010**

Excluir verifica dependências.

---

## 64. Critérios de aceite — Pessoas

Status (03/10/2026):

| Critério    | Status       | Observação                                                          |
|-------------|--------------|---------------------------------------------------------------------|
| COL-PES-001 | Parcial      | só uma pessoa; <code>multiple=true</code> é recusado                |
| COL-PES-002 | Implementado | usuários ativos da organização                                      |
| COL-PES-003 | Implementado | <code>BoardCellUser.user</code>                                     |
| COL-PES-004 | Implementado | o vínculo é por usuário                                             |
| COL-PES-005 | Parcial      | filtro por pessoa para o quadro todo, não por coluna                |
| COL-PES-006 | Parcial      | agrupa nos quadros de domínio; no Kanban dos quadros dinâmicos, não |

### COL-PES-001

A coluna pode ser configurada para uma ou várias pessoas.

### COL-PES-002

A seleção busca apenas usuários válidos da organização.

### COL-PES-003

O banco guarda referência ao usuário, não apenas nome.

### COL-PES-004

Renomear a pessoa não quebra vínculos.

### COL-PES-005

A coluna pode ser filtrada por usuário.

### COL-PES-006

Pode ser usada para agrupamento.

## 65. Critérios de aceite — Status

Status (03/10/2026):

| Critério    | Status       | Observação                                                        |
|-------------|--------------|-------------------------------------------------------------------|
| COL-STA-001 | Implementado | BoardColumnOption.pk                                              |
| COL-STA-002 | Implementado | renomear no diálogo de etiquetas e no cabeçalho da raia do Kanban |
| COL-STA-003 | Implementado | paleta de cores                                                   |
| COL-STA-004 | Implementado | Tabela, Kanban e Calendário leem a mesma etiqueta                 |
| COL-STA-005 | Parcial      | filtros e Kanban usam o id; automações não existem                |
| COL-STA-006 | Implementado | agrupamento do Kanban                                             |

### COL-STA-001

As opções possuem ID estável.

### COL-STA-002

Nome pode ser alterado inline.

### COL-STA-003

Cor pode ser alterada inline.

### COL-STA-004

Todas as visualizações refletem a mudança.

### COL-STA-005

Renomear não quebra automações ou filtros.

### COL-STA-006

Pode agrupar Kanban.



## 66. Critérios de aceite — Data

Status (03/10/2026):

| Critério    | Status       | Observação                                                        |
|-------------|--------------|-------------------------------------------------------------------|
| COL-DAT-001 | Implementado | <code>show_time=false</code> por padrão                           |
| COL-DAT-002 | Implementado | opção "Mostrar horário" nas configurações                         |
| COL-DAT-003 | Implementado | desligar o horário mantém <code>value_date</code>                 |
| COL-DAT-004 | Implementado | <code>date_field</code> do Calendário                             |
| COL-DAT-005 | Implementado | ordena por <code>value_date</code> ou <code>value_datetime</code> |

### COL-DAT-001

Data funciona sem horário.

### COL-DAT-002

Horário pode ser ativado opcionalmente.

### COL-DAT-003

Remover horário mantém a Data.

### COL-DAT-004

Pode alimentar o Calendário.

### COL-DAT-005

Ordenação é cronológica.

## 67. Critérios de aceite — Fórmula

Status (03/10/2026): COL-FOR-001 a COL-FOR-007 não implementados (o tipo `FORMULA` está apenas reservado).

### COL-FOR-001

O valor é calculado automaticamente.

### COL-FOR-002

O resultado é somente leitura.

## COL-FOR-003

Referências usam IDs de colunas.

## COL-FOR-004

Renomear coluna referenciada não quebra a Fórmula.

## COL-FOR-005

Dependência circular é bloqueada.

## COL-FOR-006

Excluir coluna referenciada é bloqueado até corrigir a Fórmula.

## COL-FOR-007

O resultado possui tipo definido.

---

## 68. Critérios de aceite — Relação

**Status (03/10/2026):** COL-REL-001 a COL-REL-005 não implementados em quadros dinâmicos. Apenas o campo de sistema `activity` (Demanda vinculada) do quadro de Tarefas guarda uma referência real, somente leitura.

### COL-REL-001

O valor referencia um objeto real.

### COL-REL-002

O usuário consegue pesquisar objetos.

### COL-REL-003

Objetos de outro tenant não aparecem.

### COL-REL-004

Clicar pode abrir o drawer do relacionado.

### COL-REL-005

Pode ser filtrada e agrupada.

---

## 69. Critérios de aceite — Arquivos

**Status (03/10/2026):** COL-ARQ-001 a COL-ARQ-004 não implementados (o tipo `FILE` está apenas reservado).

## COL-ARQ-001

Upload respeita autorização.

## COL-ARQ-002

Download exige autorização.

## COL-ARQ-003

A célula mostra quantidade de anexos.

## COL-ARQ-004

Abrir anexo não exige sair do contexto principal.

## 70. Matriz final

| Tipo           | Editável inline | Filtrar | Ordenar | Agrupar  | Resumo       | Kanban    | Calendário | Fórmula          |
|----------------|-----------------|---------|---------|----------|--------------|-----------|------------|------------------|
| Texto          | Sim             | Sim     | Sim     | Limitado | Contagem     | Sim       | Sim        | Entrada limitada |
| Número         | Sim             | Sim     | Sim     | Sim      | Sim          | Sim       | Sim        | Sim              |
| Moeda          | Sim             | Sim     | Sim     | Sim      | Sim          | Sim       | Sim        | Sim              |
| Data           | Sim             | Sim     | Sim     | Sim      | Sim          | Sim       | Principal  | Sim              |
| Pessoas        | Sim             | Sim     | Sim     | Sim      | Contagem     | Sim       | Sim        | Não direta       |
| Status         | Sim             | Sim     | Sim     | Sim      | Distribuição | Principal | Sim        | Condição         |
| Lista suspensa | Sim             | Sim     | Sim     | Sim      | Distribuição | Sim       | Sim        | Condição         |
| Prioridade     | Sim             | Sim     | Sim     | Sim      | Distribuição | Sim       | Sim        | Condição         |
| Confirmação    | Sim             | Sim     | Sim     | Sim      | Contagem     | Sim       | Sim        | Sim              |
| Arquivo        | Via painel      | Básico  | Básico  | Não      | Contagem     | Sim       | Sim        | Não              |
| Relação        | Sim             | Sim     | Sim     | Sim      | Contagem     | Sim       | Sim        | Limitado         |
| Cronograma     | Sim             | Sim     | Sim     | Não      | Duração      | Sim       | Futuro     | Sim              |
| Fórmula        | Não             | Sim     | Sim     | Depende  | Depende      | Sim       | Depende    | Resultado        |

**Matriz implementada em 03/10/2026 (quadros dinâmicos).** Apenas os 8 tipos criáveis; "Resumo" refere-se ao que existe de fato.

| Tipo   | Editável inline | Filtrar         | Ordenar | Agrupar | Resumo        | Kanban                 | Calendário      | Fórmula |
|--------|-----------------|-----------------|---------|---------|---------------|------------------------|-----------------|---------|
| Texto  | Sim             | Só busca global | Sim     | Não     | Não           | Campo do cartão        | Campo do cartão | Não     |
| Número | Sim             | Não             | Sim     | Não     | Soma por raia | Campo do cartão e soma | Campo do cartão | Não     |

| Tipo                                              | Editável inline                        | Filtrar                     | Ordenar | Agrupar                         | Resumo            | Kanban                 | Calendário       | Fórmula |
|---------------------------------------------------|----------------------------------------|-----------------------------|---------|---------------------------------|-------------------|------------------------|------------------|---------|
| Moeda                                             | Sim                                    | Não                         | Sim     | Não                             | Soma por raia     | Campo do cartão e soma | Campo do cartão  | Não     |
| Data                                              | Sim                                    | Não                         | Sim     | Não                             | Não               | Campo do cartão        | Posiciona o item | Não     |
| Pessoas                                           | Sim (uma pessoa)                       | Filtro por pessoa do quadro | Sim     | Não (só nos quadros de domínio) | Não               | Campo do cartão        | Campo do cartão  | Não     |
| Status                                            | Sim                                    | Só busca global             | Sim     | Sim (Kanban)                    | Contagem por raia | Agrupar e colore       | Colore o cartão  | Não     |
| Lista suspensa                                    | Sim                                    | Só busca global             | Sim     | Sim (Kanban)                    | Contagem por raia | Agrupar e colore       | Colore o cartão  | Não     |
| Confirmação                                       | Sim                                    | Não                         | Sim     | Não                             | Não               | Campo do cartão        | Campo do cartão  | Não     |
| Prioridade, Arquivo, Relação, Cronograma, Fórmula | Não implementados em quadros dinâmicos |                             |         |                                 |                   |                        |                  |         |

## 71. Definição final

A coluna na LPS é:

Um componente reutilizável de dados e comportamento que define como uma informação é armazenada, editada, validada, filtrada, ordenada, agrupada, calculada e apresentada nas diferentes visualizações do sistema.

As 13 colunas-base oficiais são:

```

Texto
Número
Moeda
Data
Pessoas
Status
Lista suspensa
Prioridade
Confirmação
Arquivo
Relação
Cronograma
Fórmula

```

Campos específicos da construção e da operação da Biasi devem nascer da composição desses tipos, e não da criação indiscriminada de novos tipos técnicos.

Essa é a base para a LPS ser dinâmica sem virar um sistema tecnicamente desorganizado.

**Status (03/10/2026):** a definição é o alvo. Hoje o motor entrega 8 dos 13 tipos-base (Texto, Número, Moeda, Data, Pessoas, Status, Lista suspensa e Confirmação); os cinco restantes (Prioridade, Arquivo, Relação, Cronograma e Fórmula) continuam como requisito futuro.

# Apêndice C — Especificação da Visualização por Calendário

## Status de implementação (03/10/2026)

A visualização Calendário está **em grande parte implementada** como visão do motor de Quadros ( `BoardView` do tipo `Calendário`), com mês, cartões, criar no dia, arrastar entre dias, gaveta, hora opcional, busca, filtro por pessoa e configuração que salva sozinha; cobre quase toda a "primeira versão obrigatória" (§113). Ela vale para o **Quadro de cada Demanda** (itens = tarefas). As telas de lista `/demandas/calendario/` e `/tarefas/calendario/` **não** usam essa visão: a primeira é uma lista de demandas por prazo e a segunda é uma semana somente de leitura. Na interface, Atividade virou **Demanda**, "Situação" virou **Etap**a e "Condição" virou **Status** (o código segue `Activity / Task`). Legenda: **Implementado**, **Parcial**, **Não implementado**, **Divergente**.

| Requisito / bloco                                                                     | Status       | Onde está no código                                                                                                                                                     |
|---------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calendário como visão dos mesmos itens, sem dados próprios (§1, §2, §4)               | Implementado | <code>boards/calendar_view.py</code> (docstring do módulo), <code>BoardView.Type.CALENDAR</code> em <code>boards/models.py</code>                                       |
| Escolha da coluna de Data; exigir coluna de Data (§5, §56)                            | Implementado | <code>clean_calendar_settings</code> , <code>complete_new_settings</code> ; aba "+" cria a coluna de Data no ato ( <code>boards.js</code> )                             |
| Múltiplos Calendários com Datas diferentes (§57)                                      | Implementado | Várias visões por quadro ( <code>ViewService</code> , <code>board-view-create</code> ); <code>boards/test_calendar.py</code>                                            |
| Data com ou sem hora, fuso local (§6–§8, §42–§44, §96–§97)                            | Implementado | <code>item_when</code> , configuração "Exibir horário" da coluna; pop-over de data em <code>boards.js</code>                                                            |
| Escala temporal (§15)                                                                 | Parcial      | Só <b>Mês</b> ( <code>PERIODS = ("month",)</code> ); semana/dia/agenda não existem na visão de Quadro                                                                   |
| Grade mensal, dias de outros meses, hoje, Hoje/anterior/próximo (§14, §16–§18, §89)   | Implementado | <code>build_month</code> , <code>grid_range</code> , <code>shift_month</code> ; <code>board-view-calendar</code> troca o mês sem recarregar ( <code>?mes=</code> )      |
| Hover "+ Adicionar", criar no dia, janela por cima do calendário (§19–§24)            | Implementado | <code>board_calendar.html</code> , <code>_calendar_day.html</code> , <code>boards.js</code> ; <code>board-item-create</code> com <code>initial</code> (tudo ou nada)    |
| Cartão: título, campos, limite, personalização (§27–§31)                              | Implementado | <code>_calendar_card.html</code> ; <code>card_fields</code> (até 6, padrão 3, <code>MAX_CARD_FIELDS</code> )                                                            |
| Cor por Status/Lista, grupo ou sem cor; cor nunca única informação (§32–§34, §79–§80) | Implementado | <code>color_by</code> , <code>entry_color</code> ; renomear/recolorir a etiqueta reflete (testes)                                                                       |
| Clique no cartão abre a gaveta; clique no campo edita (§35–§37)                       | Implementado | <code>boards/_item_drawer.html</code> , <code>board-item-detail</code>                                                                                                  |
| Arrastar entre dias, otimista, autosave, volta com aviso (§38–§41, §74)               | Implementado | <code>boards.js</code> (modo <code>[data-cal-body]</code> ), grava só a célula de data pelo serviço da tabela; <code>BOARD_CELL_UPDATED</code> com data anterior e nova |
| Remover hora / remover data (§44, §45)                                                | Implementado | Pop-over de data ("Limpar horário", "Limpar data")                                                                                                                      |
| Itens sem Data e colocar no calendário (§46, §47)                                     | Parcial      | Contador e lista "Sem data" ( <code>PANEL_LIMIT = 100</code> ); colocar no calendário é editando a data ou criando pelo dia                                             |
| Muitos itens no dia e + N mais (§48, §49)                                             | Implementado | <code>MAX_VISIBLE_PER_DAY = 3</code> , lista em pop-over                                                                                                                |
| Pesquisa e filtro por Pessoa; mantidos ao mudar de mês (§50–§51)                      | Implementado | <code>?q</code> e <code>?pessoa</code> nos links do mês                                                                                                                 |

| Requisito / bloco                                                           | Status           | Onde está no código                                                                                                                                                     |
|-----------------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Filtros genéricos, composição e filtros salvos (§52–§54)                    | Não implementado | Só busca e pessoa; no Quadro não há filtros por Status/Grupo salvos                                                                                                     |
| Finais de semana ocultos e aviso (§58–§59)                                  | Implementado     | <code>show_weekends</code> , aviso e dias bloqueados                                                                                                                    |
| Concluídos (§60)                                                            | Implementado     | <code>show_completed</code> (só o Status decide); atrasados fora do mês em lista própria                                                                                |
| Cancelados (§61)                                                            | Não implementado | Não há tratamento próprio para itens cancelados                                                                                                                         |
| Tarefas: prazo solicitado × comprometido (§3.2, §62, §64–§65)               | Divergente       | O quadro tem as colunas de Data que o modelo criar ( <code>starter_templates.py</code> : "Prazo"); os dois prazos de <code>Task</code> só existem no modelo legado      |
| Templates padrão de Demandas e Tarefas com Calendário (§63, §64)            | Parcial          | O Kanban nasce em todo quadro; o Calendário <b>não</b> nasce pronto, é criado pela aba "+"                                                                              |
| Atraso e priorização visual (§66–§68)                                       | Parcial          | Marcador "Atrasada" só em coluna de prazo ( <code>is_deadline</code> ); sem priorização visual própria                                                                  |
| Reagendamento e auditoria (§69–§72)                                         | Implementado     | Mover grava a data anterior e a nova no log; item não é concluído ao mudar de dia                                                                                       |
| Permissões e multi-organização (§73–§75)                                    | Implementado     | <code>quadro.editar_item</code> / <code>quadro.criar_item</code> ; leitor não arrasta; 404 entre organizações ( <code>boards/test_calendar.py</code> )                  |
| Atualização entre visualizações (§76–§80)                                   | Implementado     | Calendário, Tabela e Kanban leem as mesmas células                                                                                                                      |
| Tempo estimado e sessões de trabalho (§82, §83)                             | Não implementado | —                                                                                                                                                                       |
| Conflito de edição (§92)                                                    | Parcial          | Há controle de versão nas visões de domínio ( <code>DomainBoardConflict</code> , <code>data-work-updated-at</code> ); o Calendário de Quadro não trata conflito         |
| Teclado e acessibilidade (§93, §94)                                         | Parcial          | Foco devolvido, Escape em duas etapas, "+ Adicionar" visível sem mouse; não há navegação por setas entre os dias do calendário (as setas existem nas células da tabela) |
| Duplicar e compartilhar visualização (§102, §103)                           | Não implementado | Existem criar, renomear e excluir ( <code>board-view-*</code> )                                                                                                         |
| Excluir a visão não apaga itens; renomear (§100, §101)                      | Implementado     | <code>ViewDeleteView</code> , <code>ViewUpdateView</code>                                                                                                               |
| Integrações externas, recorrência, Gantt (§110–§112)                        | Não implementado | Fora da primeira versão, como a especificação prevê                                                                                                                     |
| Calendário de Demandas na lista /demandas/calendario/ (§3.1, §104, §123)    | Divergente       | Lista de cartões por <code>requested_deadline</code> ( <code>demand_work_board.html</code> , <code>DemandWorkBoardView</code> ), sem grade nem arrastar                 |
| Calendário de Tarefas de todos os quadros /tarefas/calendario/ (§105, §124) | Divergente       | Semana somente de leitura, com contador "Sem prazo" ( <code>TaskCenterQuery.week</code> , <code>task_center.html</code> )                                               |

**Divergência:** a especificação trata Demandas e Tarefas como dois Quadros, cada um com seu Calendário. No código, o Calendário completo existe nos Quadros (hoje um por Demanda); as telas globais de Demandas e de Tarefas oferecem apenas as versões simplificadas descritas acima.

## 1. Objetivo

A visualização por **Calendário** organiza os mesmos itens do Quadro em uma linha do tempo baseada em uma coluna do tipo Data.

Na LPS, o Calendário não deve ser um módulo separado nem possuir dados próprios. Ele é uma **visualização configurável dos mesmos itens** exibidos na Tabela (Quadro) e no Kanban.

Exemplo:

| Quadro: Demandas   |             |            |                    |
|--------------------|-------------|------------|--------------------|
| Demanda            | Responsável | Prazo      | Status             |
| Arena Center Norte | Ryan        | 06/10/2026 | Em andamento       |
| GEHAKA             | Luan        | 12/10/2026 | Aguardando retorno |
| Ahlstrom           | Jennifer    | 14/10/2026 | Confirmada         |

Calendário:

|                    |
|--------------------|
| 06/10              |
| Arena Center Norte |
| 12/10              |
| GEHAKA             |
| 14/10              |
| Ahlstrom           |

Nenhum item foi duplicado.

## 2. Princípio central

O dado pertence ao Quadro. O Calendário é apenas uma maneira temporal de enxergar e editar esse dado.

Consequências:

- a mesma Demanda ou Tarefa aparece na Tabela, Kanban e Calendário;
- alterar a data no Calendário altera o mesmo campo utilizado nas demais visualizações;
- alterar Status, Pessoa, Setor ou outra informação pelo Calendário deve refletir imediatamente nas outras visualizações;
- o Calendário não deve criar cópias dos itens;
- pequenas alterações devem acontecer sem o usuário sair da tela;
- a visualização deve privilegiar compromissos, prazos e entregas, e não vigilância de pessoas.

## 3. Relação com a arquitetura atual da LPS

Hoje a LPS já possui calendários para Demandas (antes Atividades) e Tarefas: os dois simplificados em `/demandas/calendario/` e `/tarefas/calendario/` e o Calendário configurável dos Quadros (app `boards`).

A documentação atual estabelece que Lista, Kanban e Calendário de Demandas utilizam o mesmo filtro de dados ( `activities/filtering.py` , barra `_workspace_filter_toolbar.html` ).

A evolução proposta mantém esse princípio, mas transforma o Calendário em uma visualização configurável do novo motor de Quadros.



### 3.1 Demandas

Na interface, **Atividade** passa a ser chamada de **Demanda** (já é assim desde 01/10/2026, inclusive nas URLs `/demandas/...`).

Internamente, enquanto a migração arquitetural não estiver concluída:

```
Demanda = Activity
```

O campo nativo disponível atualmente para prazo da Demanda é:

```
Activity.requested_deadline
```

Esse campo pode ser usado como Data padrão da visualização de Calendário de Demandas.

### 3.2 Tarefas

A Tarefa possui atualmente dois conceitos de prazo:

```
requested_deadline
committed_deadline
```

Significados:

```
Prazo solicitado = prazo pedido por quem solicitou
Prazo comprometido = prazo assumido pelo executor
```

A visualização deve permitir escolher qual coluna temporal será utilizada.

No template padrão de Tarefas, a preferência deve ser:

```
Prazo comprometido
```

Se a organização quiser trabalhar visualmente com Prazo solicitado, deve poder criar outra visualização de Calendário usando esse campo.

Não esconder a diferença conceitual entre os dois prazos.

---

## 4. Regra estrutural da visualização

O Calendário precisa estar vinculado a:

```
Board
└─ BoardView
 └─ type = CALENDAR
 └─ settings
```

Exemplo conceitual:

```
{
 "name": "Calendário de prazos",
 "type": "calendar",
 "settings": {
 "date_field": "prazo",
 "period": "month",
 "color_by": "status",
 "show_weekends": true
 }
}
```

A visualização salva **configuração**, não cópia de dados.

## 5. Pré-requisito para criar um Calendário

Uma visualização de Calendário precisa de pelo menos uma coluna compatível com Data.

Tipos compatíveis inicialmente:

Data  
Data + hora

Tipos futuros:

Intervalo de datas  
Cronograma

Se o Quadro não tiver uma coluna de Data:

Criar visualização → Calendário

A LPS deve orientar:

Para usar o Calendário, escolha ou crie uma coluna de Data.

[ + Criar coluna de Data ]

O usuário não deve precisar sair da tela.

## 6. Uma coluna Data, duas formas de uso

Data e Data + hora não devem ser tratadas como dois campos completamente independentes para o usuário.

A coluna deve possuir configuração:

Data  
Exibir horário: Não

Ao ativar horário:

```
Data
Exibir horário: Sim
```

Exemplo:

```
15/10/2026
```

passa a aceitar:

```
15/10/2026 14:00
```

## 6.1 Regra

A hora é opcional e pertence à mesma informação de Data.

Não exigir horário para usar o Calendário.

## 7. Item sem horário

Se o item possuir:

```
Data = 15/10/2026
Hora = vazia
```

o Calendário deve tratá-lo como compromisso do dia.

Exemplo:

```
15
```

```
Revisar proposta comercial
```

Não inventar horário.

## 8. Item com horário

Se o item possuir:

```
Data = 15/10/2026
Hora = 14:00
```

o Calendário pode mostrar:

```
14:00 Visita técnica
```

Na visão mensal, o horário deve ser compacto.

Na futura visão diária/semanal, o item pode ocupar a faixa correspondente ao horário.

---

## 9. Estrutura principal da tela

A visualização deve possuir cinco áreas:

1. cabeçalho;
  2. abas de visualização;
  3. barra de ações;
  4. navegação temporal;
  5. grade do Calendário.
- 

## 10. Cabeçalho

Exemplo para Demandas:

```
Demandas
Gestão das demandas internas e intersetoriais.
```

Exemplo para Tarefas:

```
Tarefas
Execução operacional das tarefas vinculadas às demandas e processos.
```

A identidade do módulo continua sendo a mesma.

Apenas a visualização muda.

---

## 11. Abas de visualização

Exemplo:

```
[Quadro principal] [Kanban] [Calendário] [+]
```

Regras:

- `Calendário` deve ficar destacado quando ativo;
- mudar de visualização não altera o conjunto de dados;
- filtros salvos podem ser próprios de cada visualização;
- o usuário pode ter mais de um Calendário no mesmo Quadro.

Exemplo:

```
Calendário de prazos
Calendário de visitas técnicas
Calendário de entregas
```

Os três podem usar os mesmos itens, mas colunas de Data diferentes.

---

## 12. Barra de ações

Exemplo:

```
[Criar demanda] [Adicionar ferramenta] Pesquisar Pessoa Filtro ...
```

ou:

```
[Criar tarefa] [Adicionar ferramenta] Pesquisar Pessoa Filtro ...
```

Itens principais:

- Criar item;
- Pesquisar;
- Pessoa / Responsável;
- Filtro;
- Hoje;
- período anterior;
- período seguinte;
- mês/período atual;
- seletor de visualização temporal;
- Configurações;
- Mais ações.

## 13. Botão Criar

O nome do botão deve respeitar o contexto.

```
Demandas → Criar demanda
Tarefas → Criar tarefa
Genérico → Criar item
```

O botão não deve mandar o usuário para outra página.

Deve abrir o formulário por cima da visualização atual.

## 14. Navegação temporal

A barra deve conter:

```
[Hoje] < > outubro 2026 [Mês v]
```

### 14.1 Hoje

**Hoje** leva a visualização diretamente ao período que contém a data atual.

## 14.2 Anterior

Na visualização Mês:

< = mês anterior

## 14.3 Próximo

Na visualização Mês:

> = mês seguinte

## 14.4 Título do período

Exemplo:

outubro 2026

Deve refletir o período atualmente carregado.

## 15. Escala temporal

Primeira entrega obrigatória:

Mês

Estrutura preparada para:

Semana  
Dia  
Agenda

A expansão para Semana e Dia não deve exigir criação de outro conjunto de dados.

## 16. Grade mensal

A visão Mês deve usar sete colunas:

Seg | Ter | Qua | Qui | Sex | Sáb | Dom

Cada célula representa um dia.

Cada célula deve suportar:

- número do dia;
- itens daquele dia;
- estado de hover;
- criação rápida;
- limite visual de itens;
- indicador de itens adicionais;

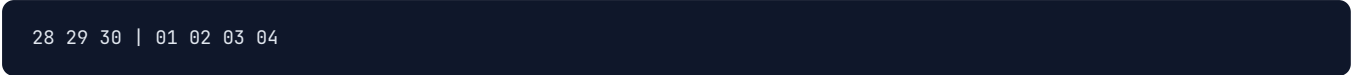
- drag-and-drop de itens entre datas.

## 17. Dias de outros meses

Na primeira e última semana podem aparecer dias do mês anterior ou seguinte.

Devem ser visualmente discretos.

Exemplo:



28 29 30 | 01 02 03 04

Os dias fora do mês ativo continuam clicáveis.

## 18. Data atual

O dia atual deve receber destaque discreto.

Exemplo:



[ 02 ]

Não usar uma cor excessivamente forte que concorra com os cartões.

## 19. Hover sobre o dia

Essa é uma regra importante da experiência.

Quando o usuário não está interagindo:



15

Ao passar o mouse:



15  
+ Adicionar

### 19.1 Regra

A ação deve aparecer no contexto onde o usuário pretende agir.

Não manter **+ Adicionar** permanentemente em todas as células.

Isso poluiria a tela.

## 20. Criar item clicando no dia

Ao clicar em:



+ Adicionar

dentro de 15/10/2026 , o formulário deve abrir sem sair do Calendário.

A Data deve vir preenchida automaticamente:

```
Data = 15/10/2026
```

O usuário não deve informar novamente algo que o contexto já determinou.

---

## 21. Criação contextual

A regra é:

```
O gesto do usuário pode preencher informações do item.
```

Exemplos:

```
Clicou no dia 15
→ Data = 15/10/2026
```

```
Criou dentro de uma visualização filtrada por Setor Comercial
→ a LPS pode sugerir Comercial, sem gravar automaticamente se houver ambiguidade
```

```
Arrastou um item de 15 para 18
→ Data = 18/10/2026
```

Não preencher campos de forma implícita quando o contexto não for inequívoco.

---

## 22. Modal de criação de Demanda

Exemplo de campos:

```
Criar demanda

Grupo
Responsável
Status
Prioridade
Setor
Cliente/Obra
Data
Descrição curta
```

Os campos exibidos devem vir da configuração do Quadro.

A lista acima é um template inicial, não uma estrutura rígida.



## 23. Modal de criação de Tarefa

Exemplo:

```
Criar tarefa

Demanda vinculada
Responsável
Setor
Status
Prioridade
Prazo
Hora (opcional)
Checklist rápido
```

Novamente, os campos devem ser configuráveis.

A Tarefa precisa continuar vinculada às regras reais de `TaskService`.

## 24. Editores dentro do modal

Mesmo dentro de um modal maior, campos simples não devem abrir novas páginas.

Exemplo:

```
Responsável
[Paulo v]
```

clicou:

```
Pesquisar pessoa...

Paulo
Ryan
Luan
Jennifer
```

Selecionou:

```
Paulo
```

O popover fecha.

## 25. Níveis de interação

A LPS deve trabalhar com três níveis.

### 25.1 Edição direta

Para:

- título;
- texto curto;
- número.

Fluxo:

clicar → editar → salvar automaticamente

## 25.2 Popover contextual

Para:

- Data;
- Pessoa;
- Status;
- Prioridade;
- Setor;
- Lista suspensa;
- Cor.

Fluxo:

clicar → popover → selecionar → pronto

## 25.3 Modal ou drawer

Para:

- criação completa;
- detalhes;
- checklist;
- comentários;
- anexos;
- histórico;
- configurações avançadas.

## 26. Regra global de navegação

Não tirar o usuário do Calendário para editar uma informação que pode ser alterada no próprio contexto.

O usuário deve continuar enxergando o Calendário atrás do editor.

## 27. Cartão do Calendário

Cada item deve possuir um cartão compacto.

Exemplo:

Arena Center Norte – Levantamento  
● Planejamento  
LT Lucas Tavares  
Arena Center Norte

O cartão não deve tentar mostrar todos os campos do item.

---

## 28. Título do cartão

O título é obrigatório.

Demandas:



Título da demanda

Tarefas:



Título da tarefa

O título deve ser a informação visual principal.

---

## 29. Campos visíveis no cartão

A visualização deve permitir configurar quais campos aparecem.

Exemplos:

- Status;
  - Responsável;
  - Setor;
  - Prioridade;
  - Cliente;
  - Obra;
  - Demanda vinculada;
  - horário;
  - Tags;
  - outros campos configuráveis.
- 

## 30. Limite de informação no cartão

O cartão mensal deve ser compacto.

Recomendação:



Título  
+ até 3 informações secundárias

Informações adicionais ficam disponíveis ao clicar.

Evitar transformar cada célula do Calendário em um formulário.

---

## 31. Personalização do cartão

A configuração pode possuir:

```
Personalizar item do Calendário
```

- ☒ Status
- ☒ Responsável
- ☒ Cliente/Obra
- ☐ Setor
- ☐ Prioridade
- ☐ Descrição

Os campos devem poder ser reordenados.

## 32. Cor dos itens

A visualização deve permitir:

```
Colorir por:
[Status v]
```

Possíveis fontes:

- Status;
- Prioridade;
- Setor;
- Grupo;
- outra coluna categórica compatível.

## 33. Cor não é identidade do item

A cor é apenas uma representação visual.

Se:

```
Status = Em andamento
Cor = azul
```

e a cor for alterada para verde:

- Quadro deve atualizar;
- Kanban deve atualizar;
- Calendário deve atualizar;
- filtros continuam funcionando;
- o item continua sendo o mesmo.

## 34. Status com identificador estável

Nunca ligar regras ao texto visual.

Errado:

```
if status.name == "Em andamento":
```

Correto conceitualmente:

```
status_option_id = 42
```

O usuário pode renomear:

```
Em andamento
```

para:

```
Em execução
```

sem quebrar:

- Calendário;
- filtros;
- automações;
- Kanban;
- relatórios.

---

## 35. Clique em um item

Ao clicar em um cartão:

```
Arena Center Norte – Levantamento
```

não abrir outra página.

Abrir um drawer lateral ou modal de detalhe.

---

## 36. Drawer de Demanda

Pode conter:

- título;
- Status;
- Responsável;
- Solicitante;
- Setor;
- Prazo;
- Cliente;
- Obra;
- descrição;
- checklist/etapas;
- Tarefas vinculadas;
- comentários;

- arquivos;
- histórico.

O Calendário continua visível atrás.

---

## 37. Drawer de Tarefa

Pode conter:

- título;
  - Demanda vinculada;
  - Responsável;
  - Setor;
  - Status;
  - prazo;
  - horário;
  - checklist;
  - participantes;
  - comentários;
  - anexos;
  - histórico;
  - ações operacionais.
- 

## 38. Drag-and-drop entre dias

O usuário deve poder segurar um item e mover para outro dia.

Exemplo:

```
15/10
Visita técnica
```

arrastado para:

```
17/10
```

Resultado:

```
Data = 17/10/2026
```

---

## 39. Regra do drag-and-drop

Mover no Calendário altera **somente o campo de Data** que alimenta aquela visualização.

Não alterar automaticamente:

- Status;
- Responsável;
- Setor;
- Prioridade.

A menos que exista uma automação explícita configurada para isso.

---

## 40. Atualização otimista

Ao soltar o item:

1. mover visualmente imediatamente;
2. salvar em segundo plano;
3. confirmar silenciosamente;
4. se falhar, voltar ao dia anterior;
5. mostrar mensagem curta de erro.

Isso evita sensação de lentidão.

---

## 41. Autosave

Não criar botão:



Salvar Calendário

Pequenas alterações devem ser gravadas automaticamente.

Exemplos:

- mudar Data;
  - mudar hora;
  - mudar Status;
  - trocar Responsável;
  - alterar Prioridade;
  - mover item.
- 

## 42. Data + hora inline

Ao clicar na Data, abrir um popover.

Exemplo:



Hoje

15/10/2026

outubro 2026

Seg Ter Qua Qui Sex Sáb Dom

...

Mostrar ícone de relógio:



## 43. Adicionar hora

Ao clicar no relógio:

```
15/10/2026 | 14:00
```

Mostrar:

```
Horário opcional
```

```
08:00
```

```
09:00
```

```
10:00
```

```
14:00
```

```
15:00
```

```
16:00
```

O usuário também pode digitar.

## 44. Remover hora

A interface deve permitir:

```
Limpar horário
```

Sem apagar a Data.

Resultado:

```
15/10/2026
```

## 45. Remover Data

Se o usuário limpar a Data:

```
Data = vazia
```

o item deixa de aparecer na grade principal.

Ele não deve ser excluído.

## 46. Itens sem Data

A visualização deve oferecer acesso aos itens sem Data.

Exemplo:

```
Sem data 7
```



Ao clicar:

Itens sem data

- Revisar contrato
- Validar projeto
- Solicitar cotação

Isso evita que itens desapareçam da gestão.

---

## 47. Colocar item sem Data no Calendário

Da lista `Sem data`, o usuário pode:

- abrir e definir a Data;
- futuramente arrastar para um dia.

Ao definir:

18/10/2026

o item passa a aparecer imediatamente no dia 18.

---

## 48. Muitos itens no mesmo dia

Se a célula não comportar todos:

15

Demanda A  
Demanda B  
Demanda C  
+ 5 mais

Não aumentar indefinidamente a altura da semana.

---

## 49. + N mais

Ao clicar:

+ 5 mais

abrir um popover ou painel com todos os itens do dia.

Não navegar para outra tela.

---

## 50. Busca

A barra deve permitir pesquisa textual.

Exemplos:

```
Santa Isabel
Arena
Ahlstrom
SPDA
```

O Calendário mostra apenas itens compatíveis.

---

## 51. Filtro por Pessoa

Exemplo:

```
Pessoa
[Ryan]
```

Mostrar somente itens relacionados ao Ryan conforme o campo usado pela visualização.

A regra de relacionamento precisa ser explícita.

Demandas:

```
Responsável/Dono
Solicitante
Participante, se configurado
```

Tarefas:

```
Responsável
Participante
```

Não misturar papéis sem informar.

---

## 52. Filtros disponíveis

A visualização deve reaproveitar o motor de filtros do Quadro.

Exemplos:

- Responsável;
  - Solicitante;
  - Setor;
  - Status;
  - Prioridade;
  - Cliente;
  - Obra;
  - Centro de custo;
  - Grupo;
  - Tags;
  - Demanda vinculada;
  - campos customizados.
-

## 53. Composição de filtros

Permitir regras como:

```
Setor = Comercial
E
Status ≠ Concluído
E
Responsável = Ryan
```

Não criar um motor separado exclusivamente para o Calendário.

## 54. Filtros temporários e filtros salvos

Filtros aplicados apenas durante a navegação podem ser temporários.

Quando o usuário salvar a configuração:

```
Calendário Comercial
```

o conjunto de filtros passa a fazer parte da configuração da visualização.

## 55. Configuração da visualização

Painel sugerido:

```
Configurações do Calendário

Data utilizada
[Prazo v]

Colorir por
[Status v]

Campos do cartão
[Configurar]

Mostrar finais de semana
[Sim]

Mostrar concluídos
[Sim]

Itens sem data
[Mostrar contador]

Escala
[Mês]
```

## 56. Escolher a Data utilizada

Essa é a configuração mais importante.

Exemplos em Demandas:

```
Prazo
Visita técnica
Data de entrega
Data de envio
```

Exemplos em Tarefas:

```
Prazo solicitado
Prazo comprometido
Data de execução
```

O Calendário deve ler exatamente a coluna escolhida.

---

## 57. Múltiplos Calendários

O mesmo Quadro pode possuir:

```
Calendário de prazos
Calendário de visitas técnicas
Calendário de entregas ao cliente
```

Todos exibindo os mesmos itens.

Isso é um dos principais ganhos do modelo de visualizações.

---

## 58. Finais de semana

Configuração:

```
Mostrar finais de semana: Sim/Não
```

Se desativado:

```
Seg | Ter | Qua | Qui | Sex
```

Não alterar as Datas dos itens.

É apenas visual.

---

## 59. Datas em final de semana oculto

Se um item estiver em sábado e os finais de semana estiverem ocultos:

- não mover automaticamente para sexta ou segunda;
- não alterar o banco;
- indicar que existem itens em dias ocultos;
- oferecer forma de exibi-los.

## 60. Concluídos

A visualização pode possuir:

```
Mostrar concluídos: Sim/Não
```

Ocultar concluídos é um filtro visual.

Não excluir nem alterar o item.

## 61. Cancelados

Mesma regra.

A visualização pode ocultar cancelados por padrão.

O comportamento deve estar documentado na configuração da view.

## 62. Tarefas: prazo solicitado x comprometido

A LPS não deve apagar essa diferença.

Exemplo:

```
Prazo solicitado: 15/10
Prazo comprometido: 18/10
```

Um Calendário pode mostrar:

```
Calendário – prazo comprometido
```

Outra visualização pode mostrar:

```
Calendário – prazo solicitado
```

Evitar escolher silenciosamente um deles em todos os contextos.

## 63. Template padrão de Demandas

Configuração inicial sugerida:

```
Nome: Calendário
Data: Prazo
Colorir por: Status
Escala: Mês
Mostrar finais de semana: Sim
```

Cartão:

```
Título
Status
Responsável
Cliente/Obra
```

---

## 64. Template padrão de Tarefas

Configuração inicial sugerida:

```
Nome: Calendário
Data: Prazo comprometido
Colorir por: Status
Escala: Mês
Mostrar finais de semana: Sim
```

Cartão:

```
Título
Status
Responsável
Demanda vinculada
```

Se `Prazo comprometido` ainda não existir para a Tarefa, a interface deve indicar isso claramente.

---

## 65. Tarefa sem prazo comprometido

Não inventar prazo.

Possibilidades:

```
Sem prazo comprometido
```

ou, se a visualização tiver sido configurada explicitamente:

```
Usar prazo solicitado enquanto não houver prazo comprometido
```

Essa opção deve ser visível na configuração.

Não aplicar fallback oculto.

---

## 66. Priorização visual

Prioridade pode ser mostrada como chip:

```
Alta
Média
Baixa
```

Mas a cor principal do cartão deve seguir a regra `Colorir por`.

Não usar três códigos de cor concorrentes no mesmo cartão.

---

## 67. Atraso

Um item atrasado pode receber um indicador discreto.

Exemplo:

▲ Atrasada

ou borda específica.

Não transformar todo o cartão em vermelho se isso prejudicar a leitura do código de cores configurado.

---

## 68. Hoje + atraso

Ao abrir o Calendário no dia atual, itens anteriores ainda abertos podem ser acessados por:

- filtro Atrasadas ;
- painel lateral;
- indicador/resumo.

O Calendário não deve esconder trabalho vencido.

---

## 69. Reagendamento não é conclusão

Mover:

15/10 → 18/10

significa reagendar.

Não significa:

Status = Concluído

---

## 70. Reagendamento não deve apagar histórico

A alteração de Data deve registrar:

Data anterior  
Nova Data  
Usuário  
Data/hora da alteração

---

## 71. Auditoria

Mudanças feitas pelo Calendário precisam passar pelo mesmo padrão da arquitetura da LPS:

URL → View → Service → Model

O Calendário não deve gravar diretamente no model ignorando Services.

## 72. Alterações auditáveis

Pelo menos:

- mudança de Data;
- mudança de hora;
- troca de Responsável;
- mudança de Status;
- mudança de Prioridade;
- criação;
- edição relevante;
- exclusão/inativação conforme regra do domínio.

## 73. Permissões

Ver o botão não significa ter autorização.

O Calendário deve respeitar o motor:

```
SUJEITO + AÇÃO + ESCOPO + ORIGEM
```

Exemplos:

```
atividade.editar
tarefa.editar
tarefa.criar
atividade.criar
```

## 74. Drag-and-drop e autorização

Antes de aceitar um movimento de Data:

1. verificar organização;
2. verificar acesso ao recurso;
3. verificar permissão para editar;
4. validar o novo valor;
5. salvar por Service;
6. auditar.

Nunca repetir o problema atual do movimento de estágio do Kanban, que hoje possui lacunas de autorização/auditoria documentadas.

## 75. Multi-organização

O Calendário deve respeitar `Organization`.

Nenhum item de outra organização pode:

- aparecer;
- ser pesquisado;
- ser movido;
- ser aberto;



- ser editado.

---

## 76. Atualização entre visualizações

Exemplo:

Calendário:

Visita técnica  
15/10

Usuário arrasta para:

17/10

Ao abrir a Tabela:

Prazo = 17/10/2026

Não existe sincronização posterior.

É o mesmo dado.

---

## 77. Atualização de Status

Se o usuário clicar no cartão e alterar:

Em andamento → Concluída

o Kanban deve refletir automaticamente.

Se o Kanban estiver agrupado por Status, o cartão muda de coluna.

---

## 78. Atualização de Pessoa

Se trocar:

Ryan → Luan

Filtros e outras visualizações devem refletir imediatamente.

---

## 79. Renomear Status

Se:

Em andamento

for renomeado para:

Em execução

os cartões do Calendário passam a mostrar:

Em execução

sem migração manual de itens.

## 80. Alterar cor do Status

Se a cor mudar:

Azul → Verde

todos os cartões que usam aquela opção devem mudar imediatamente.

## 81. Calendário não é agenda de microgerenciamento

A finalidade é mostrar:

- compromissos;
- entregas;
- prazos;
- concentração de demandas;
- conflitos;
- sobreposição;
- datas críticas.

Não deve ser usado por padrão para medir:

- minutos trabalhados;
- presença contínua;
- cada pausa;
- cada movimento de funcionário.

## 82. Tempo estimado

Se existir como campo de Tarefa, pode aparecer no cartão.

Exemplo:

Estimativa: 2h

Mas não deve ser obrigatório para o Calendário funcionar.

## 83. Sessões de trabalho

`WorkSession` não deve determinar onde um item aparece no Calendário.

Calendário trata de Data/Prazo.

Cronômetro trata de execução.

São conceitos diferentes.

## 84. Visão gerencial

O Calendário pode ajudar o gestor a enxergar:

- dias com excesso de entregas;
- concentração de visitas;
- prazos simultâneos;
- itens atrasados;
- capacidade temporal;
- compromissos por responsável;
- demanda por setor.

A visualização deve ajudar a decidir onde agir.

---

## 85. Visão do executor

O executor pode usar filtros:

Responsável = Eu

e enxergar:

meus compromissos da semana/mês

Sem exposição desnecessária de informações de outros setores quando a permissão não permitir.

---

## 86. Densidade

Em meses muito carregados, o sistema deve continuar legível.

Regras:

- cartões compactos;
  - altura máxima por célula;
  - + N mais ;
  - tooltip/popover;
  - filtros rápidos;
  - não reduzir fonte indefinidamente.
- 

## 87. Responsividade

Desktop é a experiência principal da primeira entrega.

Em telas menores:

- menu lateral pode recolher;
- Calendário pode horizontalizar/rolar;
- drawer pode ocupar a tela inteira;
- ações continuam acessíveis.

Não tentar comprimir sete dias em uma largura ilegível.

---

## 88. Performance

A visualização deve buscar somente o intervalo necessário.

Exemplo:

```
outubro 2026
```

Carregar:

- dias visíveis do mês;
- margem necessária para a primeira/última semana.

Não carregar todo o histórico do Quadro.

## 89. Mudança de mês

Ao clicar em `>` :

1. atualizar título;
2. buscar apenas o novo intervalo;
3. manter filtros;
4. manter configuração;
5. evitar reload completo quando possível.

## 90. Estado vazio

Sem itens no mês:

```
Nenhum item neste período.

Passe o mouse sobre um dia para adicionar.
```

Não tratar como erro.

## 91. Erro ao salvar

Exemplo:

```
Não foi possível alterar a data.
0 item voltou para 15/10/2026.
```

A mensagem precisa ser curta e explicativa.

## 92. Conflito de edição

Se outro usuário alterar o mesmo item simultaneamente:

- não sobrescrever silenciosamente uma versão mais recente;
- atualizar o cartão;
- informar conflito quando necessário;
- preservar a informação mais recente validada pelo servidor.

---

## 93. Acessibilidade

A experiência não pode depender exclusivamente de cor.

Exemplo:

● Concluída

e não apenas uma faixa verde sem texto.

Itens interativos precisam suportar:

- foco de teclado;
- Enter;
- Esc;
- labels;
- contraste adequado;
- tooltip acessível.

---

## 94. Teclado

Comportamentos esperados:

```
Enter = abrir/confirmar
Esc = fechar popover/modal
Tab = navegar
```

Drag-and-drop deve possuir alternativa acessível futura.

---

## 95. Formatação de Data

Interface brasileira:

DD/MM/AAAA

Exemplo:

15/10/2026

No cartão mensal, formas compactas podem ser usadas sem ambiguidade.

---

## 96. Fuso horário

Horas devem respeitar o fuso configurado para a organização/usuário.

Não assumir UTC na interface.

Armazenamento técnico pode utilizar horário timezone-aware, mas a exibição deve respeitar o contexto local.

---

## 97. Data sem hora x DateTime

A implementação precisa preservar a diferença sem inventar horário.

Conceitualmente:

```
15/10/2026
```

não é automaticamente:

```
15/10/2026 00:00
```

para fins de experiência do usuário.

O sistema deve saber que aquele item é `all day`.

---

## 98. Configuração salva

Uma visualização deve preservar:

- nome;
  - tipo;
  - Data utilizada;
  - escala;
  - filtros salvos;
  - cor por;
  - campos visíveis;
  - ordem dos campos;
  - finais de semana;
  - exibição de concluídos;
  - outras configurações visuais.
- 

## 99. Configuração não deve alterar o dado

Trocar:

```
Colorir por: Status
```

para:

```
Colorir por: Setor
```

não deve modificar nenhum item.

Apenas a representação visual.

---

## 100. Exclusão da visualização

Excluir:

```
Calendário de visitas
```

---

remove apenas a `BoardView` .

Não apagar:

- Demandas;
- Tarefas;
- Datas;
- Status;
- Pessoas;
- histórico.

---

## 101. Renomear a visualização

Exemplo:

Calendário

para:

Prazos Comercial

O rename deve ser inline e autosave.

---

## 102. Duplicar visualização

O usuário pode duplicar:

Calendário de prazos

para criar:

Calendário Comercial

A nova view copia configurações, não itens.

---

## 103. Compartilhamento

Uma visualização pode ser compartilhada conforme as permissões do Quadro.

Não conceder acesso a dados que o usuário não poderia visualizar pela regra de autorização.

---

## 104. Regras específicas para Demandas

O Calendário de Demandas deve privilegiar visão macro.

Informações úteis:

- Demanda;
- prazo;
- responsável;
- Status;

- Cliente/Obra;
- Setor.

Evitar excesso de checklist ou subtarefas no cartão principal.

---

## 105. Regras específicas para Tarefas

O Calendário de Tarefas é mais operacional.

Informações úteis:

- Tarefa;
- Demanda vinculada;
- Responsável;
- prazo;
- horário, se houver;
- Status;
- Prioridade.

Checklist deve aparecer apenas de forma resumida:

3/5

---

## 106. Demanda e suas Tarefas

Uma Demanda e suas Tarefas são objetos diferentes.

Exemplo:

Demanda:  
Ahlstrom – Visita técnica

Tarefas:

Confirmar acesso  
Separar documentos  
Realizar levantamento  
Emitir relatório

O Calendário de Demandas pode mostrar a Demanda.

O Calendário de Tarefas pode mostrar cada Tarefa.

Não duplicar automaticamente a Demanda como se fosse uma Tarefa.

---

## 107. Mostrar Demanda vinculada no cartão de Tarefa

Exemplo:

Realizar levantamento  
Ahlstrom – Visita técnica

Isso ajuda a manter contexto sem abrir a Demanda.



---

## 108. Clique na Demanda vinculada

Deve abrir o drawer/contexto da Demanda.

Evitar abandonar o Calendário.

---

## 109. Calendário e notificações

Apenas visualizar o Calendário não gera notificação.

Alterações podem gerar notificações se a regra de negócio já determinar isso.

Exemplo:

- mudança de responsável;
- negociação de prazo;
- conclusão;
- bloqueio.

Não notificar por toda pequena mudança visual.

---

## 110. Calendário e automações

Automações podem reagir a mudanças no campo de Data.

Exemplo:

```
Quando Prazo mudar
→ recalcular indicador de atraso
```

ou:

```
2 dias antes do Prazo
→ criar lembrete
```

Essas regras pertencem ao motor de automações, não à visualização em si.

---

## 111. Calendário e integração externa

Sincronização com agendas externas pode ser evolução futura.

Exemplos:

```
Google Calendar
Microsoft Outlook
```

Não faz parte da primeira entrega obrigatória.

Quando existir, precisa definir claramente:

- leitura;
  - escrita;
  - direção da sincronização;
  - conflitos;
-

- privacidade;
  - quais campos sincronizam.
- 

## 112. O que não pertence à primeira versão

Não tornar obrigatório inicialmente:

- sincronização com Google;
- sincronização com Outlook;
- recorrência avançada;
- intervalos multi-dia;
- dependências visuais;
- Gantt;
- reservas de sala;
- gestão de recursos;
- timeboxing obrigatório;
- controle de ponto.

Primeiro entregar o núcleo simples e confiável.

---

## 113. Primeira versão obrigatória

A primeira versão funcional deve possuir:

1. visualização Mês;
  2. escolha do campo de Data;
  3. cartões no dia correspondente;
  4. hover + Adicionar ;
  5. criação contextual;
  6. modal sem sair da tela;
  7. edição de Data via popover;
  8. hora opcional;
  9. click no cartão → drawer;
  10. drag-and-drop entre dias;
  11. Pesquisa;
  12. filtro por Pessoa;
  13. filtros genéricos;
  14. Hoje;
  15. mês anterior/próximo;
  16. personalização básica de cartões;
  17. cor por Status;
  18. itens sem Data;
  19. autosave;
  20. autorização e auditoria.
-

## 114. Critérios de aceite — criação

### CA-CAL-001

Ao passar o mouse sobre um dia vazio:



+ Adicionar

deve aparecer.

### CA-CAL-002

Ao clicar em , a LPS abre o formulário sem sair do Calendário.

### CA-CAL-003

A Data do dia clicado deve vir preenchida.

### CA-CAL-004

Cancelar fecha o formulário e mantém o Calendário exatamente no mesmo período.

### CA-CAL-005

Criar salva o item e o exibe no dia correto sem reload completo da página.

---

## 115. Critérios de aceite — Data e hora

### CA-CAL-006

Clicar na Data abre um seletor contextual.

### CA-CAL-007

O horário não é obrigatório.

### CA-CAL-008

Ativar horário permite selecionar ou digitar uma hora.

### CA-CAL-009

Remover a hora mantém a Data.

### CA-CAL-010

Remover a Data retira o item da grade, mas não exclui o item.

---

## 116. Critérios de aceite — edição

### CA-CAL-011

Clicar no Status permite alterar sem sair da tela.

### CA-CAL-012

Clicar em Responsável permite pesquisar e seleccionar sem sair da tela.

### CA-CAL-013

Alterações pequenas usam autosave.

### CA-CAL-014

Ao salvar, outras visualizações passam a refletir o novo valor.

---

## 117. Critérios de aceite — drag-and-drop

### CA-CAL-015

Um item pode ser arrastado de uma Data para outra.

### CA-CAL-016

O movimento altera somente a Data utilizada pela visualização.

### CA-CAL-017

O item deve mover visualmente antes da confirmação do servidor.

### CA-CAL-018

Falha no servidor deve restaurar a Data anterior.

### CA-CAL-019

A mudança deve passar por autorização e auditoria.

---

## 118. Critérios de aceite — filtros

### CA-CAL-020

Pesquisar filtra os itens visíveis.

### CA-CAL-021

Filtro de Responsável funciona sem alterar os dados.

---

## CA-CAL-022

Combinações de filtros são suportadas.

## CA-CAL-023

Trocar o mês preserva os filtros ativos.

---

## 119. Critérios de aceite — consistência

### CA-CAL-024

Um item criado pelo Calendário deve aparecer na Tabela.

### CA-CAL-025

Se possuir Status compatível, deve aparecer corretamente no Kanban.

### CA-CAL-026

Alterar a Data na Tabela reposiciona o item no Calendário.

### CA-CAL-027

Renomear uma opção de Status atualiza seu texto no Calendário.

### CA-CAL-028

Mudar a cor da opção atualiza sua representação visual.

---

## 120. Critérios de aceite — itens sem Data

### CA-CAL-029

Itens sem Data continuam existentes.

### CA-CAL-030

A visualização mostra contador/acesso para `Sem data`.

### CA-CAL-031

Definir uma Data faz o item aparecer imediatamente na grade.

---

## 121. Critérios de aceite — segurança

### CA-CAL-032

Usuário de outra organização não acessa o item.

---

## CA-CAL-033

Usuário sem permissão de edição não consegue alterar Data via drag-and-drop.

## CA-CAL-034

Esconder botão não substitui a verificação no servidor.

## CA-CAL-035

A atualização deve passar por Service.

---

## 122. Critérios de aceite — experiência

### CA-CAL-036

Operações comuns não devem redirecionar para uma nova página.

### CA-CAL-037

A tela permanece visível atrás de popovers, modais e drawers.

### CA-CAL-038

Ações secundárias aparecem principalmente por contexto/hover.

### CA-CAL-039

A interface não deve exigir botão geral `Salvar Calendário`.

### CA-CAL-040

A densidade deve permanecer legível com vários itens.

---

## 123. Exemplo completo — Demandas

Quadro:

| Título             | Prazo      | Status             | Responsável |
|--------------------|------------|--------------------|-------------|
| Arena Center Norte | 06/10/2026 | Planejamento       | Ryan        |
| GEHAKA             | 12/10/2026 | Aguardando retorno | Luan        |
| Ahlstrom           | 14/10/2026 | Confirmada         | Jennifer    |
| Martinichi         | 16/10/2026 | Em andamento       | Ryan        |

Calendário:

|                    |
|--------------------|
| 06                 |
| Arena Center Norte |
| Planejamento       |
| Ryan               |
| 12                 |
| GEHAKA             |
| Aguardando retorno |
| Luan               |
| 14                 |
| Ahlstrom           |
| Confirmada         |
| Jennifer           |
| 16                 |
| Martinichi         |
| Em andamento       |
| Ryan               |

## 124. Exemplo completo — Tarefas

Quadro:

| Tarefa                       | Prazo      | Responsável | Status       |
|------------------------------|------------|-------------|--------------|
| Agendar visita técnica       | 06/10/2026 | Jennifer    | Planejamento |
| Conferir projetos elétricos  | 07/10/2026 | Ryan        | Em andamento |
| Enviar devolutiva ao cliente | 08/10/2026 | Luan        | Planejamento |
| Lançar pedido de compra      | 09/10/2026 | Ryan        | Em andamento |

Calendário:

|                              |
|------------------------------|
| 06                           |
| Agendar visita técnica       |
| Jennifer                     |
| Planejamento                 |
| 07                           |
| Conferir projetos elétricos  |
| Ryan                         |
| Em andamento                 |
| 08                           |
| Enviar devolutiva ao cliente |
| Luan                         |
| Planejamento                 |
| 09                           |
| Lançar pedido de compra      |
| Ryan                         |
| Em andamento                 |

## 125. Regra de produto

A experiência deve seguir:

ver → clicar → alterar → continuar

e não:

ver → abrir página → procurar formulário → editar → salvar → voltar

## 126. Regra de simplicidade

A tela permanece limpa enquanto o usuário não demonstra intenção.

Por isso:

- **+ Adicionar** aparece no hover;
- menus avançados ficam escondidos;
- detalhes abrem sob demanda;
- cartões mostram apenas o necessário;
- configurações ficam em painel próprio.

## 127. Regra de contexto

O usuário deve executar a ação no lugar em que percebeu a necessidade.

Exemplos:

Viu um espaço no dia 15  
→ cria no dia 15

Viu prazo errado  
→ arrasta para o dia correto

Viu Status errado  
→ clica no Status e corrige

## 128. Regra contra microgerenciamento

O Calendário deve responder perguntas como:

- O que vence esta semana?
- Onde temos muitas entregas no mesmo dia?
- Quais visitas estão programadas?
- Quais compromissos estão atrasados?
- Qual setor concentra entregas?
- Quem possui compromissos simultâneos?



Ele não deve responder, por padrão:

- Quem ficou parado por 12 minutos?
- Quem clicou menos?
- Quem manteve o cronômetro aberto?
- Quem trabalhou cada minuto do dia?

## 129. Regra arquitetural final

Tabela, Kanban e Calendário devem ser três lentes sobre o mesmo trabalho.



Uma mudança em qualquer uma delas atualiza a mesma fonte de verdade.

## 130. Definição final

A visualização por Calendário da LPS é:

Uma representação temporal configurável dos itens de um Quadro, baseada em uma coluna de Data, que permite visualizar, criar, editar e reagendar Demandas e Tarefas diretamente no contexto do período exibido, sem duplicar dados e sem tirar o usuário da tela.

Ela deve unir:

- simplicidade;
- contexto;
- autosave;
- edição inline;
- filtros;
- Data + hora opcional;
- múltiplas visualizações;
- segurança;
- auditoria;
- integração com o restante da LPS.

O Calendário não é outro módulo.

É mais uma lente sobre o mesmo trabalho.

# Apêndice D — Especificação da Visualização por Kanban

## Status de implementação (03/10/2026)

**Resumo.** O Kanban existe hoje em quatro superfícies, todas lendo os mesmos dados da fonte de verdade (nenhuma guarda cartões próprios): (1) **Kanban dos Quadros dinâmicos** ( `/quadros/visoes/<id>/` ), o que mais se aproxima desta especificação (cerca de 80% implementado); (2) **Kanban de Demandas** ( `/demandas/kanban/` ), parcial e com configuração própria; (3) **Kanban de Tarefas** ( `/tarefas/kanban/` ), somente leitura, com três raias por estado derivado; (4) **Kanban legado de Demandas por setor** ( `/demandas/kanban-legado/` ), que agrupa por Etapa do setor, tem limite de coluna e gaveta (a rota `/tarefas/kanban-legado/` existe, mas é interceptada pela regra de rotas desativadas de Tarefas e responde "410 Gone"). As seções 1 a 45 abaixo descrevem a intenção; onde o código seguiu outro caminho há uma nota **Divergência**. Itens futuros foram mantidos no texto.

| Requisito / bloco                                                                                                  | Status                                                                                                                                              | Onde está no código                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visualização configurável sobre o mesmo quadro (sem duplicar itens), vários Kanbans por quadro, renomear e excluir | Implementado                                                                                                                                        | <code>boards/models.py</code> ( <code>BoardView</code> ), <code>boards/services.py</code> ( <code>ViewService</code> ), <code>boards/views.py</code> ( <code>ViewCreateView</code> , <code>ViewUpdateView</code> , <code>ViewDeleteView</code> ), <code>templates/boards/_board_view_tabs.html</code> , <code>static/js/boards.js</code> ( <code>openViewDialog</code> ) |
| Agrupar por Status ou Lista suspensa, com troca sem recriar a visualização                                         | Implementado                                                                                                                                        | <code>boards/kanban.py</code> ( <code>GROUPABLE_TYPES</code> , <code>clean_kanban_settings</code> , <code>resolve_group_column</code> ), <code>templates/boards/board_kanban.html</code>                                                                                                                                                                                 |
| Agrupar por Setor, Prioridade, Pessoa ou Etapa nos Quadros                                                         | Não implementado (só colunas de Status e Lista suspensa; Setor e Prioridade funcionam se o quadro tiver uma coluna de Lista suspensa com esse nome) | <code>boards/kanban.py</code> (comentário de <code>GROUPABLE_TYPES</code> )                                                                                                                                                                                                                                                                                              |
| Raias com contagem, raias vazias, raia "Em branco" (automática, sempre, nunca) e soma de Número/Moeda no cabeçalho | Implementado                                                                                                                                        | <code>boards/kanban.py</code> ( <code>build_lanes</code> , <code>format_total</code> ), <code>templates/boards/_kanban_lanes.html</code>                                                                                                                                                                                                                                 |
| Resumo por média, contagem configurável e densidade do cartão                                                      | Não implementado (há contagem fixa e soma opcional)                                                                                                 | <code>boards/kanban.py</code> ( <code>DEFAULT_SETTINGS</code> )                                                                                                                                                                                                                                                                                                          |
| Renomear e recolorir etiqueta pela raia, editar etiquetas, paleta de 16 cores, definir etiqueta padrão             | Implementado                                                                                                                                        | <code>static/js/boards.js</code> ( <code>renameLane</code> , <code>recolorLane</code> , <code>openLabels</code> ), <code>boards/views.py</code> ( <code>OptionUpdateView</code> ), <code>boards/presentation.py</code> ( <code>COLOR_PALETTE</code> )                                                                                                                    |
| Alterar o Status direto no cartão                                                                                  | Implementado (clique abre o editor da célula)                                                                                                       | <code>templates/boards/_kanban_card.html</code> , <code>static/js/boards.js</code> ( <code>openCellEditor</code> ), <code>boards/views.py</code> ( <code>CellUpdateView</code> )                                                                                                                                                                                         |
| Arrastar cartão entre raias (interface otimista, volta se o servidor recusar)                                      | Implementado                                                                                                                                        | <code>static/js/boards.js</code> ( <code>moveCard</code> , evento <code>drop</code> ), <code>boards/services.py</code> ( <code>CellService.set_value</code> )                                                                                                                                                                                                            |
| Regras ricas no arrastar (pedir motivo, destino bloqueado destacado, reordenar dentro da raia)                     | Não implementado                                                                                                                                    | <code>static/js/boards.js</code> ( <code>moveCard</code> grava só a célula agrupadora)                                                                                                                                                                                                                                                                                   |

| Requisito / bloco                                                                   | Status                                                                                                                 | Onde está no código                                                                                                                                 |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternativa ao arrastar ("Mover para")                                              | Implementado nos Quadros; no Kanban de Demandas novo não verificado/ausente                                            | <code>static/js/boards.js ( openCardMenu )</code>                                                                                                   |
| Configurar cartões (campos, ordem, nome do campo, pré-visualização, até 12 campos)  | Implementado, com setas em vez de arrastar campos; sem capa                                                            | <code>static/js/boards.js ( openKanbanConfig ), boards/kanban.py ( MAX_CARD_FIELDS , resolve_card_columns )</code>                                  |
| Ordenação dos cartões (ordem do quadro, título, criação, qualquer coluna)           | Implementado                                                                                                           | <code>boards/kanban.py ( sort_choice , sort_options ), boards/queries.py ( kanban_items )</code>                                                    |
| Pesquisa e filtro por Pessoa                                                        | Implementado                                                                                                           | <code>boards/queries.py ( search_filter , _filtered_items ), templates/boards/board_kanban.html</code>                                              |
| Filtros por valor de coluna (Responsável, Setor, Prazo, Status)                     | Não implementado (botão "Filtros" desabilitado, "em breve")                                                            | <code>templates/boards/board_kanban.html</code>                                                                                                     |
| Criar item dentro da raia já com o agrupador preenchido                             | Implementado nos Quadros                                                                                               | <code>static/js/boards.js ( addCard ), boards/views.py ( ItemCreateView , parâmetro initial )</code>                                                |
| Gaveta lateral com detalhes ao clicar no cartão                                     | Não implementado no Kanban dos Quadros (o clique no título renomeia); existe no Calendário e no Kanban legado          | <code>static/js/boards.js ( gaveta só no bloco cal ), activities/kanban.py ( ActivityKanbanDrawerView )</code>                                      |
| Permissões no servidor e auditoria                                                  | Implementado                                                                                                           | <code>boards/services.py ( _require , _audit , ações QUADRO_EDITAR e QUADRO_EDITAR_ITEM ), boards/presentation.py</code>                            |
| Autosave das configurações leves                                                    | Implementado nos Quadros; no Kanban de Demandas é por botão "Salvar cartões"                                           | <code>static/js/boards.js ( updateView , debounce de 250 ms), static/js/work-board.js ( submitCardSettings )</code>                                 |
| Kanban de Demandas                                                                  | Parcial / Divergente (raias por Estágio ou Status; cartão com título, responsável, setor, prioridade, prazo e tarefas) | <code>boards/work_views.py ( DemandWorkBoardView ), boards/domain_defaults.py , templates/boards/_work_kanban.html , static/js/work-board.js</code> |
| Kanban de Tarefas                                                                   | Divergente (três raias por estado derivado, somente leitura)                                                           | <code>boards/task_center.py ( kanban_columns ), boards/task_center_views.py , templates/boards/task_center.html</code>                              |
| Kanban legado de Demandas por setor (Etapas do setor, limite informativo de coluna) | Existe, fora desta especificação (a versão de Tarefas está desativada)                                                 | <code>activities/kanban.py , activities/kanban_urls.py , activities/urls.py ( demandas/kanban-legado/ ), static/js/kanban.js</code>                 |
| Responsividade, performance por raia e acessibilidade                               | Parcial (rolagem horizontal; sem carga por raia; sem layout móvel dedicado verificado)                                 | <code>static/css/boards.css , boards/views.py ( kanban_context )</code>                                                                             |

## 1. Objetivo

**Divergência:** "Quadro" na LPS tem duas camadas. Os **Quadros dinâmicos** (app `boards` , `Board / BoardView` ) são o que esta especificação descreve. Já Demandas e Tarefas têm telas próprias: o Kanban de Demandas lê o `DomainBoard` (configuração de lentes sobre `Activity` ) e o Kanban de Tarefas lê itens dos Quadros de cada Demanda. Veja o resumo no início do documento.

A visualização **Kanban** organiza os mesmos itens do Quadro em colunas visuais, normalmente usando um campo categórico como Status, Setor, Prioridade ou Lista suspensa.

A principal referência de experiência é o comportamento observado no Monday: o Kanban não possui dados próprios. Ele lê as colunas do quadro, permite escolher por qual campo agrupar e permite configurar quais informações aparecem nos cartões.

Na LPS, o Kanban deve ser uma **visualização configurável**, e não um módulo independente.

## 2. Princípio central

O Kanban não cria outro conjunto de dados. Ele reorganiza os mesmos itens do quadro.

Exemplo do Quadro:

|                       |                   |             |
|-----------------------|-------------------|-------------|
| Demanda               | Status            | Setor       |
| Validar Go/No-Go      | Trabalhando Nisso | Comercial   |
| Conferir documentação | Done              | Suprimentos |

Kanban agrupado por **Status** :

```
Trabalhando Nisso
- Validar Go/No-Go

Done
- Conferir documentação
```

Kanban agrupado por **Setor** :

```
Comercial
- Validar Go/No-Go

Suprimentos
- Conferir documentação
```

Nenhum item foi duplicado.

## 3. Relação entre Quadro e Kanban

O Quadro controla os campos.

O Kanban decide:

- por qual campo agrupar;
- quais campos mostrar no cartão;
- em qual ordem mostrar;
- filtros;
- ordenação;
- campos visíveis;
- opções visuais próprias da visualização.

## 4. Criação da visualização Kanban

**Divergência:** a janela "Adicionar visualização" (botão `+` nas abas, visível só a quem tem `quadro.editar`) lista Tabela (já existe: é o "Quadro principal"), Kanban, Calendário, Linha do tempo e Dashboard. **Linha do tempo e Dashboard aparecem desabilitados ("Em breve")**. O nome sugerido é "Kanban"; repetido, vira "Kanban 2", "Kanban 3" etc. Todo quadro já nasce com um Kanban ( `ViewService.create_default_kanban` ) e o modelo "Orçamentos" traz um Kanban por Status com soma do Valor.

A partir das abas:

```
[Quadro principal] [Calendário] [+]
```

Ao clicar em `+` :

Adicionar visualização

Tabela  
Kanban  
Calendário  
Linha do tempo  
Dashboard

Escolheu Kanban:

Nome da visualização: Kanban Comercial  
[Criar visualização]

### 4.1 Regra

Criar uma visualização não deve duplicar itens.

Deve criar apenas uma configuração de visualização vinculada ao quadro.

## 5. Estrutura visual

A tela Kanban deve ter:

1. cabeçalho do quadro;
2. abas de visualização;
3. barra de ações;
4. colunas do Kanban;
5. cartões;
6. configurações da visualização;
7. configuração dos cartões.

## 6. Barra de ações

**Divergência:** a barra dos Quadros tem Buscar (campo de texto e botão), filtro **Pessoa**, **Agrupar por**, **Ordenar** e **Configurar cartões**. Agrupar, Ordenar e Configurar só aparecem a quem pode editar o quadro; os demais veem apenas o selo "Agrupado por ...". **"Filtros" está desabilitado ("em breve")**. Não há "Mais ações" e o botão de criar fica no rodapé de cada raia ("Adicionar" + título da coluna principal do quadro), não na barra. No Kanban de Demandas a barra é a de filtros da própria tela de Demandas (abas de escopo, busca, Atrasadas, Vencem hoje, Setor, Etapa, Status e Pessoas).

Exemplo:

```
[Criar demanda] Pesquisar Pessoa Filtro Ordenar Agrupar por: Status Configurar cartões ...
```

Botões principais:

- Criar item;
- Pesquisar;
- Pessoa;
- Filtro;
- Ordenar;
- Agrupar por;
- Configurar cartões;
- Configurações da visualização;
- Mais ações.

## 7. Agrupamento

**Divergência:** nos Quadros só colunas de **Status** e **Lista suspensa** viram raias ( `GROUPABLE_TYPES` ); Pessoa, Prioridade e Setor como tipos próprios ficam para uma próxima versão. Se o quadro não tem nenhuma dessas colunas, a tela mostra "Escolha por qual coluna agrupar". No Kanban de Demandas o servidor aceita agrupar por `stage`, `condition`, `sector`, `owner` e `urgency` (Tarefas: `stage`, `sector`, `responsavel`, `priority`), mas o painel de configuração só oferece **Estágio** e **Status**.

O agrupamento é a regra que define as colunas do Kanban.

Exemplo:

```
Agrupar por: Status
```

Opções compatíveis podem incluir:

- Status;
- Lista suspensa;
- Setor;
- Prioridade;
- Pessoa, em algumas visões;
- outros campos categóricos suportados.

## 7.1 Exemplo observado

**Divergência:** o seletor "Agrupar por" dos Quadros lista só as colunas de Status e Lista suspensa do quadro. "Etapa" como agrupamento existe apenas no Kanban de Demandas e no Kanban legado (Etapas do setor).

O usuário escolhe:

```
Coluna Kanban
[Status v]
```

E o seletor pode listar:

```
Status
Setor
Prioridade
Etapa
```

A LPS deve seguir esse conceito.

## 8. Colunas do Kanban

Cada valor da coluna agrupadora se transforma em uma coluna visual.

Exemplo:

Status possui:

```
Trabalhando Nisso
Stuck
Done
```

Kanban:

```
[Trabalhando Nisso] [Stuck] [Done]
```

### 8.1 Coluna vazia

**Divergência:** a opção é "Mostrar raias vazias" ( `show_empty` , padrão ligado). A raia "Em branco" tem controle próprio (seção 29).

A configuração pode permitir:

```
Mostrar colunas vazias: Sim/Não
```

Se ligado, `Stuck` aparece mesmo sem cartões.

## 8.2 Quantidade

O cabeçalho pode mostrar:

```
Trabalhando Nisso 12
```

## 8.3 Resumo financeiro ou numérico

**Divergência:** implementado como "Somar valor no cabeçalho da raia" ( `sum_column` ), aceitando colunas de **Número** ou **Moeda**, formatado como a coluna (símbolo, unidade, casas decimais). A raia sem itens não mostra total.

Se existir uma coluna de valor compatível:

```
Comercial 8
R$ 4.850.000,00
```

Configuração:

```
Somar valor por coluna: Sim/Não
```

## 9. Renomear uma coluna do Kanban

**Divergência:** o nome da raia é, de fato, o rótulo da etiqueta ( `BoardColumnOption.Label` ); não há outro cadastro. Renomeia-se com duplo clique no título da raia ou pelo menu da raia ("Renomear etiqueta"), e a raia é redesenhada pelo servidor ( `refreshKanban` ), refletindo também na tabela. No Kanban de Demandas, as raia herdada o nome da Etapa ou Status do setor ( `ActivityStage / WorkflowStatus` ), que se edita em Configurações.

No comportamento observado, a coluna do Kanban herda o nome da etiqueta do campo usado para agrupamento.

Isso é fundamental.

Se a etiqueta:

```
Working on it
```

for renomeada inline para:

```
Trabalhando Nisso
```

A coluna do Kanban deve mudar imediatamente.

Não deve existir outro cadastro do nome da coluna do Kanban.

## 10. Alterar a cor da coluna

A cor da coluna deve ser herdada da opção usada para agrupamento.



Exemplo:

```
StatusOption
Nome: Trabalhando Nisso
Cor: rosa
```

A etiqueta na Tabela fica rosa.

O cabeçalho correspondente do Kanban também pode refletir a cor.

A mudança ocorre sem sair da tela.

## 11. Edição inline da etiqueta

**Divergência:** clicar no Status dentro do cartão abre o editor padrão da célula (lista de etiquetas), com o botão "Editar etiquetas". O cartão dos Quadros tem os campos editáveis no lugar; o título do cartão também (clique ou Enter).

Ao clicar em um Status no cartão, abrir um menu rápido.

Exemplo:

```
Trabalhando Nisso
Stuck
Done

Editar etiquetas
```

O usuário pode selecionar outro Status imediatamente.

## 12. Edição de etiquetas

**Divergência:** o painel "Editar etiquetas" permite renomear, mudar a cor, adicionar, reordenar, excluir e marcar a etiqueta padrão ("Padrão para novos itens"); também há "representa conclusão" ( `is_done` ), usado pelo Calendário e pela tela de Tarefas. Excluir uma etiqueta envia seus itens para a raia "Em branco".

A ação `Editar etiquetas` abre um painel compacto.

Possibilidades:

- renomear opção;
- mudar cor;
- adicionar etiqueta;
- reordenar;
- definir padrão;
- aplicar.

Exemplo:

```
[rosa] Trabalhando Nisso ...
[vermelho] Stuck ...
[verde] Done ...
[cinza] Etiqueta padrão ...

+ Nova etiqueta
```

## 13. Paleta de cores

**Divergência:** a paleta oferece 16 cores fixas ( `COLOR_PALETTE` ); não há seletor de cor livre. A cor é propriedade da etiqueta e vale na tabela, no Kanban e no Calendário. Não existe "filtro" que use as etiquetas ainda (filtros por valor são futuros).

O usuário deve conseguir clicar na cor de uma etiqueta e escolher uma nova em uma paleta.

A ação deve:

- atualizar imediatamente;
- refletir em Tabela;
- refletir em Kanban;
- refletir em filtros;
- refletir em outros componentes que usam a mesma opção.

A cor é propriedade da opção do campo, não do cartão.

## 14. Arrastar cartão

**Divergência:** nos Quadros, arrastar grava a etiqueta da coluna agrupadora pelo mesmo endpoint da célula ( `board-cell-update` ), então valem as mesmas validações e a mesma auditoria da edição na tabela; soltar na raia "Em branco" limpa o valor (recusado se a coluna for obrigatória) e quem só visualiza não arrasta. **Reordenar cartões dentro da mesma raia não é persistido** (a ordem vem da ordenação escolhida). No Kanban de Demandas, soltar chama `DomainBoardMutationService.set_value`, que executa `ActivityService.set_stage` (ou `set_condition`, `change_owner` etc.) e só permite arrastar quem tem a ação correspondente ( `can_move_kanban` ).

Drag-and-drop é uma ação central do Kanban.

Exemplo:

```
Trabalhando Nisso Done
[Demanda A] ---> [Demanda A]
```

Semântica:

Se Kanban está agrupado por `Status` :

```
mover cartão → altera Status
```

Se agrupado por `Setor` :

```
mover cartão → solicita/efetua alteração de Setor
```

### 14.1 Regra crítica

**Divergência:** a regra "arrastar é gesto, a regra continua no serviço" vale no Kanban de Demandas e no legado. Nos Quadros não há regra de negócio além das validações da célula (obrigatória, etiqueta ativa da mesma coluna, permissão `quadro.editar_item` ).

Nem todo agrupamento pode permitir alteração por drag automaticamente.

Exemplos:

## Status

Normalmente pode alterar diretamente, respeitando regras de negócio.

## Setor

Pode exigir serviço específico, autorização, notificação, atualização de fila ou confirmação.

## Responsável

Pode exigir regra de atribuição.

Portanto:

Arrastar é gesto visual. A regra de negócio continua sendo executada pelo serviço correspondente.

## 15. Feedback durante drag

**Divergência:** hoje a raia de destino recebe apenas a marcação `is-drop-target` ; não há mensagem "Solte o item aqui..." nem indicação de destino bloqueado. Se o servidor recusar, o cartão volta à posição anterior e aparece uma mensagem (toast).

Ao arrastar:

- cartão deve ganhar elevação;
- destino deve ser destacado;
- coluna deve indicar que aceita o item;
- se destino não for permitido, indicar bloqueio;
- após soltar, atualizar imediatamente;
- se o servidor recusar, devolver à posição anterior.

Exemplo:

Solte o item aqui  
para mover para Suprimentos

## 16. Configuração da visualização

**Divergência:** a configuração é uma **janela modal larga "Configurar cartões"**, e não um drawer. Ela reúne: mostrar raia vazias, raia "Em branco", somar valor, mostrar o nome de cada campo e a lista de campos do cartão com pré-visualização. "Agrupar por" e "Ordenar" ficam na barra.

Botão:

Configurações

ou ícone de engrenagem.

Drawer recomendado:

#### Configurações da visualização

##### Kanban

Agrupar por: Status

Mostrar colunas vazias: [on]

Ordenar cartões por: Prazo

Somar valor por coluna: [on]

## 17. Configurações recomendadas do Kanban

### Agrupar por

Seleciona a coluna responsável pelas lanes.

### Mostrar colunas vazias

Exibe opções sem itens.

### Ordenar cartões por

Exemplos:

- ordem manual;
- prazo;
- prioridade;
- data de criação;
- título;
- valor.

### Mostrar resumo por coluna

**Divergência:** só a **soma** (e a contagem fixa no cabeçalho). Média e demais resumos não foram implementados.

Exemplos:

- soma;
- contagem;
- média.

### Densidade

**Divergência:** não implementado (continua como evolução futura).

Possível evolução:

- compacta;
- normal;
- detalhada.

## 18. Personalização do cartão

**Divergência:** o cartão aceita até **12 campos** ( `MAX_CARD_FIELDS` ). Sem escolha, mostra os **4 primeiros campos visíveis** do quadro (menos o agrupador). O nome do campo só aparece se "Mostrar o nome de cada campo" estiver ligado (padrão desligado). Os campos são as colunas do quadro; "Comentários", "Anexos" e "Tarefas vinculadas" não existem como campo de cartão nos Quadros.

O botão `Configurar cartões` deve abrir um painel lateral.

Exemplo:

```
Personalizar cartão Kanban
```

```
Campos visíveis
```

```
[x] Responsável
```

```
[x] Setor
```

```
[x] Prioridade
```

```
[x] Prazo
```

```
[x] Tarefas vinculadas
```

```
[x] Comentários
```

```
[x] Anexos
```

```
[] Valor estimado
```

```
[] Obra vinculada
```

## 19. Arrastar campos dentro do cartão

**Divergência:** a ordem dos campos do cartão se altera com os botões de seta (subir/descer), não arrastando; o salvamento é automático (250 ms).

O usuário deve poder reorganizar os campos.

Exemplo:

```
ANTES
```

```
Status
```

```
Prazo
```

```
Setor
```

```
Valor
```

```
DEPOIS
```

```
Prazo
```

```
Valor
```

```
Status
```

```
Setor
```

A pré-visualização deve atualizar imediatamente.

## 20. Pré-visualização do cartão

**Divergência:** a pré-visualização mostra uma cópia do primeiro cartão real da visualização (ou a mensagem "Ainda não há cartões para mostrar"). "Exibir imagem de capa" não existe.

O painel de configuração deve mostrar um cartão de exemplo.

Exemplo:

```
Validar Go/No-Go Santa Isabel
```

```
Status: Em andamento
```

```
Prazo: 20/09/2026
```

```
Setor: Comercial
```

```
Responsável: Camila Santos
```

Configurações podem incluir:

```
Mostrar nome da coluna
```

```
Exibir imagem de capa
```

## 21. Card da Demanda

**Divergência:** o cartão do Kanban de Demandas vem de `DomainBoardField` : padrão com **Título, Responsável, Setor, Prioridade, Prazo e Tarefas** (progresso feitas/total). Solicitante, Comentários e Anexos não estão no cartão. O nome dos campos aparece por padrão ( `show_field_names` = verdadeiro). A configuração é salva com o botão "Salvar cartões" e a página recarrega.

Configuração inicial recomendada para Demandas:

```
Título
Solicitante
Responsável
Setor
Prioridade
Prazo
Tarefas vinculadas
Comentários
Anexos
```

Campos adicionais opcionais:

```
Obra
Cliente
Valor estimado
Tags
Descrição resumida
```

## 22. Card da Tarefa

**Divergência:** a Tarefa hoje é um **item do Quadro da Demanda** ( `BoardItem` ); a `Task` antiga ficou como histórico. O cartão do Kanban de Tarefas mostra nome, Demanda, cliente e obra, prazo, Status (etiqueta), responsável e link "Abrir quadro". Não há checklist, bloqueio nem tempo estimado nesse cartão.

Configuração inicial recomendada:

Título  
Demanda vinculada  
Responsável  
Prazo  
Prioridade  
Checklist

Campos adicionais:

Setor  
Participantes  
Bloqueio  
Comentários  
Anexos  
Tempo estimado  
Observações

Tempo deve ser opcional e secundário.

## 23. Abrir detalhes do item

**Divergência:** no Kanban dos Quadros clicar no título do cartão **renomeia o item**; não há gaveta (o endpoint `board-item-detail` existe e é usado pelo Calendário). No Kanban de Demandas o título é um link para a ficha da Demanda. O Kanban legado abre a gaveta ( `activity-kanban-drawer` ) sem sair do quadro.

Clicar no título/cartão deve abrir um drawer lateral.

Evitar abandonar o Kanban.

Para Demanda:

- resumo;
- contexto;
- tarefas;
- arquivos;
- comentários;
- histórico.

Para Tarefa:

- Demanda vinculada;
- responsável;
- prioridade;
- prazo;
- Status;
- checklist;
- comentários;
- arquivos;
- histórico.

## 24. Criar item na coluna

**Divergência:** nos Quadros, "Adicionar" na raia cria o item no primeiro grupo do quadro, já com a etiqueta da raia (ou vazia, na raia "Em branco"), e põe o título em edição. No Kanban de Demandas "Adicionar demanda" abre o assistente de criação (sem pré-preencher a Etapa). O Kanban legado de Demandas cria a Demanda direto na coluna (somente o nome; setor e etapa vêm do quadro, o responsável é quem criou). Criar Tarefa direto numa coluna do Kanban de Tarefas atual é feito pelo botão "Adicionar" da raia (A fazer, Em andamento, Concluídas), que escolhe o Quadro da Demanda e já usa a etiqueta correspondente quando existe.

Cada coluna pode apresentar:

```
+ Adicionar demanda
```

ou:

```
+ Adicionar tarefa
```

Ao criar dentro de uma coluna, o campo agrupador já deve nascer preenchido.

Exemplo:

Criou dentro de `Em andamento` :

```
Status = Em andamento
```

## 25. Filtros

**Divergência:** só há busca por texto e filtro por Pessoa na URL ( `?q=` , `?pessoa=` ); eles não são salvos na visualização. Filtros por valor de coluna estão desabilitados. O Kanban de Demandas tem os filtros da tela de Demandas (abas de escopo, Setor, Etapa, Status, Atrasadas, Vencem hoje e Pessoas); o legado acrescenta cliente, obra e etiqueta.

O Kanban utiliza filtros da visualização.

Exemplos:

```
Responsável = Camila Santos
Setor = Comercial
Prioridade = Alta
Prazo ≤ 7 dias
Status ≠ Concluído
```

O filtro não altera os dados, apenas a visualização.

## 26. Filtro por Pessoa

**Divergência:** nos Quadros é uma lista de pessoas que aparecem em colunas de Pessoa do quadro ("Todas as pessoas" ou uma). As opções "Eu", solicitante, participante e "pessoas do meu setor" não existem; elas existem como abas de escopo na tela de Demandas.



Atalho útil:

Pessoa

Pode permitir:

- Eu;
- responsável específico;
- solicitante;
- participante;
- pessoas do meu setor.

## 27. Pesquisa

**Divergência:** a busca dos Quadros procura em título do item, textos de células, rótulos de etiquetas e nomes de pessoas (não há campo de código, cliente ou obra nos Quadros genéricos).

Pesquisar deve filtrar cartões dinamicamente.

Campos típicos:

- título;
- código;
- cliente;
- obra;
- responsável;
- etiquetas.

## 28. Ordenação

**Divergência:** opções: "Ordem do quadro", Título A→Z/Z→A, Mais recentes/antigos e qualquer coluna do quadro crescente ou decrescente (Prazo, Valor e Prioridade entram se existirem como coluna). A ordem vale dentro de cada raia e fica salva na visualização. Arrastar dentro da raia não vira prioridade manual.

A ordenação pode ser global por coluna Kanban ou por lane.

Opções:

- Manual;
- Prazo crescente;
- Prazo decrescente;
- Prioridade;
- Título;
- Data de criação;
- Valor.

Se ordenação automática estiver ativa, drag dentro da mesma coluna pode ser limitado ou tratado como prioridade manual.

## 29. Coluna "Em branco"

**Divergência:** implementada com o rótulo "Em branco" e três modos ( `auto` : só quando houver itens; `always` ; `never` ). Mover um item para ela limpa a etiqueta, exceto se a coluna agrupadora for obrigatória.

Itens sem valor no campo agrupador devem aparecer em:

Em branco

A visualização pode permitir ocultar essa coluna.

Mover um item para `Em branco` significa limpar o valor, se a regra permitir.

## 30. Alterar agrupamento sem recriar o Kanban

Este é um dos pontos mais importantes.

Exemplo:

Agrupar por: Status

muda para:

Agrupar por: Setor

O Kanban é reorganizado imediatamente.

Não criar outra tela.

## 31. Múltiplos Kanbans do mesmo quadro

**Divergência:** implementado nos Quadros (cada `BoardView` tem `settings` próprio: coluna agrupadora, soma, campos, ordenação). "Kanban por Responsável" não é possível enquanto Pessoa não for agrupável.

O mesmo quadro pode possuir:

Kanban por Status  
Kanban por Setor  
Kanban por Responsável

Cada um tem configurações próprias.

Os itens continuam sendo os mesmos.

## 32. Uso para gestão sem microgerenciamento

O Kanban deve ajudar o gestor a enxergar:

- onde o trabalho está;
- o que está parado;
- onde existe concentração;
- quais itens estão bloqueados;

- quais prazos estão próximos;
- qual setor acumulou demandas;
- quais entregas precisam de intervenção.

Ele não deve ser desenhado para responder:

Quem ficou 17 minutos sem mexer numa tarefa?

A LPS deve favorecer **gestão por exceção**, não vigilância.

## 33. Exemplos de Kanban de Demandas

**Divergência:** no Kanban de Demandas as raias por padrão são as **Etapas** da organização ( `ActivityStage` , ordenadas por `order` ), mais a raia "Sem estágio" (a interface ainda mistura "Etapa" e "Estágio", e o campo padrão do quadro de domínio chama-se "Status" tanto para a Etapa quanto para o Status, em `boards/domain_defaults.py` ); agrupar por Status usa os Status de Demanda. "Por Setor" e "Por Prioridade" têm suporte no servidor, mas sem controle na tela.

### Por Status

Novas | Em andamento | Aguardando retorno | Concluídas

### Por Setor

Comercial | Engenharia | Suprimentos | Financeiro | RH

### Por Prioridade

Alta | Média | Baixa

## 34. Exemplos de Kanban de Tarefas

**Divergência:** o Kanban de Tarefas ( `/tarefas/kanban/` ) tem três raias fixas derivadas da etiqueta de Status do item: **A fazer** (sem Status ou etiqueta padrão), **Em andamento** e **Concluídas** (etiqueta que "representa conclusão"). Agrupar por Setor ou Responsável não existe nessa tela. Cada raia mostra no máximo 50 cartões ( `KANBAN_CAP` ).

### Por Status

A fazer | Em execução | Bloqueadas | Concluídas

### Por Setor

Comercial | Suprimentos | Engenharia | Obras

## Por Responsável

Paulo | Ryan | Luan | Jennifer

Essa última visão é útil para carga de trabalho, mas deve mostrar volume e compromissos, não transformar ausência de interação em julgamento de produtividade.

## 35. Relação com regras de negócio

**Divergência:** o fluxo "soltar, pedir motivo, validar" **não existe** no arrastar. No Kanban legado de Demandas, a ação que pede motivo ("Cancelar demanda") abre janela pelo menu do cartão, não pelo arrastar; as ações de Tarefa do legado (bloquear, mover de setor) foram desativadas. O Kanban de Tarefas atual não permite arrastar.

A visualização não deve ignorar serviços existentes.

Exemplo conceitual:

Usuário arrasta tarefa para Bloqueada

Se a regra exigir motivo:

1. card é solto;
2. sistema abre popup de motivo;
3. usuário informa;
4. service valida;
5. servidor persiste;
6. auditoria e notificação são geradas;
7. card permanece em Bloqueada.

O Kanban não pode contornar regras apenas porque o gesto foi drag-and-drop.

## 36. Permissões

**Divergência:** nos Quadros valem `quadro.visualizar`, `quadro.editar` (criar, renomear, configurar e excluir visualizações), `quadro.gerir_colunas` (criar, editar, reordenar e excluir etiquetas), `quadro.criar_item`, `quadro.editar_item` e `quadro.excluir_item`. Quadros de Demanda usam `DemandBoardAccess`. A conferência é feita no servidor a cada requisição.

Verificar no servidor:

- visualizar item;
- editar item;
- mover Status;
- mudar Setor;
- atribuir responsável;
- gerenciar etiquetas;
- editar visualização;
- personalizar cartões;
- compartilhar.

O fato de um botão estar escondido não substitui autorização.

## 37. Auditoria

**Divergência:** alterações de célula (inclusive o arrastar) geram `BOARD_CELL_UPDATED` com valor antigo e novo; criação, edição e exclusão de visualização geram `BOARD_VIEW_*`. O histórico do quadro fica em `board-history`. No Kanban de Demandas a auditoria é a dos `ActivityService`.

Movimentos que alteram dado de negócio devem ser auditáveis.

Exemplos:

- Status alterado;
- Setor alterado;
- Responsável alterado;
- Prioridade alterada;
- prazo alterado.

Movimentos puramente visuais podem ser tratados separadamente.

## 38. Autosave

**Divergência:** nos Quadros todas as configurações leves (agrupamento, ordenação, raias vazias, soma, campos do cartão) salvam ao alterar, sem botão. No Kanban de Demandas o painel "Configurar cartões" exige "Salvar cartões" e recarrega a página.

Configurações leves devem salvar automaticamente:

- escolha de campos do cartão;
- ordem dos campos;
- mostrar/ocultar campo;
- agrupamento;
- mostrar colunas vazias;
- opção de resumo;
- cor de etiqueta;
- renomeação.

Botão explícito pode existir para configurações maiores, mas não deve ser necessário para cada pequena interação.

## 39. Interface otimista

**Divergência:** o cartão se move na hora e volta se o servidor recusar. Depois de gravar, o navegador pede o HTML das raias de novo ( `board-view-lanes` ) para refletir contagens, soma e ordenação; não se atualiza só o cartão alterado.

Ao mover um cartão:

1. UI move imediatamente;
2. requisição é enviada;
3. servidor valida;
4. se sucesso, mantém;
5. se erro, retorna e mostra mensagem.

O usuário não deve esperar uma página recarregar.

## 40. Performance

**Divergência:** o número de consultas do Kanban dos Quadros não cresce com o número de cartões (células pré-carregadas), mas **todas as raia e cartões são carregados de uma vez**; não há carga por raia nem renderização por viewport. A tela de Tarefas limita 50 cartões por raia e 2.000 itens lidos.

Para muitos cartões:

- carregar por lane;
- limitar renderização fora do viewport;
- atualizar somente cartão alterado;
- buscar contadores de forma eficiente;
- evitar recarregar o board completo;
- debounce nas configurações;
- manter consultas isoladas por organização.

## 41. Responsividade

**Divergência:** há rolagem horizontal entre raia e ajustes de CSS abaixo de 720 px; drawer em tela cheia e gesto de swipe dedicados **não verificados**.

No desktop:

- scroll horizontal entre lanes;
- largura estável das colunas;
- cabeçalho de lane permanece legível.

No mobile:

- uma ou poucas lanes por vez;
- swipe/scroll horizontal;
- drawer ocupa tela;
- ações rápidas continuam acessíveis.

## 42. Acessibilidade

**Divergência:** o menu "Mover para" (alternativa ao arrastar) existe no cartão dos Quadros e "Mover para etapa..." no legado; o Kanban de Demandas atual não tem alternativa ao arrastar. Títulos e campos do cartão são focáveis por teclado (Enter edita); etiquetas têm texto, não só cor.

Drag-and-drop deve possuir alternativa.

Menu do cartão:

```
Mover para
 A fazer
 Em execução
 Bloqueada
 Concluída
```

Também:

- navegação por teclado;
  - foco visível;
  - contraste das cores;
  - não depender apenas da cor para indicar estado.
- 

## 43. O que não fazer

Evitar:

- criar dados exclusivos do Kanban;
  - cadastrar novamente as mesmas etiquetas;
  - fazer o usuário sair da tela para renomear Status;
  - exigir salvar manual após mudar cor;
  - transformar cada agrupamento em módulo separado;
  - permitir drag que viole regra de negócio;
  - esconder falha de persistência;
  - usar texto da etiqueta como identificador interno;
  - mostrar todos os campos no cartão por padrão;
  - usar tempo como principal métrica de produtividade.
- 

## 44. Critérios de aceite

A visualização Kanban está pronta quando:

- ☒ pode ser criada a partir do mesmo quadro; (implementado)
- ☒ não duplica itens; (implementado)
- ☒ permite escolher a coluna de agrupamento; (implementado)
- ☒ suporta Status e Lista suspensa; (implementado)
- ☐ suporta outros agrupamentos aprovados pela LPS; (Quadros: não; Demandas: Estágio e Status na tela, outros no servidor)
- ☒ reflete renomeação de etiqueta imediatamente; (implementado)
- ☒ reflete mudança de cor imediatamente; (implementado)
- ☒ permite selecionar Status diretamente no cartão; (implementado nos Quadros)
- ☒ permite editar etiquetas sem sair do Kanban; (implementado nos Quadros)
- ☒ permite arrastar cartões; (implementado)
- ☐ drag executa regra de negócio correta; (parcial: nos Quadros só a validação da célula; Demandas usa os services)
- ☒ drag inválido é revertido; (implementado)
- ☒ permite mostrar colunas vazias; (implementado)
- ☒ possui coluna Em branco para valores vazios; (implementado)
- ☒ permite ordenar cartões; (implementado)
- ☐ permite filtrar; (parcial: só busca e Pessoa; filtros por valor em breve)
- ☒ permite pesquisar; (implementado)
- ☒ permite configurar campos do cartão; (implementado)
- ☒ permite reordenar campos do cartão; (implementado, por setas)
- ☒ mostra pré-visualização; (implementado)
- ☒ permite múltiplos Kanbans do mesmo quadro; (implementado)
- ☒ alterações leves usam autosave; (implementado nos Quadros; Demandas usa botão)

- ☒ permissões são verificadas no servidor; (implementado)
- ☒ alterações de negócio são auditáveis. (implementado)

---

## 45. Resumo da filosofia

O Kanban da LPS deve transmitir a sensação:

Eu escolho como quero enxergar o trabalho, e os mesmos dados se reorganizam na hora.

A regra central é:

Mesmo dado + outra lente = outra visualização

O Kanban não é um banco separado, não é uma tela rígida e não deve exigir manutenção duplicada.

Ele é uma lente visual sobre o quadro, configurável pelo usuário e sempre conectado à mesma fonte de verdade.



# Apêndice E — Especificação da Visualização por Tabela (Quadro)

## Status de implementação (03/10/2026)

**Resumo.** A Tabela do **Quadro dinâmico** ( /quadros/<id>/ , o "Quadro principal") está implementada em cerca de 65% desta especificação: criação de colunas por + com 8 tipos, edição inline com autosave e interface otimista, renomear, mover (arrastando), redimensionar, ocultar, duplicar, alterar tipo e excluir colunas, etiquetas editáveis na própria tela, ordenação por coluna, busca, filtro por Pessoa, grupos, criar item, Data com horário opcional, permissões no servidor e auditoria. **Não existem ainda:** filtros por condições, agrupar por coluna, resumo de coluna, gaveta de detalhes na Tabela, tipos estruturais e avançados (Relação, Arquivos, Fórmula etc.), compartilhar/automatizar/integrar. As telas de **Demandas** ( /demandas/ ) e de **Tarefas** ( /tarefas/ ) não são a Tabela genérica: têm colunas fixas no HTML e seguem outro desenho (veja a tabela e as notas das seções 29 e 30). Itens ainda não implementados continuam descritos abaixo como requisito futuro.

| Requisito / bloco                                                                                     | Status                                                                                                           | Onde está no código                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modelo Quadro, Grupo, Coluna, Etiqueta, Item e Célula tipada; isolamento por organização              | Implementado                                                                                                     | boards/models.py ( Board , BoardGroup , BoardColumn , BoardColumnOption , BoardItem , BoardCell ), boards/views.py ( get_board e demais get_* filtram por organização) |
| Abas de visualização (Quadro principal, Kanban, Calendário, + )                                       | Implementado (o Quadro principal é implícito, sem registro; Linha do tempo e Dashboard aparecem como "Em breve") | templates/boards/_board_view_tabs.html , boards/models.py ( BoardView ), static/js/boards.js ( openViewDialog )                                                        |
| Cabeçalho do quadro (nome editável, histórico, excluir)                                               | Parcial (sem descrição na tela, favorito, compartilhar, automatizar, integrar)                                   | templates/boards/_board_head.html , boards/views.py ( BoardRenameView , BoardHistoryView )                                                                             |
| Barra de ações (busca, Pessoa, colunas ocultas)                                                       | Parcial (sem Filtro, Agrupar por, Ordenar global nem Configurar visualização)                                    | templates/boards/_board_workspace.html                                                                                                                                 |
| Criar coluna pelo + e escolher o tipo                                                                 | Implementado (8 tipos, sem campo de pesquisa no seletor)                                                         | boards/presentation.py ( TYPE_INFO , type_catalog ), boards/services.py ( ColumnService.create ), static/js/boards.js ( openTypePicker , addColumn )                   |
| Tipos: Status, Lista suspensa, Texto, Data, Pessoa, Número, Moeda, Sinal de confirmação               | Implementado (seleção única; uma pessoa por célula)                                                              | boards/models.py ( BoardColumn.ACTIVE_TYPES ), boards/validators.py ( normalize_cell_value )                                                                           |
| Tipos estruturais e avançados (Relação, Obra, Cliente, Arquivos, Fórmula, Cronograma, Prioridade, IA) | Não implementado (os tipos existem como reservados no catálogo, sem criação nem edição)                          | boards/models.py ( BoardColumn.Type , comentário "Reservados para fases futuras")                                                                                      |
| Edição inline por tipo, autosave e interface otimista com desfazer em caso de erro                    | Implementado                                                                                                     | static/js/boards.js ( openCellEditor , saveCell , inlineEdit ), boards/views.py ( CellUpdateView ), boards/services.py ( CellService.set_value )                       |
| Renomear coluna no lugar                                                                              | Implementado (duplo clique no nome, ou menu "Renomear")                                                          | static/js/boards.js ( startColumnRename ), boards/services.py ( ColumnService.rename )                                                                                 |
| Mover colunas arrastando o cabeçalho                                                                  | Implementado (posição decimal; sem alternativa "mover para a esquerda/direita" no menu)                          | static/js/boards.js (eventos dragstart / drop ), boards/services.py ( ColumnService.reorder , PositionService )                                                        |

| Requisito / bloco                                                                                                              | Status                                                                                                                          | Onde está no código                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Redimensionar colunas (mínimo 96 px, máximo 640 px; teclado)                                                                   | Implementado (sem duplo clique para ajuste automático)                                                                          | <code>static/js/boards.js</code> ( <code>startResize</code> , <code>resizeByKey</code> ),<br><code>boards/models.py</code> ( <code>MIN_COLUMN_WIDTH</code> , <code>MAX_COLUMN_WIDTH</code> )                         |
| Menu da coluna (Configurações, Etiquetas, Ordenar, Duplicar, Adicionar à direita, Alterar tipo, Renomear, Ocultar, Excluir)    | Parcial (sem Filtro, Recolher, Agrupar por nem ações de IA)                                                                     | <code>static/js/boards.js</code> ( <code>openColumnMenu</code> ),<br><code>boards/views.py</code> ( <code>Column*View</code> )                                                                                       |
| Configurações por tipo (descrição, obrigatória, horário, fins de semana, prazo, casas decimais, unidade, mínimo/máximo, moeda) | Parcial (sem validação personalizada, lembretes, restringir edição/visualização, resumo)                                        | <code>static/js/boards.js</code> ( <code>openSettings</code> ),<br><code>boards/validators.py</code> ( <code>clean_column_settings</code> )                                                                          |
| Etiquetas com identificador próprio, cor, ordem, padrão e "representa conclusão"                                               | Implementado                                                                                                                    | <code>boards/models.py</code> ( <code>BoardColumnOption</code> ),<br><code>boards/services.py</code> ( <code>OptionService</code> ),<br><code>static/js/boards.js</code> ( <code>openLabels</code> )                 |
| Ordenação por coluna (crescente, decrescente, remover)                                                                         | Implementado (por parâmetros da URL, recarrega a página; não é salva)                                                           | <code>boards/queries.py</code> ( <code>sort_expression</code> , <code>items</code> ),<br><code>static/js/boards.js</code> ( <code>sortUrl</code> )                                                                   |
| Filtros por condição (operadores por tipo, múltiplos filtros com E)                                                            | Não implementado                                                                                                                | não existe no código                                                                                                                                                                                                 |
| Pesquisa e filtro por Pessoa                                                                                                   | Implementado (sem destaque dos resultados; pessoa em qualquer coluna Pessoa)                                                    | <code>boards/queries.py</code> ( <code>search_filter</code> , <code>_filtered_items</code> )                                                                                                                         |
| Agrupar por coluna (Status, Setor, Responsável...)                                                                             | Divergente (existem Grupos manuais do quadro, não agrupamento por coluna)                                                       | <code>boards/models.py</code> ( <code>BoardGroup</code> ),<br><code>boards/services.py</code> ( <code>GroupService</code> ),<br><code>templates/boards/_group.html</code>                                            |
| Ocultar colunas sem apagar                                                                                                     | Implementado (vale para o quadro todo, não por visualização; recarrega a página)                                                | <code>boards/models.py</code> ( <code>BoardColumn.is_visible</code> ),<br><code>boards/services.py</code> ( <code>ColumnService.set_visible</code> )                                                                 |
| Resumo de coluna (soma, média, contagem por Status)                                                                            | Não implementado na Tabela (só soma por raia no Kanban)                                                                         | <code>boards/kanban.py</code> ( <code>build_lanes</code> )                                                                                                                                                           |
| Criar item inline e menu do item                                                                                               | Parcial (Adicionar, Renomear, Mover para grupo, Excluir; sem Duplicar, Vincular, Arquivar, Copiar link, Ver histórico por item) | <code>static/js/boards.js</code> ( <code>addItem</code> , <code>openItemMenu</code> ),<br><code>boards/services.py</code> ( <code>ItemService</code> )                                                               |
| Gaveta lateral de detalhes ao clicar no título                                                                                 | Não implementado na Tabela (existe no Calendário)                                                                               | <code>boards/views.py</code> ( <code>ItemDetailView</code> ),<br><code>templates/boards/_item_drawer.html</code>                                                                                                     |
| Data com horário opcional (um só tipo)                                                                                         | Implementado (configuração "Mostrar horário" da coluna)                                                                         | <code>boards/validators.py</code> ( <code>DEFAULT_SETTINGS</code> , <code>_normalize_date</code> ), <code>boards/models.py</code> ( <code>BoardCell.value_datetime</code> )                                          |
| Reflexo imediato no Kanban e no Calendário (mesmo dado, mesma etiqueta)                                                        | Implementado                                                                                                                    | <code>boards/kanban.py</code> , <code>boards/calendar_view.py</code> ,<br><code>static/js/boards.js</code> ( <code>refreshKanban</code> , <code>refreshCalendar</code> )                                             |
| Permissões no servidor e auditoria                                                                                             | Implementado                                                                                                                    | <code>boards/services.py</code> ( <code>_require</code> , <code>_audit</code> ),<br><code>boards/presentation.py</code> ( <code>board_permissions</code> ),<br><code>audit/models.py</code> ( <code>BOARD_*</code> ) |
| Performance (paginação ou carga incremental)                                                                                   | Parcial (atualizações por fragmento; sem paginação da tabela)                                                                   | <code>boards/views.py</code> ( <code>BoardDetailView</code> ),<br><code>boards/queries.py</code> ( <code>with_cells</code> , <code>attach_cells</code> )                                                             |

| Requisito / bloco                              | Status                                                                                                        | Onde está no código                                                                                                                                                                                                               |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsividade e acessibilidade                | Parcial (rolagem horizontal, 1ª coluna fixa, teclado nas células; sem alternativa por menu para mover coluna) | <code>static/css/boards.css</code> , <code>static/js/boards.js</code> ( <code>moveFocus</code> , <code>resizeByKey</code> )                                                                                                       |
| Tabela de Demandas ( <code>/demandas/</code> ) | Divergente (colunas fixas no HTML, edição inline própria)                                                     | <code>boards/work_views.py</code> ( <code>DemandWorkBoardView</code> ), <code>templates/boards/demand_work_board.html</code> , <code>static/js/activity-inline-edit.js</code> , <code>static/js/activity-inline-options.js</code> |
| Tabela de Tarefas ( <code>/tarefas/</code> )   | Divergente (lista somente leitura sobre itens dos Quadros de Demanda)                                         | <code>boards/task_center.py</code> , <code>boards/task_center_views.py</code> , <code>templates/boards/task_center.html</code>                                                                                                    |

## 1. Objetivo

**Divergência:** "Quadro" na LPS tem duas camadas. O que esta especificação descreve é o **Quadro dinâmico** (app `boards`): `Board` → `BoardGroup` / `BoardColumn` → `BoardItem` → `BoardCell`. Demandas e Tarefas **não** são mais um quadro genérico configurável: a lista de Demandas é uma tela própria sobre `Activity`, e a "Tarefa" passou a ser um **item do Quadro de cada Demanda** (`Board.kind = DEMAND`, um por Demanda). Os termos da interface são Demanda (antes Atividade), Etapa (antes Situação) e Status (antes Condição); na lista de Demandas a coluna da Etapa ainda aparece como "Estágio".

A visualização por **Tabela (Quadro)** é a visão-base dos dados na LPS.

Ela deve funcionar como uma planilha inteligente e configurável: o usuário enxerga os itens em linhas, os campos em colunas e consegue alterar praticamente tudo sem sair da tela.

A principal referência de experiência é o comportamento observado no Monday: criar colunas, editar valores, renomear, mudar cores, filtrar, ordenar, agrupar, redimensionar e mover colunas diretamente no quadro.

Na LPS, a Tabela não deve ser tratada como uma tela específica de Demandas, Tarefas, Orçamentos ou Obras. Ela deve ser uma **visualização dos mesmos dados do quadro**.

Exemplo:

Quadro: Demandas

Título | Solicitante | Responsável | Setor | Prioridade | Prazo | Status

O mesmo conjunto de dados pode depois ser exibido como Kanban, Calendário ou outra visualização.

## 2. Princípio central

**Divergência:** o princípio está implementado: a etiqueta é uma linha própria (`BoardColumnOption`) referenciada por chave estrangeira pelas células (`BoardCellOption`); renomear ou recolorir não toca nas células, e Tabela, Kanban e Calendário leem a mesma etiqueta. O que ainda **não existe** é o consumo por dashboards, relatórios e automações (não há motor de automações no código dos Quadros).

O dado pertence ao quadro. A Tabela é apenas uma forma de enxergar e editar esse dado.

Consequências:

- não existe um "Status da Tabela" diferente do "Status do Kanban";

- uma alteração feita na Tabela precisa aparecer imediatamente em todas as outras visualizações;
- o usuário não deve preencher a mesma informação em duas telas;
- o sistema deve evitar navegação desnecessária;
- pequenas alterações devem ser salvas automaticamente.

Exemplo:

Status: Working on it

O usuário renomeia para:

Trabalhando Nisso

A LPS deve atualizar automaticamente:

- a célula da Tabela;
- o título da coluna correspondente no Kanban, se o Kanban estiver agrupado por esse Status;
- filtros;
- agrupamentos;
- dashboards;
- relatórios;
- automações relacionadas àquela opção.

A automação não deve depender do texto "Trabalhando Nisso". Ela deve depender do identificador interno da opção.

### 3. Estrutura visual do Quadro

A tela deve ser dividida em cinco áreas principais.

#### 3.1 Cabeçalho do quadro

**Divergência:** o cabeçalho mostra o link "Quadros", o **nome editável no lugar**, o indicador de salvamento, "Histórico" e "Excluir quadro" (conforme permissão). Descrição curta, favorito, Compartilhar, Automatizar, Integrar e menu de mais ações **não existem**. A descrição aparece só na lista de quadros ( /quadros/ ).

Exemplo:

Demandas Integrar | Automatizar | Compartilhar | ...  
Gestão das demandas internas e intersetoriais.

Elementos esperados:

- nome do quadro;
- descrição curta;
- favorito;
- ações do quadro;
- compartilhar;
- automatizar;
- integrar;
- menu de mais ações.

## 3.2 Abas de visualização

**Divergência:** a aba "Quadro principal" é implícita (não tem registro e não se duplica nem se filtra); só Kanban e Calendário podem ter várias visualizações. Filtros e configurações próprias existem nas visualizações Kanban e Calendário (campo `settings` do `BoardView`); busca e Pessoa vão pela URL e não são salvos. Cada quadro nasce com um Kanban; os Quadros de Demanda nascem com Kanban e Calendário.

Exemplo:

```
[Quadro principal] [Kanban] [Calendário] [+]
```

### Comportamento

- cada aba representa uma visualização dos mesmos itens;
- a aba ativa recebe destaque;
- `+` abre a criação de uma nova visualização;
- o usuário pode ter várias visualizações do mesmo tipo;
- cada visualização pode possuir filtros e configurações próprias.

Exemplo:

```
Quadro principal
Minhas demandas
Demandas do Comercial
Kanban por Status
Kanban por Setor
Calendário de prazos
```

## 3.3 Barra de ações

**Divergência:** a barra tem **Buscar** (campo e botão), **Pessoa**, a lista "Colunas ocultas (n)" com "Mostrar" e "Limpar busca e ordem". **Filtro, Agrupar por, Ordenar global e Configurar visualização não existem** (a ordenação é feita pelo menu de cada coluna). O botão de criar item fica no rodapé de cada grupo, como "Adicionar" + o título da coluna principal ( `item_label` , padrão "Nome da Tarefa"); em Quadros de Demanda o rótulo é "Adicionar tarefa".

Abaixo das abas deve existir uma barra de ações leve e sempre acessível.

Exemplo:

```
[Criar demanda] Pesquisar Pessoa Filtro Ordenar Agrupar por Ocultar ...
```

Botões principais:

- Criar item;
- Pesquisar;
- Pessoa;
- Filtro;
- Ordenar;
- Agrupar por;
- Ocultar colunas;
- Configurar visualização;

- Mais ações.

O nome do botão principal deve respeitar o contexto:

```
Criar demanda
Criar tarefa
Criar orçamento
Criar item
```

### 3.4 Cabeçalho de colunas

**Divergência:** nome, menu `...`, arrastar e redimensionar estão implementados. A linha de cabeçalho se repete **dentro de cada grupo**, e a primeira coluna ("Nome da Tarefa" por padrão, ajustável por quadro) é fixa e não tem menu: ela é o nome do item, não uma coluna configurável.

Cada coluna deve apresentar:

- nome;
- área clicável;
- menu `...`;
- suporte a arrastar;
- suporte a redimensionar;
- feedback visual durante interação.

Exemplo:

```
| Demanda | Responsável | Status | Prazo | Setor | + |
```

O `+` no final cria uma nova coluna.

### 3.5 Linhas / itens

**Divergência:** a edição é direta: clicar na célula abre o editor do tipo; o nome do item edita-se no lugar (clique ou Enter). A primeira coluna fica fixa à esquerda na rolagem horizontal.

Cada linha representa um item.

Exemplo:

```
Validar Go/No-Go Santa Isabel | Camila Santos | Em andamento | 20/09/2026 | Comercial
```

O comportamento deve ser de edição direta.

O usuário clica na célula, altera e continua trabalhando.

## 4. Criação de colunas

### 4.1 Ação principal

No final das colunas deve existir:

+

Tooltip:

Adicionar coluna

Ao clicar, abre um seletor de tipos.

### 4.2 Seletor de tipo de coluna

**Divergência:** o seletor mostra os **8 tipos ativos**, nesta ordem: Status, Lista suspensa, Texto, Data, Pessoa, Número, Moeda e Sinal de confirmação, com descrição curta. Não há campo "Pesquise ou descreva sua coluna" nem divisão em Essenciais/Mais úteis. Nomes padrão das novas colunas: "Status", "Lista suspensa", "Texto", "Data", "Pessoa", "Números", "Valor" e "Confirmação"; nomes repetidos ganham sufixo ("Texto 2").

Referência observada:

Pesquise ou descreva sua coluna

Essenciais

|         |                |
|---------|----------------|
| Status  | Lista suspensa |
| Texto   | Data           |
| Pessoas | Números        |

Mais úteis

|                  |                      |
|------------------|----------------------|
| Arquivos         | Extração por IA      |
| Documento        | Fórmula              |
| Conectar quadros | Sinal de confirmação |
| Cronograma       | Prioridade           |

Para a LPS, o catálogo inicial recomendado é:

#### Essenciais

- Status;
- Texto;
- Pessoas;
- Lista suspensa;
- Data;
- Números;
- Moeda;
- Checkbox.

## Estruturais

**Divergência:** nenhum tipo estrutural existe. Relação com outro quadro está apenas reservada no catálogo ( `RELATION` , "Conectar quadros"); Obra, Cliente, Setor, Centro de custo, Demanda vinculada e Tarefa vinculada podem ser imitados com Texto ou Lista suspensa, sem vínculo real. (Nas telas de Demandas e Tarefas, Cliente, Obra e Setor são campos da própria Demanda.)

- Relação com outro quadro;
- Obra;
- Cliente;
- Setor;
- Centro de custo;
- Demanda vinculada;
- Tarefa vinculada.

## Avançadas

**Divergência:** não implementadas. Existem como tipos reservados, que não se criam nem se editam: Arquivo, Cronograma, Prioridade, Confirmação, Conectar quadros, Fórmula e Extração por IA. "Sinal de confirmação" (checkbox) já está ativo, e "Progresso" e "Campo calculado" não têm tipo reservado.

- Arquivos;
- Fórmula;
- Cronograma;
- Progresso;
- Prioridade;
- Sinal de confirmação;
- Campo calculado;
- Extração por Inteligência Artificial.

## 4.3 Fluxo ideal

**Divergência:** o fluxo `+ → tipo → coluna aparece → renomear inline` está implementado: a coluna entra na hora e o campo de nome já abre para digitar.

```
+ → escolher tipo → coluna aparece → renomear inline → usar
```

Evitar:

```
+ → abrir página → preencher formulário → salvar → voltar
```

## 5. Tipos de coluna e comportamento

Cada tipo de coluna deve saber:

- como exibir;
- como editar;
- como validar;



- como ordenar;
- como filtrar;
- como agrupar;
- como resumir;
- como aparecer em um cartão Kanban;
- como ser usado em automações.

## 5.1 Status

**Divergência:** implementado como `BoardColumnOption` (id próprio e imutável, rótulo, cor, posição, `is_default`, `is_done`, `is_active`), com criar, renomear, recolorir, reordenar, excluir (a etiqueta fica inativa e as células que a usavam são limpas), definir padrão (os itens novos já nascem com ela) e marcar "representa conclusão". "Campo obrigatório" existe na coluna. **Validação** não existe. Um Status novo nasce com "Não iniciado" (padrão), "Em andamento" e "Concluído" (conclusão).

Exemplo:

```
Em andamento
Aguardando retorno
Concluído
```

Configurações:

- nome da coluna;
- criar etiqueta;
- renomear etiqueta;
- mudar cor;
- reordenar etiquetas;
- excluir/desativar etiqueta;
- definir etiqueta padrão;
- indicar quais etiquetas representam conclusão;
- validação;
- campo obrigatório.

Regra importante:

Cada etiqueta deve possuir um identificador interno imutável.

```
ID: status_01
Nome: Trabalhando Nisso
Cor: rosa
```

Renomear o texto ou mudar a cor não pode quebrar filtros, automações ou histórico.

## 5.2 Lista suspensa

**Divergência:** só **seleção única** (a célula guarda uma etiqueta, embora a tabela de apoio aceite várias); sem valor padrão próprio além da "etiqueta padrão", sem seleção múltipla, sem validação.

Permite uma ou mais opções, conforme configuração.

Configurações:

- opções;
- cores;
- seleção única ou múltipla;
- valor padrão;
- obrigatório;
- validação.

## 5.3 Texto

**Divergência:** um único tipo de texto de até **5.000 caracteres**, com campo de uma linha na edição. Não há "curto/longo", tamanho máximo configurável, placeholder nem validação.

Configurações:

- texto curto ou longo;
- tamanho máximo;
- obrigatório;
- placeholder;
- validação.

Edição:

clique → digite → Enter

## 5.4 Pessoas

**Divergência:** a célula guarda **uma pessoa** da mesma organização, com busca no servidor; marcar "várias pessoas" é recusado ("Várias pessoas ficam para uma versão futura"). Restrição por setor ou papel não existe.

Permite selecionar usuários da organização.

Configurações:

- uma pessoa;
- múltiplas pessoas;
- restringir por setor;
- restringir por papel;
- obrigatório.

O seletor deve permitir pesquisa.

## 5.5 Data

**Divergência:** é um só tipo de Data, com configurações: **Mostrar horário** ( `show_time` ), **Permitir sábado e domingo** ( `allow_weekends` ), **Tratar como prazo** ( `is_deadline` , destaca "vencido") e formato ( `DD/MM/YYYY` , `DD/MM/YY` , `YYYY-MM-DD` , aceito pelo servidor). A edição abre um seletor com "Hoje" e "Limpar". Lembretes, sincronização com calendário externo, número da semana e ícone de data não existem.

A Data é um dos melhores exemplos de configuração inline observada.

Estado simples:

```
17/09/2026
```

O usuário pode habilitar horário clicando no relógio.

Estado completo:

```
17/09/2026 09:00
```

A LPS não precisa criar dois tipos diferentes de campo.

Deve existir uma única coluna do tipo Data com configuração:

```
Exibir horário: sim/não
```

Configurações possíveis:

- exibir horário;
- formato da data;
- habilitar final de semana;
- definir como prazo;
- lembretes;
- sincronização com calendário no futuro;
- mostrar número da semana;
- ícone de data.

---

## 5.6 Número

**Divergência:** implementados casas decimais (0 a 6), **unidade** (até 12 caracteres, por exemplo "%"), mínimo e máximo. Formato e as operações de resumo (soma, média, mínimo, máximo, contagem) **não existem** na Tabela; a soma só é usada no cabeçalho das raias do Kanban.

Configurações:

- casas decimais;
- mínimo;
- máximo;
- unidade;
- formato;
- validação.

Operações de resumo:

- soma;
- média;
- mínimo;
- máximo;
- contagem.

## 5.7 Moeda

**Divergência:** moedas aceitas: Real (R\$), Dólar (US\$) e Euro (€); casas decimais de 0 a 6; mínimo e máximo; exibição no padrão brasileiro ( R\$ 1.250.000,00 ). Resumo por soma/média na Tabela não existe.

Configurações:

- moeda;
- casas decimais;
- separador;
- resumo por soma/média.

Exemplo:

R\$ 1.250.000,00

## 6. Edição inline

A edição inline é uma regra obrigatória do produto.

### 6.1 Renomear coluna

**Divergência:** o gesto é **duplo clique** no nome (ou "Renomear" no menu); um clique simples no nome não edita. Enter confirma e Esc cancela; se o servidor recusar, o nome anterior volta.

Fluxo:

clicar no nome → editar → Enter

ou:

... → Renomear

Sem abrir nova página.

### 6.2 Editar célula

**Divergência:** Status e Lista abrem uma lista de etiquetas com "Limpar" e "Editar etiquetas"; Pessoa abre busca; Data abre seletor; Texto e Número abrem campo na própria célula (Enter confirma, Esc cancela, setas navegam entre células, F2 e Enter abrem, Delete limpa); a confirmação alterna ao clique ou à barra de espaço.

Exemplos:

#### Status

clique → opções → selecionar

## Pessoa

clique → pesquisar pessoa → seleccionar

## Data

clique → calendário → data

## Texto

clique → digitar → Enter

## Número

clique → digitar → Enter

## 7. Autosave

**Divergência:** todas as ações listadas salvam sozinhas, **com duas exceções de comportamento:** "Ocultar coluna" e "Mostrar coluna" recarregam a página, e a **ordenação** é feita por navegação (parâmetros `sort` e `dir` na URL), sem ser gravada. O texto é gravado ao confirmar (Enter ou sair do campo), sem debounce a cada tecla.

Pequenas alterações devem ser persistidas automaticamente.

Não utilizar botão genérico:

Salvar quadro

para operações rotineiras.

Exemplos de autosave:

- renomear coluna;
- editar célula;
- mudar cor de etiqueta;
- criar etiqueta;
- redimensionar coluna;
- mover coluna;
- ocultar coluna;
- habilitar horário na Data;
- mudar ordenação.

### 7.1 Interface otimista

**Divergência:** implementado: a célula muda na hora e volta ao valor anterior, com mensagem, se o servidor recusar; o servidor devolve o HTML final da célula, do cabeçalho ou da linha.

A interface deve responder antes da confirmação do servidor.

Exemplo:

1. usuário altera `Em andamento` para `Em execução` ;
2. a tela muda imediatamente;
3. a LPS envia a alteração;
4. se sucesso: nenhuma interrupção;
5. se erro: desfazer e mostrar mensagem.

## 8. Mover colunas

**Divergência:** a ordem ( `position` , decimal com passo 1000) e a largura pertencem à **coluna do quadro**, e valem para todas as visualizações; não há ordem por visualização. Soltar mostra o lado de destino (antes/depois) e uma falha devolve a coluna ao lugar anterior. Não há alternativa por menu ("Mover coluna para a esquerda/direita").

O cabeçalho deve ser arrastável.

Exemplo:

```
ANTES
Demanda | Pessoa | Data | Status

DEPOIS
Demanda | Status | Pessoa | Data
```

O movimento precisa:

- ser fluido;
- mostrar a posição de destino;
- preservar largura;
- persistir automaticamente;
- ser independente por visualização, se essa for a decisão do produto.

## 9. Redimensionar colunas

**Divergência:** largura mínima 96 px e máxima 640 px (padrão 160 px), gravada por coluna; também funciona pelo teclado (setas, 16 px por passo, 48 px com Shift). **Duplo clique para ajuste automático não existe.**

O usuário deve poder arrastar a borda lateral do cabeçalho.

Exemplo:

```
| Responsável |
```

vira:

```
| Responsável |
```

Persistir `width` automaticamente.

Recomendado:

- largura mínima;

- largura máxima;
- duplo clique para ajuste automático;
- cursor visual de resize.

## 10. Menu de coluna

**Divergência:** o menu real tem: Configurações da coluna e Editar etiquetas (Status e Lista), **Ordenar ascendente**, **Ordenar descendente**, Remover ordenação, **Duplicar coluna**, **Adicionar coluna à direita**, **Alterar tipo** (com aviso de quantos valores serão apagados), Renomear, **Ocultar coluna** e Excluir coluna. As ações de gestão só aparecem a quem tem `quadro.gerir_colunas`. Não existem "Ações assistidas por IA", Filtro, Recolher nem Agrupar por.

Cada coluna deve possuir menu `...`.

Estrutura recomendada:

```
Configurações >
Ações assistidas por IA >
Filtro
Ordenar >
Recolher
Agrupar por

Duplicar coluna
Adicionar coluna à direita
Alterar tipo de coluna

Renomear
Excluir
```

Na LPS, algumas opções podem variar conforme permissão e tipo.

## 11. Configurações específicas da coluna

**Divergência:** a janela "Configurações" tem **descrição** (até 500 caracteres, aparece ao passar o mouse), **valor obrigatório** e as opções do tipo (Data: horário, fins de semana, prazo; Número e Moeda: casas, unidade, moeda, mínimo e máximo; Status e Lista: botão "Editar etiquetas"). Validação personalizada, lembretes, restringir edição ou visualização e resumo da coluna **não existem**.

O submenu de Configurações precisa variar pelo tipo.

### Exemplo: Data

```
Personalizar coluna de Data
Adicionar descrição
Definir como obrigatória
Definir validação
Definir como prazo
Adicionar/editar lembretes
Restringir edição
Restringir visualização
Exibir resumo da coluna
```

## Exemplo: Status

- Personalizar coluna de Status
- Adicionar descrição
- Definir como obrigatória
- Definir validação
- Restringir edição
- Restringir visualização
- Exibir/ocultar resumo

## 12. Ordenação

**Divergência:** a ordenação é aplicada **dentro de cada grupo** (os grupos mantêm sua ordem), com vazios por último, e Status/Lista ordenam pela ordem das etiquetas e Pessoa pelo primeiro nome. Vale só até sair da página (não é salva).

Toda coluna compatível deve oferecer Ordenar.

Modos:

- Crescente
- Decrescente
- Remover ordenação

A lógica depende do tipo.

### Texto

Ordem alfabética.

### Número

Ordem numérica.

### Moeda

Ordem pelo valor numérico.

### Data

Ordem cronológica.

### Pessoas

Ordem pelo nome exibido.

### Status / Lista

Preferencialmente pela ordem configurada das etiquetas, e não pelo nome alfabético.



## 13. Filtros

**Divergência: não implementado.** Hoje só existem a busca por texto e o filtro por Pessoa. Os operadores de 13.1 e a combinação de 13.2 continuam como requisito futuro (o Kanban também exibe "Filtros" desabilitado, "em breve").

O botão global `Filtro` deve permitir construir condições sem sair da visualização.

Exemplos:

```
Status = Em andamento
Responsável = Paulo
Setor = Comercial
Prazo antes de 30/09/2026
Prioridade = Alta
```

### 13.1 Operadores

#### Texto

- contém;
- não contém;
- é igual;
- está vazio;
- não está vazio.

#### Número / Moeda

- igual;
- maior que;
- menor que;
- entre;
- vazio.

#### Data

- hoje;
- amanhã;
- antes de;
- depois de;
- entre;
- atrasado;
- vazio.

#### Pessoas

- é;
- não é;
- contém;
- vazio.

#### Status / Lista

- é;

- não é;
- qualquer um entre;
- vazio.

---

## 13.2 Múltiplos filtros

Permitir combinação:

```
Setor = Comercial
E
Status ≠ Concluído
E
Prazo ≤ hoje + 7 dias
```

---

## 14. Pesquisa

**Divergência:** a busca (até 120 caracteres) procura no nome do item, em textos de células, em rótulos de etiquetas e em nomes de pessoas. Não há destaque dos resultados nem campos "cliente", "obra" ou "código" nos Quadros genéricos.

A busca deve pesquisar rapidamente os dados visíveis do quadro.

Pode incluir:

- título;
- texto;
- pessoas;
- cliente;
- obra;
- etiquetas;
- código.

Resultados devem ser destacados sem mudar de página.

---

## 15. Filtro por pessoa

**Divergência:** o seletor lista as pessoas que aparecem em colunas de Pessoa do quadro e filtra itens em que a pessoa aparece em **qualquer** coluna Pessoa. Não há "Eu", "Meu setor" nem escolha entre Responsável, Solicitante e Participante (nas telas de Demandas e Tarefas existem abas e filtros de escopo próprios).

O atalho `Pessoa` deve facilitar visões pessoais.

Exemplos:

```
Eu
Paulo Bonfim
Camila Santos
Meu setor
```

Pode filtrar por campos como:

- Responsável;

- Solicitante;
- Participante.

## 16. Agrupar por

**Divergência:** a Tabela dos Quadros usa **Grupos manuais** ( BoardGroup : nome, cor, ordem), não agrupamento por uma coluna. Cada grupo tem cabeçalho com expandir/recolher (a escolha é lembrada no navegador), nome editável, contagem ("N tarefas"), cor e menu (renomear, mudar a cor, mover para cima/baixo, excluir; só exclui grupo vazio). **Agrupar por Status/Setor/Responsável não existe**, e o cabeçalho do grupo não mostra somas.

A Tabela pode ser dividida em grupos.

Exemplo por Status:

```
Novas
 Demanda 1
 Demanda 2

Em andamento
 Demanda 3

Concluídas
 Demanda 4
```

Possíveis colunas de agrupamento:

- Status;
- Setor;
- Responsável;
- Prioridade;
- Lista suspensa;
- outras colunas categóricas compatíveis.

### 16.1 Cabeçalho de grupo

**Divergência:** implementados nome, contagem, cor, menu e expandir/recolher; somas e resumos não.

Pode mostrar:

- nome;
- quantidade de itens;
- somas/resumos;
- cor;
- menu;
- expandir/recolher.

Exemplo:

```
Em andamento 12 | R$ 3.250.000,00
```

## 17. Ocultar colunas

**Divergência:** "Ocultar coluna" está no menu da coluna e as ocultas ficam na lista "Colunas ocultas (n)", com "Mostrar". O estado ( `BoardColumn.is_visible` ) pertence ao **quadro**, não à visualização, e a página recarrega.

O usuário deve poder esconder colunas sem apagá-las.

Exemplo:

```
Ocultar:
[] Cliente
[x] Observações internas
[] Prazo
```

A configuração pertence à visualização.

## 18. Resumo de coluna

**Divergência: não implementado** na Tabela. Só há a soma de uma coluna de Número/Moeda no cabeçalho das raías do Kanban.

Dependendo do tipo, o rodapé ou cabeçalho pode mostrar resumo.

Exemplos:

### Moeda

Soma: R\$ 8.250.000,00

### Número

Média: 42%

### Status

3 Em andamento | 2 Concluídos | 1 Bloqueado

## 19. Criar item

**Divergência:** implementado: "+ Adicionar ..." no rodapé do grupo cria a linha (já com a etiqueta padrão nas colunas de Status/Lista) e abre o nome para digitar; Enter confirma e os demais campos são editados na linha. Não há formulário completo alternativo nos Quadros genéricos; em Quadros de Demanda, a exclusão de item pede o **motivo do cancelamento**.

O usuário deve poder criar item sem formulário complexo.

Exemplo:

+ Adicionar demanda

Ao clicar:

- linha aparece;
- foco vai para Título;
- usuário digita;
- Enter confirma;
- demais campos podem ser preenchidos inline.

Formulário completo continua disponível para cadastros com contexto complexo.

## 20. Menu do item

**Divergência:** o menu do item tem **Mover para o grupo**, **Renomear** e **Excluir item** (conforme permissão). Abrir detalhes, Duplicar, Vincular, Arquivar, Copiar link e Ver histórico por item **não existem** (o histórico do quadro inteiro está em `Histórico`). A linha também se move arrastando a alça, o que persiste grupo e posição.

Cada linha pode ter `...`.

Possíveis ações:

- Abrir detalhes;
- Duplicar;
- Mover para grupo;
- Vincular;
- Arquivar;
- Excluir, conforme regra;
- Copiar link;
- Ver histórico.

## 21. Abertura de detalhes

**Divergência: não implementado na Tabela:** clicar no nome renomeia o item. A gaveta do item (campos editáveis no lugar e últimas alterações) existe no **Calendário** ( `board-item-detail` ). Em Demandas, o título leva à ficha da Demanda; a gaveta com resumo, tarefas, comunicação e histórico existe no Kanban legado ( `activity-kanban-drawer` ).

Clicar no título pode abrir um drawer lateral, mantendo o quadro no fundo.

Para Demandas, o drawer pode mostrar:

- Resumo;
- Contexto;
- Tarefas vinculadas;
- Comentários;
- Arquivos;
- Histórico.

Para Tarefas:

- Demanda vinculada;
- Responsável;
- Prioridade;

- Prazo;
- Status;
- Checklist;
- Comentários;
- Arquivos;
- Histórico.

A intenção é evitar abandonar o contexto do quadro.

## 22. Data + hora

**Divergência:** implementado como **configuração da coluna** ("Mostrar horário", em "Configurações"), e não por um relógio dentro do seletor. A mesma coluna guarda data ( `value_date` ) e, quando exhibe horário, também data e hora ( `value_datetime` ); a hora é opcional por célula e nunca se inventa 00:00.

Comportamento obrigatório baseado no fluxo observado:

1. coluna Data começa simples;
2. usuário abre o seletor;
3. calendário aparece;
4. existe ação de relógio / adicionar horário;
5. ao habilitar, o seletor apresenta data + hora;
6. a mesma célula passa a armazenar horário.

Exemplo:

Antes: 17/09/2026  
Depois: 17/09/2026 09:00

Não criar outra coluna só para isso.

## 23. Relação com Kanban

**Divergência:** implementado: Kanban e Calendário leem as mesmas células e etiquetas, e renomear ou recolorir a etiqueta muda a raia e a cor nas três visões.

A Tabela é a melhor visão para configurar e editar os dados.

O Kanban lê esses mesmos dados.

Exemplo:

Tabela:

| Demanda | Status            |
|---------|-------------------|
| A       | Trabalhando Nisso |
| B       | Done              |

Kanban agrupado por Status:

```
Trabalhando Nisso
- A

Done
- B
```

Se o usuário renomear `Trabalhando Nisso` para `Em execução`, a coluna do Kanban precisa mudar automaticamente.

## 24. Permissões

**Divergência:** ações verificadas no servidor ( `quadro.visualizar`, `quadro.editar`, `quadro.gerir_colunas`, `quadro.criar_item`, `quadro.editar_item`, `quadro.excluir_item`, `quadro.excluir`, `quadro.criar` ); criar, renomear, configurar e excluir visualizações pedem `quadro.editar`. "Compartilhar visualização" não existe (as visualizações são compartilhadas por todos que veem o quadro). Quadros de Demanda usam também a relação da pessoa com a Demanda ( `DemandBoardAccess` ).

A interface pode esconder ações, mas isso não é segurança.

O servidor precisa validar permissões para:

- visualizar quadro;
- editar item;
- editar coluna;
- criar coluna;
- excluir coluna;
- mudar configuração;
- gerenciar opções;
- criar visualização;
- compartilhar visualização.

No modelo da LPS, a autorização deve continuar seguindo o motor de ação + escopo.

## 25. Histórico e auditoria

**Divergência:** registrados ( `audit.AuditLog`, ações `BOARD_*` ): quadro, grupo, coluna (criação, alteração, **movimento**, **redimensionamento**, exclusão), item (criação, alteração, movimento, exclusão), célula (valor antigo e novo) e visualização. Mudanças visuais (largura, ordem) **são auditadas** em vez de ignoradas. A tela "Histórico" do quadro é paginada.

Alterações relevantes devem ter histórico.

Recomendado auditar:

- criação/exclusão de coluna;
- mudança de tipo;
- renomeação de etiqueta;
- mudança de campo relevante;
- movimentação de item que altera um dado de negócio;
- mudança de responsável;
- mudança de prazo;

- mudança de prioridade;
- mudança de Status.

Mudanças puramente visuais podem ter nível de auditoria diferente:

- largura;
- ordem visual;
- coluna escondida.

---

## 26. Performance

**Divergência:** as gravações atualizam só a célula, o cabeçalho ou a linha (fragmentos HTML) e o número de consultas não cresce com o número de itens (células pré-carregadas), mas a Tabela **carrega todos os itens do quadro de uma vez, sem paginação nem carga incremental**. Os metadados das colunas vão num `json_script` na própria página.

A Tabela precisa continuar fluida com muitos itens.

Regras recomendadas:

- paginação ou carregamento incremental;
- atualizações parciais;
- evitar recarregar a página inteira;
- debounce em autosave de texto;
- operações de drag com resposta local;
- cache de metadados de colunas;
- queries filtradas por organização.

---

## 27. Responsividade

**Divergência:** há rolagem horizontal e a primeira coluna e o cabeçalho ficam fixos; o drawer em tela cheia no celular **não se aplica** (não há drawer na Tabela). Ajustes de CSS abaixo de 720 e 760 px existem.

Desktop é a experiência principal.

Em telas menores:

- permitir scroll horizontal;
- preservar a primeira coluna quando útil;
- menus devem caber no viewport;
- drawer pode ocupar tela inteira no mobile;
- evitar comprimir colunas a ponto de ficarem ilegíveis.

---

## 28. Acessibilidade

**Divergência:** teclado: Enter e F2 editam, Esc cancela, setas navegam entre células, setas redimensionam a coluna; há rótulos ARIA e aviso de status. **Faltam** as alternativas por menu para mover coluna.

Além de arrastar com mouse, oferecer alternativas por menu.



Exemplo:

```
Mover coluna para esquerda
Mover coluna para direita
```

Também:

- foco de teclado;
- Enter para editar;
- Escape para cancelar;
- contraste suficiente;
- tooltips;
- ícones acompanhados de rótulos nas ações críticas.

## 29. Regras específicas para Demandas

**Divergência:** a lista de Demandas ( /demandas/ , modo "Lista") tem **colunas fixas no HTML**, e não colunas configuráveis: Demanda (título com código, cliente e obra), Cliente / Obra, Setor, Responsável, Tarefas (progresso "feitas de total" e percentual), Prazo, **Estágio** (Etapa) e **Status**. **Solicitante e Prioridade não aparecem como colunas**. A edição inline cobre título, responsável, prazo, setor, Estágio e Status (e Cliente / Obra abre a janela "Editar demanda" na etapa 2); quem não tem permissão vê a célula de leitura. Abas de escopo: "Sob minha responsabilidade", "Do meu setor", "Em que participo", "Concluídas" e "Todas as demandas" (conforme permissão), com filtros por busca, Setor, Etapa, Status, pessoas e atalhos "Atrasadas" e "Vencem hoje". A configuração de campos e visões do `DomainBoard` ( `DomainBoardField` , `DomainBoardView` ) existe, mas hoje só alimenta o cartão do Kanban de Demandas.

Colunas iniciais sugeridas:

```
Título da demanda
Solicitante
Responsável
Setor
Prioridade
Prazo
Status
Tarefas vinculadas
Obra
Cliente
```

O administrador pode adicionar outras.

O usuário não deve ser obrigado a manter todas visíveis.

## 30. Regras específicas para Tarefas

**Divergência:** a Tarefa é um **item do Quadro da Demanda** ( `BoardItem` ), que nasce com as colunas **Responsável** (Pessoa), **Status** (Não iniciado, Em andamento, Concluído) e **Data**; o quadro pode ganhar outras colunas e vir de um modelo. A tela /tarefas/ é **somente leitura**: lista tarefas em que a pessoa é responsável (ou, com permissão, de outras pessoas) com Nome da Tarefa, Demanda, Cliente / Obra, Responsável, Status, Prazo e "Abrir quadro", com abas Lista, Kanban e Calendário semanal, filtros de busca, estado (A fazer, Em andamento, Concluídas) e prazo, e 50 linhas por página. Dependência, tempo estimado, tempo registrado, tags e bloqueio **não existem** nesse modelo. O modelo antigo `Task` ficou como histórico.

Colunas iniciais sugeridas:

Título da tarefa  
Demanda vinculada  
Responsável  
Setor  
Prioridade  
Prazo  
Status  
Checklist

Campos opcionais:

Participantes  
Dependência  
Tempo estimado  
Tempo registrado  
Tags  
Bloqueio

Tempo deve ser informação auxiliar, não o centro do gerenciamento.

---

## 31. O que não fazer

Evitar:

- formulário para toda pequena edição;
- status duplicados por visualização;
- salvar manualmente cada alteração;
- tabela rígida por módulo;
- colunas codificadas diretamente no HTML;
- usar o nome da etiqueta como chave de automação;
- recarregar a página após editar uma célula;
- criar `Data` e `Data + Hora` como tipos independentes;
- obrigar o gestor a visualizar todos os campos;
- transformar o quadro em instrumento de vigilância individual.

---

## 32. Critérios de aceite

A visualização Tabela está pronta quando:

- ☒ o usuário pode criar coluna pelo `+`; (implementado)
- ☒ pode escolher o tipo; (8 tipos)
- ☒ pode renomear sem sair da tela; (implementado)
- ☒ pode editar células inline; (implementado)
- ☒ pode mover colunas; (arrastando; sem alternativa por menu)
- ☒ pode redimensionar colunas; (implementado)
- ☒ pode ocultar colunas; (por quadro, não por visualização)
- ☒ pode ordenar qualquer tipo compatível; (por parâmetros da URL)
- ☐ pode filtrar por múltiplas condições; (não implementado; só busca e Pessoa)
- ☐ pode agrupar por coluna compatível; (não implementado; há grupos manuais)
- ☒ pode editar etiquetas de Status na própria tela; (implementado)

- ☒ pode mudar a cor das etiquetas sem sair da tela; (implementado)
- ☒ mudanças são refletidas no Kanban automaticamente; (implementado)
- ☒ Data pode opcionalmente receber horário; (configuração da coluna)
- ☒ pequenas alterações usam autosave; (implementado)
- ☒ o usuário recebe feedback de erro sem perder o contexto; (implementado)
- ☒ permissões são verificadas no servidor; (implementado)
- ☒ o tenant da organização é respeitado; (implementado)
- ☒ alterações relevantes possuem histórico. (implementado)

---

## 33. Resumo da filosofia

A visualização Tabela da LPS deve transmitir esta sensação:

Eu clico no dado que quero mudar, altero e continuo trabalhando.

Ela é o local mais completo para estruturar os dados, mas não deve parecer um cadastro burocrático.

A regra é:

`selecionar > editar > continuar`

não:

`abrir tela > preencher formulário > salvar > voltar`

A Tabela é a fundação do quadro. O Kanban, o Calendário e as demais visualizações são outras lentes sobre os mesmos dados.